SFUSE_Documentation

다음에 의해 생성됨 : Doxygen 1.14.0

1 SFUSE 1
1.0.0.1 SFUSE 미리보기 . . . 1
1.0.1 사용 방법 . . . 1
1.0.1.1 1. 다운로드 . . . 1
1.0.1.2 2. 코드 컴파일 및 마운트 . . . 1
1.0.1.3 3. 언마운트 방법 . . . 1
1.0.1.4 Reference . . . 2
1.0.1.5 NOTE . . . 2

2 SFUSE HISTORY 3
2.1 v0.0.1 (2024년 12월 30일) . . . 3
2.1.1 New Features . . . 3
2.1.2 Bug Fixes . . . 3

3 실험 결과 5
3.0.1 실험 결과 그래프 . . . 5

4 fio 워크플로우 7
4.0.1 SFUSE의 워크플로우 결과 . . . 7
4.0.2 Ext4의 워크플로우 결과 . . . 9

5 데이터 구조 색인 11
5.1 데이터 구조 . . . 11

6 파일 색인 13
6.1 파일 목록 . . . 13

7 데이터 구조 문서화 15
7.1 sfuse_dirent 구조체 참조 . . . 15
7.1.1 상세한 설명 . . . 15
7.1.2 필드 문서화 . . . 15
7.1.2.1 ino . . . 15
7.1.2.2 name . . . 15
7.2 sfuse_fs 구조체 참조 . . . 15
7.2.1 상세한 설명 . . . 16
7.2.2 필드 문서화 . . . 16
7.2.2.1 backing_fd . . . 16
7.2.2.2 block_map . . . 16
7.2.2.3 inode_map . . . 16
7.2.2.4 sb . . . 17
7.3 sfuse_inode 구조체 참조 . . . 17
7.3.1 상세한 설명 . . . 17
7.3.2 필드 문서화 . . . 17
7.3.2.1 atime . . . 17
7.3.2.2 ctime . . . 18

7.3.2.3 direct . . . 18
7.3.2.4 double_indirect . . . 18
7.3.2.5 gid . . . 18
7.3.2.6 indirect . . . 18
7.3.2.7 links . . . 18
7.3.2.8 mode . . . 18
7.3.2.9 mtime . . . 18
7.3.2.10 size . . . 18
7.3.2.11 uid . . . 18
7.4 sfuse_super 구조체 참조 . . . 19
7.4.1 상세한 설명 . . . 19
7.4.2 필드 문서화 . . . 19
7.4.2.1 block_bitmap_start . . . 19
7.4.2.2 blocks_count . . . 19
7.4.2.3 data_block_start . . . 19
7.4.2.4 free_blocks . . . 19
7.4.2.5 free_inodes . . . 19
7.4.2.6 inode_bitmap_start . . . 20
7.4.2.7 inode_table_start . . . 20
7.4.2.8 inodes_count . . . 20
7.4.2.9 magic . . . 20

8 파일 문서화 21
8.1 benchmark/result.md 파일 참조 . . . 21
8.2 benchmark/workflow.md 파일 참조 . . . 21
8.3 HISTORY.md 파일 참조 . . . 21
8.4 include/bitmap.h 파일 참조 . . . 21
8.4.1 상세한 설명 . . . 22
8.4.2 함수 문서화 . . . 22
8.4.2.1 alloc_block() . . . 22
8.4.2.2 alloc_inode() . . . 24
8.4.2.3 bitmap_load() . . . 25
8.4.2.4 bitmap_sync() . . . 26
8.4.2.5 free_block() . . . 28
8.4.2.6 free_inode() . . . 29
8.5 bitmap.h . . . 30
8.6 include/block.h 파일 참조 . . . 31
8.6.1 상세한 설명 . . . 31
8.6.2 함수 문서화 . . . 31
8.6.2.1 read_block() . . . 31
8.6.2.2 write_block() . . . 34
8.7 block.h . . . 36
8.8 include/dir.h 파일 참조 . . . 36

8.8.1 상세한 설명 . . . 37
8.8.2 매크로 문서화 . . . 37
8.8.2.1 SFUSE_NAME_LEN . . . 37
8.8.3 함수 문서화 . . . 37
8.8.3.1 dir_add_entry() . . . 37
8.8.3.2 dir_load() . . . 40
8.8.3.3 dir_remove_entry() . . . 42
8.9 dir.h . . . 44
8.10 include/disk.h 파일 참조 . . . 45
8.10.1 상세한 설명 . . . 45
8.10.2 함수 문서화 . . . 46
8.10.2.1 disk_read() . . . 46
8.10.2.2 disk_write() . . . 47
8.11 disk.h . . . 49
8.12 include/fs.h 파일 참조 . . . 50
8.12.1 상세한 설명 . . . 51
8.12.2 매크로 문서화 . . . 51
8.12.2.1 SFUSE_ROOT_INO . . . 51
8.12.3 함수 문서화 . . . 51
8.12.3.1 fs_destroy() . . . 51
8.12.3.2 fs_initialize() . . . 52
8.12.3.3 fs_resolve_path() . . . 57
8.12.3.4 fs_split_path() . . . 61
8.12.3.5 get_fs_context() . . . 63
8.13 fs.h . . . 64
8.14 include/inode.h 파일 참조 . . . 65
8.14.1 상세한 설명 . . . 66
8.14.2 매크로 문서화 . . . 66
8.14.2.1 SFUSE_NDIR_BLOCKS . . . 66
8.14.3 함수 문서화 . . . 67
8.14.3.1 fs_init_inode() . . . 67
8.14.3.2 inode_load() . . . 68
8.14.3.3 inode_sync() . . . 70
8.14.3.4 logical_to_physical() . . . 72
8.15 inode.h . . . 75
8.16 include/ops.h 파일 참조 . . . 76
8.16.1 변수 문서화 . . . 76
8.16.1.1 sfuse_ops . . . 76
8.17 ops.h . . . 77
8.18 include/super.h 파일 참조 . . . 77
8.18.1 상세한 설명 . . . 78
8.18.2 매크로 문서화 . . . 78
8.18.2.1 SFUSE_BLOCK_BITMAP_BLOCK . . . 78

8.18.2.2 SFUSE_BLOCK_SIZE . . . 78
8.18.2.3 SFUSE_BLOCKS_COUNT . . . 79
8.18.2.4 SFUSE_DATA_BLOCK_START . . . 79
8.18.2.5 SFUSE_INODE_BITMAP_BLOCK . . . 79
8.18.2.6 SFUSE_INODE_TABLE_BLOCK . . . 79
8.18.2.7 SFUSE_INODES_COUNT . . . 79
8.18.2.8 SFUSE_MAGIC . . . 79
8.18.2.9 SFUSE_SUPERBLOCK_OFFSET . . . 79
8.18.3 함수 문서화 . . . 79
8.18.3.1 sb_format() . . . 79
8.18.3.2 sb_load() . . . 80
8.18.3.3 sb_sync() . . . 81
8.19 super.h . . . 83
8.20 benchmark/README.md 파일 참조 . . . 84
8.21 README.md 파일 참조 . . . 84
8.22 src/bitmap.c 파일 참조 . . . 84
8.22.1 상세한 설명 . . . 85
8.22.2 함수 문서화 . . . 85
8.22.2.1 alloc_block() . . . 85
8.22.2.2 alloc_inode() . . . 86
8.22.2.3 bitmap_load() . . . 87
8.22.2.4 bitmap_sync() . . . 88
8.22.2.5 free_block() . . . 89
8.22.2.6 free_inode() . . . 90
8.23 bitmap.c . . . 91
8.24 src/block.c 파일 참조 . . . 93
8.24.1 상세한 설명 . . . 93
8.24.2 함수 문서화 . . . 93
8.24.2.1 read_block() . . . 93
8.24.2.2 write_block() . . . 95
8.25 block.c . . . 96
8.26 src/dir.c 파일 참조 . . . 97
8.26.1 상세한 설명 . . . 98
8.26.2 함수 문서화 . . . 98
8.26.2.1 dir_add_entry() . . . 98
8.26.2.2 dir_load() . . . 100
8.26.2.3 dir_remove_entry() . . . 102
8.27 dir.c . . . 103
8.28 src/disk.c 파일 참조 . . . 105
8.28.1 상세한 설명 . . . 106
8.28.2 함수 문서화 . . . 106
8.28.2.1 disk_read() . . . 106
8.28.2.2 disk_write() . . . 107

8.29 disk.c . . . 108
8.30 src/fs.c 파일 참조 . . . 109
8.30.1 상세한 설명 . . . 110
8.30.2 함수 문서화 . . . 110
8.30.2.1 fs_destroy() . . . 110
8.30.2.2 fs_initialize() . . . 111
8.30.2.3 fs_resolve_path() . . . 116
8.30.2.4 fs_split_path() . . . 118
8.30.2.5 get_fs_context() . . . 120
8.31 fs.c . . . 121
8.32 src/inode.c 파일 참조 . . . 127
8.32.1 상세한 설명 . . . 128
8.32.2 함수 문서화 . . . 128
8.32.2.1 fs_init_inode() . . . 128
8.32.2.2 inode_load() . . . 129
8.32.2.3 inode_sync() . . . 131
8.32.2.4 logical_to_physical() . . . 133
8.33 inode.c . . . 135
8.34 src/main.c 파일 참조 . . . 137
8.34.1 상세한 설명 . . . 138
8.34.2 함수 문서화 . . . 138
8.34.2.1 main() . . . 138
8.35 main.c . . . 140
8.36 src/ops.c 파일 참조 . . . 141
8.36.1 함수 문서화 . . . 142
8.36.1.1 sfuse_access_cb() . . . 142
8.36.1.2 sfuse_create_cb() . . . 143
8.36.1.3 sfuse_destroy_cb() . . . 143
8.36.1.4 sfuse_flush_cb() . . . 144
8.36.1.5 sfuse_fsync_cb() . . . 144
8.36.1.6 sfuse_getattr_cb() . . . 145
8.36.1.7 sfuse_getxattr_cb() . . . 145
8.36.1.8 sfuse_init_cb() . . . 146
8.36.1.9 sfuse_listxattr_cb() . . . 146
8.36.1.10 sfuse_mkdir_cb() . . . 147
8.36.1.11 sfuse_open_cb() . . . 148
8.36.1.12 sfuse_read_cb() . . . 148
8.36.1.13 sfuse_readdir_cb() . . . 149
8.36.1.14 sfuse_rename_cb() . . . 150
8.36.1.15 sfuse_rmdir_cb() . . . 151
8.36.1.16 sfuse_statfs_cb() . . . 152
8.36.1.17 sfuse_truncate_cb() . . . 153
8.36.1.18 sfuse_unlink_cb() . . . 154

8.36.1.19 sfuse_utimens_cb() 155
8.36.1.20 sfuse_write_cb() 156
8.36.2 변수 문서화 157
8.36.2.1 sfuse_ops 157
8.37 ops.c 157
8.38 src/super.c 파일 참조 164
8.38.1 상세한 설명 165
8.38.2 함수 문서화 165
8.38.2.1 sb_format() 165
8.38.2.2 sb_load() 166
8.38.2.3 sb_sync() 167
8.39 super.c 168

Index 171

Chapter 1

SFUSE

SFUSE는 Simple FUSE의 약자입니다.

파일 읽기 및 삭제, 디렉토리 탐색과 같은 기본적인 파일 시스템의 동작을 FUSE(Filesystem in Userspace)를 사용하여 구현한 간단한 파일 시스템입니다.

주요 기능 및 코드 설명은 SFUSE Doxygen 문서를 참고바랍니다.
SFUSE의 버전에 따른 특징은 HISTORY.md 문서를 참고바랍니다.

1.0.0.1 SFUSE 미리보기

1.0.1 사용 방법

1.0.1.1 1. 다운로드

이 코드를 실행하기 위한 **필요한 패키지 설치**와 **프로젝트 다운로드**를 하는 명령어입니다.

- Ubuntu:

```
sudo apt install git build-essential cmake pkg-config fuse3 libfuse3-dev -y && \
git clone https://github.com/2daeeun/SFUSE.git && \
cd SFUSE
```

- Arch Linux:

```
sudo pacman -S git cmake fuse3 && \
git clone https://github.com/2daeeun/SFUSE.git && \
cd SFUSE
```

1.0.1.2 2. 코드 컴파일 및 마운트

run.sh는 컴파일과 마운트 작업을 도와주는 셸 스크립트입니다.

```
./run.sh
```

1.0.1.3 3. 언마운트 방법

```
umount sfuse_filesystem && rmdir sfuse_filesystem
```

1.0.1.4 Reference

- **참조한** VSFS **소스코드**
  - OS-Simple-File-System
  - tiny-file-system
  - fisopfs (Origin)
  - fisopfs (Korean)
- **참조한** VSFS **관련 문서**
  - Project 06: Simple File System: 노터데임 대학교 CSE 30341 운영 체제 원리(2024년 가을)
- Doxygen **설치** (HTML and PDF)
  - Arch Linux:

```
sudo pacman -S doxygen graphviz
yay -S texlive-core texlive-bin texlive-fontsextra texlive-latexextra texlive-latexrecommended texlive-luatex
texlive-xetex texlive-lang texlive-listofitems texlive-fontsrecommended ttf-texgyre
# fmt 생성
sudo fmtutil-sys --all
# pdf 생성
cd docs/latex
make clean
make
```

1.0.1.5 NOTE

- 본 프로젝트의 실험은 Arch Linux의 FUSE v3.16.2에서 진행하였으며, Ubuntu 24.04의 FUSE v3.14에서도 작동을 확인하였습니다. (2024년 11월 20일 확인)
- 본 프로젝트는 추후 파일 쓰기와 삭제 기능을 추가 할 예정이며, 더 나아가 I/O 최적화와 로그 시스템 연구에 쓰일 예정입니다.

Chapter 2

SFUSE HISTORY

2.1 v0.0.1 (2024년 12월 30일)

2.1.1 New Features

-

2.1.2 Bug Fixes

-

Chapter 3

실험 결과

워크로드	파일시스템	ioengine / Depth	IOPS	BW (MiB/s)	평균 지연 (µs)	지연 표준편차 (µs)	실행 시간 (ms)
순차 쓰기	SFUSE	sync / 1	321	322	3085	650.9	311
순차 쓰기	Ext4	sync / 1	429	429	2319	142.0	233
순차 읽기	SFUSE	sync / 1	440	441	2248	828.7	227
순차 읽기	Ext4	sync / 1	469	469	2114	101.8	213
랜덤 쓰기	SFUSE	libaio / 16	30.9k	121	516	85.6	829
랜덤 쓰기	Ext4	libaio / 16	34.6k	135	461	707.3	740
랜덤 읽기	SFUSE	libaio / 16	48.8k	190	327	77.9	525
랜덤 읽기	Ext4	libaio / 16	66.1k	258	241	58.5	387

3.0.1 실험 결과 그래프

Chapter 4

fio 워크플로우

1. **순차 쓰기 → 순차 읽기**

```
# 1) 순차 쓰기
fio --name=seq-write \
    --filename=/run/media/leedaeeun/sfuse/fiotest.dat \
    --size=100M --bs=1M --rw=write --direct=1 --ioengine=sync --iodepth=1

# 2) 캐시 비우기
echo 3 | sudo tee /proc/sys/vm/drop_caches

# 3) 순차 읽기 (같은 파일)
fio --name=seq-read \
    --filename=/run/media/leedaeeun/sfuse/fiotest.dat \
    --size=100M --bs=1M --rw=read --direct=1 --ioengine=sync --iodepth=1
```

2. **랜덤 쓰기 → 랜덤 읽기**

```
# 1) 랜덤 쓰기
fio --name=rand-write \
    --filename=/run/media/leedaeeun/sfuse/fiotest-rand.dat \
    --size=100M --bs=4k --rw=randwrite --direct=1 --ioengine=libaio --iodepth=16

# 2) 캐시 비우기
echo 3 | sudo tee /proc/sys/vm/drop_caches

# 3) 랜덤 읽기 (같은 파일)
fio --name=rand-read \
    --filename=/run/media/leedaeeun/sfuse/fiotest-rand.dat \
    --size=100M --bs=4k --rw=randread --direct=1 --ioengine=libaio --iodepth=16
```

4.0.1 SFUSE의 워크플로우 결과

1. SFUSE의 "**순차 쓰기 → 순차 읽기**"

```
root@arch:/run/media/leedaeeun/sfuse #
fio --name=seq-write \
    --filename=/run/media/leedaeeun/sfuse/fiotest.dat \
    --size=100M --bs=1M --rw=write --direct=1 --ioengine=sync --iodepth=1
seq-write: (g=0): rw=write, bs=(R) 1024KiB-1024KiB, (W) 1024KiB-1024KiB, (T) 1024KiB-1024KiB, ioengine=sync,
iodepth=1
fio-3.39
Starting 1 process
seq-write: Laying out IO file (1 file / 100MiB)
seq-write: (groupid=0, jobs=1): err= 0: pid=3896: Thu Jun 19 14:43:11 2025
  write: IOPS=321, BW=322MiB/s (337MB/s)(100MiB/311msec); 0 zone resets
    clat (usec): min=1891, max=6456, avg=3059.86, stdev=650.89
     lat (usec): min=1927, max=6494, avg=3085.39, stdev=650.82
    clat percentiles (usec):
     |  1.00th=[ 1893],  5.00th=[ 2278], 10.00th=[ 2376], 20.00th=[ 2507],
     | 30.00th=[ 2606], 40.00th=[ 2835], 50.00th=[ 2999], 60.00th=[ 3195],
     | 70.00th=[ 3359], 80.00th=[ 3490], 90.00th=[ 3687], 95.00th=[ 4228],
     | 99.00th=[ 4359], 99.50th=[ 6456], 99.90th=[ 6456], 99.95th=[ 6456],
     | 99.99th=[ 6456]
   bw (   KiB/s): min=204800, max=204800, per=62.20%, avg=204800.00, stdev= 0.00, samples=1
   iops        : min=  200, max=  200, avg=200.00, stdev= 0.00, samples=1
  lat (msec)   : 2=1.00%, 4=91.00%, 10=8.00%
  cpu          : usr=0.65%, sys=1.29%, ctx=102, majf=1, minf=9
  IO depths    : 1=100.0%, 2=0.0%, 4=0.0%, 8=0.0%, 16=0.0%, 32=0.0%, >=64=0.0%
     submit    : 0=0.0%, 4=100.0%, 8=0.0%, 16=0.0%, 32=0.0%, 64=0.0%, >=64=0.0%
     complete  : 0=0.0%, 4=100.0%, 8=0.0%, 16=0.0%, 32=0.0%, 64=0.0%, >=64=0.0%
     issued rwts: total=0,100,0,0 short=0,0,0,0 dropped=0,0,0,0
     latency   : target=0, window=0, percentile=100.00%, depth=1

Run status group 0 (all jobs):
```

```
    WRITE: bw=322MiB/s (337MB/s), 322MiB/s-322MiB/s (337MB/s-337MB/s), io=100MiB (105MB), run=311-311msec

root@arch:/run/media/leedaeeun/sfuse #
echo 3 | sudo tee /proc/sys/vm/drop_caches
3
root@arch:/run/media/leedaeeun/sfuse #
fio --name=seq-read \
    --filename=/run/media/leedaeeun/sfuse/fiotest.dat \
    --size=100M --bs=1M --rw=read --direct=1 --ioengine=sync --iodepth=1
seq-read: (g=0): rw=read, bs=(R) 1024KiB-1024KiB, (W) 1024KiB-1024KiB, (T) 1024KiB-1024KiB, ioengine=sync, iodepth=1
fio-3.39
Starting 1 process
seq-read: (groupid=0, jobs=1): err= 0: pid=3918: Thu Jun 19 14:43:22 2025
  read: IOPS=440, BW=441MiB/s (462MB/s)(100MiB/227msec)
    clat (usec): min=1938, max=7356, avg=2247.42, stdev=828.74
     lat (usec): min=1938, max=7358, avg=2247.63, stdev=828.93
    clat percentiles (usec):
     |  1.00th=[ 1942],  5.00th=[ 1958], 10.00th=[ 1975], 20.00th=[ 1975],
     | 30.00th=[ 1991], 40.00th=[ 1991], 50.00th=[ 2008], 60.00th=[ 2024],
     | 70.00th=[ 2024], 80.00th=[ 2073], 90.00th=[ 2311], 95.00th=[ 3982],
     | 99.00th=[ 5669], 99.50th=[ 7373], 99.90th=[ 7373], 99.95th=[ 7373],
     | 99.99th=[ 7373]
  lat (msec)   : 2=47.00%, 4=48.00%, 10=5.00%
  cpu          : usr=0.00%, sys=1.77%, ctx=102, majf=1, minf=263
  IO depths    : 1=100.0%, 2=0.0%, 4=0.0%, 8=0.0%, 16=0.0%, 32=0.0%, >=64=0.0%
     submit    : 0=0.0%, 4=100.0%, 8=0.0%, 16=0.0%, 32=0.0%, 64=0.0%, >=64=0.0%
     complete  : 0=0.0%, 4=100.0%, 8=0.0%, 16=0.0%, 32=0.0%, 64=0.0%, >=64=0.0%
     issued rwts: total=100,0,0,0 short=0,0,0,0 dropped=0,0,0,0
     latency   : target=0, window=0, percentile=100.00%, depth=1

Run status group 0 (all jobs):
   READ: bw=441MiB/s (462MB/s), 441MiB/s-441MiB/s (462MB/s-462MB/s), io=100MiB (105MB), run=227-227msec
```

1. SFUSE의 "랜덤 쓰기 → 랜덤 읽기"

```
root@arch:/run/media/leedaeeun/sfuse #
fio --name=rand-write \
    --filename=/run/media/leedaeeun/sfuse/fiotest-rand.dat \
    --size=100M --bs=4k --rw=randwrite --direct=1 --ioengine=libaio --iodepth=16
rand-write: (g=0): rw=randwrite, bs=(R) 4096B-4096B, (W) 4096B-4096B, (T) 4096B-4096B, ioengine=libaio, iodepth=16
fio-3.39
Starting 1 process
rand-write: Laying out IO file (1 file / 100MiB)
Jobs: 1 (f=1)
rand-write: (groupid=0, jobs=1): err= 0: pid=4129: Thu Jun 19 14:43:59 2025
  write: IOPS=30.9k, BW=121MiB/s (126MB/s)(100MiB/829msec); 0 zone resets
    slat (nsec): min=1392, max=915191, avg=30083.21, stdev=12825.93
    clat (usec): min=90, max=2657, avg=486.02, stdev=81.22
     lat (usec): min=116, max=2808, avg=516.10, stdev=85.55
    clat percentiles (usec):
     |  1.00th=[  355],  5.00th=[  392], 10.00th=[  412], 20.00th=[  437],
     | 30.00th=[  453], 40.00th=[  469], 50.00th=[  482], 60.00th=[  494],
     | 70.00th=[  510], 80.00th=[  529], 90.00th=[  562], 95.00th=[  586],
     | 99.00th=[  644], 99.50th=[  693], 99.90th=[ 1418], 99.95th=[ 2089],
     | 99.99th=[ 2376]
   bw (  KiB/s): min=77912, max=126888, per=82.90%, avg=102400.00, stdev=34631.26, samples=2
   iops        : min=19478, max=31722, avg=25600.00, stdev=8657.82, samples=2
  lat (usec)   : 100=0.01%, 250=0.17%, 500=62.55%, 750=36.96%, 1000=0.16%
  lat (msec)   : 2=0.10%, 4=0.06%
  cpu          : usr=8.70%, sys=14.73%, ctx=25511, majf=0, minf=11
  IO depths    : 1=0.1%, 2=0.1%, 4=0.1%, 8=0.1%, 16=99.9%, 32=0.0%, >=64=0.0%
     submit    : 0=0.0%, 4=100.0%, 8=0.0%, 16=0.0%, 32=0.0%, 64=0.0%, >=64=0.0%
     complete  : 0=0.0%, 4=100.0%, 8=0.0%, 16=0.1%, 32=0.0%, 64=0.0%, >=64=0.0%
     issued rwts: total=0,25600,0,0 short=0,0,0,0 dropped=0,0,0,0
     latency   : target=0, window=0, percentile=100.00%, depth=16

Run status group 0 (all jobs):
  WRITE: bw=121MiB/s (126MB/s), 121MiB/s-121MiB/s (126MB/s-126MB/s), io=100MiB (105MB), run=829-829msec

root@arch:/run/media/leedaeeun/sfuse #
echo 3 | sudo tee /proc/sys/vm/drop_caches
3
root@arch:/run/media/leedaeeun/sfuse #
fio --name=rand-read \
    --filename=/run/media/leedaeeun/sfuse/fiotest-rand.dat \
    --size=100M --bs=4k --rw=randread --direct=1 --ioengine=libaio --iodepth=16
rand-read: (g=0): rw=randread, bs=(R) 4096B-4096B, (W) 4096B-4096B, (T) 4096B-4096B, ioengine=libaio, iodepth=16
fio-3.39
Starting 1 process

rand-read: (groupid=0, jobs=1): err= 0: pid=4149: Thu Jun 19 14:44:08 2025
  read: IOPS=48.8k, BW=190MiB/s (200MB/s)(100MiB/525msec)
    slat (nsec): min=1267, max=590484, avg=18333.00, stdev=8627.64
    clat (usec): min=81, max=2057, avg=308.36, stdev=73.79
     lat (usec): min=99, max=2268, avg=326.69, stdev=77.85
```

```
    clat percentiles (usec):
     |  1.00th=[  206],  5.00th=[  235], 10.00th=[  243], 20.00th=[  265],
     | 30.00th=[  293], 40.00th=[  302], 50.00th=[  306], 60.00th=[  318],
     | 70.00th=[  326], 80.00th=[  334], 90.00th=[  363], 95.00th=[  396],
     | 99.00th=[  437], 99.50th=[  461], 99.90th=[ 1483], 99.95th=[ 1614],
     | 99.99th=[ 1942]
   bw (  KiB/s): min=195424, max=195424, per=100.00%, avg=195424.00, stdev= 0.00, samples=1
   iops        : min=48856, max=48856, avg=48856.00, stdev= 0.00, samples=1
  lat (usec)   : 100=0.02%, 250=15.32%, 500=84.29%, 750=0.13%, 1000=0.02%
  lat (msec)   : 2=0.22%, 4=0.01%
  cpu          : usr=11.83%, sys=22.52%, ctx=25543, majf=0, minf=25
  IO depths    : 1=0.1%, 2=0.1%, 4=0.1%, 8=0.1%, 16=99.9%, 32=0.0%, >=64=0.0%
     submit    : 0=0.0%, 4=100.0%, 8=0.0%, 16=0.0%, 32=0.0%, 64=0.0%, >=64=0.0%
     complete  : 0=0.0%, 4=100.0%, 8=0.0%, 16=0.1%, 32=0.0%, 64=0.0%, >=64=0.0%
     issued rwts: total=25600,0,0,0 short=0,0,0,0 dropped=0,0,0,0
     latency   : target=0, window=0, percentile=100.00%, depth=16

Run status group 0 (all jobs):
   READ: bw=190MiB/s (200MB/s), 190MiB/s-190MiB/s (200MB/s-200MB/s), io=100MiB (105MB), run=525-525msec
```

4.0.2 Ext4의 워크플로우 결과

1. Ext4의 "순차 쓰기 → 순차 읽기"

```
root@arch:/run/media/leedaeeun/sfuse #
fio --name=seq-write \
    --filename=/run/media/leedaeeun/sfuse/fiotest.dat \
    --size=100M --bs=1M --rw=write --direct=1 --ioengine=sync --iodepth=1
seq-write: (g=0): rw=write, bs=(R) 1024KiB-1024KiB, (W) 1024KiB-1024KiB, (T) 1024KiB-1024KiB, ioengine=sync, iodepth=1
fio-3.39
Starting 1 process
seq-write: Laying out IO file (1 file / 100MiB)
seq-write: (groupid=0, jobs=1): err= 0: pid=4302: Thu Jun 19 14:45:59 2025
  write: IOPS=429, BW=429MiB/s (450MB/s)(100MiB/233msec); 0 zone resets
    clat (usec): min=2143, max=3641, avg=2245.26, stdev=142.03
     lat (usec): min=2195, max=3681, avg=2318.92, stdev=139.31
    clat percentiles (usec):
     |  1.00th=[ 2147],  5.00th=[ 2212], 10.00th=[ 2212], 20.00th=[ 2212],
     | 30.00th=[ 2212], 40.00th=[ 2245], 50.00th=[ 2245], 60.00th=[ 2245],
     | 70.00th=[ 2245], 80.00th=[ 2245], 90.00th=[ 2245], 95.00th=[ 2245],
     | 99.00th=[ 2278], 99.50th=[ 3654], 99.90th=[ 3654], 99.95th=[ 3654],
     | 99.99th=[ 3654]
  lat (msec)   : 4=100.00%
  cpu          : usr=4.74%, sys=8.19%, ctx=100, majf=0, minf=11
  IO depths    : 1=100.0%, 2=0.0%, 4=0.0%, 8=0.0%, 16=0.0%, 32=0.0%, >=64=0.0%
     submit    : 0=0.0%, 4=100.0%, 8=0.0%, 16=0.0%, 32=0.0%, 64=0.0%, >=64=0.0%
     complete  : 0=0.0%, 4=100.0%, 8=0.0%, 16=0.0%, 32=0.0%, 64=0.0%, >=64=0.0%
     issued rwts: total=0,100,0,0 short=0,0,0,0 dropped=0,0,0,0
     latency   : target=0, window=0, percentile=100.00%, depth=1

Run status group 0 (all jobs):
  WRITE: bw=429MiB/s (450MB/s), 429MiB/s-429MiB/s (450MB/s-450MB/s), io=100MiB (105MB), run=233-233msec

Disk stats (read/write):
  sdb: ios=0/61, sectors=0/124928, merge=0/0, ticks=0/130, in_queue=130, util=51.42%
```

```
root@arch:/run/media/leedaeeun/sfuse #
echo 3 | sudo tee /proc/sys/vm/drop_caches
3
```

```
root@arch:/run/media/leedaeeun/sfuse #
fio --name=seq-read \
    --filename=/run/media/leedaeeun/sfuse/fiotest.dat \
    --size=100M --bs=1M --rw=read --direct=1 --ioengine=sync --iodepth=1
seq-read: (g=0): rw=read, bs=(R) 1024KiB-1024KiB, (W) 1024KiB-1024KiB, (T) 1024KiB-1024KiB, ioengine=sync, iodepth=1
fio-3.39
Starting 1 process
seq-read: (groupid=0, jobs=1): err= 0: pid=4321: Thu Jun 19 14:46:09 2025
  read: IOPS=469, BW=469MiB/s (492MB/s)(100MiB/213msec)
    clat (usec): min=1911, max=2298, avg=2114.15, stdev=101.81
     lat (usec): min=1911, max=2298, avg=2114.67, stdev=102.04
    clat percentiles (usec):
     |  1.00th=[ 1909],  5.00th=[ 1942], 10.00th=[ 1958], 20.00th=[ 2008],
     | 30.00th=[ 2040], 40.00th=[ 2114], 50.00th=[ 2147], 60.00th=[ 2147],
     | 70.00th=[ 2147], 80.00th=[ 2212], 90.00th=[ 2245], 95.00th=[ 2245],
     | 99.00th=[ 2311], 99.50th=[ 2311], 99.90th=[ 2311], 99.95th=[ 2311],
     | 99.99th=[ 2311]
  lat (msec)   : 2=16.00%, 4=84.00%
  cpu          : usr=2.83%, sys=3.30%, ctx=101, majf=1, minf=266
  IO depths    : 1=100.0%, 2=0.0%, 4=0.0%, 8=0.0%, 16=0.0%, 32=0.0%, >=64=0.0%
     submit    : 0=0.0%, 4=100.0%, 8=0.0%, 16=0.0%, 32=0.0%, 64=0.0%, >=64=0.0%
     complete  : 0=0.0%, 4=100.0%, 8=0.0%, 16=0.0%, 32=0.0%, 64=0.0%, >=64=0.0%
     issued rwts: total=100,0,0,0 short=0,0,0,0 dropped=0,0,0,0
     latency   : target=0, window=0, percentile=100.00%, depth=1

Run status group 0 (all jobs):
```

```
   READ: bw=469MiB/s (492MB/s), 469MiB/s-469MiB/s (492MB/s-492MB/s), io=100MiB (105MB), run=213-213msec

Disk stats (read/write):
  sdb: ios=142/0, sectors=145408/0, merge=0/0, ticks=237/0, in_queue=237, util=56.22%
```

1. Ext4의 "랜덤 쓰기 → 랜덤 읽기"

```
root@arch:/run/media/leedaeeun/sfuse #
fio --name=rand-write \
    --filename=/run/media/leedaeeun/sfuse/fiotest-rand.dat \
    --size=100M --bs=4k --rw=randwrite --direct=1 --ioengine=libaio --iodepth=16
rand-write: (g=0): rw=randwrite, bs=(R) 4096B-4096B, (W) 4096B-4096B, (T) 4096B-4096B, ioengine=libaio, iodepth=16
fio-3.39
Starting 1 process
rand-write: Laying out IO file (1 file / 100MiB)
rand-write: (groupid=0, jobs=1): err= 0: pid=4386: Thu Jun 19 14:46:57 2025
  write: IOPS=34.6k, BW=135MiB/s (142MB/s)(100MiB/740msec); 0 zone resets
    slat (usec): min=4, max=105, avg= 7.73, stdev= 2.63
    clat (usec): min=102, max=8738, avg=453.81, stdev=707.30
     lat (usec): min=108, max=8746, avg=461.53, stdev=707.33
    clat percentiles (usec):
     |  1.00th=[  184],  5.00th=[  212], 10.00th=[  217], 20.00th=[  221],
     | 30.00th=[  227], 40.00th=[  231], 50.00th=[  241], 60.00th=[  251],
     | 70.00th=[  265], 80.00th=[  347], 90.00th=[  725], 95.00th=[ 1876],
     | 99.00th=[ 3654], 99.50th=[ 4752], 99.90th=[ 7308], 99.95th=[ 8717],
     | 99.99th=[ 8717]
   bw (  KiB/s): min=136520, max=136520, per=98.66%, avg=136520.00, stdev= 0.00, samples=1
   iops        : min=34130, max=34130, avg=34130.00, stdev= 0.00, samples=1
  lat (usec)   : 250=59.57%, 500=26.28%, 750=4.40%, 1000=1.43%
  lat (msec)   : 2=3.86%, 4=3.66%, 10=0.79%
  cpu          : usr=7.98%, sys=29.91%, ctx=21227, majf=0, minf=11
  IO depths    : 1=0.1%, 2=0.1%, 4=0.1%, 8=0.1%, 16=99.9%, 32=0.0%, >=64=0.0%
     submit    : 0=0.0%, 4=100.0%, 8=0.0%, 16=0.0%, 32=0.0%, 64=0.0%, >=64=0.0%
     complete  : 0=0.0%, 4=100.0%, 8=0.0%, 16=0.1%, 32=0.0%, 64=0.0%, >=64=0.0%
     issued rwts: total=0,25600,0,0 short=0,0,0,0 dropped=0,0,0,0
     latency   : target=0, window=0, percentile=100.00%, depth=16

Run status group 0 (all jobs):
  WRITE: bw=135MiB/s (142MB/s), 135MiB/s-135MiB/s (142MB/s-142MB/s), io=100MiB (105MB), run=740-740msec

Disk stats (read/write):
  sdb: ios=4/22024, sectors=72/176192, merge=0/0, ticks=4/9836, in_queue=9839, util=71.05%


root@arch:/run/media/leedaeeun/sfuse #
echo 3 | sudo tee /proc/sys/vm/drop_caches
3

root@arch:/run/media/leedaeeun/sfuse #
fio --name=rand-read \
    --filename=/run/media/leedaeeun/sfuse/fiotest-rand.dat \
    --size=100M --bs=4k --rw=randread --direct=1 --ioengine=libaio --iodepth=16
rand-read: (g=0): rw=randread, bs=(R) 4096B-4096B, (W) 4096B-4096B, (T) 4096B-4096B, ioengine=libaio, iodepth=16
fio-3.39
Starting 1 process

rand-read: (groupid=0, jobs=1): err= 0: pid=4405: Thu Jun 19 14:47:03 2025
  read: IOPS=66.1k, BW=258MiB/s (271MB/s)(100MiB/387msec)
    slat (usec): min=3, max=102, avg= 4.71, stdev= 1.55
    clat (usec): min=109, max=1754, avg=236.32, stdev=58.07
     lat (usec): min=123, max=1778, avg=241.03, stdev=58.54
    clat percentiles (usec):
     |  1.00th=[  155],  5.00th=[  172], 10.00th=[  184], 20.00th=[  200],
     | 30.00th=[  212], 40.00th=[  223], 50.00th=[  231], 60.00th=[  241],
     | 70.00th=[  253], 80.00th=[  269], 90.00th=[  289], 95.00th=[  318],
     | 99.00th=[  363], 99.50th=[  392], 99.90th=[  510], 99.95th=[ 1729],
     | 99.99th=[ 1745]
  lat (usec)   : 250=67.60%, 500=32.26%, 750=0.08%
  lat (msec)   : 2=0.06%
  cpu          : usr=14.77%, sys=34.72%, ctx=21136, majf=0, minf=28
  IO depths    : 1=0.1%, 2=0.1%, 4=0.1%, 8=0.1%, 16=99.9%, 32=0.0%, >=64=0.0%
     submit    : 0=0.0%, 4=100.0%, 8=0.0%, 16=0.0%, 32=0.0%, 64=0.0%, >=64=0.0%
     complete  : 0=0.0%, 4=100.0%, 8=0.0%, 16=0.1%, 32=0.0%, 64=0.0%, >=64=0.0%
     issued rwts: total=25600,0,0,0 short=0,0,0,0 dropped=0,0,0,0
     latency   : target=0, window=0, percentile=100.00%, depth=16

Run status group 0 (all jobs):
   READ: bw=258MiB/s (271MB/s), 258MiB/s-258MiB/s (271MB/s-271MB/s), io=100MiB (105MB), run=387-387msec

Disk stats (read/write):
  sdb: ios=9701/0, sectors=77608/0, merge=0/0, ticks=2248/0, in_queue=2248, util=40.16%
```

Chapter 5

데이터 구조 색인

5.1 데이터 구조

다음은 데이터 구조들입니다. (간략한 설명만을 보여줍니다) :

sfuse_dirent
디렉터리 엔트리 정보를 저장하는 구조체 15
sfuse_fs
SFUSE 파일 시스템의 전역 컨텍스트를 나타내는 구조체 15
sfuse_inode
파일의 메타데이터를 관리하는 아이노드 구조체 17
sfuse_super
파일 시스템 메타데이터를 저장하는 슈퍼블록 구조체 19

Chapter 6

파일 색인

6.1 파일 목록

다음은 모든 파일에 대한 목록입니다. (간략한 설명만을 보여줍니다) :

include/bitmap.h
비트맵 관리 함수 선언 21
include/block.h
블록 단위 읽기 및 쓰기를 위한 인터페이스 정의 31
include/dir.h
파일 시스템의 디렉터리 관리 기능 정의 36
include/disk.h
원시 디바이스에 대한 저수준 I/O 연산 함수 선언 45
include/fs.h
SFUSE 파일 시스템의 핵심 구조 및 연산 정의 50
include/inode.h
아이노드(inode) 구조체 정의 및 관리 함수 선언 65
include/ops.h 76
include/super.h
SFUSE 파일 시스템의 슈퍼블록 관련 구조 및 함수 정의 77
src/bitmap.c
비트맵 관리 함수 구현 84
src/block.c
블록 단위 입출력 헬퍼 함수 구현 93
src/dir.c
디렉터리 엔트리 로드, 추가, 삭제 기능 구현 97
src/disk.c
원시 디바이스에 직접 접근하는 저수준 I/O 연산 구현 105
src/fs.c
SFUSE 파일시스템 초기화, 관리 및 경로 분석 함수의 구현 109
src/inode.c
아이노드 로드, 동기화 및 블록 주소 변환 함수 구현 127
src/main.c
SFUSE 파일시스템의 메인 진입점 137
src/ops.c 141
src/super.c
SFUSE 파일 시스템의 슈퍼블록 관리 함수 구현 164

Chapter 7

데이터 구조 문서화

7.1 sfuse_dirent 구조체 참조

디렉터리 엔트리 정보를 저장하는 구조체
#include <dir.h>

데이터 필드

- uint32_t ino
- char name [255+1]

7.1.1 상세한 설명

디렉터리 엔트리 정보를 저장하는 구조체
각 디렉터리 엔트리는 연결된 아이노드 번호와 해당 엔트리의 이름을 포함한다. 빈 엔트리는 아이노드 번호(ino)가 0이다.
dir.h 파일의 27 번째 라인에서 정의되었습니다.

7.1.2 필드 문서화

7.1.2.1 ino

uint32_t sfuse_dirent::ino
디렉터리 엔트리에 연결된 아이노드 번호 (0이면 빈 슬롯)
dir.h 파일의 28 번째 라인에서 정의되었습니다.

7.1.2.2 name

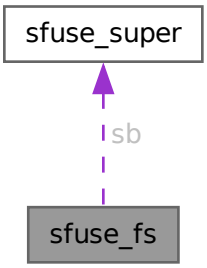
char sfuse_dirent::name[255+1]
NULL 종단 문자열 형태의 엔트리 이름 (최대 SFUSE_NAME_LEN)
dir.h 파일의 29 번째 라인에서 정의되었습니다.
이 구조체에 대한 문서화 페이지는 다음의 파일로부터 생성되었습니다.:

- include/dir.h

7.2 sfuse_fs 구조체 참조

SFUSE 파일 시스템의 전역 컨텍스트를 나타내는 구조체
#include <fs.h>

sfuse_fs에 대한 협력 다이어그램:



데이터 필드

- int backing_fd
- struct sfuse_super sb
- uint8_t * block_map
- uint8_t * inode_map

7.2.1 상세한 설명

SFUSE 파일 시스템의 전역 컨텍스트를 나타내는 구조체
SFUSE 파일 시스템의 핵심 상태와 데이터를 관리하는 중심 구조체이다. 블록 디바이스 접근을 위한 파일 디스크립터(backing_fd)를 저장하며, 슈퍼블록(sb)을 통해 파일 시스템의 메타데이터를 관리한다. 또한 블록과 아이노드의 할당 상태를 각각 비트맵 형태(block_map, inode_map)로 저장하여, 데이터 블록과 아이노드의 효율적인 관리와 할당을 지원한다. 이 구조체는 파일 시스템의 마운트 시 초기화되어 FUSE 연산 수행 시 전역적으로 사용된다.
fs.h 파일의 42 번째 라인에서 정의되었습니다.

7.2.2 필드 문서화

7.2.2.1 backing_fd

int sfuse_fs::backing_fd
블록 디바이스의 파일 디스크립터
fs.h 파일의 43 번째 라인에서 정의되었습니다.

7.2.2.2 block_map

uint8_t* sfuse_fs::block_map
블록 비트맵 버퍼 포인터
fs.h 파일의 45 번째 라인에서 정의되었습니다.

7.2.2.3 inode_map

uint8_t* sfuse_fs::inode_map
아이노드 비트맵 버퍼 포인터
fs.h 파일의 46 번째 라인에서 정의되었습니다.

7.2.2.4 sb

struct sfuse_super sfuse_fs::sb
슈퍼블록 구조체
fs.h 파일의 44 번째 라인에서 정의되었습니다.
이 구조체에 대한 문서화 페이지는 다음의 파일로부터 생성되었습니다.:

- include/fs.h

7.3 sfuse_inode 구조체 참조

파일의 메타데이터를 관리하는 아이노드 구조체
#include <inode.h>

데이터 필드

- mode_t mode
  파일 타입 및 접근 권한
- uid_t uid
  파일 소유자의 사용자 ID
- gid_t gid
  파일 소유자의 그룹 ID
- uint32_t size
  파일 크기 (바이트 단위)
- uint32_t atime
  마지막 접근 시간 (Access Time)
- uint32_t mtime
  마지막 수정 시간 (Modification Time)
- uint32_t ctime
  상태 변경 시간 (Change Time)
- uint32_t direct [12]
  직접 참조 블록 포인터 배열
- uint32_t indirect
  Single Indirect 블록 포인터
- uint32_t double_indirect
  Double Indirect 블록 포인터
- uint32_t links
  파일에 연결된 링크 수

7.3.1 상세한 설명

파일의 메타데이터를 관리하는 아이노드 구조체
아이노드는 파일의 권한, 소유자, 크기, 타임스탬프, 블록 포인터 등의 메타데이터를 저장한다.
inode.h 파일의 27 번째 라인에서 정의되었습니다.

7.3.2 필드 문서화

7.3.2.1 atime

uint32_t sfuse_inode::atime
마지막 접근 시간 (Access Time)
inode.h 파일의 32 번째 라인에서 정의되었습니다.

7.3.2.2 ctime

uint32_t sfuse_inode::ctime
상태 변경 시간 (Change Time)
inode.h 파일의 34 번째 라인에서 정의되었습니다.

7.3.2.3 direct

uint32_t sfuse_inode::direct[12]
직접 참조 블록 포인터 배열
inode.h 파일의 35 번째 라인에서 정의되었습니다.

7.3.2.4 double_indirect

uint32_t sfuse_inode::double_indirect
Double Indirect 블록 포인터
inode.h 파일의 37 번째 라인에서 정의되었습니다.

7.3.2.5 gid

gid_t sfuse_inode::gid
파일 소유자의 그룹 ID
inode.h 파일의 30 번째 라인에서 정의되었습니다.

7.3.2.6 indirect

uint32_t sfuse_inode::indirect
Single Indirect 블록 포인터
inode.h 파일의 36 번째 라인에서 정의되었습니다.

7.3.2.7 links

uint32_t sfuse_inode::links
파일에 연결된 링크 수
inode.h 파일의 38 번째 라인에서 정의되었습니다.

7.3.2.8 mode

mode_t sfuse_inode::mode
파일 타입 및 접근 권한
inode.h 파일의 28 번째 라인에서 정의되었습니다.

7.3.2.9 mtime

uint32_t sfuse_inode::mtime
마지막 수정 시간 (Modification Time)
inode.h 파일의 33 번째 라인에서 정의되었습니다.

7.3.2.10 size

uint32_t sfuse_inode::size
파일 크기 (바이트 단위)
inode.h 파일의 31 번째 라인에서 정의되었습니다.

7.3.2.11 uid

uid_t sfuse_inode::uid
파일 소유자의 사용자 ID
inode.h 파일의 29 번째 라인에서 정의되었습니다.

이 구조체에 대한 문서화 페이지는 다음의 파일로부터 생성되었습니다.:

- include/inode.h

7.4 sfuse_super 구조체 참조

파일 시스템 메타데이터를 저장하는 슈퍼블록 구조체
#include <super.h>

데이터 필드

- uint32_t magic
- uint32_t inodes_count
- uint32_t blocks_count
- uint32_t free_inodes
- uint32_t free_blocks
- uint32_t inode_bitmap_start
- uint32_t block_bitmap_start
- uint32_t inode_table_start
- uint32_t data_block_start

7.4.1 상세한 설명

파일 시스템 메타데이터를 저장하는 슈퍼블록 구조체
super.h 파일의 45 번째 라인에서 정의되었습니다.

7.4.2 필드 문서화

7.4.2.1 block_bitmap_start

uint32_t sfuse_super::block_bitmap_start
블록 비트맵이 시작되는 블록 번호
super.h 파일의 52 번째 라인에서 정의되었습니다.

7.4.2.2 blocks_count

uint32_t sfuse_super::blocks_count
파일 시스템의 전체 블록 수
super.h 파일의 48 번째 라인에서 정의되었습니다.

7.4.2.3 data_block_start

uint32_t sfuse_super::data_block_start
실제 데이터 블록이 시작되는 블록 번호
super.h 파일의 54 번째 라인에서 정의되었습니다.

7.4.2.4 free_blocks

uint32_t sfuse_super::free_blocks
사용할 수 있는 데이터 블록의 수
super.h 파일의 50 번째 라인에서 정의되었습니다.

7.4.2.5 free_inodes

uint32_t sfuse_super::free_inodes
사용할 수 있는 아이노드의 수
super.h 파일의 49 번째 라인에서 정의되었습니다.

7.4.2.6 inode_bitmap_start

uint32_t sfuse_super::inode_bitmap_start
아이노드 비트맵이 시작되는 블록 번호
super.h 파일의 51 번째 라인에서 정의되었습니다.

7.4.2.7 inode_table_start

uint32_t sfuse_super::inode_table_start
아이노드 테이블이 시작되는 블록 번호
super.h 파일의 53 번째 라인에서 정의되었습니다.

7.4.2.8 inodes_count

uint32_t sfuse_super::inodes_count
파일 시스템에 존재하는 전체 아이노드 수
super.h 파일의 47 번째 라인에서 정의되었습니다.

7.4.2.9 magic

uint32_t sfuse_super::magic
슈퍼블록 유효성 확인을 위한 매직 넘버
super.h 파일의 46 번째 라인에서 정의되었습니다.
이 구조체에 대한 문서화 페이지는 다음의 파일로부터 생성되었습니다.:

- include/super.h

Chapter 8

파일 문서화

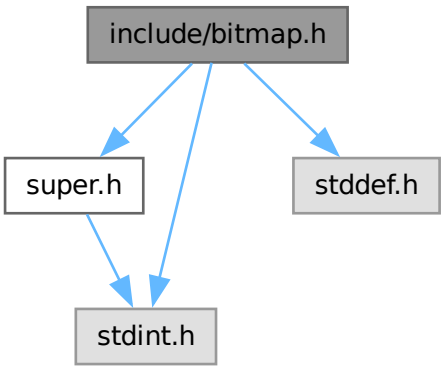
8.1 benchmark/result.md 파일 참조

8.2 benchmark/workflow.md 파일 참조

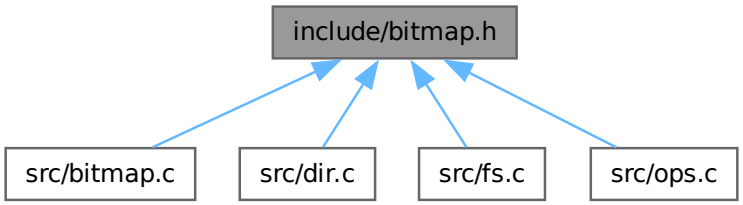
8.3 HISTORY.md 파일 참조

8.4 include/bitmap.h 파일 참조

비트맵 관리 함수 선언
#include "super.h"
#include <stddef.h>
#include <stdint.h>
bitmap.h에 대한 include 의존 그래프



이 그래프는 이 파일을 직/간접적으로 include 하는 파일들을 보여줍니다.:



함수

- int bitmap_load (int fd, uint32_t block_no, uint8_t *map, size_t map_size)
  비트맵 데이터를 디스크에서 로드
- int bitmap_sync (int fd, uint32_t block_no, uint8_t *map, size_t map_size)
  비트맵 데이터를 디스크에 기록
- int alloc_block (struct sfuse_super *sb, uint8_t *block_map)
  데이터 블록 할당
- void free_block (struct sfuse_super *sb, uint8_t *block_map, uint32_t offset)
  데이터 블록 해제
- int alloc_inode (struct sfuse_super *sb, uint8_t *inode_map)
  아이노드 할당
- void free_inode (struct sfuse_super *sb, uint8_t *inode_map, uint32_t ino)
  아이노드 해제

8.4.1 상세한 설명

비트맵 관리 함수 선언
bitmap.h에서는 데이터 블록과 아이노드를 관리하기 위한 비트맵 관련 함수들의 인터페이스를 정의한다.
비트맵 로딩, 디스크 동기화 및 블록과 아이노드 할당/해제 기능을 제공한다.
bitmap.h 파일에서 정의되었습니다.

8.4.2 함수 문서화

8.4.2.1 alloc_block()

```
int alloc_block (
            struct sfuse_super * sb,
            uint8_t * block_map)
```

데이터 블록 할당
사용 가능한 빈 블록을 찾아 할당하고, 비트맵을 업데이트한다.

매개변수

sb	슈퍼블록 구조체 포인터
block_map	블록 비트맵 버퍼

반환값

할당된 블록의 오프셋(0부터 시작), 실패 시 -ENOSPC

데이터 블록 할당
이 함수는 파일 시스템에서 데이터 저장을 위해 사용되지 않은 블록을 찾아 할당한다. 블록의 사용 여부는 비트맵(block_map)에 비트 단위로 저장되어 있으며, 이 비트맵을 순차적으로 탐색해 첫 번째 빈 블록을 찾아 할당한다.
블록을 할당하면 해당 블록의 비트를 '사용 중(1)'으로 설정하고, 슈퍼블록의 'free_blocks' 필드 값을 감소시켜 남은 가용 블록 수를 갱신한다.

매개변수

sb	슈퍼블록 구조체 포인터 (파일시스템의 전체적인 상태를 관리)
block_map	블록 할당 여부를 나타내는 비트맵 버퍼 포인터

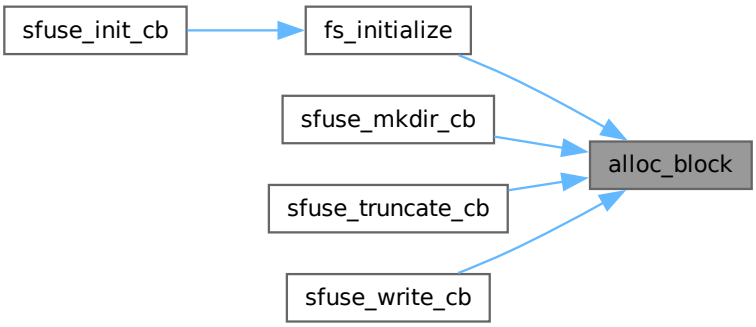
반환값

성공 시 할당된 블록의 오프셋(0부터 시작)을 반환하며, 가용 블록이 없으면 공간 부족을 의미하는 -ENOSPC 반환

bitmap.c 파일의 99 번째 라인에서 정의되었습니다.

```
00099                                                {
00100   // 데이터 블록 영역에서 사용 가능한 총 블록 개수를 계산
00101   uint32_t total = sb->blocks_count - sb->data_block_start;
00102
00103   // 비트맵을 처음부터 끝까지 탐색하며 빈 블록을 찾음
00104   for (uint32_t i = 0; i < total; i++) {
00105     uint32_t byte_idx = i / 8; // 비트맵의 바이트 단위 인덱스
00106     uint32_t bit_idx = i % 8;  // 바이트 내의 비트 위치 (0~7)
00107
00108     // 현재 블록(i)이 사용 가능한 상태인지 확인 (비트 값이 0이면 사용 가능)
00109     if (!(block_map[byte_idx] & (1 « bit_idx))) {
00110       // 사용 가능한 블록 발견 시 해당 비트를 '1'로 설정하여 사용 중으로 표시
00111       block_map[byte_idx] |= (1 « bit_idx);
00112
00113       // 슈퍼블록에서 사용 가능한 블록 개수 1 감소
00114       sb->free_blocks--;
00115
00116       // 할당된 블록의 오프셋 반환 (0부터 시작)
00117       return (int)i;
00118     }
00119   }
00120
00121   // 모든 블록이 이미 사용 중이면 ENOSPC(공간 부족 오류) 반환
00122   return -ENOSPC;
00123 }
```

이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.4.2.2 alloc_inode()

```
int alloc_inode (
            struct sfuse_super * sb,
            uint8_t * inode_map)
```

아이노드 할당
비어 있는 아이노드를 찾아 할당하고, 아이노드 비트맵을 업데이트한다.

매개변수

sb	슈퍼블록 구조체 포인터
inode_map	아이노드 비트맵 버퍼

반환값

할당된 아이노드 번호, 실패 시 -ENOSPC

아이노드 할당
파일 시스템에서 새로운 파일이나 디렉토리를 생성할 때 사용할 빈 아이노드를 탐색하여 할당하는 함수이다.
아이노드 번호 0은 특수 용도로 예약되어 있으므로, 할당 가능한 아이노드의 탐색은 1번 아이노드부터 시작한다.
사용 가능한 아이노드를 찾으면 아이노드 비트맵에서 해당 비트를 사용 중(1)으로 설정하고, 슈퍼블록의 'free_inodes' 값을 감소시켜 파일 시스템의 상태를 업데이트한다.

매개변수

sb	슈퍼블록 구조체 포인터 (파일 시스템 상태 정보 관리)
inode_map	아이노드 할당 상태를 나타내는 비트맵 버퍼 포인터

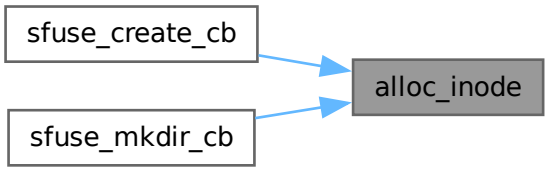
반환값

성공 시 할당된 아이노드 번호(1 이상)를 반환하고, 사용 가능한 아이노드가 없을 경우 -ENOSPC 반환

bitmap.c 파일의 172 번째 라인에서 정의되었습니다.

```
{
  // 아이노드 0은 예약이므로 1부터 탐색을 시작
  for (uint32_t i = 1; i < sb->inodes_count; i++) {
    uint32_t byte_idx = i / 8; // 비트맵 내의 바이트 인덱스 계산
    uint32_t bit_idx = i % 8;  // 바이트 내의 비트 위치 계산 (0~7)

    // 현재 아이노드(i)가 빈 상태인지 검사 (0이면 빈 상태)
    if (!(inode_map[byte_idx] & (1 « bit_idx))) {
      // 사용 가능한 아이노드를 발견하면, 비트맵에 사용 중으로 표시 (비트=1)
      inode_map[byte_idx] |= (1 « bit_idx);

      // 슈퍼블록의 남은 빈 아이노드 수를 1 감소시킴
      sb->free_inodes--;

      // 할당된 아이노드 번호를 반환 (번호는 1부터 시작)
      return (int)i;
    }
  }

  // 빈 아이노드를 찾지 못했을 때 공간 부족(-ENOSPC) 오류 반환
  return -ENOSPC;
}
```

이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.4.2.3 bitmap_load()

```
int bitmap_load (
            int fd,
            uint32_t block_no,
            uint8_t * map,
            size_t map_size)
```

비트맵 데이터를 디스크에서 로드
디스크의 특정 블록 번호에서 비트맵 데이터를 읽어 메모리에 저장한다.

매개변수

fd	디바이스 파일 디스크립터
block_no	비트맵이 저장된 블록 번호
map	읽어온 비트맵 데이터를 저장할 버퍼
map_size	버퍼의 크기 (바이트 단위)

반환값

성공 시 0, 실패 시 음수 오류 코드 반환

비트맵 데이터를 디스크에서 로드
파일 시스템에서 블록 또는 아이노드의 할당 여부를 나타내는 비트맵 데이터를 디스크의 특정 블록에서 읽어들여 메모리의 버퍼에 저장한다.
이 함수는 파일 시스템 초기화 및 비트맵 동기화 시 주로 사용된다.

매개변수

fd	디바이스 파일 디스크립터 (읽을 대상 디스크 장치)
block_no	비트맵이 저장된 디스크 내의 블록 번호
map	비트맵 데이터를 저장할 메모리 버퍼 포인터
map_size	비트맵 데이터의 크기(바이트 단위)

반환값

성공 시 0을 반환하며, 오류 발생 시 음수 값으로 오류 코드 반환: -EIO (I/O 오류), 기타 disk_read 가 반환하는 음수 값

bitmap.c 파일의 35 번째 라인에서 정의되었습니다.

```
00035                                                   {
00036   ssize_t ret;
00037
00038   // 디스크에서 비트맵 데이터를 읽어 메모리(map)로 로드한다.
00039   ret = disk_read(fd, map, map_size, (off_t)block_no * SFUSE_BLOCK_SIZE);
00040   if (ret < 0)
00041     return (int)ret; // disk_read 실패: 읽기 자체가 실패함
00042
00043   // 읽은 데이터 크기가 요청한 크기와 일치하는지 검증한다.
00044   if ((size_t)ret != map_size)
00045     return -EIO; // 읽은 데이터 크기가 예상 크기와 불일치
00046
00047   return 0; // 정상 종료
00048 }
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:

이 함수를 호출하는 함수들에 대한 그래프입니다.:

8.4.2.4 bitmap_sync()

```
int bitmap_sync (
            int fd,
            uint32_t block_no,
            uint8_t * map,
            size_t map_size)
```

비트맵 데이터를 디스크에 기록
메모리에 있는 비트맵 데이터를 지정된 디스크 블록에 저장한다.

매개변수

fd	디바이스 파일 디스크립터
block_no	비트맵을 기록할 블록 번호
map	기록할 비트맵 데이터를 담고 있는 버퍼
map_size	버퍼의 크기 (바이트 단위)

반환값

성공 시 0, 실패 시 음수 오류 코드 반환

비트맵 데이터를 디스크에 기록
파일 시스템에서 현재 메모리에 저장된 비트맵 데이터를 지정된 디스크의 블록 위치로 기록한다.
비트맵의 변경 사항(블록 또는 아이노드 할당 상태 등)을 디스크에 반영하여, 시스템이 재부팅되거나 파일 시스템이 다시 마운트될 때 정확한 상태를 유지하도록 한다.

매개변수

fd	디바이스 파일 디스크립터 (기록할 대상 디스크 장치)
block_no	비트맵을 기록할 디스크 내의 블록 번호
map	디스크에 기록할 비트맵 데이터가 저장된 메모리 버퍼 포인터
map_size	비트맵 데이터의 크기(바이트 단위)

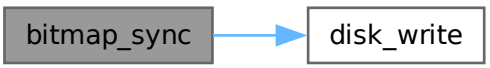
반환값

성공 시 0을 반환하며, 오류 발생 시 음수 값으로 오류 코드 반환: -EIO (I/O 오류), 또는 disk_write 가 반환하는 기타 음수 값

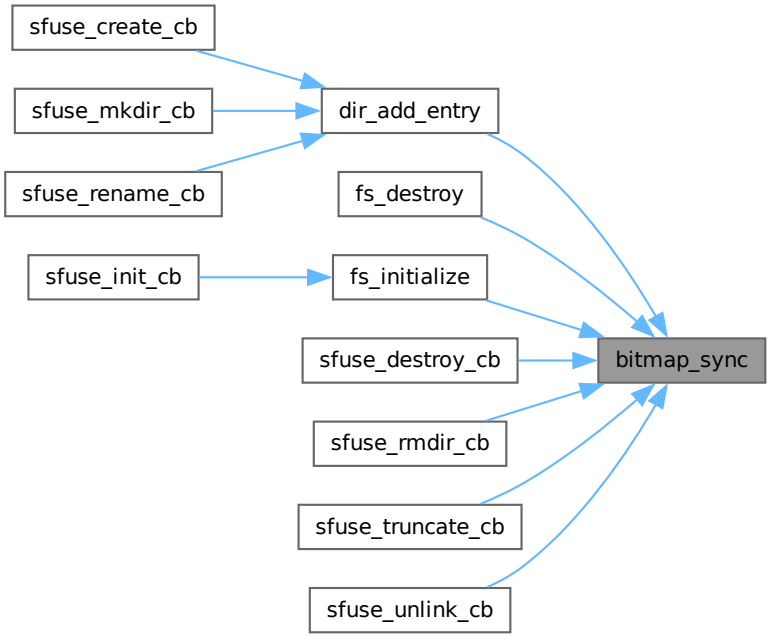
bitmap.c 파일의 68 번째 라인에서 정의되었습니다.

```
00068                                                   {
00069   ssize_t ret;
00070
00071   // 메모리(map)의 비트맵 데이터를 디스크의 지정된 위치에 기록한다.
00072   ret = disk_write(fd, map, map_size, (off_t)block_no * SFUSE_BLOCK_SIZE);
00073   if (ret < 0)
00074     return (int)ret; // disk_write 실패: 기록 자체가 실패함
00075
00076   // 기록한 데이터 크기가 요청한 크기와 일치하는지 검증한다.
00077   if ((size_t)ret != map_size)
00078     return -EIO; // 기록한 데이터 크기가 예상 크기와 불일치 (부분적 기록 발생)
00079
00080   return 0; // 정상적으로 비트맵 데이터를 디스크에 기록 완료
00081 }
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.4.2.5 free_block()

```
void free_block (
            struct sfuse_super * sb,
            uint8_t * block_map,
            uint32_t offset)
```

데이터 블록 해제
할당된 블록을 해제하고 비트맵을 업데이트한다.

매개변수

sb	슈퍼블록 구조체 포인터
block_map	블록 비트맵 버퍼
offset	해제할 블록의 오프셋(0부터 시작)

데이터 블록 해제
이 함수는 파일 시스템에서 더 이상 사용하지 않는 데이터 블록을 다시 사용 가능한 상태로 되돌리기 위해 해제(free)한다.
주어진 블록 오프셋(offset)의 위치를 비트맵에서 찾아서, 해당 비트를 '0'으로 설정함으로써 빈 블록임을 표시하고, 슈퍼블록의 'free_blocks' 값을 1 증가시켜 남은 가용 블록 수를 갱신한다.
이 함수는 파일 삭제 또는 파일 크기 축소 시 호출되어, 디스크 공간을 효율적으로 재사용할 수 있도록 도와준다.

매개변수

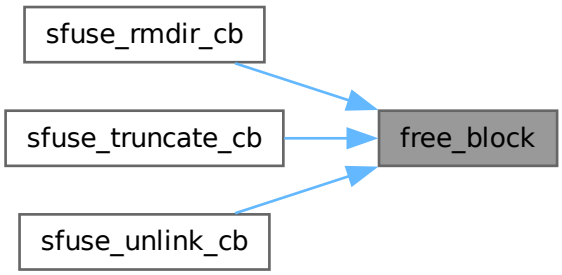
sb	슈퍼블록 구조체 포인터 (파일 시스템의 전체 상태 관리)
block_map	블록 할당 여부를 나타내는 비트맵 버퍼 포인터

offset	해제할 블록의 오프셋(0부터 시작)

bitmap.c 파일의 142 번째 라인에서 정의되었습니다.

```
00142                                                   {
00143   uint32_t byte_idx = offset / 8; // 비트맵 내의 바이트 단위 인덱스 계산
00144   uint32_t bit_idx = offset % 8;  // 바이트 내에서의 비트 위치 계산 (0~7)
00145
00146   // 비트맵에서 해당 블록의 비트를 '0'으로 설정 (빈 상태로 표시)
00147   block_map[byte_idx] &= ~(1 « bit_idx);
00148
00149   // 슈퍼블록 내 빈 블록 개수를 1 증가 (가용 블록 수 갱신)
00150   sb->free_blocks++;
00151 }
```

이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.4.2.6 free_inode()

```
void free_inode (
            struct sfuse_super * sb,
            uint8_t * inode_map,
            uint32_t ino)
```

아이노드 해제
할당된 아이노드를 해제하고, 아이노드 비트맵을 업데이트한다.

매개변수

sb	슈퍼블록 구조체 포인터
inode_map	아이노드 비트맵 버퍼
ino	해제할 아이노드 번호

아이노드 해제
이 함수는 파일이나 디렉토리가 삭제될 때 호출되며, 사용하던 아이노드를 해제하여 다시 사용 가능한 상태로 되돌린다.
주어진 아이노드 번호(ino)를 검사하여 유효한 범위인지 확인한 후, 해당 아이노드의 비트를 아이노드 비트맵(inode_map)에서 0으로 설정하여 빈 상태를 나타내고, 슈퍼블록의 'free_inodes'를 증가시켜 사용 가능한 아이노드 수를 갱신한다.
아이노드 번호가 유효하지 않은 경우(0이거나 총 아이노드 수를 초과하는 경우), 함수는 아무 작업도 수행하지 않고 즉시 종료된다.

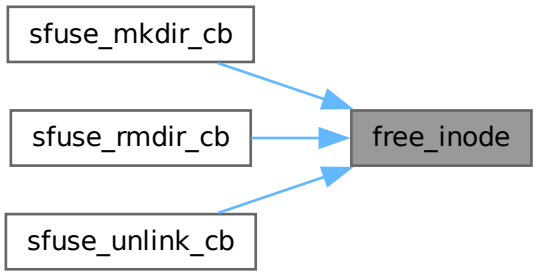
매개변수

sb	슈퍼블록 구조체 포인터 (파일 시스템의 전역 상태 관리)
inode_map	아이노드 할당 상태를 나타내는 비트맵 버퍼 포인터
ino	해제할 아이노드 번호 (유효 범위: 1 ~ inodes_count-1)

bitmap.c 파일의 213 번째 라인에서 정의되었습니다.

```
{
// 아이노드 번호 유효성 검사 (0번 아이노드 및 유효 범위 초과 시 무시)
if (ino == 0 || ino >= sb->inodes_count)
  return; // 잘못된 아이노드 번호이므로 즉시 종료

uint32_t byte_idx = ino / 8; // 비트맵의 바이트 인덱스 계산
uint32_t bit_idx = ino % 8;  // 해당 바이트 내의 비트 위치 계산 (0~7)

// 비트맵에서 해당 아이노드의 비트를 0으로 설정하여 빈 상태로 변경
inode_map[byte_idx] &= ~(1 « bit_idx);

// 슈퍼블록의 빈 아이노드 개수를 1 증가시킴
sb->free_inodes++;
}
```

이 함수를 호출하는 함수들에 대한 그래프입니다.:



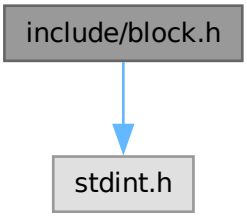
8.5 bitmap.h

이 파일의 문서화 페이지로 가기

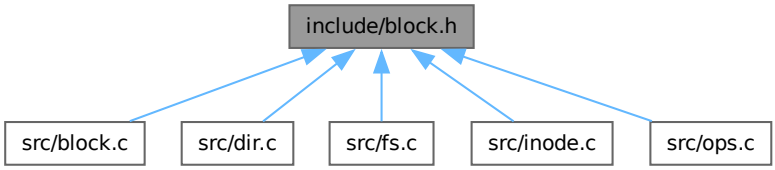
```
#ifndef SFUSE_BITMAP_H
#define SFUSE_BITMAP_H

#include "super.h" // struct sfuse_super 정의
#include <stddef.h>
#include <stdint.h>

int bitmap_load(int fd, uint32_t block_no, uint8_t *map, size_t map_size);

int bitmap_sync(int fd, uint32_t block_no, uint8_t *map, size_t map_size);

int alloc_block(struct sfuse_super *sb, uint8_t *block_map);

void free_block(struct sfuse_super *sb, uint8_t *block_map, uint32_t offset);

int alloc_inode(struct sfuse_super *sb, uint8_t *inode_map);

void free_inode(struct sfuse_super *sb, uint8_t *inode_map, uint32_t ino);

#endif // SFUSE_BITMAP_H
```

8.6 include/block.h 파일 참조

블록 단위 읽기 및 쓰기를 위한 인터페이스 정의
#include <stdint.h>
block.h에 대한 include 의존 그래프



이 그래프는 이 파일을 직/간접적으로 include 하는 파일들을 보여줍니다.:



함수

- int read_block (int fd, uint32_t block_no, void *buf)
  디바이스에서 블록 단위(4KB)로 읽기
- int write_block (int fd, uint32_t block_no, const void *buf)
  디바이스에 블록 단위(4KB)로 쓰기

8.6.1 상세한 설명

블록 단위 읽기 및 쓰기를 위한 인터페이스 정의
디바이스와의 입출력을 블록 단위로 처리하기 위한 인터페이스를 제공한다. 블록 크기는 SFUSE_BLOCK_SIZE (일반적으로 4KB)로 고정된다.
block.h 파일에서 정의되었습니다.

8.6.2 함수 문서화

8.6.2.1 read_block()

```
int read_block (
            int fd,
            uint32_t block_no,
            void * buf)
```

디바이스에서 블록 단위(4KB)로 읽기

매개변수

fd	디바이스 파일 디스크립터
block_no	읽을 블록 번호 (0부터 시작)
buf	SFUSE_BLOCK_SIZE 바이트 버퍼

반환값

0 성공, 음수 오류코드

디바이스에서 블록 단위(4KB)로 읽기
블록 번호를 이용하여 디바이스 파일의 정확한 위치(offset)를 계산한 후, disk_read()를 호출하여 해당 위치에서 블록 단위의 데이터를 읽는다. 읽기 작업은 SFUSE_BLOCK_SIZE 단위로 이루어진다.

매개변수

fd	디바이스 파일 디스크립터
block_no	읽을 블록 번호 (0부터 시작)
buf	읽은 데이터를 저장할 SFUSE_BLOCK_SIZE 크기의 버퍼

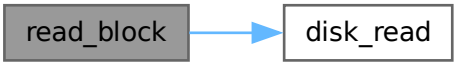
반환값

0 성공, 음수 오류 코드 -EIO: 읽은 데이터 크기가 SFUSE_BLOCK_SIZE와 다를 경우 disk_read()에서 반환된 음수의 오류 코드

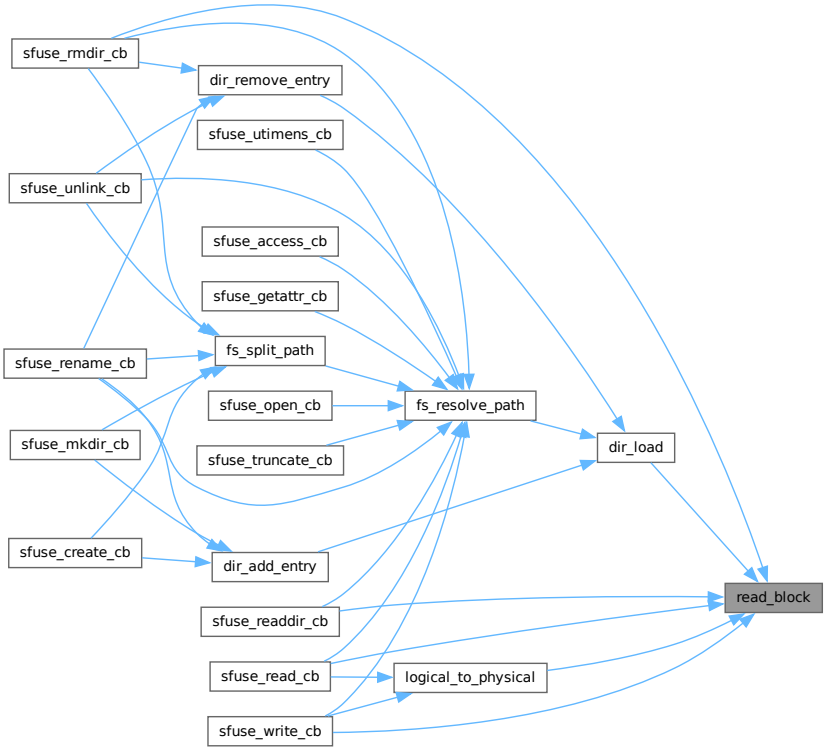
block.c 파일의 30 번째 라인에서 정의되었습니다.

```
00030                                                   {
00031   // 블록 번호를 실제 파일의 바이트 오프셋으로 변환
00032   off_t offset = (off_t)block_no * SFUSE_BLOCK_SIZE;
00033
00034   // 계산된 오프셋에서 블록 크기만큼 데이터를 읽기
00035   ssize_t ret = disk_read(fd, buf, SFUSE_BLOCK_SIZE, offset);
00036
00037   // disk_read 실패 시 오류 코드 반환
00038   if (ret < 0)
00039     return (int)ret;
00040
00041   // 읽은 데이터가 요청한 블록 크기와 일치하지 않을 경우 EIO 오류 반환
00042   if ((size_t)ret != SFUSE_BLOCK_SIZE)
00043     return -EIO;
00044
00045   return 0;
00046 }
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.6.2.2 write_block()

int write_block (
int fd,
uint32_t block_no,
const void * buf)

디바이스에 블록 단위(4KB)로 쓰기

매개변수

fd	디바이스 파일 디스크립터
block_no	쓸 블록 번호 (0부터 시작)
buf	SFUSE_BLOCK_SIZE 바이트 데이터 버퍼

반환값

0 성공, 음수 오류코드

디바이스에 블록 단위(4KB)로 쓰기
블록 번호를 기반으로 디바이스 파일 내의 정확한 위치(offset)를 계산한 후, disk_write()를 호출하여 해당 위치에 블록 크기의 데이터를 기록한다. 쓰기 작업은 SFUSE_BLOCK_SIZE 단위로 이루어진다.

매개변수

fd	디바이스 파일 디스크립터
block_no	기록할 블록 번호 (0부터 시작)

buf	기록할 데이터를 담고 있는 SFUSE_BLOCK_SIZE 크기의 버퍼

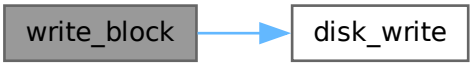
반환값

0 성공, 음수 오류 코드 -EIO: 기록한 데이터 크기가 SFUSE_BLOCK_SIZE와 다를 경우 disk_write()에서 반환된 음수의 오류 코드

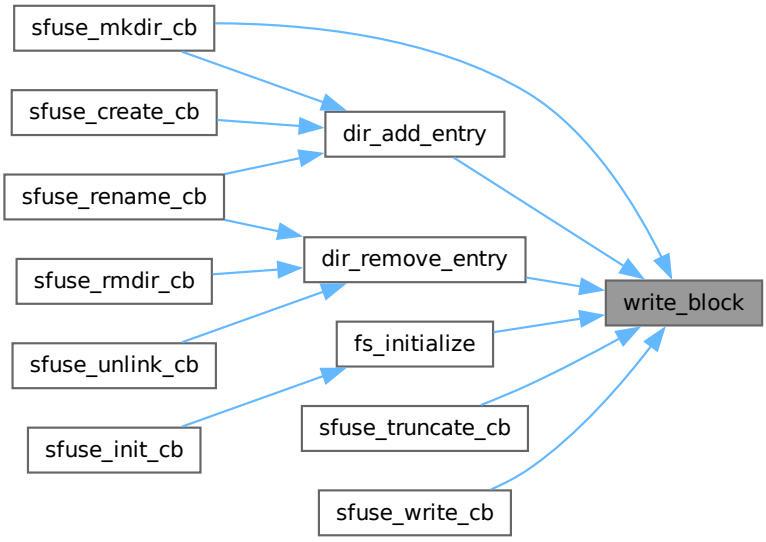
block.c 파일의 62 번째 라인에서 정의되었습니다.

```
00062                                                   {
00063   // 블록 번호를 실제 파일의 바이트 오프셋으로 변환
00064   off_t offset = (off_t)block_no * SFUSE_BLOCK_SIZE;
00065
00066   // 계산된 오프셋에서 블록 크기만큼 데이터를 쓰기
00067   ssize_t ret = disk_write(fd, buf, SFUSE_BLOCK_SIZE, offset);
00068
00069   // disk_write 실패 시 오류 코드 반환
00070   if (ret < 0)
00071     return (int)ret;
00072
00073   // 기록한 데이터가 요청한 블록 크기와 일치하지 않을 경우 EIO 오류 반환
00074   if ((size_t)ret != SFUSE_BLOCK_SIZE)
00075     return -EIO;
00076
00077   return 0;
00078 }
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



이 함수를 호출하는 함수들에 대한 그래프입니다.:



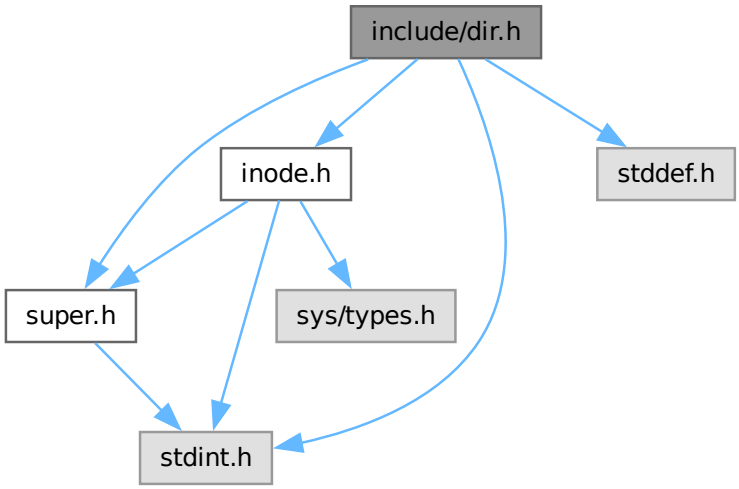
8.7 block.h

이 파일의 문서화 페이지로 가기

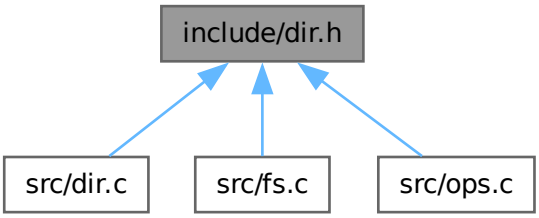
```
00001
00008
00009 #ifndef SFUSE_BLOCK_H
00010 #define SFUSE_BLOCK_H
00011
00012 #include <stdint.h>
00013
00021 int read_block(int fd, uint32_t block_no, void *buf);
00022
00030 int write_block(int fd, uint32_t block_no, const void *buf);
00031
00032 #endif // SFUSE_BLOCK_H
```

8.8 include/dir.h 파일 참조

파일 시스템의 디렉터리 관리 기능 정의
#include "inode.h"
#include "super.h"
#include <stddef.h>
#include <stdint.h>
dir.h에 대한 include 의존 그래프



이 그래프는 이 파일을 직/간접적으로 include 하는 파일들을 보여줍니다.:



데이터 구조

- struct sfuse_dirent

  디렉터리 엔트리 정보를 저장하는 구조체

매크로

- #define SFUSE_NAME_LEN 255

함수

- int dir_load (int fd, const struct sfuse_super *sb, uint32_t ino, void *buf)

  디렉터리의 모든 블록을 디스크에서 읽어들인다.
- int dir_add_entry (int fd, const struct sfuse_super *sb, uint32_t ino, const char *name, uint32_t child_ino, uint8_t *block_map, uint8_t *inode_map, struct sfuse_super *supermap)

  디렉터리에 새로운 엔트리를 추가한다.
- int dir_remove_entry (int fd, const struct sfuse_super *sb, uint32_t ino, const char *name)

  디렉터리에서 지정된 이름의 엔트리를 삭제한다.

8.8.1 상세한 설명

파일 시스템의 디렉터리 관리 기능 정의
디렉터리의 생성, 읽기, 추가 및 삭제와 같은 관리 작업을 처리하는 함수와 구조체 정의. 디렉터리 엔트리는 파일이나 하위 디렉터리를 나타낸다.
dir.h 파일에서 정의되었습니다.

8.8.2 매크로 문서화

8.8.2.1 SFUSE_NAME_LEN

#define SFUSE_NAME_LEN 255
dir.h 파일의 18 번째 라인에서 정의되었습니다.

8.8.3 함수 문서화

8.8.3.1 dir_add_entry()

```
int dir_add_entry (
            int fd,
            const struct sfuse_super * sb,
```

uint32_t ino,
const char * name,
uint32_t child_ino,
uint8_t * block_map,
uint8_t * inode_map,
struct sfuse_super * supermap)

디렉터리에 새로운 엔트리를 추가한다.
부모 디렉터리에 파일 또는 디렉터리를 나타내는 엔트리를 추가하며, 추가 과정에서 블록 및 아이노드 비트맵과 슈퍼블록을 적절히 갱신한다.

매개변수

fd	디바이스 파일 디스크립터
sb	슈퍼블록 정보 구조체 포인터
ino	부모 디렉터리의 아이노드 번호
name	추가할 파일 또는 디렉터리의 이름
child_ino	추가될 엔트리에 연결될 새로운 아이노드 번호
block_map	블록 비트맵 버퍼 (사용된 블록 관리용)
inode_map	아이노드 비트맵 버퍼 (사용된 아이노드 관리용)
supermap	갱신 후 슈퍼블록의 변경 사항 동기화를 위한 슈퍼블록 복사본

반환값

성공 시 0 반환, 실패 시 음수의 오류 코드 반환

디렉터리에 새로운 엔트리를 추가한다.
지정된 디렉터리 아이노드가 관리하는 직접 블록에서 빈 슬롯을 찾아, 새로운 파일 또는 디렉터리 엔트리를 추가한다. 엔트리 추가 후 수정된 디렉터리 데이터를 디스크에 기록하고, 파일시스템의 슈퍼블록 및 블록/아이노드 비트맵도 최신 상태로 동기화한다.

매개변수

fd	디바이스 파일 디스크립터 (디스크 접근용)
sb	슈퍼블록 구조체 포인터 (파일시스템의 정보 참조용)
ino	부모 디렉터리 아이노드 번호 (엔트리를 추가할 디렉터리)
name	추가할 새 엔트리의 이름
child_ino	새 엔트리에 연결될 아이노드 번호
block_map	블록 사용 정보를 나타내는 비트맵 버퍼
inode_map	아이노드 사용 정보를 나타내는 비트맵 버퍼
supermap	슈퍼블록 복사본 (변경 사항 동기화 시 사용)

반환값

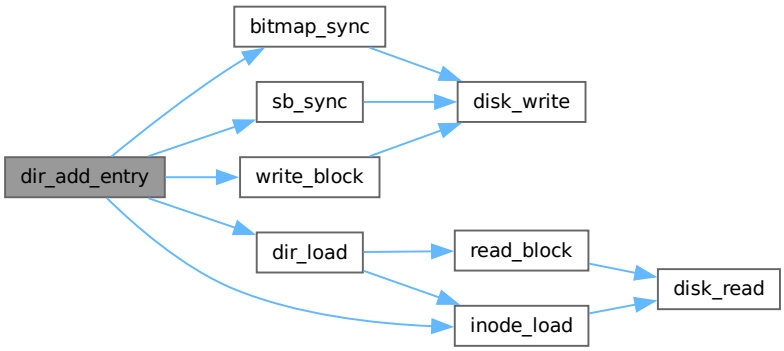
성공 시 0 반환, 빈 슬롯이 없어 추가가 불가능하면 -ENOSPC 반환, 메모리 할당 실패 시 -ENOMEM, 기타 입출력 오류 발생 시 음수 오류 코드 반환

dir.c 파일의 89 번째 라인에서 정의되었습니다.

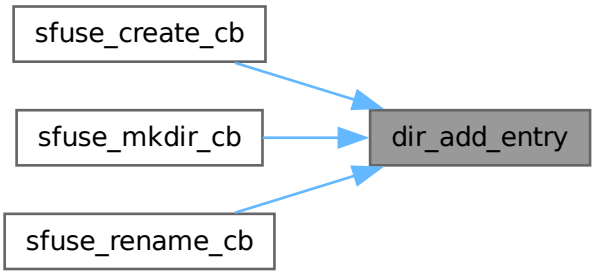
```
00091                                                   {
00092   // 부모 디렉터리의 아이노드를 로드하여 inode 구조체에 저장한다.
00093   struct sfuse_inode inode;
00094   int res = inode_load(fd, sb, ino, &inode);
00095   if (res < 0)
00096     return res; // 아이노드 로딩에 실패하면 즉시 종료한다.
00097
00098   // 디렉터리 데이터 전체를 저장할 메모리 버퍼를 할당한다.
00099   size_t buf_size = SFUSE_NDIR_BLOCKS * SFUSE_BLOCK_SIZE;
00100   void *buf = malloc(buf_size);
```

```
if (!buf)
  return -ENOMEM; // 메모리 할당 실패 시 오류 코드 반환

// 디스크로부터 디렉터리의 모든 직접 블록 데이터를 읽어온다.
res = dir_load(fd, sb, ino, buf);
if (res < 0) {
  free(buf);
  return res; // 블록 로드 실패 시 메모리 해제 후 종료한다.
}

// 버퍼를 디렉터리 엔트리 배열로 해석하여 최대 엔트리 개수 산정
struct sfuse_dirent *ents = (struct sfuse_dirent *)buf;
uint32_t max_entries = buf_size / sizeof(*ents);

// 디렉터리 내의 빈 슬롯을 탐색하여 새 엔트리를 추가한다.
for (uint32_t i = 0; i < max_entries; i++) {
  if (ents[i].ino == 0) {    // 빈 슬롯 발견 (아이노드 번호가 0이면 빈 슬롯)
    ents[i].ino = child_ino; // 새 아이노드 번호를 할당
    strncpy(ents[i].name, name, sizeof(ents[i].name) - 1); // 엔트리 이름 복사
    ents[i].name[sizeof(ents[i].name) - 1] = '\0'; // 문자열의 NULL 종단 보장

    // 수정된 전체 디렉터리 데이터를 디스크의 직접 블록에 다시 기록한다.
    for (uint32_t b = 0; b < SFUSE_NDIR_BLOCKS; b++) {
      if (inode.direct[b] == 0)
        continue; // 사용되지 않은 블록은 건너뛴다.

      res = write_block(fd, inode.direct[b],
                        (char *)buf + b * SFUSE_BLOCK_SIZE);
      if (res < 0) {
        free(buf);
        return res; // 블록 기록 실패 시 메모리 해제 후 종료한다.
      }
    }

    // 변경된 슈퍼블록과 블록 및 아이노드 비트맵을 디스크에 동기화한다.
    sb_sync(fd, supermap);
    bitmap_sync(fd, sb->block_bitmap_start, block_map, sb->blocks_count / 8);
    bitmap_sync(fd, sb->inode_bitmap_start, inode_map, sb->inodes_count / 8);

    free(buf); // 작업이 완료되었으므로 메모리 해제
    return 0;  // 성공적으로 엔트리 추가 완료
  }
}

// 디렉터리 내 모든 슬롯이 사용 중이어서 추가 불가 (공간 부족)
free(buf);
return -ENOSPC; // 추가 공간 부족 오류 반환
}
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.8.3.2 dir_load()

```
int dir_load (
            int fd,
            const struct sfuse_super * sb,
            uint32_t ino,
            void * buf)
```

디렉터리의 모든 블록을 디스크에서 읽어들인다.
지정된 아이노드 번호(디렉터리)에 해당하는 데이터를 연속적인 블록으로 읽어서 버퍼에 저장한다.

매개변수

fd	디바이스 파일 디스크립터
sb	슈퍼블록 정보 구조체 포인터
ino	디렉터리 아이노드 번호
buf	읽은 디렉터리 데이터(연속 블록)를 저장할 버퍼

반환값

성공 시 0 반환, 실패 시 음수의 오류 코드 반환

디렉터리의 모든 블록을 디스크에서 읽어들인다.
주어진 아이노드 번호(ino)에 해당하는 디렉터리의 아이노드 정보를 먼저 디스크에서 읽어온다. 이 아이노드에는 해당 디렉터리의 실제 데이터가 저장된 데이터 블록의 번호들이 저장되어 있다.
이 함수는 직접(direct) 블록 포인터만을 처리하며, 각 블록이 0인 경우(미할당)는 무시한다. 각 블록을 순서대로 디스크에서 읽어, 버퍼의 알맞은 위치에 복사하여 최종적으로 메모리에 디렉터리 전체 데이터를 적재한다.

매개변수

fd	디바이스 파일 디스크립터 (데이터를 읽을 원본 디바이스를 나타냄)
sb	슈퍼블록 정보 구조체 포인터 (파일 시스템의 레이아웃 정보 제공)
ino	데이터를 읽어올 디렉터리의 아이노드 번호
buf	읽어온 디렉터리 데이터를 저장할 버퍼 (충분한 크기가 미리 할당되어 있어야 함)

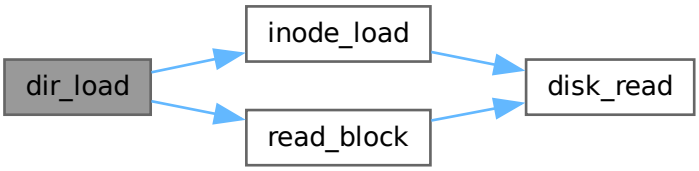
반환값

성공적으로 모든 블록을 읽었다면 0을 반환하고, inode_load 또는 read_block 수행 중 오류가 발생하면 음수의 오류 코드 반환

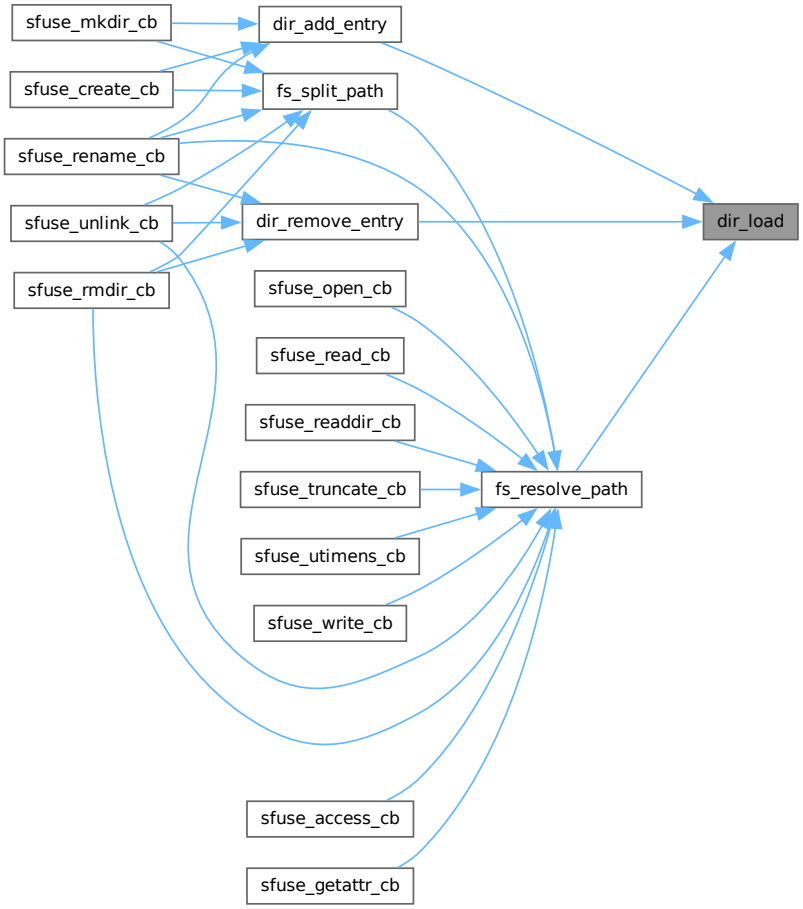
dir.c 파일의 42 번째 라인에서 정의되었습니다.

```
00042                                                   {
00043   // 디스크에서 주어진 아이노드 번호(ino)의 아이노드 메타데이터를 로드하여 inode
00044   // 변수에 저장
00045   struct sfuse_inode inode;
00046   int res = inode_load(fd, sb, ino, &inode);
00047   if (res < 0)
00048     return res; // 아이노드 로딩 실패 시 즉시 오류 코드 반환
00049
00050   // 아이노드가 참조하는 모든 직접(direct) 블록들을 순회하며 디스크에서 읽는다
00051   for (uint32_t i = 0; i < SFUSE_NDIR_BLOCKS; i++) {
00052     // 현재 블록 번호가 0이면 미할당 상태이므로 건너뜀
00053     if (inode.direct[i] == 0)
00054       continue;
00055
00056     // i번째 블록을 읽어 buf의 i번째 블록 위치로 이동한 메모리 주소에 저장
00057     // 각 블록 크기(SFUSE_BLOCK_SIZE)를 곱해 정확한 위치 계산
00058     res = read_block(fd, inode.direct[i], (char *)buf + i * SFUSE_BLOCK_SIZE);
00059     if (res < 0)
00060       return res; // 블록 읽기 중 오류 발생 시 즉시 오류 코드 반환
00061   }
00062
00063   // 모든 직접 블록을 성공적으로 읽었으면 0 반환
00064   return 0;
00065 }
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.8.3.3 dir_remove_entry()

```
int dir_remove_entry (
            int fd,
            const struct sfuse_super * sb,
            uint32_t ino,
            const char * name)
```

디렉터리에서 지정된 이름의 엔트리를 삭제한다.
부모 디렉터리에서 지정된 이름의 엔트리를 삭제하여 파일이나 하위 디렉터리를 제거한다. 삭제 시 해당 엔트리가 차지한 블록과 아이노드는 별도의 처리가 필요하다.

매개변수

fd	디바이스 파일 디스크립터
sb	슈퍼블록 정보 구조체 포인터
ino	부모 디렉터리의 아이노드 번호
name	삭제할 디렉터리 엔트리의 이름

반환값

성공 시 0 반환, 실패 시 음수의 오류 코드 반환

디렉터리에서 지정된 이름의 엔트리를 삭제한다.
이름이 일치하는 엔트리를 찾아 해당 슬롯을 비운 후 디스크에 데이터를 기록한다.

매개변수

fd	디바이스 파일 디스크립터
sb	슈퍼블록 정보 구조체 포인터
ino	부모 디렉터리 아이노드 번호
name	삭제할 엔트리의 이름

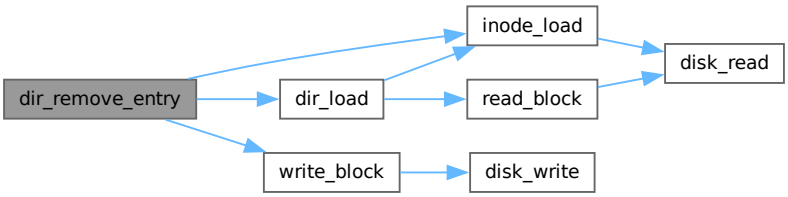
반환값

성공 시 0, 엔트리가 존재하지 않으면 -ENOENT, 기타 오류 발생 시 음수 오류 코드

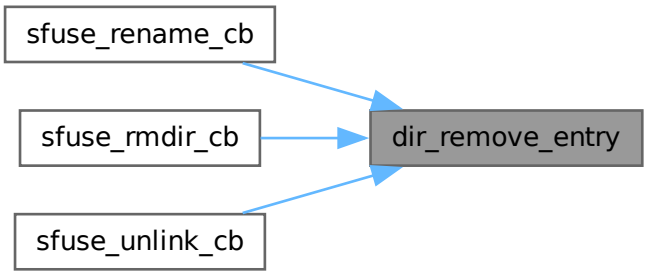
dir.c 파일의 162 번째 라인에서 정의되었습니다.

```
{
// 디렉터리 아이노드 로드
struct sfuse_inode inode;
int res = inode_load(fd, sb, ino, &inode);
if (res < 0)
  return res;

// 디렉터리 데이터 블록 메모리에 로드
size_t buf_size = SFUSE_NDIR_BLOCKS * SFUSE_BLOCK_SIZE;
void *buf = malloc(buf_size);
if (!buf)
  return -ENOMEM;

res = dir_load(fd, sb, ino, buf);
if (res < 0) {
  free(buf);
  return res;
}

struct sfuse_dirent *ents = (struct sfuse_dirent *)buf;
uint32_t max_entries = buf_size / sizeof(*ents);

// 이름이 일치하는 엔트리 탐색 및 삭제
for (uint32_t i = 0; i < max_entries; i++) {
  if (ents[i].ino != 0 && strcmp(ents[i].name, name) == 0) {
    ents[i].ino = 0;
    ents[i].name[0] = '\0';

    // 수정된 디렉터리 데이터를 디스크에 기록
    for (uint32_t b = 0; b < SFUSE_NDIR_BLOCKS; b++) {
      if (inode.direct[b] == 0)
        continue;

      res = write_block(fd, inode.direct[b],
                        (char *)buf + b * SFUSE_BLOCK_SIZE);
      if (res < 0) {
        free(buf);
        return res;
      }
    }

    free(buf);
    return 0;
  }
}

// 해당 이름의 엔트리가 없을 경우 메모리 해제 후 오류 반환
free(buf);
return -ENOENT;
}
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



이 함수를 호출하는 함수들에 대한 그래프입니다.:



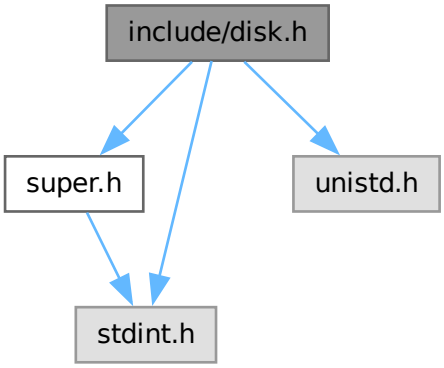
8.9 dir.h

이 파일의 문서화 페이지로 가기

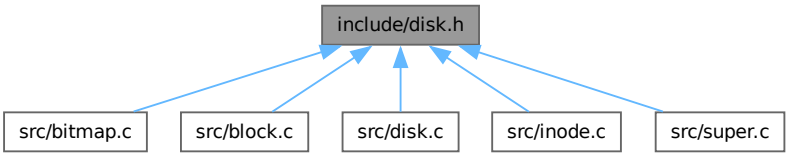
```
#ifndef SFUSE_DIR_H
#define SFUSE_DIR_H

#include "inode.h"
#include "super.h"
#include <stddef.h>
#include <stdint.h>

// 디렉터리 엔트리의 최대 이름 길이
#define SFUSE_NAME_LEN 255

struct sfuse_dirent {
  uint32_t ino;
  char name[SFUSE_NAME_LEN + 1];
};

int dir_load(int fd, const struct sfuse_super *sb, uint32_t ino, void *buf);

int dir_add_entry(int fd, const struct sfuse_super *sb, uint32_t ino,
                  const char *name, uint32_t child_ino, uint8_t *block_map,
                  uint8_t *inode_map, struct sfuse_super *supermap);

int dir_remove_entry(int fd, const struct sfuse_super *sb, uint32_t ino,
                     const char *name);

#endif // SFUSE_DIR_H
```

8.10 include/disk.h 파일 참조

원시 디바이스에 대한 저수준 I/O 연산 함수 선언
#include "super.h"
#include <stdint.h>
#include <unistd.h>
disk.h에 대한 include 의존 그래프



이 그래프는 이 파일을 직/간접적으로 include 하는 파일들을 보여줍니다.:



함수

- ssize_t disk_read (int fd, void *buf, size_t size, off_t offset)

  원시 디바이스에서 특정 offset으로부터 데이터를 읽는다.
- ssize_t disk_write (int fd, const void *buf, size_t size, off_t offset)

  원시 디바이스의 특정 오프셋(offset)에 데이터를 쓴다.

8.10.1 상세한 설명

원시 디바이스에 대한 저수준 I/O 연산 함수 선언
본 파일은 원시 디바이스에 직접 접근하여 데이터를 읽고 쓰는 기능을 제공한다.
디스크 접근은 보통 슈퍼블록, 아이노드, 데이터 블록 등의 읽기/쓰기에 사용되며, 다른 상위 레벨 함수들에서 활용된다.

주의

read_block()과 write_block() 함수는 중복 코드를 방지하기 위해 별도의 파일(block.c)에 구현되어 있다.

disk.h 파일에서 정의되었습니다.

8.10.2 함수 문서화

8.10.2.1 disk_read()

```
ssize_t disk_read (
            int fd,
            void * buf,
            size_t count,
            off_t off)
```

원시 디바이스에서 특정 offset으로부터 데이터를 읽는다.
이 함수는 주어진 파일 디스크립터(fd)를 사용하여 지정된 offset 위치에서부터 size 바이트만큼 데이터를 읽어 buf가 가리키는 메모리 영역에 저장한다.
내부적으로는 디바이스 내 offset 위치로 lseek를 수행한 뒤 read 시스템 호출을 통해 읽기를 수행한다.

매개변수

fd	원시 디바이스 파일 디스크립터 (open으로 열림)
buf	데이터를 저장할 메모리 버퍼의 포인터
size	읽으려는 데이터의 크기(바이트 단위)
offset	읽기를 시작할 원시 디바이스 내의 바이트 단위 오프셋

반환값

성공 시 실제로 읽은 바이트 수를 반환한다. 실패 시 음수(-errno)를 반환하며, errno는 오류를 나타낸다.

반환값

-EIO	입출력 오류가 발생한 경우
-EBADF	유효하지 않은 파일 디스크립터를 전달한 경우

원시 디바이스에서 특정 offset으로부터 데이터를 읽는다.
본 함수는 lseek를 통해 디스크 내 원하는 위치(offset)로 이동한 뒤, read를 통해 데이터를 읽는다. offset은 파일의 시작 위치로부터 바이트 단위로 계산된다.

매개변수

fd	디바이스 파일 디스크립터
buf	읽은 데이터를 저장할 버퍼 포인터
count	읽을 데이터의 크기(바이트 단위)
off	읽기 시작할 위치 (바이트 단위 오프셋)

반환값

성공 시 실제로 읽은 바이트 수 반환. 실패 시 음수(-errno) 반환.

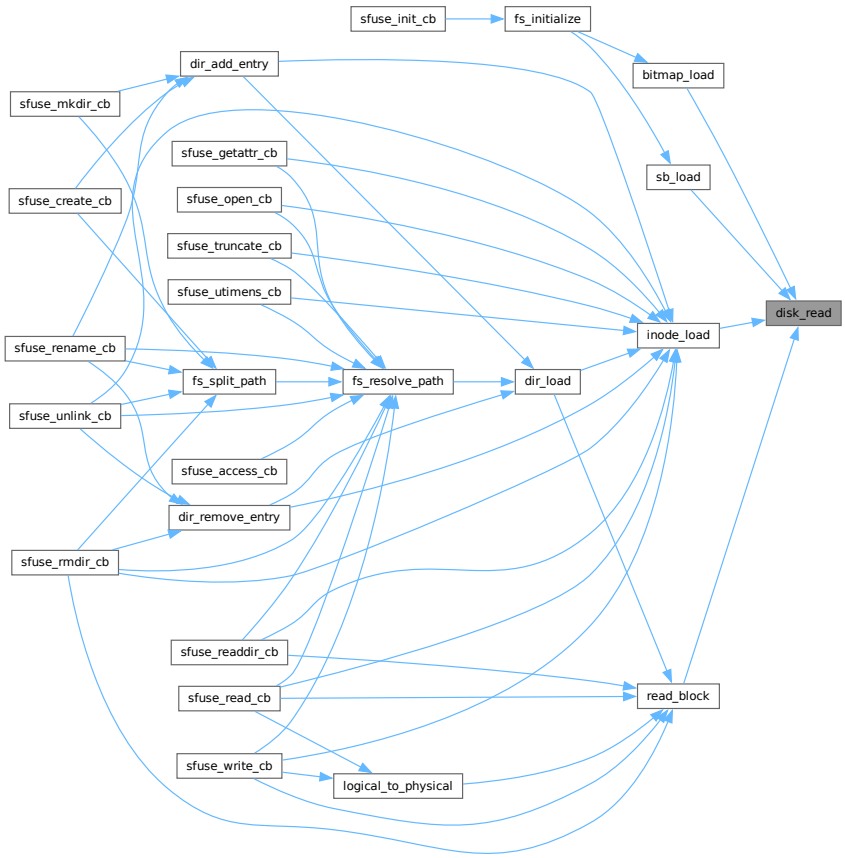
주의

반환된 바이트 수는 요청한 count 값보다 작을 수 있다.

disk.c 파일의 32 번째 라인에서 정의되었습니다.

```
00032                                   {
00033  // 지정된 위치로 디스크 포인터 이동
00034  if (lseek(fd, off, SEEK_SET) < 0)
00035    return -errno; // lseek 실패 시 errno 반환
00036
00037  // 데이터를 실제로 읽음
00038  ssize_t ret = read(fd, buf, count);
00039  if (ret < 0)
00040    return -errno; // 읽기 실패 시 errno 반환
00041
00042  return ret; // 읽기 성공 시 읽은 바이트 수 반환
00043 }
```

이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.10.2.2 disk_write()

```
ssize_t disk_write (
            int fd,
            const void * buf,
            size_t count,
            off_t off)
```

원시 디바이스의 특정 오프셋(offset)에 데이터를 쓴다.
이 함수는 주어진 파일 디스크립터(fd)를 사용하여 지정된 offset 위치에서부터 buf가 가리키는 데이터를 size 바이트만큼 디스크에 기록한다.
내부적으로는 디바이스 내 offset 위치로 lseek를 수행한 뒤 write 시스템 호출을 통해 기록을 수행한다.

매개변수

fd	원시 디바이스 파일 디스크립터 (open으로 열림)
buf	기록할 데이터를 담고 있는 메모리 버퍼의 포인터
size	기록할 데이터의 크기(바이트 단위)
offset	기록을 시작할 원시 디바이스 내의 바이트 단위 오프셋

반환값

성공 시 실제로 기록된 바이트 수를 반환한다. 실패 시 음수(-errno)를 반환하며, errno는 오류를 나타낸다.

반환값

-EIO	입출력 오류가 발생한 경우
-EBADF	유효하지 않은 파일 디스크립터를 전달한 경우
-ENOSPC	디스크 공간이 부족한 경우

원시 디바이스의 특정 오프셋(offset)에 데이터를 쓴다.
본 함수는 lseek를 통해 디스크 내 원하는 위치(offset)로 이동한 뒤, write를 통해 데이터를 쓴다. offset은 파일의 시작 위치로부터 바이트 단위로 계산된다.

매개변수

fd	디바이스 파일 디스크립터
buf	기록할 데이터가 저장된 버퍼 포인터
count	기록할 데이터 크기(바이트 단위)
off	기록을 시작할 위치 (바이트 단위 오프셋)

반환값

성공 시 실제로 기록된 바이트 수 반환. 실패 시 음수(-errno) 반환.

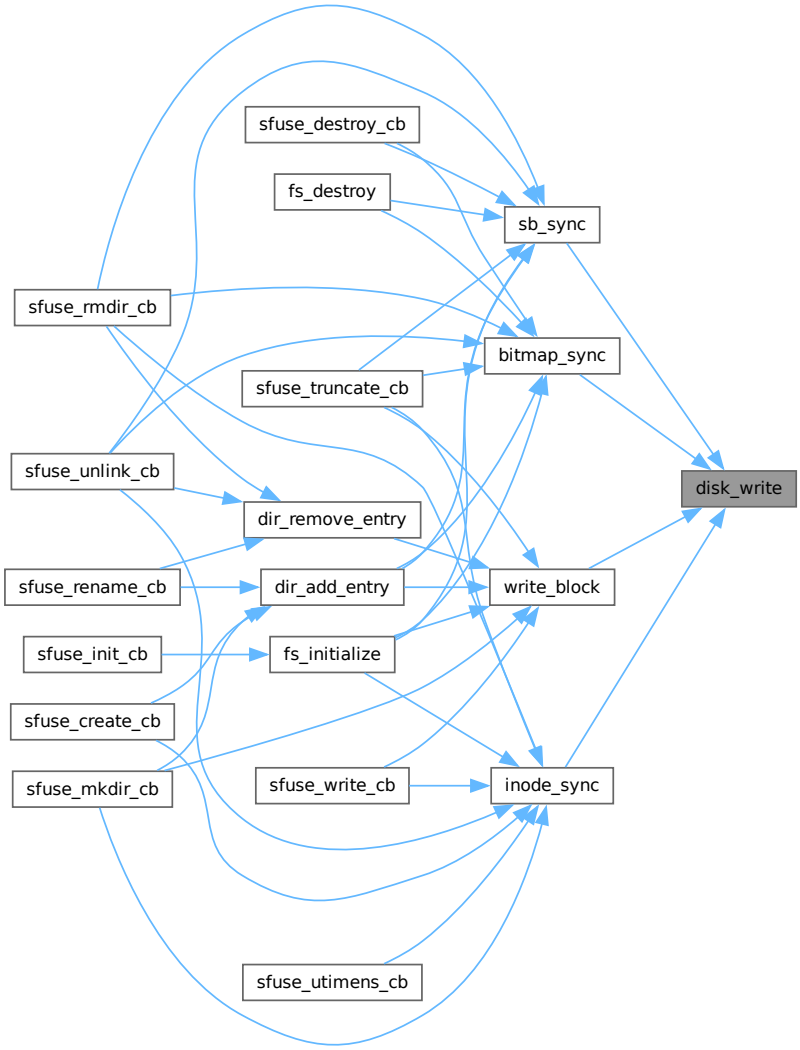
주의

기록된 바이트 수가 요청한 count 값보다 작을 수 있다. 디스크가 꽉 차는 등의 상황에서는 ENOSPC 오류가 발생할 수 있다.

disk.c 파일의 61 번째 라인에서 정의되었습니다.

```
00061                                                   {
00062   // 지정된 위치로 디스크 포인터 이동
00063   if (lseek(fd, off, SEEK_SET) < 0)
00064     return -errno; // lseek 실패 시 errno 반환
00065
00066   // 데이터를 실제로 기록함
00067   ssize_t ret = write(fd, buf, count);
00068   if (ret < 0)
00069     return -errno; // 기록 실패 시 errno 반환
00070
00071   return ret; // 기록 성공 시 기록한 바이트 수 반환
00072 }
```

이 함수를 호출하는 함수들에 대한 그래프입니다.:



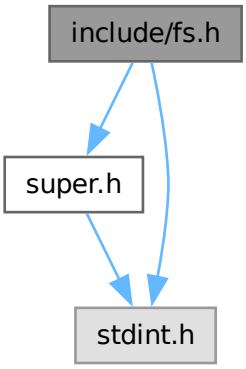
8.11 disk.h

이 파일의 문서화 페이지로 가기

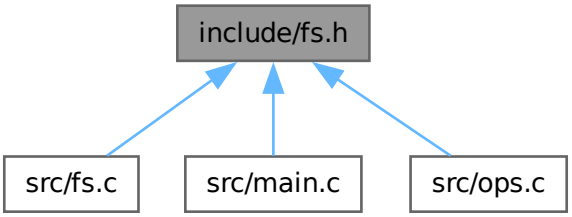
```
00001
00013
00014 #ifndef SFUSE_DISK_H
00015 #define SFUSE_DISK_H
00016
00017 #include "super.h" // SFUSE_BLOCK_SIZE, SFUSE_SUPERBLOCK_OFFSET
00018 #include <stdint.h>
00019 #include <unistd.h>
00020
00040 ssize_t disk_read(int fd, void *buf, size_t size, off_t offset);
00041
00062 ssize_t disk_write(int fd, const void *buf, size_t size, off_t offset);
00063
00064 #endif // SFUSE_DISK_H
00065
00066 // NOTE: read_block()과 write_block() 함수는 중복 코드를 방지하기 위해
00067 // block.c에구현되어 있다.
```

8.12 include/fs.h 파일 참조

SFUSE 파일 시스템의 핵심 구조 및 연산 정의
#include "super.h"
#include <stdint.h>
fs.h에 대한 include 의존 그래프



이 그래프는 이 파일을 직/간접적으로 include 하는 파일들을 보여줍니다.:



데이터 구조

- struct sfuse_fs
  SFUSE 파일 시스템의 전역 컨텍스트를 나타내는 구조체

매크로

- #define SFUSE_ROOT_INO 1
  파일 시스템 루트 디렉터리의 고정된 아이노드(inode) 번호 (일반적으로 1)

함수

- struct sfuse_fs * get_fs_context (void)
  FUSE 콜백 함수 내에서 전역 파일 시스템 컨텍스트를 얻는다.

- int fs_initialize (int backing_fd)

  SFUSE 파일 시스템을 초기화하고 슈퍼블록 및 비트맵을 로드한다.
- void fs_destroy (void *private_data)

  SFUSE 파일 시스템을 정리하고 언마운트한다.
- int fs_resolve_path (struct sfuse_fs *fs, const char *path, uint32_t *ino_out)

  주어진 경로를 아이노드 번호로 변환한다.
- char * fs_split_path (const char *fullpath, uint32_t *parent_ino)

  전체 경로를 부모 디렉터리의 아이노드와 마지막 구성요소의 이름으로 분할한다.

8.12.1 상세한 설명

SFUSE 파일 시스템의 핵심 구조 및 연산 정의
fs.h 파일에서 정의되었습니다.

8.12.2 매크로 문서화

8.12.2.1 SFUSE_ROOT_INO

#define SFUSE_ROOT_INO 1
파일 시스템 루트 디렉터리의 고정된 아이노드(inode) 번호 (일반적으로 1)
SFUSE 파일 시스템의 루트 디렉터리('/')를 나타내기 위한 아이노드 번호로, 파일 시스템 내부에서 모든 경로 탐색 및 해석의 출발점(entry point)으로 활용된다. 루트 아이노드는 파일 시스템 초기화 시 반드시 생성되어야 하며, 아이노드 테이블 내에서 항상 아이노드 번호 1번에 위치한다.

주의

> 대부분의 Unix 기반 파일 시스템에서는 관례적으로 루트 디렉터리의 아이노드 번호를 1로 고정한다. 이러한 고정 번호를 사용하면 루트 디렉터리를 참조할 때 추가적인 탐색 과정 없이 즉각적으로 접근할 수 있어 파일 시스템 성능 및 효율성을 높이는 데 기여한다.

fs.h 파일의 27 번째 라인에서 정의되었습니다.

8.12.3 함수 문서화

8.12.3.1 fs_destroy()

void fs_destroy (
void * private_data)
SFUSE 파일 시스템을 정리하고 언마운트한다.

매개변수

private_data	get_fs_context()가 반환한 파일 시스템 전역 컨텍스트 포인터

SFUSE 파일 시스템을 정리하고 언마운트한다.
파일 시스템 종료 시 호출되며, 메모리에 존재하는 파일 시스템의 메타데이터(슈퍼블록, 블록 및 inode 비트맵)를 디스크에 동기화하고, 할당된 메모리 리소스를 해제한다.

매개변수

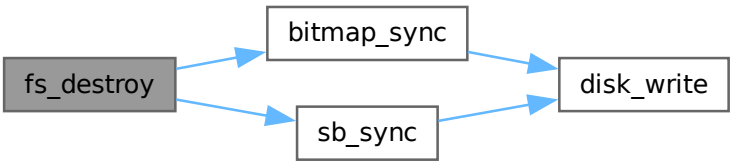
private_data	종료될 파일 시스템 컨텍스트(struct sfuse_fs)의 포인터.

fs.c 파일의 377 번째 라인에서 정의되었습니다.

```
{
// 전달받은 private_data 포인터를 struct sfuse_fs 타입으로 변환
struct sfuse_fs *fs = private_data;

// 비트맵 데이터를 디스크에 동기화하기 위한 메모리 크기 계산
// 블록 비트맵 크기: 전체 블록 수(blocks_count)를 비트맵 표현을 위해 바이트로
```

```
// 환산
size_t bmap_bytes = fs->sb.blocks_count / 8;

// inode 비트맵 크기: 전체 inode 수(inodes_count)를 비트맵 표현을 위해
// 바이트로 환산
size_t imap_bytes = fs->sb.inodes_count / 8;

// 메모리에 존재하는 블록 비트맵 상태를 디스크에 동기화한다.
// 이 작업을 통해 파일 시스템 종료 시 블록의 사용 상태가 디스크에 정확히
// 반영된다.
bitmap_sync(fs->backing_fd, fs->sb.block_bitmap_start, fs->block_map,
            bmap_bytes);

// 메모리에 존재하는 inode 비트맵 상태를 디스크에 동기화한다.
// 파일 시스템 종료 시 inode의 사용 여부를 디스크에 저장한다.
bitmap_sync(fs->backing_fd, fs->sb.inode_bitmap_start, fs->inode_map,
            imap_bytes);

// 슈퍼블록 상태를 디스크에 동기화하여 최신의 파일 시스템 메타데이터를
// 유지한다.
sb_sync(fs->backing_fd, &fs->sb);

// 파일 시스템의 종료 작업 이후 메모리에 할당된 비트맵 메모리를 해제하여,
// 메모리 누수를 방지한다.
free(fs->block_map); // 블록 비트맵 메모리 해제
free(fs->inode_map); // inode 비트맵 메모리 해제
}
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



8.12.3.2 fs_initialize()

```
int fs_initialize (
            int backing_fd)
```

SFUSE 파일 시스템을 초기화하고 슈퍼블록 및 비트맵을 로드한다.

매개변수

backing_fd	초기화할 블록 디바이스의 파일 디스크립터

반환값

성공 시 0, 실패 시 음수 오류 코드

SFUSE 파일 시스템을 초기화하고 슈퍼블록 및 비트맵을 로드한다.
이 함수는 주어진 블록 장치 파일 디스크립터를 이용하여 SFUSE 파일 시스템을 초기화한다. 먼저 장치의 크기를 계산하고, 슈퍼블록이 존재하지 않으면 파일 시스템을 포맷하여 새로 생성하며, 슈퍼블록 및 루트 디렉터리를 초기화한다. 만약 슈퍼블록이 이미 존재한다면 이를 로드하고 기존 비트맵 데이터를 불러온다.

매개변수

backing_fd	파일 시스템을 저장하는 장치의 파일 디스크립터.

반환값

성공 시 0, 실패 시 음수의 에러 코드.

SFUSE 파일 시스템을 초기화하고 슈퍼블록 및 메타데이터를 준비한다.
파일 시스템의 슈퍼블록, inode 테이블, 비트맵 등을 초기화하거나 기존 데이터를 로드하여 파일 시스템 사용 준비를 완료한다.

매개변수

backing_fd	파일 시스템을 저장하는 블록 장치 파일 디스크립터.

반환값

성공 시 0, 실패 시 음수의 에러 코드.

fs.c 파일의 75 번째 라인에서 정의되었습니다.

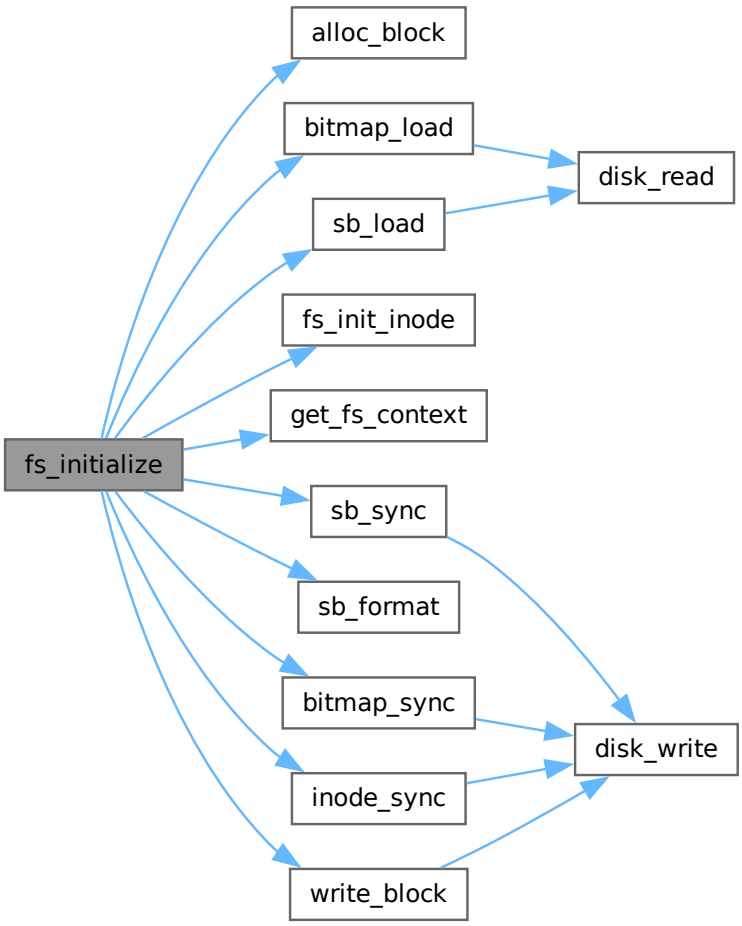
```
00075                                   {
00076  // 현재 사용 중인 파일 시스템 컨텍스트 획득
00077  struct sfuse_fs *fs = get_fs_context();
00078
00079  // 얻은 블록 장치 파일 디스크립터 저장
00080  fs->backing_fd = backing_fd;
00081
00082  /*
00083  블록 장치(backing_fd)의 크기를 바이트 단위로 얻는다.
00084  파일의 크기를 얻으려면 파일의 가장 끝으로 파일 오프셋(offset)을 이동시킨다.
00085  이때 사용되는 함수가 바로 `lseek'이다.
00086
00087  lseek 함수 원형:
00088  off_t lseek(int fd, off_t offset, int whence);
00089
00090  인자 설명:
00091  - fd: 오프셋을 이동시킬 파일 디스크립터.
00092  - offset: 이동할 오프셋의 크기.
00093  - whence: 오프셋 이동 기준을 설정하는 플래그.
00094    - SEEK_SET: 파일의 시작 지점으로부터 offset 바이트 이동.
00095    - SEEK_CUR: 현재 위치에서 offset 바이트 이동.
00096    - SEEK_END: 파일 끝에서부터 offset 바이트 이동.
00097
00098  여기서는 파일의 크기를 알고자 하므로,
00099  SEEK_END를 이용해 파일의 끝으로 이동(offset은 0)을 지정한다.
00100  그러면 lseek는 파일의 총 크기(바이트 단위)를 반환한다.
00101 */
00102  off_t device_size = lseek(backing_fd, 0, SEEK_END);
00103
00104  /*
00105  장치 크기(device_size)는 바이트 단위이기 때문에,
00106  이를 SFUSE 파일 시스템이 사용하는 블록 크기(SFUSE_BLOCK_SIZE)로 나누어
00107  블록 장치가 가진 전체 블록의 수를 계산한다.
00108
00109  예시로, 만약 장치 크기가 8192바이트이고 블록 크기가 1024바이트라면,
00110  총 블록 수(total_blocks)는 8192 / 1024 = 8이 된다.
00111
00112  total_blocks 값은 파일 시스템의 메타데이터와 데이터 블록을 배치할 때
00113  사용된다.
00114  */
00115  uint32_t total_blocks = device_size / SFUSE_BLOCK_SIZE;
00116
00117  // 슈퍼블록을 디스크에서 로드(load) 시도.
00118  // sb_load()가 성공하면 이미 파일 시스템이 포맷되어 있다는 뜻이다.
00119  // sb_load()는 디스크에서 슈퍼블록 데이터를 읽어 fs->sb 구조체에 저장한다.
00120  // 성공 시 0을 반환하며, 실패 시 음수를 반환한다.
00121  //
00122  // sb_load()가 실패하면 (즉, 음수 반환 시),
00123  // 이는 슈퍼블록이 존재하지 않거나 손상된 상태를 나타내므로,
00124  // 새로운 슈퍼블록을 포맷하고 초기화하는 과정을 수행해야 한다.
00125  if (sb_load(backing_fd, &fs->sb) < 0) {
00126
00127    // 슈퍼블록 포맷: 슈퍼블록을 깨끗한 초기 상태로 설정.
00128    // 이 함수는 슈퍼블록의 모든 값을 기본값 또는 초기 상태로 설정한다.
00129    sb_format(&fs->sb);
00130
00131    // 슈퍼블록에 파일 시스템의 전체 블록 개수(total_blocks)를 설정한다.
00132    // 이 값은 파일 시스템이 관리할 수 있는 블록의 최대 개수를 나타낸다.
00133    fs->sb.blocks_count = total_blocks;
00134
00135    // inode 개수 설정: 파일 시스템이 관리할 수 있는 inode의 총 개수를 설정.
00136    // inode는 파일이나 디렉터리 같은 객체의 메타데이터를 저장하는 단위이다.
00137    fs->sb.inodes_count = SFUSE_INODES_COUNT;
```

```

// --- 메타데이터 영역의 크기 계산 시작 ---

// 1. 블록 비트맵(block bitmap)의 크기를 블록 단위로 계산
//
// 블록 비트맵은 각 블록이 사용 중인지 여부를 비트 단위로 관리한다.
// 하나의 블록을 1비트(bit)로 관리하므로,
// 전체 블록 수를 8로 나누면 필요한 바이트(byte) 수가 된다.
//
// 이 바이트 수를 블록 단위(SFUSE_BLOCK_SIZE)로 변환하기 위해 다음 수식을
// 사용:
//
// (fs->sb.blocks_count / 8 + SFUSE_BLOCK_SIZE - 1) / SFUSE_BLOCK_SIZE
//
// 이 수식은 '올림(ceil)'의 효과를 내는 표현으로,
// 정확히 나누어 떨어지지 않을 경우 필요한 블록을 추가로 확보하기 위한
// 계산법이다.
uint32_t block_bitmap_blocks =
    (fs->sb.blocks_count / 8 + SFUSE_BLOCK_SIZE - 1) / SFUSE_BLOCK_SIZE;

// 2. inode 비트맵(inode bitmap)의 크기를 블록 단위로 계산
//
// inode 비트맵은 inode들이 사용 중인지 여부를 관리하는 비트맵이다.
// 전체 inode 개수를 8로 나누면 바이트 수가 되며,
// 이것을 블록 단위로 변환하는 계산법은 블록 비트맵 계산법과 동일하다.
uint32_t inode_bitmap_blocks =
    (fs->sb.inodes_count / 8 + SFUSE_BLOCK_SIZE - 1) / SFUSE_BLOCK_SIZE;

// 3. inode 테이블(inode table)의 크기를 블록 단위로 계산
//
// inode 테이블은 실제 inode 구조체들을 저장하는 영역이다.
// 각 inode는 'struct sfuse_inode'의 크기만큼 메모리를 차지하므로,
// 전체 inode 개수에 inode 크기를 곱하면 inode 테이블의 총 크기가 바이트
// 단위로 나온다.
//
// 이것을 다시 블록 크기(SFUSE_BLOCK_SIZE)로 나누어 몇 개의 블록이 필요한지
// 계산한다. 여기서도 블록이 정수로 딱 떨어지지 않을 수 있으므로, 올림
// 효과를 내도록 처리한다.
uint32_t inode_table_blocks =
    (fs->sb.inodes_count * sizeof(struct sfuse_inode) + SFUSE_BLOCK_SIZE -
     1) /
    SFUSE_BLOCK_SIZE;

// 슈퍼블록에서 관리하는 각 영역의 위치(블록 번호)를 설정한다.
// SFUSE 파일 시스템의 디스크 레이아웃은 일반적으로 다음과 같다:
//
//    파일 시스템 블록 구조:
// +-------------------------+
// | 블록 0 (슈퍼블록)        |
// +-------------------------+
// | 블록 1 (예비 블록)       |
// +-------------------------+
// | 블록 2 ~ N               |
// | (block bitmap)           |
// | [블록 사용 여부 표시]    |
// +-------------------------+
// | inode bitmap             |
// | [inode 사용 여부 표시]   |
// +-------------------------+
// | inode tables             |
// | [파일 메타데이터 저장]   |
// +-------------------------+
// | data block               |
// | [실제 사용자 데이터 저장]|
// +-------------------------+
//
// block 비트맵의 시작 위치는 슈퍼블록(블록 0)과 예비 블록(블록 1) 뒤에 위치
fs->sb.block_bitmap_start = 2;

// inode 비트맵은 block 비트맵 다음에 위치
fs->sb.inode_bitmap_start = fs->sb.block_bitmap_start + block_bitmap_blocks;

// inode 테이블은 inode 비트맵 다음에 위치
fs->sb.inode_table_start = fs->sb.inode_bitmap_start + inode_bitmap_blocks;

// 실제 사용자 데이터 블록은 inode 테이블 다음에 위치
fs->sb.data_block_start = fs->sb.inode_table_start + inode_table_blocks;

// 데이터 블록 시작 위치 이후의 블록만 실제 데이터 저장을 위해 사용
// 가능하므로, 전체 블록 수(total_blocks)에서 데이터 블록 시작 위치를 빼면
// 여유 블록 수를 구할 수 있다.
fs->sb.free_blocks = total_blocks - fs->sb.data_block_start;

// inode 중 루트 디렉터리를 제외한 나머지 inode들이 사용 가능
fs->sb.free_inodes = fs->sb.inodes_count - 1;

// 위에서 설정한 슈퍼블록 데이터를 디스크에 즉시 동기화하여 저장
```

```
if (sb_sync(backing_fd, &fs->sb) < 0)
  return -EIO; // 동기화 실패 시 입출력 에러 반환

// ---- 메모리에 블록 비트맵과 inode 비트맵을 관리할 공간을 확보 ----

// 비트맵 크기 계산:
// 비트맵은 비트 단위로 블록이나 inode의 사용 여부를 관리한다.
// 1 바이트(byte)는 8 비트(bit)를 가지므로, 블록과 inode의 개수를 각각 8로
// 나누면 비트맵이 필요한 메모리 크기가 바이트 단위로 나온다.
size_t bmap_bytes = fs->sb.blocks_count / 8;
size_t imap_bytes = fs->sb.inodes_count / 8;

// 위에서 계산한 크기만큼 메모리를 할당하고, 그 공간을 0으로 초기화한다.
// calloc(1, size)는 size 크기만큼 메모리를 할당한 후 모든 비트를 0으로
// 설정한다. 이렇게 하면 비트맵이 초기 상태에서는 모든 블록과 inode가 "사용
// 가능"(free)으로 표시된다.
fs->block_map = calloc(1, bmap_bytes);
fs->inode_map = calloc(1, imap_bytes);

// 메모리 할당 실패 시 메모리 부족 에러 반환
if (!fs->block_map || !fs->inode_map)
  return -ENOMEM;

// 비트맵의 초기 상태(모든 블록 및 inode가 사용 가능 상태)를 디스크에 동기화
bitmap_sync(backing_fd, fs->sb.block_bitmap_start, fs->block_map,
            bmap_bytes);
bitmap_sync(backing_fd, fs->sb.inode_bitmap_start, fs->inode_map,
            imap_bytes);

// ---- 루트 디렉터리 inode 초기화 ----

// 루트 inode 번호 설정 (1번으로 정의됨)
uint32_t root = SFUSE_ROOT_INO;

// inode 비트맵에서 루트 inode를 사용 중으로 표시하는 코드
// 비트맵(bitmap)은 각 inode의 사용 여부를 관리하는 데이터 구조로,
// 1비트가 1개의 inode의 상태를 나타낸다.
// 비트값이 0이면 해당 inode는 사용 가능(free), 1이면 사용
// 중(allocated)이다.

// inode 비트맵은 바이트(byte) 배열 형태로 관리된다.
// 배열의 각 요소(바이트)마다 8개의 inode 상태를 표현할 수 있다.
// 특정 inode가 속한 바이트의 위치는 inode 번호를 8로 나눈 값(root / 8)이며,
// 바이트 내의 정확한 비트 위치는 inode 번호를 8로 나눈 나머지(root % 8)가
// 된다.

// 예시:
// 루트 inode(root)가 5라면, inode_map[0] (바이트의 첫 번째 요소)의 5번째
// 비트를 설정한다.

// 이 코드의 동작 방식:
// 1. (1 « (root % 8)): 1을 root % 8 만큼 왼쪽으로 시프트(shift)하여 원하는
// 비트 위치로 설정한다.
// 2. inode_map[root / 8] |= (...): OR 연산으로 해당 비트를 설정하여 inode를
// '사용 중'으로 표시한다.
//
// 비트 연산을 사용하는 이유는 매우 빠르고 공간을 절약하기 때문이다.
fs->inode_map[root / 8] |= (1 « (root % 8));

// 위에서 inode 비트맵에 루트 inode를 할당했기 때문에,
// 슈퍼블록에서 관리하는 사용 가능한(free) inode의 개수도 하나 감소시켜야
// 한다.
fs->sb.free_inodes--;

// 루트 inode 구조체 초기화 (초기화 전에 메모리 전체를 0으로 설정)
struct sfuse_inode root_inode;
memset(&root_inode, 0, sizeof(root_inode));

// 루트 inode 메타데이터 설정 (디렉터리, 접근 권한 0755, 소유자 UID/GID
// 설정)
fs_init_inode(&fs->sb, root, S_IFDIR | 0755, fuse_get_context()->uid,
              fuse_get_context()->gid, &root_inode);

// ---- 루트 디렉터리의 데이터 블록 할당 및 초기화 ----

// 블록 할당 함수를 호출하여, 첫 번째 데이터 블록 인덱스를 할당 받음
int blk_index = alloc_block(&fs->sb, fs->block_map);
if (blk_index < 0)
  return -ENOSPC; // 사용 가능한 블록 공간 부족 시 에러 반환

// 실제 블록 번호는 데이터 블록의 시작 위치에서 blk_index만큼 더한 값
uint32_t phys_block = fs->sb.data_block_start + blk_index;

// inode가 관리하는 직접(direct) 블록 포인터 중 첫 번째 항목을 설정
root_inode.direct[0] = phys_block;
root_inode.size =
    SFUSE_BLOCK_SIZE; // 루트 디렉터리 크기를 블록 크기만큼 설정
```

```
    // 데이터 블록 내에 루트 디렉터리의 기본 엔트리(".” 및 "..")를 생성하기 위한
    // 초기화
    uint8_t block[SFUSE_BLOCK_SIZE] = {0}; // 블록 크기만큼 0으로 초기화
    struct sfuse_dirent *entries =
        (struct sfuse_dirent *)block; // 디렉터리 엔트리 구조체로 사용

    // ".”(현재 디렉터리)는 루트 inode를 가리킴
    entries[0].ino = root;
    strcpy(entries[0].name, ".”);

    // "..”(상위 디렉터리) 역시 루트 inode를 가리킴 (루트 디렉터리는 자신을
    // 가리킴)
    entries[1].ino = root;
    strcpy(entries[1].name, "..”);

    // 구성된 디렉터리 데이터를 디스크의 실제 블록에 기록
    if (write_block(backing_fd, phys_block, block) < 0)
      return -EIO; // 블록 기록 실패 시 입출력 에러 반환

    // 구성된 루트 inode를 디스크에 저장하여 동기화
    if (inode_sync(backing_fd, &fs->sb, root, &root_inode) < 0)
      return -EIO; // inode 동기화 실패 시 입출력 에러 반환

    // ---- 모든 메타데이터의 최종 상태를 디스크에 동기화 ----

    // 블록 비트맵과 inode 비트맵의 최종 상태를 디스크에 저장
    bitmap_sync(backing_fd, fs->sb.block_bitmap_start, fs->block_map,
                bmap_bytes);
    bitmap_sync(backing_fd, fs->sb.inode_bitmap_start, fs->inode_map,
                imap_bytes);

    // 슈퍼블록의 최종 상태를 디스크에 저장하여 파일 시스템 초기화 완료
    sb_sync(backing_fd, &fs->sb);

  } else {
    // 슈퍼블록이 이미 존재(정상 로드 성공) 시, 기존 메타데이터를 디스크에서
    // 로드

    size_t bmap_bytes = fs->sb.blocks_count / 8;
    size_t imap_bytes = fs->sb.inodes_count / 8;

    fs->block_map = calloc(1, bmap_bytes);
    fs->inode_map = calloc(1, imap_bytes);

    // 기존 비트맵 데이터를 디스크에서 메모리로 로드하여 파일 시스템 재구성
    bitmap_load(backing_fd, fs->sb.block_bitmap_start, fs->block_map,
                bmap_bytes);
    bitmap_load(backing_fd, fs->sb.inode_bitmap_start, fs->inode_map,
                imap_bytes);
  }

  // 초기화 과정이 모두 정상적으로 완료되었으므로 성공(0)을 반환
  return 0;
}
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



이 함수를 호출하는 함수들에 대한 그래프입니다.:

8.12.3.3 fs_resolve_path()

```
int fs_resolve_path (
            struct sfuse_fs * fs,
            const char * path,
            uint32_t * ino_out)
```

주어진 경로를 아이노드 번호로 변환한다.

매개변수

fs	파일 시스템의 전역 컨텍스트
path	변환할 경로 문자열
ino_out	결과 아이노드 번호를 저장할 포인터

반환값

성공 시 0, 실패 시 음수 오류 코드

주어진 경로를 아이노드 번호로 변환한다.
이 함수는 SFUSE 파일 시스템 내에서 주어진 경로를 순차적으로 분석하여 해당 경로에 대응하는 inode 번호를 찾고 결과를 반환한다.
예를 들어 경로 "/home/user/file.txt"가 주어지면, "/" → "home" → "user" → "file.txt" 순서로 디렉터리를 순차적으로 탐색하여 최종적으로 "file.txt"의 inode 번호를 찾는다.

매개변수

fs	파일 시스템 컨텍스트(struct sfuse_fs)의 포인터.
path	inode 번호를 찾을 경로 문자열.
ino_out	찾은 inode 번호를 저장할 포인터.

반환값

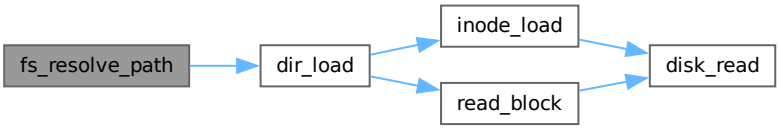
성공 시 0, 실패 시 음수의 에러 코드 반환.

fs.c 파일의 426 번째 라인에서 정의되었습니다.

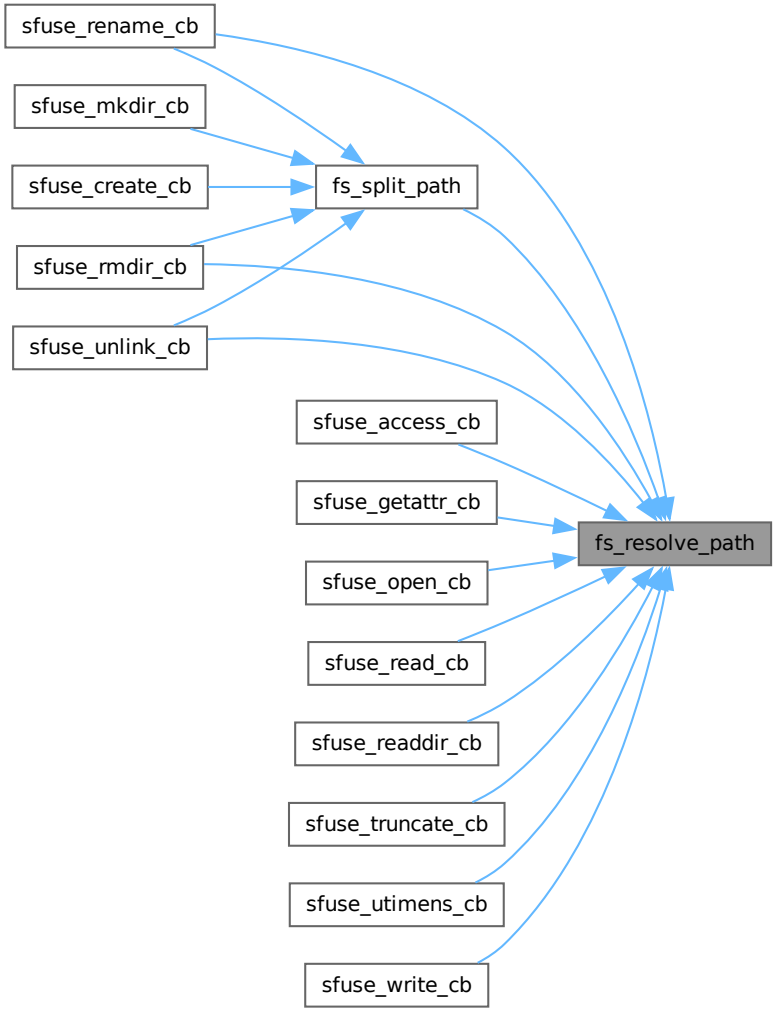
```
00426                                                   {
00427   // 시작 지점을 루트 디렉터리의 inode로 설정
00428   uint32_t cur = SFUSE_ROOT_INO;
00429
00430   // 예외 처리: 빈 문자열 또는 루트 디렉터리('/')가 들어오면 바로 루트 inode
00431   // 반환
00432   if (!path || path[0] == '\0' || strcmp(path, "/") == 0) {
00433     *ino_out = cur;
00434     return 0;
00435   }
00436
00437   // 경로 문자열을 수정 가능하도록 복제(메모리 복사)
00438   char *dup = strdup(path);
00439   if (!dup)
00440     return -ENOMEM; // 메모리 부족 시 에러 반환
00441
00442   // 경로 문자열을 "/" 기준으로 토큰(token)으로 나눈다.
00443   // 예시: "/home/user/file" → ["home", "user", "file"]
00444   char *token = strtok(dup, "/");
00445
00446   // 모든 토큰에 대해 순차적으로 디렉터리를 탐색
00447   while (token) {
00448     // 디렉터리 데이터를 메모리에 로드하기 위한 버퍼 크기를 계산한다.
00449     // 해당 버퍼는 디렉터리 엔트리 정보를 저장하는 데 사용된다.
00450     // SFUSE_NDIR_BLOCKS: 디렉터리 inode가 직접 참조할 수 있는 블록의 최대 개수.
00451     //                    직접 참조 블록 포인터 수 (12개)
00452     // SFUSE_BLOCK_SIZE:  각 블록의 크기(바이트 단위)()
00453     //
00454     // 따라서 디렉터리 데이터를 로드할 수 있는 전체 버퍼 크기는 다음과 같다:
00455     //
00456     //   buf_size = (직접 참조 블록 개수) × (각 블록 크기)
00457     //
00458     // 수식으로 표현하면:
00459     //   buf_size = SFUSE_NDIR_BLOCKS * SFUSE_BLOCK_SIZE;
00460     //
00461     // 예시:
00462     //   만약 SFUSE_NDIR_BLOCKS가 12이고 SFUSE_BLOCK_SIZE가 1024 바이트라면,
00463     //   buf_size = 12 × 1024 = 12288 바이트(12KB)가 된다.
00464     size_t buf_size = SFUSE_NDIR_BLOCKS * SFUSE_BLOCK_SIZE;
00465     void *buf = malloc(buf_size);
00466     if (!buf) {
00467       free(dup);
00468       return -ENOMEM; // 메모리 부족 시 에러 반환
00469     }
```

```

  // 현재 디렉터리의 모든 엔트리(entry)를 디스크에서 메모리로 로드한다.
  // dir_load는 cur(현재 inode 번호)에 해당하는 디렉터리 데이터를 버퍼에
  // 채운다.
  if (dir_load(fs->backing_fd, &fs->sb, cur, buf) < 0) {
    free(buf);
    free(dup);
    return -ENOENT; // 디렉터리를 읽을 수 없는 경우 존재하지 않음 에러 반환
  }

  // 버퍼의 데이터를 디렉터리 엔트리 배열로 캐스팅하여 접근 가능하도록 설정
  struct sfuse_dirent *ents = (struct sfuse_dirent *)buf;

  // 디렉터리 엔트리의 최대 개수를 계산
  size_t max_entries = buf_size / sizeof(*ents);

  // 다음 디렉터리 또는 파일의 inode 번호를 찾기 위한 임시 변수
  uint32_t next_ino = 0;

  // 현재 디렉터리 내의 엔트리 중에서 이름이 토큰과 일치하는 엔트리를 찾는다.
  for (size_t i = 0; i < max_entries; i++) {
    // ents[i].ino가 0이면 빈 엔트리이므로 무시
    // 토큰과 같은 이름의 엔트리를 찾으면 next_ino에 inode 번호 저장 후 반복문
    // 종료
    if (ents[i].ino != 0 && strcmp(ents[i].name, token) == 0) {
      next_ino = ents[i].ino;
      break;
    }
  }

  // 버퍼 사용이 끝났으므로 메모리 해제
  free(buf);

  // 해당하는 이름을 가진 엔트리를 찾지 못한 경우, 경로가 존재하지 않음을 의미
  if (next_ino == 0) {
    free(dup);
    return -ENOENT; // 경로 존재하지 않음 에러 반환
  }

  // 다음 토큰 탐색을 위해 현재 inode 번호를 방금 찾은 inode 번호로 업데이트
  cur = next_ino;

  // 다음 토큰으로 넘어감
  token = strtok(NULL, "/");
 }

 // 모든 토큰 탐색이 완료되면, 최종 inode 번호를 결과로 반환
 free(dup);      // 경로 문자열 복제본의 메모리 해제
 *ino_out = cur; // 최종 inode 번호 저장
 return 0;       // 성공적으로 경로를 해석함
}
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.12.3.4 fs_split_path()

```
char * fs_split_path (
            const char * fullpath,
            uint32_t * parent_ino)
```

전체 경로를 부모 디렉터리의 아이노드와 마지막 구성요소의 이름으로 분할한다.
예: "/foo/bar" → 부모 디렉터리 아이노드와 "bar"를 분리하여 반환한다. 반환된 이름은 동적으로 할당되므로 사용 후 free()로 메모리를 해제해야 한다.

매개변수

fullpath	분할할 전체 경로 문자열
parent_ino	부모 디렉터리 아이노드 번호를 저장할 포인터

반환값

마지막 구성요소의 이름을 담은 문자열 (동적 할당)

전체 경로를 부모 디렉터리의 아이노드와 마지막 구성요소의 이름으로 분할한다.
주어진 전체 경로에서 마지막 경로 구성 요소를 파일 또는 디렉터리 이름으로 분리하고, 나머지 앞부분 경로는 부모 디렉터리로 간주하여 해당하는 부모 디렉터리의 inode 번호를 찾는다.
예시: "/home/user/file.txt" → parent_ino: "/home/user"의 inode 번호 반환값: "file.txt"
"/file.txt" → parent_ino: 루트 디렉터리의 inode 번호 ("/") 반환값: "file.txt"

매개변수

fullpath	전체 파일 또는 디렉터리 경로 문자열
parent_ino	부모 디렉터리의 inode 번호가 저장될 포인터

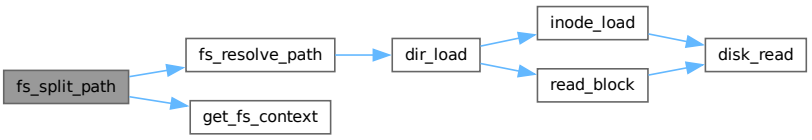
반환값

성공 시 파일명 또는 디렉터리명 문자열 포인터 반환 (반드시 호출한 곳에서 free 해야 함) 실패 시 NULL 반환

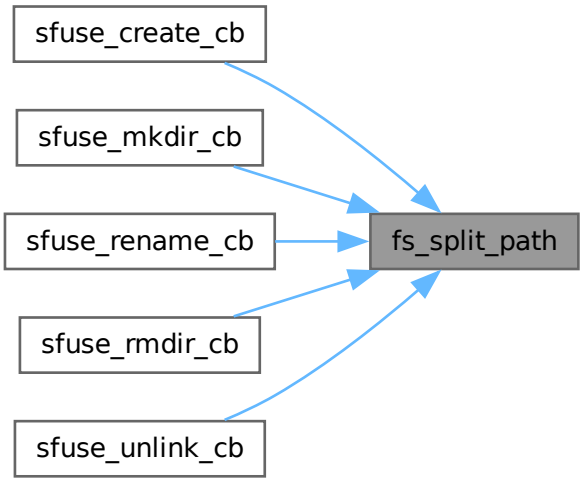
fs.c 파일의 543 번째 라인에서 정의되었습니다.

```
00543                                                   {
00544   char *dup = strdup(fullpath); // 경로를 수정하기 위해 복제본 생성
00545   if (!dup)
00546     return NULL; // 메모리 부족 시 실패 처리
00547
00548   char *slash = strrchr(dup, '/'); // 마지막 '/' 문자를 찾아 경로 분리 지점 설정
00549   char *name;
00550
00551   if (!slash) {
00552     // '/' 문자가 없을 경우 (예: "file.txt")
00553     *parent_ino = SFUSE_ROOT_INO; // 부모는 루트 디렉터리로 설정
00554     name = dup;                   // 전체 경로가 곧 파일명
00555   } else if (slash == dup) {
00556     // 경로가 루트에 바로 붙은 경우 (예: "/file.txt")
00557     *parent_ino = SFUSE_ROOT_INO; // 부모는 루트 디렉터리
00558     name = strdup(slash + 1);     // '/' 다음부터 이름 복사 ("file.txt")
00559     free(dup);                    // 최초 복제한 문자열은 더 이상 필요 없음
00560   } else {
00561     // 그 외 일반적인 경우 (예: "/home/user/file.txt")
00562     *slash =
00563         '\0'; // '/' 위치에서 문자열을 종료시켜 부모 경로만 남김 ("/home/user")
00564
00565     // 남은 부모 경로(dup)를 이용하여 부모 디렉터리의 inode 번호를 찾는다.
00566     if (fs_resolve_path(get_fs_context(), dup, parent_ino) < 0) {
00567       free(dup);   // 부모 경로 탐색 실패 시 메모리 해제
00568       return NULL; // 실패 반환
00569     }
00570
00571     // '/' 뒤의 문자열("file.txt")을 이름으로 복사하여 반환
00572     name = strdup(slash + 1);
00573     free(dup); // 부모 경로 탐색 완료 후 메모리 해제
00574   }
00575
00576   return name; // 호출한 곳에서 반드시 free(name) 수행 필요
00577 }
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.12.3.5 get_fs_context()

struct sfuse_fs * get_fs_context (void)

FUSE 콜백 함수 내에서 전역 파일 시스템 컨텍스트를 얻는다.

반환값

파일 시스템의 전역 컨텍스트(struct sfuse_fs*) 포인터

FUSE 콜백 함수 내에서 전역 파일 시스템 컨텍스트를 얻는다.
본 함수는 FUSE 라이브러리에서 제공하는 fuse_get_context()를 통해 얻은 private_data 필드를 SFUSE 파일 시스템 컨텍스트 구조체 포인터로 반환한다.
SFUSE 파일 시스템 컨텍스트(struct sfuse_fs)는 파일 시스템의 상태 정보와 메타데이터를 포함한다.
추가 설명:

- 컨텍스트(context): 파일 시스템 연산 처리 시 필요한 사용자 및 상태 정보를 저장하는 구조체로, 요청자(UID, GID, PID)의 정보와 같은 메타데이터를 포함한다.
- fuse_get_context(): 현재 처리 중인 파일 시스템 연산의 컨텍스트를 반환하는 FUSE 제공 함수.
- private_data: 파일 시스템 개발자가 정의한 데이터를 저장하는 포인터로, SFUSE에서는 파일 시스템의 슈퍼블록, 비트맵, 블록 정보 등 상태 데이터와 메타데이터 접근을 위해 사용한다.

반환값

struct sfuse_fs* 현재 파일 시스템의 컨텍스트 포인터

참고

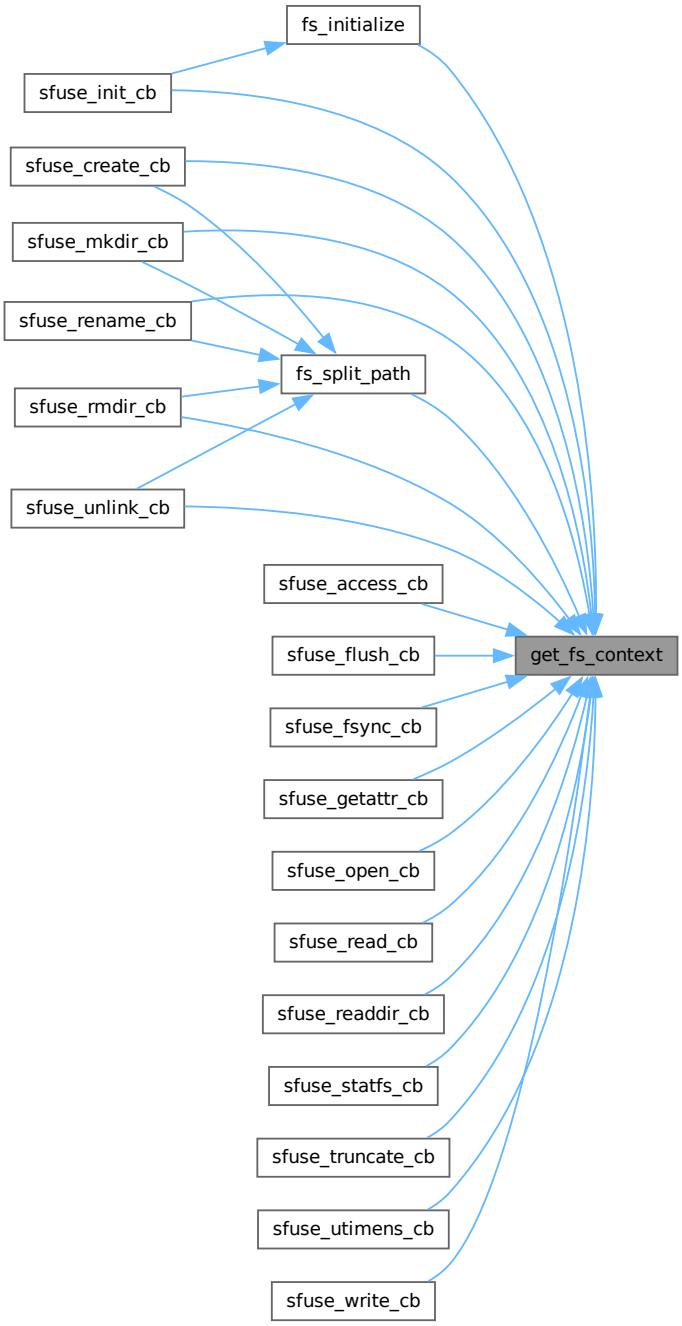
fuse_get_context()

struct sfuse_fs

fs.c 파일의 51 번째 라인에서 정의되었습니다.

```
00051                                   {
00052   return (struct sfuse_fs *)fuse_get_context()->private_data;
00053 }
```

이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.13 fs.h

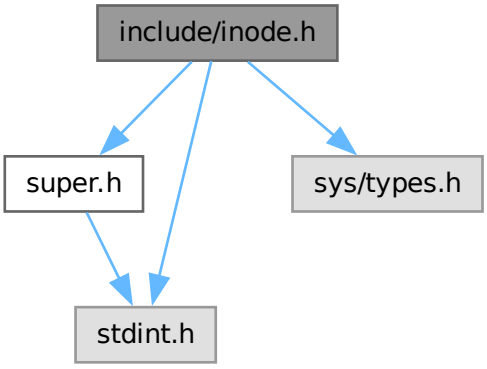
이 파일의 문서화 페이지로 가기

```
00001
00005
```

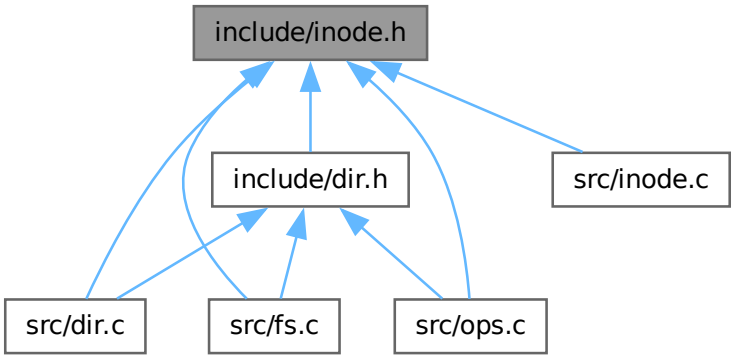
```
#ifndef SFUSE_FS_H
#define SFUSE_FS_H

#include "super.h"
#include <stdint.h>

#define SFUSE_ROOT_INO 1

struct sfuse_fs {
  int backing_fd;
  struct sfuse_super sb;
  uint8_t *block_map;
  uint8_t *inode_map;
};

struct sfuse_fs *get_fs_context(void);

int fs_initialize(int backing_fd);

void fs_destroy(void *private_data);

int fs_resolve_path(struct sfuse_fs *fs, const char *path, uint32_t *ino_out);

char *fs_split_path(const char *fullpath, uint32_t *parent_ino);

#endif /* SFUSE_FS_H */
```

8.14 include/inode.h 파일 참조

아이노드(inode) 구조체 정의 및 관리 함수 선언
#include "super.h"
#include <stdint.h>
#include <sys/types.h>
inode.h에 대한 include 의존 그래프



이 그래프는 이 파일을 직/간접적으로 include 하는 파일들을 보여줍니다.:



데이터 구조

- struct sfuse_inode
  파일의 메타데이터를 관리하는 아이노드 구조체

매크로

- #define SFUSE_NDIR_BLOCKS 12
  < struct sfuse_super, SFUSE_BLOCK_SIZE 정의

함수

- void fs_init_inode (const struct sfuse_super *sb, uint32_t ino, mode_t mode, uid_t uid, gid_t gid, struct sfuse_inode *inode)
  새로운 아이노드를 기본 값으로 초기화한다.
- int inode_load (int fd, const struct sfuse_super *sb, uint32_t ino, struct sfuse_inode *inode)
  디스크에서 지정된 아이노드를 읽어 메모리에 로드한다.
- int inode_sync (int fd, const struct sfuse_super *sb, uint32_t ino, const struct sfuse_inode *inode)
  메모리에 저장된 아이노드 정보를 디스크에 기록한다.
- int logical_to_physical (int fd, const struct sfuse_super *sb, const struct sfuse_inode *inode, uint32_t lbn, void *buf, uint32_t *pbn_out)
  논리 블록 번호(Logical Block Number)를 물리 블록 번호(Physical Block Number)로 변환한다.

8.14.1 상세한 설명

아이노드(inode) 구조체 정의 및 관리 함수 선언
이 파일은 SFUSE 파일 시스템에서 파일 메타데이터를 관리하는 아이노드의 구조체를 정의하고, 아이노드를 디스크로부터 읽거나 쓰는 기능과 논리 블록 번호를 물리 블록 번호로 변환하는 함수들을 제공한다.
inode.h 파일에서 정의되었습니다.

8.14.2 매크로 문서화

8.14.2.1 SFUSE_NDIR_BLOCKS

#define SFUSE_NDIR_BLOCKS 12

< struct sfuse_super, SFUSE_BLOCK_SIZE 정의
< mode_t, uid_t, gid_t 정의 Direct 블록 포인터 수 (12개)
inode.h 파일의 18 번째 라인에서 정의되었습니다.

8.14.3 함수 문서화

8.14.3.1 fs_init_inode()

```
void fs_init_inode (
            const struct sfuse_super * sb,
            uint32_t ino,
            mode_t mode,
            uid_t uid,
            gid_t gid,
            struct sfuse_inode * inode)
```

새로운 아이노드를 기본 값으로 초기화한다.
새 아이노드는 주어진 모드, 사용자 ID, 그룹 ID로 초기화되며, 나머지 필드는 기본값(예: 크기 0, 링크 수 1)으로 설정된다.

매개변수

sb	슈퍼블록 정보 포인터 (현재 사용되지 않음)
ino	아이노드 번호 (현재 사용되지 않음)
mode	새 파일의 타입 및 권한
uid	파일 소유자의 사용자 ID
gid	파일 소유자의 그룹 ID
inode	초기화할 아이노드 포인터

새로운 아이노드를 기본 값으로 초기화한다.
< 블록 읽기/쓰기 (read_block/write_block) 디스크 읽기/쓰기 함수 (disk_read/disk_write)
새로운 아이노드를 기본 값으로 초기화한다.
이 함수는 파일 시스템 내에서 새로운 아이노드를 생성할 때 호출된다. 아이노드는 파일 또는 디렉터리와 같은 객체의 메타데이터를 관리하는 핵심적인 구조체로, 이 함수는 아이노드를 사용 가능한 기본값으로 설정하여 초기 상태를 정의한다.
세부적으로는 다음의 작업을 수행한다:

- 아이노드 메모리 영역을 모두 0으로 초기화한다.
- 파일의 접근 권한(mode), 소유자(uid), 그룹(gid)을 설정한다.
- 파일 크기를 0으로 설정한다 (새로 생성된 파일이므로 초기 크기는 0이다).
- 접근(atime), 수정(mtime), 상태 변경(ctime) 시간을 현재 시각으로 설정한다.
- 모든 Direct 블록 포인터를 0으로 초기화하여, 아직 데이터를 가리키지 않도록 설정한다.
- Single indirect, Double indirect 블록 포인터 역시 0으로 설정하여, 추가적인 데이터 블록 참조가 없음을 나타낸다.
- 링크 수(links)를 초기화한다. 만약 디렉터리라면 기본적으로 2개(자기 자신 '.'과 상위 디렉터리 '..'), 파일이라면 1개로 설정한다.

매개변수

sb	슈퍼블록 정보 포인터 (미사용 파라미터)
ino	초기화할 아이노드 번호 (미사용 파라미터)
mode	새 아이노드의 파일 타입(파일/디렉터리 등)과 접근 권한
uid	새 아이노드를 소유할 사용자의 사용자 ID

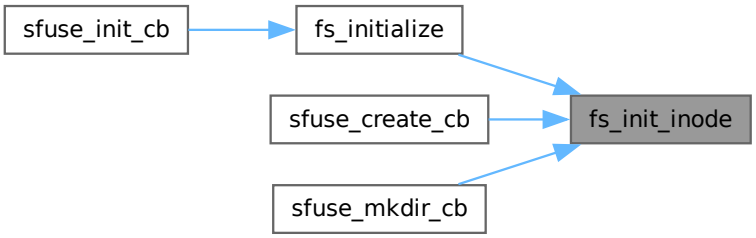
매개변수

gid	새 아이노드를 소유할 사용자의 그룹 ID
inode	실제로 초기화할 아이노드 구조체 포인터

inode.c 파일의 47 번째 라인에서 정의되었습니다.

```
00048                                                   {
00049   (void)sb;  // 미사용 파라미터로 인한 컴파일러 경고 방지
00050   (void)ino; // 미사용 파라미터로 인한 컴파일러 경고 방지
00051
00052   // inode 메모리를 0으로 초기화하여 깨끗한 상태로 시작
00053   memset(inode, 0, sizeof(*inode));
00054
00055   // 파일 타입과 접근 권한(mode)을 설정
00056   inode->mode = mode;
00057
00058   // 아이노드 소유자의 사용자(uid)와 그룹(gid) ID를 설정
00059   inode->uid = uid;
00060   inode->gid = gid;
00061
00062   // 새로 생성된 파일/디렉터리이므로 크기를 0으로 설정
00063   inode->size = 0;
00064
00065   // 접근(atime), 수정(mtime), 상태 변경(ctime) 시간을 현재 시각으로 초기화
00066   inode->atime = inode->mtime = inode->ctime = (uint32_t)time(NULL);
00067
00068   // Direct 블록 포인터를 모두 0으로 초기화하여 데이터 블록을 아직 가리키지 않음
00069   for (int i = 0; i < SFUSE_NDIR_BLOCKS; i++)
00070     inode->direct[i] = 0;
00071
00072   // Single indirect, Double indirect 블록 포인터도 데이터가 없음을 나타내기
00073   // 위해 0 설정
00074   inode->indirect = 0;
00075   inode->double_indirect = 0;
00076
00077   // 디렉터리면 기본 링크 수를 2로, 파일이면 1로 설정
00078   inode->links = (S_ISDIR(mode) ? 2 : 1);
00079 }
```

이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.14.3.2 inode_load()

```
int inode_load (
                int fd,
                const struct sfuse_super * sb,
                uint32_t ino,
                struct sfuse_inode * inode)
```

디스크에서 지정된 아이노드를 읽어 메모리에 로드한다.

매개변수

fd	디바이스 파일 디스크립터

매개변수

sb	슈퍼블록 정보 포인터
ino	읽어올 아이노드 번호
inode	읽은 아이노드를 저장할 버퍼 포인터

반환값

성공 시 0, 실패 시 음수의 오류 코드 반환

디스크에서 지정된 아이노드를 읽어 메모리에 로드한다.
파일 시스템에서 파일과 디렉터리의 메타데이터(크기, 권한, 소유자, 데이터 블록 위치 등)는 아이노드(inode)에 저장된다. 이 함수는 디스크에 저장된 아이노드 데이터를 읽어 메모리의 inode 구조체에 로드한다.
아이노드를 로드하는 과정은 다음과 같다:

1. 아이노드 번호(ino)의 유효성을 검사한다.
   - ino 번호는 0이 아니어야 하고, 슈퍼블록의 inodes_count보다 작아야 한다.
2. 디스크 내의 아이노드 위치(offset)를 계산한다.
   - 슈퍼블록의 inode_table_start는 아이노드 테이블의 시작 위치를 나타내는 블록 번호이다.
   - 아이노드의 크기(sizeof(*inode))에 아이노드 번호를 곱해 정확한 위치를 계산한다.
3. 계산된 위치에서 아이노드 데이터를 읽어 inode 버퍼에 저장한다.
   - 읽은 데이터의 크기가 아이노드 구조체 크기와 일치하지 않으면 입출력 오류(EIO)를 반환한다.

매개변수

fd	디바이스 파일 디스크립터
sb	슈퍼블록 정보 포인터 (아이노드 테이블의 위치와 총 개수 정보 포함)
ino	읽을 아이노드 번호 (0번 아이노드는 무효로 간주됨)
inode	디스크에서 읽은 아이노드를 저장할 메모리 버퍼 포인터

반환값

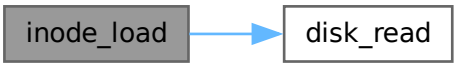
성공 시 0 반환, 실패 시 음수의 오류 코드 반환 -EINVAL : 유효하지 않은 아이노드 번호(ino가 0이거나 유효 범위를 초과) -EIO : 아이노드 데이터를 읽는 중 입출력 오류 발생

inode.c 파일의 113 번째 라인에서 정의되었습니다.

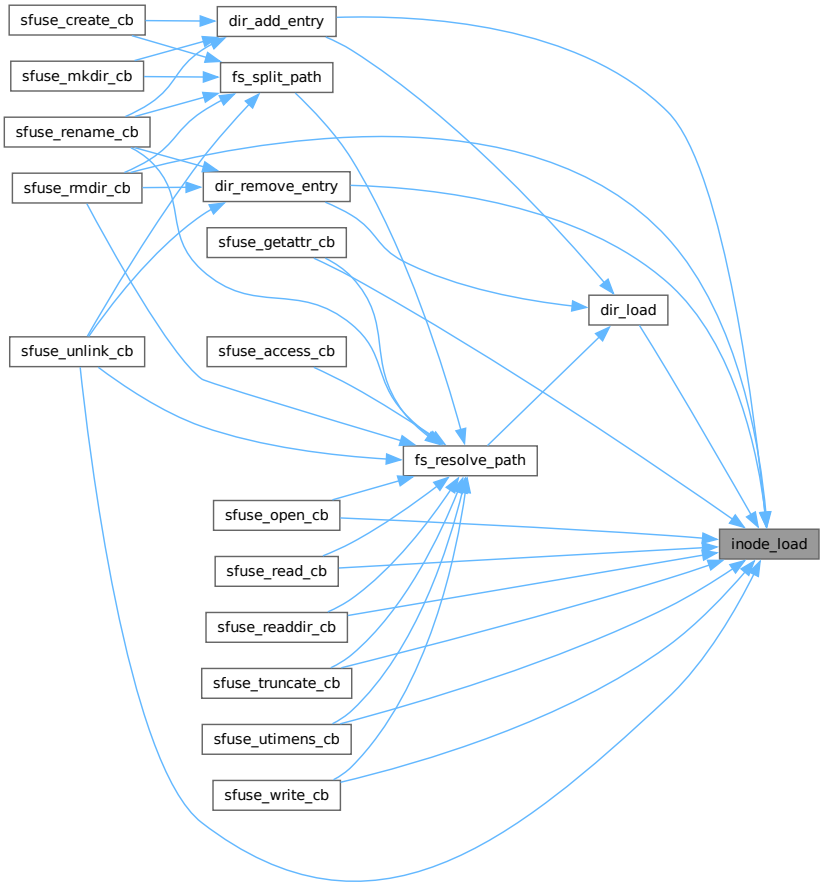
```
00114                                   {
00115   /* [1단계] 아이노드 번호(ino) 유효성 검사 */
00116   if (ino == 0 || ino >= sb->inodes_count)
00117     return -EINVAL; // ino가 유효하지 않음 (0이거나 최대 범위 초과)
00118
00119   /* [2단계] 디스크에서 아이노드가 저장된 위치(offset) 계산 */
00120   off_t off = ((off_t)sb->inode_table_start * SFUSE_BLOCK_SIZE) +
00121               (ino * sizeof(*inode));
00122   /*
00123    * inode_table_start 블록 번호를 실제 바이트 단위로 변환하기 위해 블록
00124    * 크기(SFUSE_BLOCK_SIZE)를 곱함. 거기에 inode 번호(ino)에 inode
00125    * 크기(sizeof(*inode))를 곱하여 정확한 위치 계산.
00126    */
00127
00128   /* [3단계] 디스크에서 아이노드 데이터를 읽어 메모리에 로드 */
00129   ssize_t ret = disk_read(fd, inode, sizeof(*inode), off);
00130   if (ret < 0)
00131     return (int)ret; // 디스크 읽기 오류 발생 시 해당 오류 코드 반환
00132
00133   // 읽은 데이터 크기가 아이노드 크기와 불일치 시 입출력 오류 반환
00134   if ((size_t)ret != sizeof(*inode))
00135     return -EIO;
00136
```

```
00137    return 0; // 성공적으로 아이노드 로드 완료
00138 }
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.14.3.3 inode_sync()

```
int inode_sync (
            int fd,
            const struct sfuse_super * sb,
            uint32_t ino,
            const struct sfuse_inode * inode)
```

메모리에 저장된 아이노드 정보를 디스크에 기록한다.

매개변수

fd	디바이스 파일 디스크립터
sb	슈퍼블록 정보 포인터
ino	기록할 아이노드 번호
inode	디스크에 저장할 아이노드 정보 포인터

반환값

성공 시 0, 실패 시 음수의 오류 코드 반환

메모리에 저장된 아이노드 정보를 디스크에 기록한다.
파일 시스템은 아이노드를 메모리 내에서 수정한 후, 변경된 내용을 디스크에 영구적으로 저장해야 한다.
이 함수는 메모리 상의 아이노드를 디스크의 올바른 위치에 정확하게 기록하여 파일의 메타데이터가 유지되도록 한다.
아이노드를 디스크에 기록하는 과정은 다음과 같다:

1. 아이노드 번호(ino)의 유효성을 검사한다.
   - ino 번호는 반드시 0보다 크고 슈퍼블록의 총 아이노드 개수(inodes_count)보다 작아야 한다.
2. 기록할 아이노드 데이터의 디스크 내 위치(offset)를 계산한다.
   - 아이노드 테이블의 시작 위치(inode_table_start)를 블록 단위에서 바이트 단위로 변환하고, 여기에 아이노드 번호(ino)와 아이노드의 크기(sizeof(*inode))를 곱하여 정확한 위치를 얻는다.
3. 메모리 상의 아이노드 데이터를 계산된 위치에 디스크로 기록한다.
   - 기록 후, 실제로 기록된 바이트 수가 아이노드의 크기와 정확히 일치하는지 확인한다.
   - 기록된 바이트 수가 일치하지 않거나 오류가 발생하면 적절한 오류 코드를 반환한다.

매개변수

fd	디바이스 파일 디스크립터 (디스크 접근을 위한 식별자)
sb	슈퍼블록 정보 (아이노드 테이블 위치 정보 포함)
ino	디스크에 기록할 아이노드 번호
inode	메모리에 있는, 디스크에 기록될 아이노드 데이터 포인터

반환값

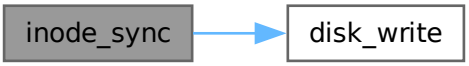
성공 시 0 반환, 실패 시 음수의 오류 코드 반환 -EINVAL : 유효하지 않은 아이노드 번호(0이거나 범위 초과) -EIO : 디스크 기록 과정에서 입출력 오류가 발생

inode.c 파일의 173 번째 라인에서 정의되었습니다.

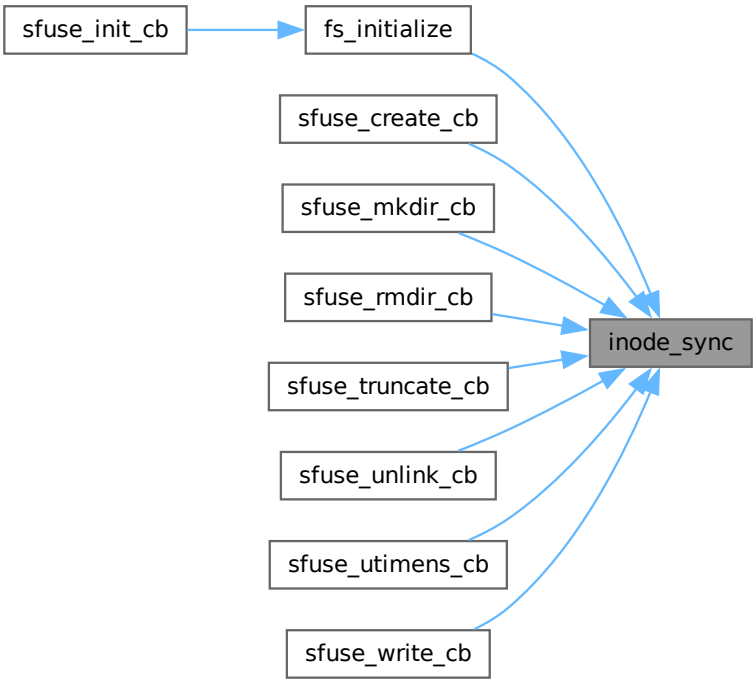
```
00174                                   {
00175   /* [1단계] 아이노드 번호(ino)의 유효성 검사 */
00176   if (ino == 0 || ino >= sb->inodes_count)
00177     return -EINVAL; // 아이노드 번호가 유효하지 않을 경우
00178
00179   /* [2단계] 아이노드 데이터를 기록할 디스크의 정확한 위치(offset) 계산 */
00180   off_t off = ((off_t)sb->inode_table_start * SFUSE_BLOCK_SIZE) +
00181               (ino * sizeof(*inode));
00182   /*
00183    * inode_table_start는 블록 번호를 나타내므로 바이트 단위로 변환하기 위해
00184    * SFUSE_BLOCK_SIZE를 곱한다.
00185    * 아이노드 번호(ino)에 아이노드 크기를 곱해 정확한 바이트 오프셋을 결정한다.
00186    */
00187
00188   /* [3단계] 메모리의 아이노드를 디스크의 해당 위치에 기록 */
00189   ssize_t ret = disk_write(fd, inode, sizeof(*inode), off);
00190   if (ret < 0)
00191     return (int)ret; // 디스크 기록 오류가 발생했을 때 오류 코드 반환
00192
```

```
00193   if ((size_t)ret != sizeof(*inode))
00194     return -EIO; // 기록된 데이터의 크기가 아이노드 크기와 다르면 입출력 오류
00195                  // 반환
00196
00197   return 0; // 성공적으로 아이노드를 디스크에 기록함
00198 }
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.14.3.4 logical_to_physical()

```
int logical_to_physical (
                int fd,
                const struct sfuse_super * sb,
                const struct sfuse_inode * inode,
                uint32_t lbn,
                void * buf,
                uint32_t * pbn_out)
```

논리 블록 번호(Logical Block Number)를 물리 블록 번호(Physical Block Number)로 변환한다.

파일 내의 논리적 블록 위치를 실제 디스크 상의 물리적 블록 위치로 변환하여 데이터를 접근할 수 있도록 한다.

매개변수

fd	디바이스 파일 디스크립터
sb	슈퍼블록 정보 포인터
inode	대상 아이노드 포인터
lbn	변환할 논리 블록 번호
buf	임시로 사용할 블록 크기의 버퍼 (크기는 SFUSE_BLOCK_SIZE)
pbn_out	결과로 변환된 물리 블록 번호를 저장할 포인터

반환값

성공 시 0, 실패 시 음수의 오류 코드 반환

논리 블록 번호(Logical Block Number)를 물리 블록 번호(Physical Block Number)로 변환한다.
파일은 여러 블록에 걸쳐 데이터를 저장하며, 파일 내에서의 블록 위치는 논리적(LBN)으로 표현된다. 그러나 실제 디스크는 논리 블록을 직접적으로 알지 못하므로, 물리적인 블록 번호(PBN)로 변환해 접근해야 한다. 이 함수는 아이노드에 저장된 블록 주소를 참조하여 LBN을 PBN으로 변환해주는 역할을 수행한다.
LBN에서 PBN 변환 방법:

- [Direct Block 처리]
  - LBN이 SFUSE_NDIR_BLOCKS(보통 12개)보다 작으면, direct 배열에서 직접 찾아 반환한다.
- [Single Indirect Block 처리]
  - Direct 블록으로 표현할 수 있는 범위를 넘어가면, Single indirect 블록에서 찾는다.
  - Single indirect 블록이 가리키는 블록 하나에는 여러 개의 블록 주소를 담을 수 있다.
- [Double Indirect Block 처리]
  - Single indirect 블록 범위도 초과하면, Double indirect 블록을 참조해야 한다.
  - Double indirect 블록은 두 단계의 포인터를 거쳐 주소를 참조한다.

매개변수

fd	디바이스 파일 디스크립터 (디스크 접근용)
sb	슈퍼블록 정보 (현재 사용하지 않음)
inode	대상 파일의 아이노드 포인터
lbn	변환하려는 논리 블록 번호
buf	블록 데이터를 읽기 위한 임시 버퍼 (크기: SFUSE_BLOCK_SIZE)
pbn_out	변환된 물리 블록 번호를 저장할 출력 포인터

반환값

성공 시 0, 실패 시 음수의 오류 코드 반환 -ENOENT : 요청한 블록이 할당되지 않아 존재하지 않음 -EIO : 블록 데이터를 디스크에서 읽을 때 입출력 오류 발생

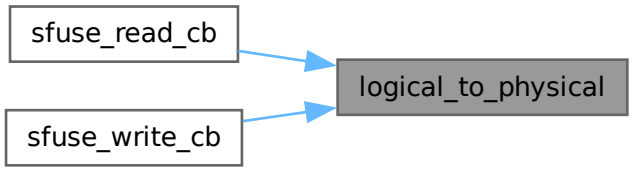
inode.c 파일의 235 번째 라인에서 정의되었습니다.

```
{

  (void)sb; // 미사용 파라미터로 인한 컴파일러 경고 방지

```

```
/* --- [1단계: Direct 블록 확인] --- */
if (lbn < SFUSE_NDIR_BLOCKS) {
  // 요청한 논리 블록 번호가 Direct 블록의 범위 내에 있음
  *pbn_out = inode->direct[lbn]; // 직접적인 배열 참조로 즉시 얻을 수 있음
  return 0;
}

/* --- [2단계: Single Indirect 블록 확인] --- */
lbn -= SFUSE_NDIR_BLOCKS; // Direct 블록의 범위를 초과했으므로, Single
                          // indirect 범위로 보정

uint32_t entries = SFUSE_BLOCK_SIZE / sizeof(uint32_t);
// 하나의 블록에 저장 가능한 블록 주소의 개수를 계산
// (예: 4096 바이트 블록 / 4 바이트 주소 = 1024개의 블록 주소 저장 가능)

if (lbn < entries) {
  // Single indirect 블록에서 찾을 수 있는 범위 내에 있음
  if (inode->indirect == 0)
    return -ENOENT; // Single indirect 블록 자체가 아직 할당되지 않은 상태

  // indirect 블록 번호가 존재하면, 해당 블록을 디스크에서 읽는다
  if (read_block(fd, inode->indirect, buf) < 0)
    return -EIO; // indirect 블록을 읽는 도중 오류가 발생하면 입출력 오류 반환

  // indirect 블록에서 블록 주소 배열을 가져옴
  uint32_t *ptrs = (uint32_t *)buf;
  *pbn_out = ptrs[lbn]; // indirect 블록의 주소 배열에서 요청한 논리 블록의
                        // 물리 블록 번호를 얻음
  return 0;
}

/* --- [3단계: Double Indirect 블록 확인] --- */
lbn -= entries; // Single indirect 블록의 범위를 초과하였으므로, Double
                // indirect 블록을 처리하기 위해 보정

uint32_t idx1 = lbn / entries; // Double indirect 첫 번째 레벨의 인덱스 계산
uint32_t idx2 = lbn % entries; // Double indirect 두 번째 레벨의 인덱스 계산

if (inode->double_indirect == 0)
  return -ENOENT; // Double indirect 블록 자체가 할당되지 않은 상태

// double indirect의 첫 번째 레벨 블록을 읽는다.
if (read_block(fd, inode->double_indirect, buf) < 0)
  return -EIO; // 첫 번째 레벨 블록을 읽을 때 오류 발생 시 입출력 오류 반환

uint32_t *ptrs1 = (uint32_t *)buf;
uint32_t block2 = ptrs1[idx1]; // 두 번째 레벨 블록 번호를 얻음

if (block2 == 0)
  return -ENOENT; // 두 번째 레벨 블록이 할당되지 않은 경우

// 두 번째 레벨 블록을 디스크에서 읽는다.
if (read_block(fd, block2, buf) < 0)
  return -EIO; // 두 번째 레벨 블록을 읽을 때 오류 발생 시 입출력 오류 반환

uint32_t *ptrs2 = (uint32_t *)buf;
*pbn_out = ptrs2[idx2]; // 최종적으로 변환된 물리 블록 번호를 얻음

return 0; // 성공적으로 물리 블록 번호 변환 완료
}
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:

이 함수를 호출하는 함수들에 대한 그래프입니다.:



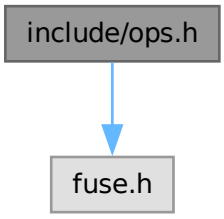
8.15 inode.h

이 파일의 문서화 페이지로 가기

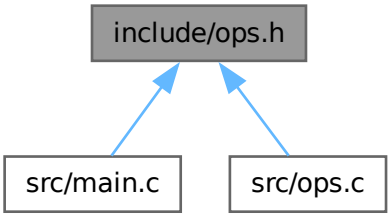
```
#ifndef SFUSE_INODE_H
#define SFUSE_INODE_H

#include "super.h"
#include <stdint.h>
#include <sys/types.h>

#define SFUSE_NDIR_BLOCKS 12

struct sfuse_inode {
  mode_t mode;
  uid_t uid;
  gid_t gid;
  uint32_t size;
  uint32_t atime;
  uint32_t mtime;
  uint32_t ctime;
  uint32_t direct[SFUSE_NDIR_BLOCKS];
  uint32_t indirect;
  uint32_t double_indirect;
  uint32_t links;
};

void fs_init_inode(const struct sfuse_super *sb, uint32_t ino, mode_t mode,
                   uid_t uid, gid_t gid, struct sfuse_inode *inode);

int inode_load(int fd, const struct sfuse_super *sb, uint32_t ino,
               struct sfuse_inode *inode);

int inode_sync(int fd, const struct sfuse_super *sb, uint32_t ino,
               const struct sfuse_inode *inode);

int logical_to_physical(int fd, const struct sfuse_super *sb,
                        const struct sfuse_inode *inode, uint32_t lbn,
                        void *buf, uint32_t *pbn_out);

#endif // SFUSE_INODE_H
```

8.16 include/ops.h 파일 참조

#include <fuse.h>
ops.h에 대한 include 의존 그래프



이 그래프는 이 파일을 직/간접적으로 include 하는 파일들을 보여줍니다.:



변수

- const struct fuse_operations sfuse_ops

8.16.1 변수 문서화

8.16.1.1 sfuse_ops

const struct fuse_operations sfuse_ops [extern]
ops.c 파일의 562 번째 라인에서 정의되었습니다.

```
00562                                   {
00563     .init = sfuse_init_cb,
00564     .destroy = sfuse_destroy_cb,
00565     .getattr = sfuse_getattr_cb,
00566     .access = sfuse_access_cb,
00567     .readdir = sfuse_readdir_cb,
00568     .open = sfuse_open_cb,
00569     .read = sfuse_read_cb,
00570     .write = sfuse_write_cb,
00571     .create = sfuse_create_cb,
00572     .mkdir = sfuse_mkdir_cb,
00573     .unlink = sfuse_unlink_cb,
00574     .rmdir = sfuse_rmdir_cb,
00575     .rename = sfuse_rename_cb,
00576     .truncate = sfuse_truncate_cb,
```

```
    .utimens = sfuse_utimens_cb,
    .flush = sfuse_flush_cb,
    .fsync = sfuse_fsync_cb,
    .statfs = sfuse_statfs_cb,
    .getxattr = sfuse_getxattr_cb,
    .listxattr = sfuse_listxattr_cb,
};
```

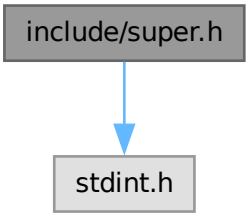
8.17 ops.h

이 파일의 문서화 페이지로 가기

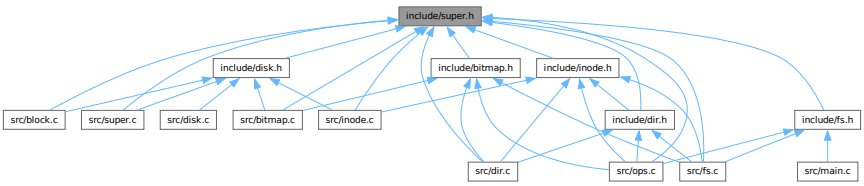
```
#ifndef SFUSE_OPS_H
#define SFUSE_OPS_H

#include <fuse.h>

#ifdef __cplusplus
extern "C" {
#endif

extern const struct fuse_operations sfuse_ops;

#ifdef __cplusplus
}
#endif

#endif // SFUSE_OPS_H
```

8.18 include/super.h 파일 참조

SFUSE 파일 시스템의 슈퍼블록 관련 구조 및 함수 정의
#include <stdint.h>
super.h에 대한 include 의존 그래프



이 그래프는 이 파일을 직/간접적으로 include 하는 파일들을 보여줍니다.:



데이터 구조

- struct sfuse_super

파일 시스템 메타데이터를 저장하는 슈퍼블록 구조체

매크로

- #define SFUSE_MAGIC 0xEF53
  슈퍼블록의 매직 넘버
- #define SFUSE_BLOCK_SIZE 4096
  블록 크기 (바이트)
- #define SFUSE_SUPERBLOCK_OFFSET 1024
  슈퍼블록이 저장되는 디스크 내의 오프셋 (바이트 단위)
- #define SFUSE_INODES_COUNT 1024
  파일 시스템의 총 아이노드 개수
- #define SFUSE_BLOCKS_COUNT 8192
  파일 시스템의 총 블록 개수
- #define SFUSE_INODE_BITMAP_BLOCK 2
  아이노드 비트맵이 시작되는 블록 번호
- #define SFUSE_BLOCK_BITMAP_BLOCK 3
  블록 비트맵이 시작되는 블록 번호
- #define SFUSE_INODE_TABLE_BLOCK 4
  아이노드 테이블이 시작되는 블록 번호
- #define SFUSE_DATA_BLOCK_START 100
  실제 데이터 블록이 시작되는 블록 번호

함수

- int sb_load (int fd, struct sfuse_super *sb)
  디스크에서 슈퍼블록을 읽어오는 함수
- int sb_sync (int fd, const struct sfuse_super *sb)
  슈퍼블록을 디스크에 동기화(쓰기)하는 함수
- void sb_format (struct sfuse_super *sb)
  슈퍼블록 구조체를 기본 값으로 초기화하는 함수

8.18.1 상세한 설명

SFUSE 파일 시스템의 슈퍼블록 관련 구조 및 함수 정의
이 헤더 파일은 SFUSE 파일 시스템에서 슈퍼블록을 관리하기 위한 구조체 및 함수 원형을 정의한다.
super.h 파일에서 정의되었습니다.

8.18.2 매크로 문서화

8.18.2.1 SFUSE_BLOCK_BITMAP_BLOCK

#define SFUSE_BLOCK_BITMAP_BLOCK 3
블록 비트맵이 시작되는 블록 번호
super.h 파일의 33 번째 라인에서 정의되었습니다.

8.18.2.2 SFUSE_BLOCK_SIZE

#define SFUSE_BLOCK_SIZE 4096
블록 크기 (바이트)
super.h 파일의 18 번째 라인에서 정의되었습니다.

8.18.2.3 SFUSE_BLOCKS_COUNT

#define SFUSE_BLOCKS_COUNT 8192
파일 시스템의 총 블록 개수
super.h 파일의 27 번째 라인에서 정의되었습니다.

8.18.2.4 SFUSE_DATA_BLOCK_START

#define SFUSE_DATA_BLOCK_START 100
실제 데이터 블록이 시작되는 블록 번호
super.h 파일의 39 번째 라인에서 정의되었습니다.

8.18.2.5 SFUSE_INODE_BITMAP_BLOCK

#define SFUSE_INODE_BITMAP_BLOCK 2
아이노드 비트맵이 시작되는 블록 번호
super.h 파일의 30 번째 라인에서 정의되었습니다.

8.18.2.6 SFUSE_INODE_TABLE_BLOCK

#define SFUSE_INODE_TABLE_BLOCK 4
아이노드 테이블이 시작되는 블록 번호
super.h 파일의 36 번째 라인에서 정의되었습니다.

8.18.2.7 SFUSE_INODES_COUNT

#define SFUSE_INODES_COUNT 1024
파일 시스템의 총 아이노드 개수
super.h 파일의 24 번째 라인에서 정의되었습니다.

8.18.2.8 SFUSE_MAGIC

#define SFUSE_MAGIC 0xEF53
슈퍼블록의 매직 넘버
super.h 파일의 15 번째 라인에서 정의되었습니다.

8.18.2.9 SFUSE_SUPERBLOCK_OFFSET

#define SFUSE_SUPERBLOCK_OFFSET 1024
슈퍼블록이 저장되는 디스크 내의 오프셋 (바이트 단위)
super.h 파일의 21 번째 라인에서 정의되었습니다.

8.18.3 함수 문서화

8.18.3.1 sb_format()

void sb_format (
struct sfuse_super * sb)
슈퍼블록 구조체를 기본 값으로 초기화하는 함수
이 함수는 새로운 파일 시스템을 만들 때 사용된다.

매개변수

sb	초기화할 슈퍼블록 구조체의 포인터

슈퍼블록 구조체를 기본 값으로 초기화하는 함수
이 함수는 파일 시스템을 처음 생성할 때 호출되며, 슈퍼블록 구조체에 초기 설정값을 채워서 파일 시스템의 메타데이터를 초기화한다.

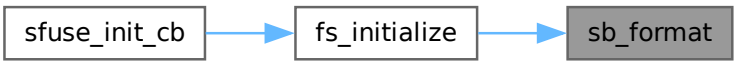
매개변수

sb	초기화할 슈퍼블록 구조체의 포인터

super.c 파일의 89 번째 라인에서 정의되었습니다.

```
00089                                   {
00090   memset(sb, 0, sizeof(*sb));
00091   sb->magic = SFUSE_MAGIC;
00092   sb->inodes_count = SFUSE_INODES_COUNT;
00093   sb->blocks_count = SFUSE_BLOCKS_COUNT;
00094   sb->free_inodes = sb->inodes_count - 1;
00095   sb->free_blocks = sb->blocks_count - SFUSE_DATA_BLOCK_START;
00096   sb->inode_bitmap_start = SFUSE_INODE_BITMAP_BLOCK;
00097   sb->block_bitmap_start = SFUSE_BLOCK_BITMAP_BLOCK;
00098   sb->inode_table_start = SFUSE_INODE_TABLE_BLOCK;
00099   sb->data_block_start = SFUSE_DATA_BLOCK_START;
00100 }
```

이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.18.3.2 sb_load()

```
int sb_load (
            int fd,
            struct sfuse_super * sb)
```

디스크에서 슈퍼블록을 읽어오는 함수
이 함수는 디스크의 특정 위치에서 슈퍼블록을 읽어 유효성을 검사한다.

매개변수

fd	디바이스 파일 디스크립터
sb	읽어온 슈퍼블록을 저장할 구조체의 포인터

반환값

성공 시 0, 실패 시 음수 오류 코드 반환

디스크의 정해진 위치에서 슈퍼블록을 읽어 메모리로 로드하고, 읽은 데이터가 정상적인지 유효성을 검사한다. 유효성 검사는 읽은 데이터 크기와 슈퍼블록 매직 넘버를 이용하여 수행된다.

매개변수

fd	디바이스 파일 디스크립터 (열린 블록 디바이스 파일)
sb	읽어온 슈퍼블록을 저장할 구조체 포인터

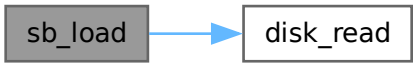
반환값

성공 시 0을 반환, 실패 시 음수 오류 코드 반환 -EIO: 디스크에서 읽은 바이트 수가 요청한 크기와 일치하지 않음 -EINVAL: 슈퍼블록의 매직 넘버가 예상 값과 다름 (슈퍼블록이 손상되었거나 올바르지 않은 디바이스) 기타 음수 값: disk_read 함수 자체가 반환한 오류 코드

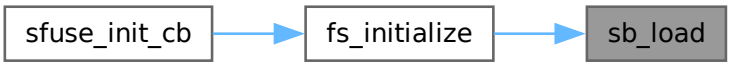
super.c 파일의 29 번째 라인에서 정의되었습니다.

```
00029                                   {
00030   ssize_t ret;
00031
00032   // 슈퍼블록을 디스크(SFUSE_SUPERBLOCK_OFFSET)에서 읽어 메모리(sb)에 저장한다.
00033   ret = disk_read(fd, sb, sizeof(*sb), SFUSE_SUPERBLOCK_OFFSET);
00034
00035   // disk_read가 실패하면 반환된 음수 오류 값을 그대로 반환한다.
00036   if (ret < 0)
00037     return ret;
00038
00039   // 요청한 슈퍼블록 크기만큼 정확히 읽었는지 확인한다.
00040   if ((size_t)ret != sizeof(*sb))
00041     return -EIO; // 읽은 데이터 크기가 슈퍼블록 크기와 다름 (입출력 오류)
00042
00043   // 슈퍼블록의 매직 넘버를 검사하여 슈퍼블록 유효성 검사를 수행한다.
00044   if (sb->magic != SFUSE_MAGIC)
00045     return -EINVAL; // 잘못된 매직 넘버
00046
00047   // 성공적으로 슈퍼블록을 읽고 유효성을 확인했으므로 0 반환.
00048   return 0;
00049 }
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.18.3.3 sb_sync()

```
int sb_sync (
            int fd,
            const struct sfuse_super * sb)
```

슈퍼블록을 디스크에 동기화(쓰기)하는 함수

- 이 함수는 메모리에 있는 슈퍼블록의 내용을 디스크에 기록한다.

매개변수

fd	디바이스 파일 디스크립터

sb	디스크에 기록할 슈퍼블록 구조체의 포인터

반환값

성공 시 0, 실패 시 음수 오류 코드 반환

슈퍼블록을 디스크에 동기화(쓰기)하는 함수
메모리 상의 슈퍼블록 구조체의 내용을 디스크의 지정된 위치에 기록하여, 파일 시스템의 메타데이터를 영구적으로 저장한다.

매개변수

fd	디바이스 파일 디스크립터
sb	디스크에 기록할 슈퍼블록 구조체의 포인터

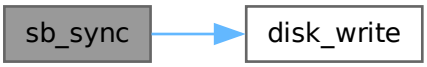
반환값

성공 시 0, 실패 시 음수 오류 코드 반환 -EIO: 기록된 바이트 수가 슈퍼블록의 크기와 일치하지 않음 기타 음수 값: disk_write 함수 자체에서 반환한 오류 코드

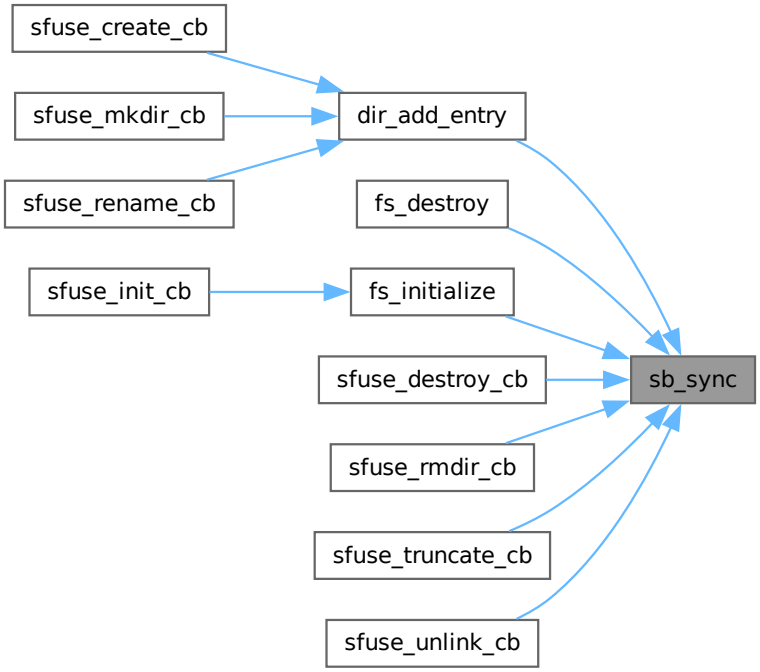
super.c 파일의 63 번째 라인에서 정의되었습니다.

```
00063                                   {
00064   ssize_t ret;
00065
00066   // 슈퍼블록 내용을 디스크(SFUSE_SUPERBLOCK_OFFSET)에 기록
00067   ret = disk_write(fd, sb, sizeof(*sb), SFUSE_SUPERBLOCK_OFFSET);
00068
00069   // 쓰기 실패 시, disk_write 함수가 반환한 음수 오류 코드를 반환
00070   if (ret < 0)
00071     return ret;
00072
00073   // 요청한 슈퍼블록 크기만큼 정확히 기록되었는지 확인
00074   if ((size_t)ret != sizeof(*sb))
00075     return -EIO;
00076
00077   // 정상적으로 슈퍼블록이 디스크에 저장됨
00078   return 0;
00079 }
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.19 super.h

이 파일의 문서화 페이지로 가기

```
00008
00009 #ifndef SFUSE_SUPER_H
00010 #define SFUSE_SUPER_H
00011
00012 #include <stdint.h>
00013
00015 #define SFUSE_MAGIC 0xEF53
00016
00018 #define SFUSE_BLOCK_SIZE 4096
00019
00021 #define SFUSE_SUPERBLOCK_OFFSET 1024
00022
00024 #define SFUSE_INODES_COUNT 1024
00025
00027 #define SFUSE_BLOCKS_COUNT 8192
00028
00030 #define SFUSE_INODE_BITMAP_BLOCK 2
00031
00033 #define SFUSE_BLOCK_BITMAP_BLOCK 3
00034
00036 #define SFUSE_INODE_TABLE_BLOCK 4
00037
00039 #define SFUSE_DATA_BLOCK_START 100
00040
00045 struct sfuse_super {
00046   uint32_t magic;
00047   uint32_t inodes_count;
00048   uint32_t blocks_count;
00049   uint32_t free_inodes;
00050   uint32_t free_blocks;
00051   uint32_t inode_bitmap_start;
00052   uint32_t block_bitmap_start;
00053   uint32_t inode_table_start;
00054   uint32_t data_block_start;
00055 };
```

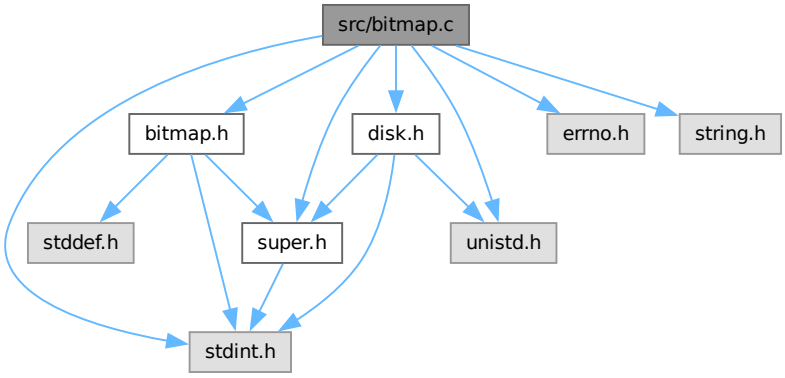
```
00056
00066 int sb_load(int fd, struct sfuse_super *sb);
00067
00077 int sb_sync(int fd, const struct sfuse_super *sb);
00078
00086 void sb_format(struct sfuse_super *sb);
00087
00088 #endif // SFUSE_SUPER_H
```

8.20 benchmark/README.md 파일 참조

8.21 README.md 파일 참조

8.22 src/bitmap.c 파일 참조

비트맵 관리 함수 구현
#include "bitmap.h"
#include "disk.h"
#include "super.h"
#include <errno.h>
#include <stdint.h>
#include <string.h>
#include <unistd.h>
bitmap.c에 대한 include 의존 그래프



함수

- int bitmap_load (int fd, uint32_t block_no, uint8_t *map, size_t map_size)
  비트맵 데이터를 디스크에서 읽어 메모리에 로드
- int bitmap_sync (int fd, uint32_t block_no, uint8_t *map, size_t map_size)
  메모리의 비트맵 데이터를 디스크에 기록
- int alloc_block (struct sfuse_super *sb, uint8_t *block_map)
  사용 가능한 데이터 블록을 찾아 할당하고, 비트맵과 슈퍼블록 상태를 갱신
- void free_block (struct sfuse_super *sb, uint8_t *block_map, uint32_t offset)
  이전에 할당된 데이터 블록을 해제하고 비트맵과 슈퍼블록 상태를 갱신
- int alloc_inode (struct sfuse_super *sb, uint8_t *inode_map)
  빈 아이노드를 찾아 할당하고 비트맵과 슈퍼블록을 갱신
- void free_inode (struct sfuse_super *sb, uint8_t *inode_map, uint32_t ino)
  할당된 아이노드를 해제하고 비트맵과 슈퍼블록을 갱신

8.22.1 상세한 설명

비트맵 관리 함수 구현
bitmpa.c는 비트맵의 디스크 I/O, 메모리 관리, 데이터 블록 및 아이노드 할당과 해제 관련 함수의 구현을 포함한다. 이 함수들은 슈퍼블록 정보를 기반으로 비트맵을 업데이트하여 파일시스템의 상태를 유지한다.
bitmap.c 파일에서 정의되었습니다.

8.22.2 함수 문서화

8.22.2.1 alloc_block()

```
int alloc_block (
            struct sfuse_super * sb,
            uint8_t * block_map)
```

사용 가능한 데이터 블록을 찾아 할당하고, 비트맵과 슈퍼블록 상태를 갱신
데이터 블록 할당
이 함수는 파일 시스템에서 데이터 저장을 위해 사용되지 않은 블록을 찾아 할당한다. 블록의 사용 여부는 비트맵(block_map)에 비트 단위로 저장되어 있으며, 이 비트맵을 순차적으로 탐색해 첫 번째 빈 블록을 찾아 할당한다.
블록을 할당하면 해당 블록의 비트를 '사용 중(1)'으로 설정하고, 슈퍼블록의 'free_blocks' 필드 값을 감소시켜 남은 가용 블록 수를 갱신한다.

매개변수

sb	슈퍼블록 구조체 포인터 (파일시스템의 전체적인 상태를 관리)
block_map	블록 할당 여부를 나타내는 비트맵 버퍼 포인터

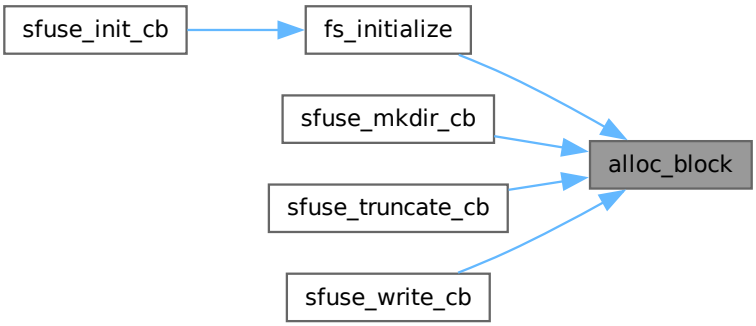
반환값

성공 시 할당된 블록의 오프셋(0부터 시작)을 반환하며, 가용 블록이 없으면 공간 부족을 의미하는 -ENOSPC 반환

bitmap.c 파일의 99 번째 라인에서 정의되었습니다.

```
99                                          {
100   // 데이터 블록 영역에서 사용 가능한 총 블록 개수를 계산
101   uint32_t total = sb->blocks_count - sb->data_block_start;
102
103   // 비트맵을 처음부터 끝까지 탐색하며 빈 블록을 찾음
104   for (uint32_t i = 0; i < total; i++) {
105     uint32_t byte_idx = i / 8; // 비트맵의 바이트 단위 인덱스
106     uint32_t bit_idx = i % 8;  // 바이트 내의 비트 위치 (0~7)
107
108     // 현재 블록(i)이 사용 가능한 상태인지 확인 (비트 값이 0이면 사용 가능)
109     if (!(block_map[byte_idx] & (1 « bit_idx))) {
110       // 사용 가능한 블록 발견 시 해당 비트를 '1'로 설정하여 사용 중으로 표시
111       block_map[byte_idx] |= (1 « bit_idx);
112
113       // 슈퍼블록에서 사용 가능한 블록 개수 1 감소
114       sb->free_blocks--;
115
116       // 할당된 블록의 오프셋 반환 (0부터 시작)
117       return (int)i;
118     }
119   }
120
121   // 모든 블록이 이미 사용 중이면 ENOSPC(공간 부족 오류) 반환
122   return -ENOSPC;
123 }
```

이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.22.2.2 alloc_inode()

```
int alloc_inode (
            struct sfuse_super * sb,
            uint8_t * inode_map)
```

빈 아이노드를 찾아 할당하고 비트맵과 슈퍼블록을 갱신
아이노드 할당
파일 시스템에서 새로운 파일이나 디렉토리를 생성할 때 사용할 빈 아이노드를 탐색하여 할당하는 함수이다.
아이노드 번호 0은 특수 용도로 예약되어 있으므로, 할당 가능한 아이노드의 탐색은 1번 아이노드부터 시작한다.
사용 가능한 아이노드를 찾으면 아이노드 비트맵에서 해당 비트를 사용 중(1)으로 설정하고, 슈퍼블록의 'free_inodes' 값을 감소시켜 파일 시스템의 상태를 업데이트한다.

매개변수

sb	슈퍼블록 구조체 포인터 (파일 시스템 상태 정보 관리)
inode_map	아이노드 할당 상태를 나타내는 비트맵 버퍼 포인터

반환값

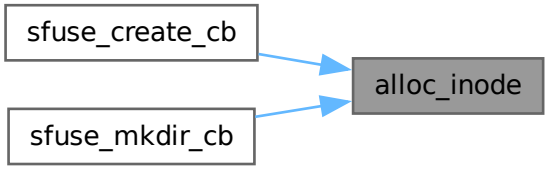
성공 시 할당된 아이노드 번호(1 이상)를 반환하고, 사용 가능한 아이노드가 없을 경우 -ENOSPC 반환

bitmap.c 파일의 172 번째 라인에서 정의되었습니다.

```
00172                                                   {
00173   // 아이노드 0은 예약이므로 1부터 탐색을 시작
00174   for (uint32_t i = 1; i < sb->inodes_count; i++) {
00175     uint32_t byte_idx = i / 8; // 비트맵 내의 바이트 인덱스 계산
00176     uint32_t bit_idx = i % 8;  // 바이트 내의 비트 위치 계산 (0~7)
00177
00178     // 현재 아이노드(i)가 빈 상태인지 검사 (0이면 빈 상태)
00179     if (!(inode_map[byte_idx] & (1 « bit_idx))) {
00180       // 사용 가능한 아이노드를 발견하면, 비트맵에 사용 중으로 표시 (비트=1)
00181       inode_map[byte_idx] |= (1 « bit_idx);
00182
00183       // 슈퍼블록의 남은 빈 아이노드 수를 1 감소시킴
00184       sb->free_inodes--;
00185
00186       // 할당된 아이노드 번호를 반환 (번호는 1부터 시작)
00187       return (int)i;
00188     }
00189   }
```

```

  // 빈 아이노드를 찾지 못했을 때 공간 부족(-ENOSPC) 오류 반환
  return -ENOSPC;
}
```

이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.22.2.3 bitmap_load()

```
int bitmap_load (
            int fd,
            uint32_t block_no,
            uint8_t * map,
            size_t map_size)
```

비트맵 데이터를 디스크에서 읽어 메모리에 로드
비트맵 데이터를 디스크에서 로드
파일 시스템에서 블록 또는 아이노드의 할당 여부를 나타내는 비트맵 데이터를 디스크의 특정 블록에서 읽어들여 메모리의 버퍼에 저장한다.
이 함수는 파일 시스템 초기화 및 비트맵 동기화 시 주로 사용된다.

매개변수

fd	디바이스 파일 디스크립터 (읽을 대상 디스크 장치)
block_no	비트맵이 저장된 디스크 내의 블록 번호
map	비트맵 데이터를 저장할 메모리 버퍼 포인터
map_size	비트맵 데이터의 크기(바이트 단위)

반환값

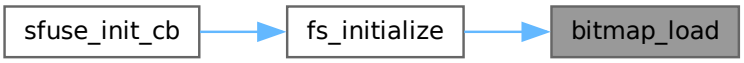
성공 시 0을 반환하며, 오류 발생 시 음수 값으로 오류 코드 반환: -EIO (I/O 오류), 기타 disk_read 가 반환하는 음수 값

bitmap.c 파일의 35 번째 라인에서 정의되었습니다.

```
{
ssize_t ret;

// 디스크에서 비트맵 데이터를 읽어 메모리(map)로 로드한다.
ret = disk_read(fd, map, map_size, (off_t)block_no * SFUSE_BLOCK_SIZE);
if (ret < 0)
  return (int)ret; // disk_read 실패: 읽기 자체가 실패함

// 읽은 데이터 크기가 요청한 크기와 일치하는지 검증한다.
if ((size_t)ret != map_size)
  return -EIO; // 읽은 데이터 크기가 예상 크기와 불일치

return 0; // 정상 종료
}
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:

이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.22.2.4 bitmap_sync()

```
int bitmap_sync (
            int fd,
            uint32_t block_no,
            uint8_t * map,
            size_t map_size)
```

메모리의 비트맵 데이터를 디스크에 기록
비트맵 데이터를 디스크에 기록
파일 시스템에서 현재 메모리에 저장된 비트맵 데이터를 지정된 디스크의 블록 위치로 기록한다.
비트맵의 변경 사항(블록 또는 아이노드 할당 상태 등)을 디스크에 반영하여, 시스템이 재부팅되거나 파일 시스템이 다시 마운트될 때 정확한 상태를 유지하도록 한다.

매개변수

fd	디바이스 파일 디스크립터 (기록할 대상 디스크 장치)
block_no	비트맵을 기록할 디스크 내의 블록 번호
map	디스크에 기록할 비트맵 데이터가 저장된 메모리 버퍼 포인터
map_size	비트맵 데이터의 크기(바이트 단위)

반환값

성공 시 0을 반환하며, 오류 발생 시 음수 값으로 오류 코드 반환: -EIO (I/O 오류), 또는 disk_write 가 반환하는 기타 음수 값

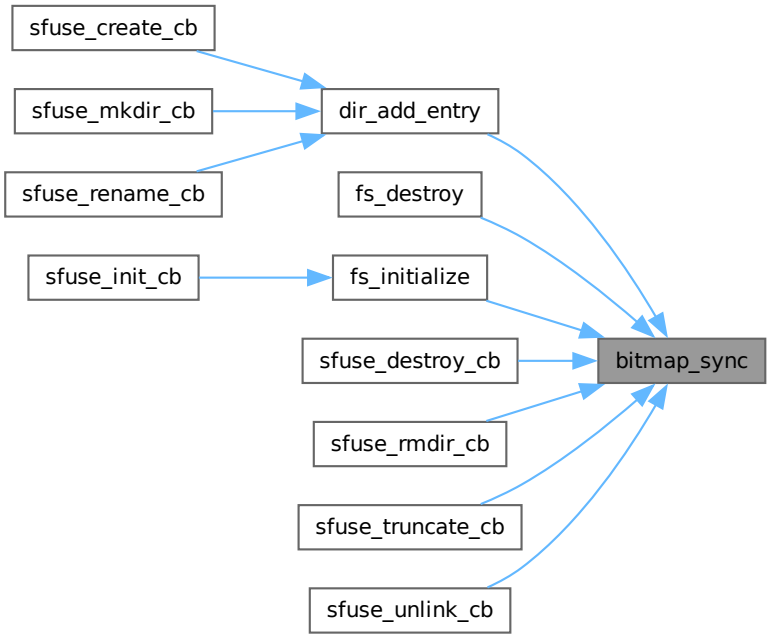
bitmap.c 파일의 68 번째 라인에서 정의되었습니다.

```
00068                                                   {
00069   ssize_t ret;
00070
00071   // 메모리(map)의 비트맵 데이터를 디스크의 지정된 위치에 기록한다.
00072   ret = disk_write(fd, map, map_size, (off_t)block_no * SFUSE_BLOCK_SIZE);
00073   if (ret < 0)
00074     return (int)ret; // disk_write 실패: 기록 자체가 실패함
00075
00076   // 기록한 데이터 크기가 요청한 크기와 일치하는지 검증한다.
```

```
00077   if ((size_t)ret != map_size)
00078     return -EIO; // 기록한 데이터 크기가 예상 크기와 불일치 (부분적 기록 발생)
00079
00080   return 0; // 정상적으로 비트맵 데이터를 디스크에 기록 완료
00081 }
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:

이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.22.2.5 free_block()

```
void free_block (
            struct sfuse_super * sb,
            uint8_t * block_map,
            uint32_t offset)
```

이전에 할당된 데이터 블록을 해제하고 비트맵과 슈퍼블록 상태를 갱신
데이터 블록 해제
이 함수는 파일 시스템에서 더 이상 사용하지 않는 데이터 블록을 다시 사용 가능한 상태로 되돌리기 위해 해제(free)한다.
주어진 블록 오프셋(offset)의 위치를 비트맵에서 찾아서, 해당 비트를 '0'으로 설정함으로써 빈 블록임을 표시하고, 슈퍼블록의 'free_blocks' 값을 1 증가시켜 남은 가용 블록 수를 갱신한다.

이 함수는 파일 삭제 또는 파일 크기 축소 시 호출되어, 디스크 공간을 효율적으로 재사용할 수 있도록 도와준다.

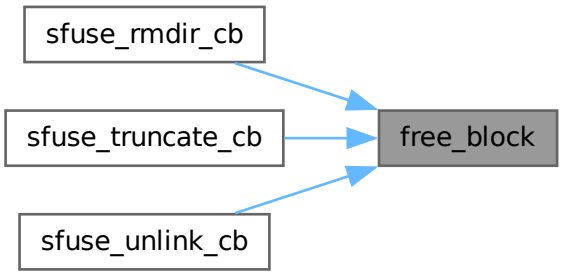
매개변수

sb	슈퍼블록 구조체 포인터 (파일 시스템의 전체 상태 관리)
block_map	블록 할당 여부를 나타내는 비트맵 버퍼 포인터
offset	해제할 블록의 오프셋(0부터 시작)

bitmap.c 파일의 142 번째 라인에서 정의되었습니다.

```
00142                                                    {
00143   uint32_t byte_idx = offset / 8; // 비트맵 내의 바이트 단위 인덱스 계산
00144   uint32_t bit_idx = offset % 8;  // 바이트 내에서의 비트 위치 계산 (0~7)
00145
00146   // 비트맵에서 해당 블록의 비트를 '0'으로 설정 (빈 상태로 표시)
00147   block_map[byte_idx] &= ~(1 « bit_idx);
00148
00149   // 슈퍼블록 내 빈 블록 개수를 1 증가 (가용 블록 수 갱신)
00150   sb->free_blocks++;
00151 }
```

이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.22.2.6 free_inode()

```
void free_inode (
            struct sfuse_super * sb,
            uint8_t * inode_map,
            uint32_t ino)
```

할당된 아이노드를 해제하고 비트맵과 슈퍼블록을 갱신
아이노드 해제
이 함수는 파일이나 디렉토리가 삭제될 때 호출되며, 사용하던 아이노드를 해제하여 다시 사용 가능한 상태로 되돌린다.
주어진 아이노드 번호(ino)를 검사하여 유효한 범위인지 확인한 후, 해당 아이노드의 비트를 아이노드 비트맵(inode_map)에서 0으로 설정하여 빈 상태를 나타내고, 슈퍼블록의 'free_inodes'를 증가시켜 사용 가능한 아이노드 수를 갱신한다.
아이노드 번호가 유효하지 않은 경우(0이거나 총 아이노드 수를 초과하는 경우), 함수는 아무 작업도 수행하지 않고 즉시 종료된다.

매개변수

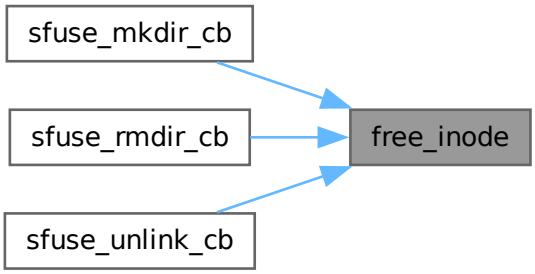
sb	슈퍼블록 구조체 포인터 (파일 시스템의 전역 상태 관리)
inode_map	아이노드 할당 상태를 나타내는 비트맵 버퍼 포인터

ino	해제할 아이노드 번호 (유효 범위: 1 ~ inodes_count-1)

bitmap.c 파일의 213 번째 라인에서 정의되었습니다.

```
00213                                                   {
00214   // 아이노드 번호 유효성 검사 (0번 아이노드 및 유효 범위 초과 시 무시)
00215   if (ino == 0 || ino >= sb->inodes_count)
00216     return; // 잘못된 아이노드 번호이므로 즉시 종료
00217
00218   uint32_t byte_idx = ino / 8; // 비트맵의 바이트 인덱스 계산
00219   uint32_t bit_idx = ino % 8;  // 해당 바이트 내의 비트 위치 계산 (0~7)
00220
00221   // 비트맵에서 해당 아이노드의 비트를 0으로 설정하여 빈 상태로 변경
00222   inode_map[byte_idx] &= ~(1 « bit_idx);
00223
00224   // 슈퍼블록의 빈 아이노드 개수를 1 증가시킴
00225   sb->free_inodes++;
00226 }
```

이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.23 bitmap.c

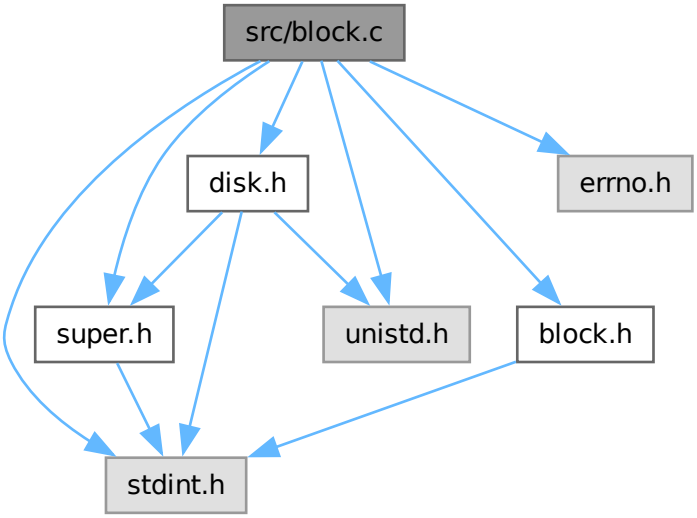
이 파일의 문서화 페이지로 가기

```
00001
00010
00011 #include "bitmap.h"
00012 #include "disk.h"
00013 #include "super.h"
00014 #include <errno.h>
00015 #include <stdint.h>
00016 #include <string.h>
00017 #include <unistd.h>
00018
00035 int bitmap_load(int fd, uint32_t block_no, uint8_t *map, size_t map_size) {
00036   ssize_t ret;
00037
00038   // 디스크에서 비트맵 데이터를 읽어 메모리(map)로 로드한다.
00039   ret = disk_read(fd, map, map_size, (off_t)block_no * SFUSE_BLOCK_SIZE);
00040   if (ret < 0)
00041     return (int)ret; // disk_read 실패: 읽기 자체가 실패함
00042
00043   // 읽은 데이터 크기가 요청한 크기와 일치하는지 검증한다.
00044   if ((size_t)ret != map_size)
00045     return -EIO; // 읽은 데이터 크기가 예상 크기와 불일치
00046
00047   return 0; // 정상 종료
00048 }
00049
00068 int bitmap_sync(int fd, uint32_t block_no, uint8_t *map, size_t map_size) {
00069   ssize_t ret;
00070
00071   // 메모리(map)의 비트맵 데이터를 디스크의 지정된 위치에 기록한다.
00072   ret = disk_write(fd, map, map_size, (off_t)block_no * SFUSE_BLOCK_SIZE);
00073   if (ret < 0)
```

```
    return (int)ret; // disk_write 실패: 기록 자체가 실패함

  // 기록한 데이터 크기가 요청한 크기와 일치하는지 검증한다.
  if ((size_t)ret != map_size)
    return -EIO; // 기록한 데이터 크기가 예상 크기와 불일치 (부분적 기록 발생)

  return 0; // 정상적으로 비트맵 데이터를 디스크에 기록 완료
}

int alloc_block(struct sfuse_super *sb, uint8_t *block_map) {
  // 데이터 블록 영역에서 사용 가능한 총 블록 개수를 계산
  uint32_t total = sb->blocks_count - sb->data_block_start;

  // 비트맵을 처음부터 끝까지 탐색하며 빈 블록을 찾음
  for (uint32_t i = 0; i < total; i++) {
    uint32_t byte_idx = i / 8; // 비트맵의 바이트 단위 인덱스
    uint32_t bit_idx = i % 8;  // 바이트 내의 비트 위치 (0~7)

    // 현재 블록(i)이 사용 가능한 상태인지 확인 (비트 값이 0이면 사용 가능)
    if (!(block_map[byte_idx] & (1 « bit_idx))) {
      // 사용 가능한 블록 발견 시 해당 비트를 '1'로 설정하여 사용 중으로 표시
      block_map[byte_idx] |= (1 « bit_idx);

      // 슈퍼블록에서 사용 가능한 블록 개수 1 감소
      sb->free_blocks--;

      // 할당된 블록의 오프셋 반환 (0부터 시작)
      return (int)i;
    }
  }

  // 모든 블록이 이미 사용 중이면 ENOSPC(공간 부족 오류) 반환
  return -ENOSPC;
}

void free_block(struct sfuse_super *sb, uint8_t *block_map, uint32_t offset) {
  uint32_t byte_idx = offset / 8; // 비트맵 내의 바이트 단위 인덱스 계산
  uint32_t bit_idx = offset % 8;  // 바이트 내에서의 비트 위치 계산 (0~7)

  // 비트맵에서 해당 블록의 비트를 '0'으로 설정 (빈 상태로 표시)
  block_map[byte_idx] &= ~(1 « bit_idx);

  // 슈퍼블록 내 빈 블록 개수를 1 증가 (가용 블록 수 갱신)
  sb->free_blocks++;
}

int alloc_inode(struct sfuse_super *sb, uint8_t *inode_map) {
  // 아이노드 0은 예약이므로 1부터 탐색을 시작
  for (uint32_t i = 1; i < sb->inodes_count; i++) {
    uint32_t byte_idx = i / 8; // 비트맵 내의 바이트 인덱스 계산
    uint32_t bit_idx = i % 8;  // 바이트 내의 비트 위치 계산 (0~7)

    // 현재 아이노드(i)가 빈 상태인지 검사 (0이면 빈 상태)
    if (!(inode_map[byte_idx] & (1 « bit_idx))) {
      // 사용 가능한 아이노드를 발견하면, 비트맵에 사용 중으로 표시 (비트=1)
      inode_map[byte_idx] |= (1 « bit_idx);

      // 슈퍼블록의 남은 빈 아이노드 수를 1 감소시킴
      sb->free_inodes--;

      // 할당된 아이노드 번호를 반환 (번호는 1부터 시작)
      return (int)i;
    }
  }

  // 빈 아이노드를 찾지 못했을 때 공간 부족(-ENOSPC) 오류 반환
  return -ENOSPC;
}

void free_inode(struct sfuse_super *sb, uint8_t *inode_map, uint32_t ino) {
  // 아이노드 번호 유효성 검사 (0번 아이노드 및 유효 범위 초과 시 무시)
  if (ino == 0 || ino >= sb->inodes_count)
    return; // 잘못된 아이노드 번호이므로 즉시 종료

  uint32_t byte_idx = ino / 8; // 비트맵의 바이트 인덱스 계산
  uint32_t bit_idx = ino % 8;  // 해당 바이트 내의 비트 위치 계산 (0~7)

  // 비트맵에서 해당 아이노드의 비트를 0으로 설정하여 빈 상태로 변경
  inode_map[byte_idx] &= ~(1 « bit_idx);

  // 슈퍼블록의 빈 아이노드 개수를 1 증가시킴
  sb->free_inodes++;
}
```

8.24 src/block.c 파일 참조

블록 단위 입출력 헬퍼 함수 구현
#include "block.h"
#include "disk.h"
#include "super.h"
#include <errno.h>
#include <stdint.h>
#include <unistd.h>
block.c에 대한 include 의존 그래프



함수

- int read_block (int fd, uint32_t block_no, void *buf)
  디바이스에서 지정된 블록 번호의 데이터를 읽는다.
- int write_block (int fd, uint32_t block_no, const void *buf)
  디바이스의 지정된 블록 번호 위치에 데이터를 기록한다.

8.24.1 상세한 설명

블록 단위 입출력 헬퍼 함수 구현
디바이스 파일에서 블록 단위로 데이터를 읽고 쓰는 함수들을 제공한다. 내부적으로 disk_read()와 disk_write()를 호출하여 실제 입출력을 수행한다.
block.c 파일에서 정의되었습니다.

8.24.2 함수 문서화

8.24.2.1 read_block()

```
int read_block (
            int fd,
            uint32_t block_no,
            void * buf)
```

디바이스에서 지정된 블록 번호의 데이터를 읽는다.
디바이스에서 블록 단위(4KB)로 읽기
블록 번호를 이용하여 디바이스 파일의 정확한 위치(offset)를 계산한 후, disk_read()를 호출하여 해당 위치에서 블록 단위의 데이터를 읽는다. 읽기 작업은 SFUSE_BLOCK_SIZE 단위로 이루어진다.

매개변수

fd	디바이스 파일 디스크립터
block_no	읽을 블록 번호 (0부터 시작)
buf	읽은 데이터를 저장할 SFUSE_BLOCK_SIZE 크기의 버퍼

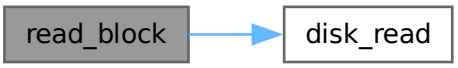
반환값

0 성공, 음수 오류 코드 -EIO: 읽은 데이터 크기가 SFUSE_BLOCK_SIZE와 다를 경우 disk_read()에서 반환된 음수의 오류 코드

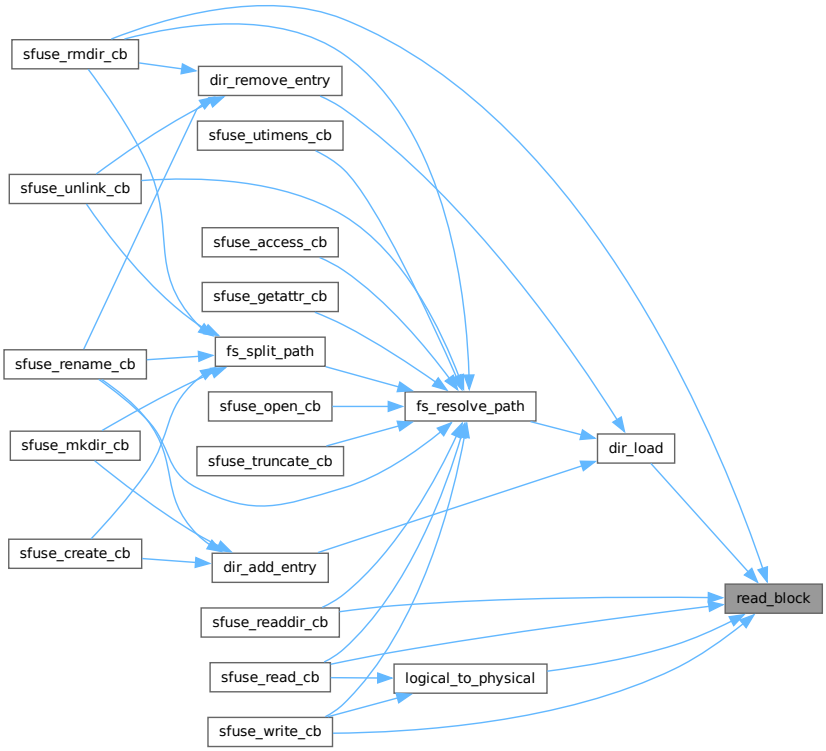
block.c 파일의 30 번째 라인에서 정의되었습니다.

```
00030                                   {
00031   // 블록 번호를 실제 파일의 바이트 오프셋으로 변환
00032   off_t offset = (off_t)block_no * SFUSE_BLOCK_SIZE;
00033
00034   // 계산된 오프셋에서 블록 크기만큼 데이터를 읽기
00035   ssize_t ret = disk_read(fd, buf, SFUSE_BLOCK_SIZE, offset);
00036
00037   // disk_read 실패 시 오류 코드 반환
00038   if (ret < 0)
00039     return (int)ret;
00040
00041   // 읽은 데이터가 요청한 블록 크기와 일치하지 않을 경우 EIO 오류 반환
00042   if ((size_t)ret != SFUSE_BLOCK_SIZE)
00043     return -EIO;
00044
00045   return 0;
00046 }
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.24.2.2 write_block()

```
int write_block (
            int fd,
            uint32_t block_no,
            const void * buf)
```

디바이스의 지정된 블록 번호 위치에 데이터를 기록한다.
디바이스에 블록 단위(4KB)로 쓰기
블록 번호를 기반으로 디바이스 파일 내의 정확한 위치(offset)를 계산한 후, disk_write()를 호출하여 해당 위치에 블록 크기의 데이터를 기록한다. 쓰기 작업은 SFUSE_BLOCK_SIZE 단위로 이루어진다.

매개변수

fd	디바이스 파일 디스크립터
block_no	기록할 블록 번호 (0부터 시작)
buf	기록할 데이터를 담고 있는 SFUSE_BLOCK_SIZE 크기의 버퍼

반환값

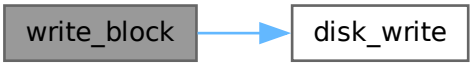
0 성공, 음수 오류 코드 -EIO: 기록한 데이터 크기가 SFUSE_BLOCK_SIZE와 다를 경우 disk_write()에서 반환된 음수의 오류 코드

block.c 파일의 62 번째 라인에서 정의되었습니다.

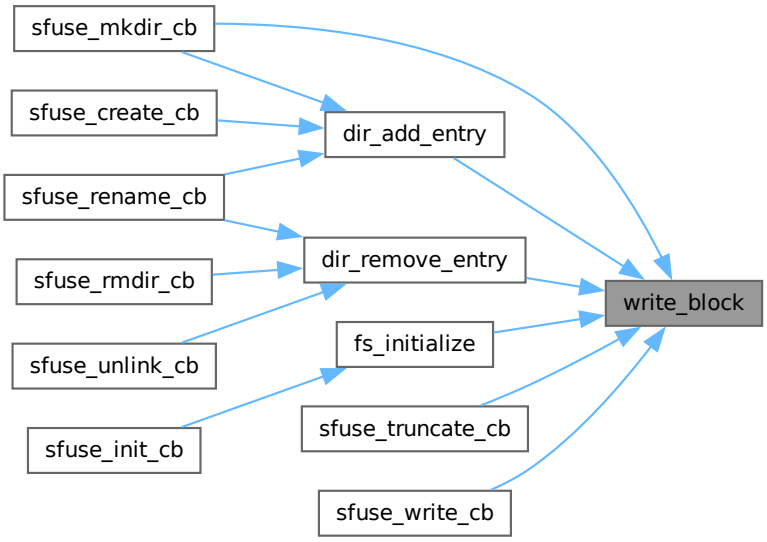
```
00062                                   {
00063   // 블록 번호를 실제 파일의 바이트 오프셋으로 변환
00064   off_t offset = (off_t)block_no * SFUSE_BLOCK_SIZE;
00065
```

```
// 계산된 오프셋에서 블록 크기만큼 데이터를 쓰기
ssize_t ret = disk_write(fd, buf, SFUSE_BLOCK_SIZE, offset);

// disk_write 실패 시 오류 코드 반환
if (ret < 0)
  return (int)ret;

// 기록한 데이터가 요청한 블록 크기와 일치하지 않을 경우 EIO 오류 반환
if ((size_t)ret != SFUSE_BLOCK_SIZE)
  return -EIO;

return 0;
}
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.25 block.c

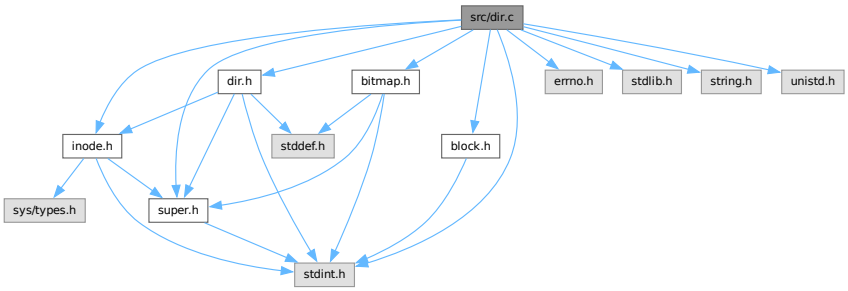
이 파일의 문서화 페이지로 가기

```
00001
00008
00009 #include "block.h"
00010 #include "disk.h"
00011 #include "super.h" // SFUSE_BLOCK_SIZE
00012 #include <errno.h>
00013 #include <stdint.h>
00014 #include <unistd.h>
00015
00030 int read_block(int fd, uint32_t block_no, void *buf) {
```

```
00031   // 블록 번호를 실제 파일의 바이트 오프셋으로 변환
00032   off_t offset = (off_t)block_no * SFUSE_BLOCK_SIZE;
00033
00034   // 계산된 오프셋에서 블록 크기만큼 데이터를 읽기
00035   ssize_t ret = disk_read(fd, buf, SFUSE_BLOCK_SIZE, offset);
00036
00037   // disk_read 실패 시 오류 코드 반환
00038   if (ret < 0)
00039     return (int)ret;
00040
00041   // 읽은 데이터가 요청한 블록 크기와 일치하지 않을 경우 EIO 오류 반환
00042   if ((size_t)ret != SFUSE_BLOCK_SIZE)
00043     return -EIO;
00044
00045   return 0;
00046 }
00047
00062 int write_block(int fd, uint32_t block_no, const void *buf) {
00063   // 블록 번호를 실제 파일의 바이트 오프셋으로 변환
00064   off_t offset = (off_t)block_no * SFUSE_BLOCK_SIZE;
00065
00066   // 계산된 오프셋에서 블록 크기만큼 데이터를 쓰기
00067   ssize_t ret = disk_write(fd, buf, SFUSE_BLOCK_SIZE, offset);
00068
00069   // disk_write 실패 시 오류 코드 반환
00070   if (ret < 0)
00071     return (int)ret;
00072
00073   // 기록한 데이터가 요청한 블록 크기와 일치하지 않을 경우 EIO 오류 반환
00074   if ((size_t)ret != SFUSE_BLOCK_SIZE)
00075     return -EIO;
00076
00077   return 0;
00078 }
```

8.26 src/dir.c 파일 참조

디렉터리 엔트리 로드, 추가, 삭제 기능 구현
#include "dir.h"
#include "bitmap.h"
#include "block.h"
#include "inode.h"
#include "super.h"
#include <errno.h>
#include <stdint.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
dir.c에 대한 include 의존 그래프



함수

- int dir_load (int fd, const struct sfuse_super *sb, uint32_t ino, void *buf)
  지정된 디렉터리 아이노드의 모든 직접(direct) 데이터 블록을 메모리에 읽어들인다.

- int dir_add_entry (int fd, const struct sfuse_super *sb, uint32_t ino, const char *name, uint32_t child_ino, uint8_t *block_map, uint8_t *inode_map, struct sfuse_super *supermap)

  디렉터리에 새 파일 또는 디렉터리 엔트리를 추가한다.
- int dir_remove_entry (int fd, const struct sfuse_super *sb, uint32_t ino, const char *name)

  디렉터리에서 특정 엔트리를 삭제한다.

8.26.1 상세한 설명

디렉터리 엔트리 로드, 추가, 삭제 기능 구현
디렉터리의 데이터를 로드하거나 새 엔트리를 추가하고, 기존의 엔트리를 삭제하는 기능을 구현한 파일이다.
dir.c 파일에서 정의되었습니다.

8.26.2 함수 문서화

8.26.2.1 dir_add_entry()

```
int dir_add_entry (
            int fd,
            const struct sfuse_super * sb,
            uint32_t ino,
            const char * name,
            uint32_t child_ino,
            uint8_t * block_map,
            uint8_t * inode_map,
            struct sfuse_super * supermap)
```

디렉터리에 새 파일 또는 디렉터리 엔트리를 추가한다.
디렉터리에 새로운 엔트리를 추가한다.
지정된 디렉터리 아이노드가 관리하는 직접 블록에서 빈 슬롯을 찾아, 새로운 파일 또는 디렉터리 엔트리를 추가한다. 엔트리 추가 후 수정된 디렉터리 데이터를 디스크에 기록하고, 파일시스템의 슈퍼블록 및 블록/아이노드 비트맵도 최신 상태로 동기화한다.

매개변수

fd	디바이스 파일 디스크립터 (디스크 접근용)
sb	슈퍼블록 구조체 포인터 (파일시스템의 정보 참조용)
ino	부모 디렉터리 아이노드 번호 (엔트리를 추가할 디렉터리)
name	추가할 새 엔트리의 이름
child_ino	새 엔트리에 연결될 아이노드 번호
block_map	블록 사용 정보를 나타내는 비트맵 버퍼
inode_map	아이노드 사용 정보를 나타내는 비트맵 버퍼
supermap	슈퍼블록 복사본 (변경 사항 동기화 시 사용)

반환값

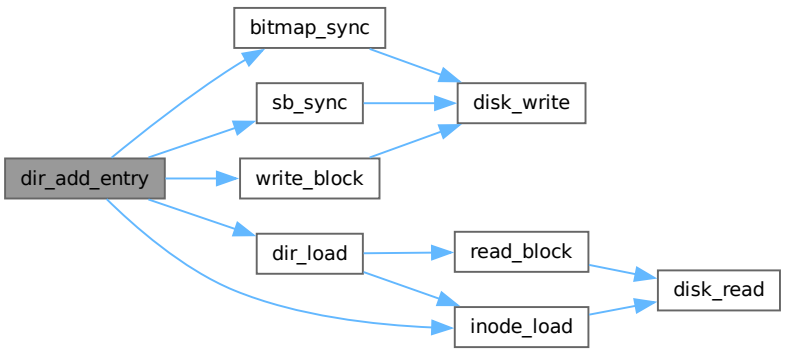
성공 시 0 반환, 빈 슬롯이 없어 추가가 불가능하면 -ENOSPC 반환, 메모리 할당 실패 시 -ENOMEM, 기타 입출력 오류 발생 시 음수 오류 코드 반환

dir.c 파일의 89 번째 라인에서 정의되었습니다.

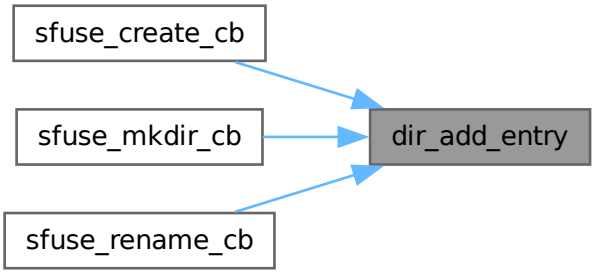
```
{
// 부모 디렉터리의 아이노드를 로드하여 inode 구조체에 저장한다.
struct sfuse_inode inode;
int res = inode_load(fd, sb, ino, &inode);
if (res < 0)
  return res; // 아이노드 로딩에 실패하면 즉시 종료한다.

// 디렉터리 데이터 전체를 저장할 메모리 버퍼를 할당한다.
size_t buf_size = SFUSE_NDIR_BLOCKS * SFUSE_BLOCK_SIZE;
```

```
void *buf = malloc(buf_size);
if (!buf)
  return -ENOMEM; // 메모리 할당 실패 시 오류 코드 반환

// 디스크로부터 디렉터리의 모든 직접 블록 데이터를 읽어온다.
res = dir_load(fd, sb, ino, buf);
if (res < 0) {
  free(buf);
  return res; // 블록 로드 실패 시 메모리 해제 후 종료한다.
}

// 버퍼를 디렉터리 엔트리 배열로 해석하여 최대 엔트리 개수 산정
struct sfuse_dirent *ents = (struct sfuse_dirent *)buf;
uint32_t max_entries = buf_size / sizeof(*ents);

// 디렉터리 내의 빈 슬롯을 탐색하여 새 엔트리를 추가한다.
for (uint32_t i = 0; i < max_entries; i++) {
  if (ents[i].ino == 0) {    // 빈 슬롯 발견 (아이노드 번호가 0이면 빈 슬롯)
    ents[i].ino = child_ino; // 새 아이노드 번호를 할당
    strncpy(ents[i].name, name, sizeof(ents[i].name) - 1); // 엔트리 이름 복사
    ents[i].name[sizeof(ents[i].name) - 1] = '\0'; // 문자열의 NULL 종단 보장

    // 수정된 전체 디렉터리 데이터를 디스크의 직접 블록에 다시 기록한다.
    for (uint32_t b = 0; b < SFUSE_NDIR_BLOCKS; b++) {
      if (inode.direct[b] == 0)
        continue; // 사용되지 않은 블록은 건너뛴다.

      res = write_block(fd, inode.direct[b],
                        (char *)buf + b * SFUSE_BLOCK_SIZE);
      if (res < 0) {
        free(buf);
        return res; // 블록 기록 실패 시 메모리 해제 후 종료한다.
      }
    }

    // 변경된 슈퍼블록과 블록 및 아이노드 비트맵을 디스크에 동기화한다.
    sb_sync(fd, supermap);
    bitmap_sync(fd, sb->block_bitmap_start, block_map, sb->blocks_count / 8);
    bitmap_sync(fd, sb->inode_bitmap_start, inode_map, sb->inodes_count / 8);

    free(buf); // 작업이 완료되었으므로 메모리 해제
    return 0;  // 성공적으로 엔트리 추가 완료
  }
}

// 디렉터리 내 모든 슬롯이 사용 중이어서 추가 불가 (공간 부족)
free(buf);
return -ENOSPC; // 추가 공간 부족 오류 반환
}
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.26.2.2 dir_load()

```
int dir_load (
            int fd,
            const struct sfuse_super * sb,
            uint32_t ino,
            void * buf)
```

지정된 디렉터리 아이노드의 모든 직접(direct) 데이터 블록을 메모리에 읽어들인다.
디렉터리의 모든 블록을 디스크에서 읽어들인다.
주어진 아이노드 번호(ino)에 해당하는 디렉터리의 아이노드 정보를 먼저 디스크에서 읽어온다. 이 아이노드에는 해당 디렉터리의 실제 데이터가 저장된 데이터 블록의 번호들이 저장되어 있다.
이 함수는 직접(direct) 블록 포인터만을 처리하며, 각 블록이 0인 경우(미할당)는 무시한다. 각 블록을 순서대로 디스크에서 읽어, 버퍼의 알맞은 위치에 복사하여 최종적으로 메모리에 디렉터리 전체 데이터를 적재한다.

매개변수

fd	디바이스 파일 디스크립터 (데이터를 읽을 원본 디바이스를 나타냄)
sb	슈퍼블록 정보 구조체 포인터 (파일 시스템의 레이아웃 정보 제공)
ino	데이터를 읽어올 디렉터리의 아이노드 번호
buf	읽어온 디렉터리 데이터를 저장할 버퍼 (충분한 크기가 미리 할당되어 있어야 함)

반환값

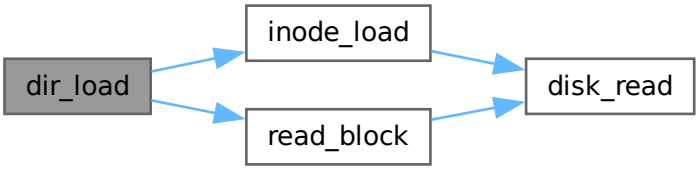
성공적으로 모든 블록을 읽었다면 0을 반환하고, inode_load 또는 read_block 수행 중 오류가 발생하면 음수의 오류 코드 반환

dir.c 파일의 42 번째 라인에서 정의되었습니다.

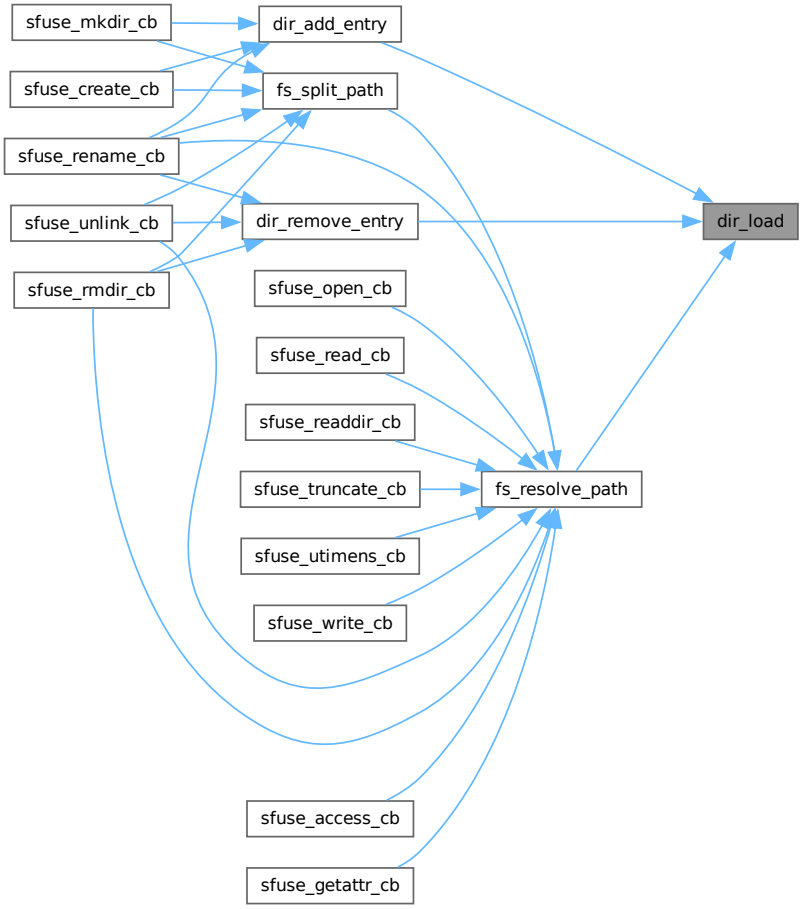
```
00042                                                   {
00043   // 디스크에서 주어진 아이노드 번호(ino)의 아이노드 메타데이터를 로드하여 inode
00044   // 변수에 저장
00045   struct sfuse_inode inode;
00046   int res = inode_load(fd, sb, ino, &inode);
00047   if (res < 0)
00048     return res; // 아이노드 로딩 실패 시 즉시 오류 코드 반환
00049
00050   // 아이노드가 참조하는 모든 직접(direct) 블록들을 순회하며 디스크에서 읽는다
00051   for (uint32_t i = 0; i < SFUSE_NDIR_BLOCKS; i++) {
00052     // 현재 블록 번호가 0이면 미할당 상태이므로 건너뜀
00053     if (inode.direct[i] == 0)
00054       continue;
00055
```

```
    // i번째 블록을 읽어 buf의 i번째 블록 위치로 이동한 메모리 주소에 저장
    // 각 블록 크기(SFUSE_BLOCK_SIZE)를 곱해 정확한 위치 계산
    res = read_block(fd, inode.direct[i], (char *)buf + i * SFUSE_BLOCK_SIZE);
    if (res < 0)
      return res; // 블록 읽기 중 오류 발생 시 즉시 오류 코드 반환
  }

  // 모든 직접 블록을 성공적으로 읽었으면 0 반환
  return 0;
}
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.26.2.3 dir_remove_entry()

```
int dir_remove_entry (
            int fd,
            const struct sfuse_super * sb,
            uint32_t ino,
            const char * name)
```

디렉터리에서 특정 엔트리를 삭제한다.
디렉터리에서 지정된 이름의 엔트리를 삭제한다.
이름이 일치하는 엔트리를 찾아 해당 슬롯을 비운 후 디스크에 데이터를 기록한다.

매개변수

fd	디바이스 파일 디스크립터
sb	슈퍼블록 정보 구조체 포인터
ino	부모 디렉터리 아이노드 번호
name	삭제할 엔트리의 이름

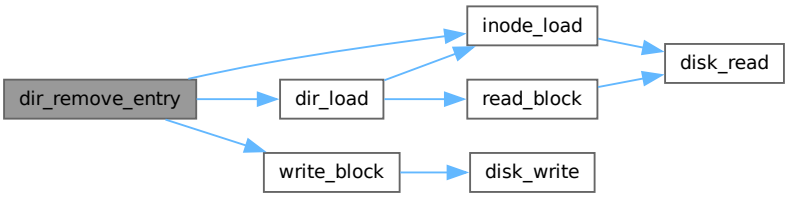
반환값

성공 시 0, 엔트리가 존재하지 않으면 -ENOENT, 기타 오류 발생 시 음수 오류 코드

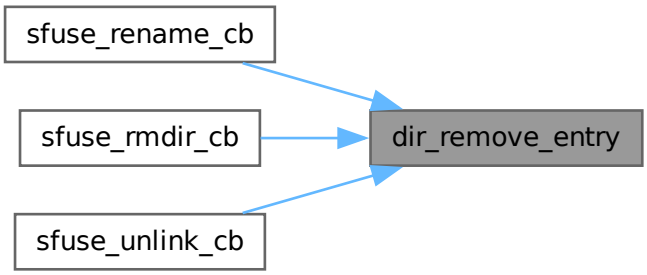
dir.c 파일의 162 번째 라인에서 정의되었습니다.

```
00163                                   {
00164   // 디렉터리 아이노드 로드
00165   struct sfuse_inode inode;
00166   int res = inode_load(fd, sb, ino, &inode);
00167   if (res < 0)
00168     return res;
00169
00170   // 디렉터리 데이터 블록 메모리에 로드
00171   size_t buf_size = SFUSE_NDIR_BLOCKS * SFUSE_BLOCK_SIZE;
00172   void *buf = malloc(buf_size);
00173   if (!buf)
00174     return -ENOMEM;
00175
00176   res = dir_load(fd, sb, ino, buf);
00177   if (res < 0) {
00178     free(buf);
00179     return res;
00180   }
00181
00182   struct sfuse_dirent *ents = (struct sfuse_dirent *)buf;
00183   uint32_t max_entries = buf_size / sizeof(*ents);
00184
00185   // 이름이 일치하는 엔트리 탐색 및 삭제
00186   for (uint32_t i = 0; i < max_entries; i++) {
00187     if (ents[i].ino != 0 && strcmp(ents[i].name, name) == 0) {
00188       ents[i].ino = 0;
00189       ents[i].name[0] = '\0';
00190
00191       // 수정된 디렉터리 데이터를 디스크에 기록
00192       for (uint32_t b = 0; b < SFUSE_NDIR_BLOCKS; b++) {
00193         if (inode.direct[b] == 0)
00194           continue;
00195
00196         res = write_block(fd, inode.direct[b],
00197                           (char *)buf + b * SFUSE_BLOCK_SIZE);
00198         if (res < 0) {
00199           free(buf);
00200           return res;
00201         }
00202       }
00203
00204       free(buf);
00205       return 0;
00206     }
00207   }
00208
00209   // 해당 이름의 엔트리가 없을 경우 메모리 해제 후 오류 반환
00210   free(buf);
00211   return -ENOENT;
00212 }
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.27 dir.c

이 파일의 문서화 페이지로 가기

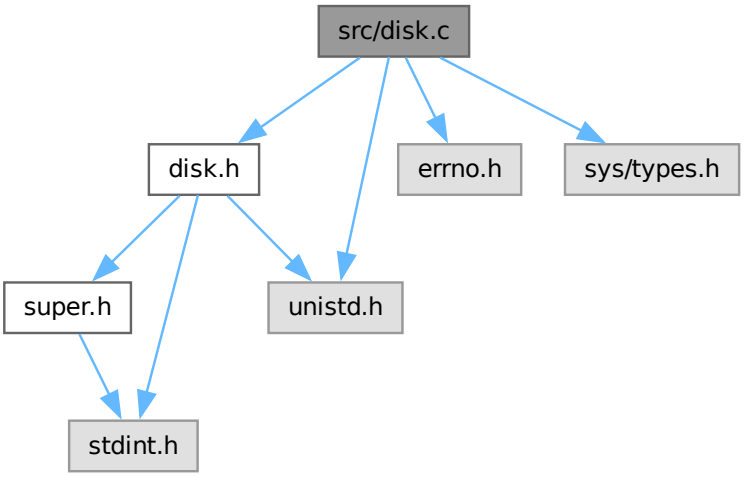
```
00001
00008
00009 #include "dir.h"
00010 #include "bitmap.h"
00011 #include "block.h"
00012 #include "inode.h"
00013 #include "super.h"
00014 #include <errno.h>
00015 #include <stdint.h>
00016 #include <stdlib.h>
00017 #include <string.h>
00018 #include <unistd.h>
00019
00042 int dir_load(int fd, const struct sfuse_super *sb, uint32_t ino, void *buf) {
00043   // 디스크에서 주어진 아이노드 번호(ino)의 아이노드 메타데이터를 로드하여 inode
00044   // 변수에 저장
00045   struct sfuse_inode inode;
00046   int res = inode_load(fd, sb, ino, &inode);
00047   if (res < 0)
00048     return res; // 아이노드 로딩 실패 시 즉시 오류 코드 반환
00049
00050   // 아이노드가 참조하는 모든 직접(direct) 블록들을 순회하며 디스크에서 읽는다
00051   for (uint32_t i = 0; i < SFUSE_NDIR_BLOCKS; i++) {
00052     // 현재 블록 번호가 0이면 미할당 상태이므로 건너뜀
00053     if (inode.direct[i] == 0)
00054       continue;
00055
00056     // i번째 블록을 읽어 buf의 i번째 블록 위치로 이동한 메모리 주소에 저장
00057     // 각 블록 크기(SFUSE_BLOCK_SIZE)를 곱해 정확한 위치 계산
00058     res = read_block(fd, inode.direct[i], (char *)buf + i * SFUSE_BLOCK_SIZE);
00059     if (res < 0)
```

```
    return res; // 블록 읽기 중 오류 발생 시 즉시 오류 코드 반환
  }

  // 모든 직접 블록을 성공적으로 읽었으면 0 반환
  return 0;
}

int dir_add_entry(int fd, const struct sfuse_super *sb, uint32_t ino,
                  const char *name, uint32_t child_ino, uint8_t *block_map,
                  uint8_t *inode_map, struct sfuse_super *supermap) {
  // 부모 디렉터리의 아이노드를 로드하여 inode 구조체에 저장한다.
  struct sfuse_inode inode;
  int res = inode_load(fd, sb, ino, &inode);
  if (res < 0)
    return res; // 아이노드 로딩에 실패하면 즉시 종료한다.

  // 디렉터리 데이터 전체를 저장할 메모리 버퍼를 할당한다.
  size_t buf_size = SFUSE_NDIR_BLOCKS * SFUSE_BLOCK_SIZE;
  void *buf = malloc(buf_size);
  if (!buf)
    return -ENOMEM; // 메모리 할당 실패 시 오류 코드 반환

  // 디스크로부터 디렉터리의 모든 직접 블록 데이터를 읽어온다.
  res = dir_load(fd, sb, ino, buf);
  if (res < 0) {
    free(buf);
    return res; // 블록 로드 실패 시 메모리 해제 후 종료한다.
  }

  // 버퍼를 디렉터리 엔트리 배열로 해석하여 최대 엔트리 개수 산정
  struct sfuse_dirent *ents = (struct sfuse_dirent *)buf;
  uint32_t max_entries = buf_size / sizeof(*ents);

  // 디렉터리 내의 빈 슬롯을 탐색하여 새 엔트리를 추가한다.
  for (uint32_t i = 0; i < max_entries; i++) {
    if (ents[i].ino == 0) {    // 빈 슬롯 발견 (아이노드 번호가 0이면 빈 슬롯)
      ents[i].ino = child_ino; // 새 아이노드 번호를 할당
      strncpy(ents[i].name, name, sizeof(ents[i].name) - 1); // 엔트리 이름 복사
      ents[i].name[sizeof(ents[i].name) - 1] = '\0'; // 문자열의 NULL 종단 보장

      // 수정된 전체 디렉터리 데이터를 디스크의 직접 블록에 다시 기록한다.
      for (uint32_t b = 0; b < SFUSE_NDIR_BLOCKS; b++) {
        if (inode.direct[b] == 0)
          continue; // 사용되지 않은 블록은 건너뛴다.

        res = write_block(fd, inode.direct[b],
                          (char *)buf + b * SFUSE_BLOCK_SIZE);
        if (res < 0) {
          free(buf);
          return res; // 블록 기록 실패 시 메모리 해제 후 종료한다.
        }
      }

      // 변경된 슈퍼블록과 블록 및 아이노드 비트맵을 디스크에 동기화한다.
      sb_sync(fd, supermap);
      bitmap_sync(fd, sb->block_bitmap_start, block_map, sb->blocks_count / 8);
      bitmap_sync(fd, sb->inode_bitmap_start, inode_map, sb->inodes_count / 8);

      free(buf); // 작업이 완료되었으므로 메모리 해제
      return 0;  // 성공적으로 엔트리 추가 완료
    }
  }

  // 디렉터리 내 모든 슬롯이 사용 중이어서 추가 불가 (공간 부족)
  free(buf);
  return -ENOSPC; // 추가 공간 부족 오류 반환
}

int dir_remove_entry(int fd, const struct sfuse_super *sb, uint32_t ino,
                     const char *name) {
  // 디렉터리 아이노드 로드
  struct sfuse_inode inode;
  int res = inode_load(fd, sb, ino, &inode);
  if (res < 0)
    return res;

  // 디렉터리 데이터 블록 메모리에 로드
  size_t buf_size = SFUSE_NDIR_BLOCKS * SFUSE_BLOCK_SIZE;
  void *buf = malloc(buf_size);
  if (!buf)
    return -ENOMEM;

  res = dir_load(fd, sb, ino, buf);
  if (res < 0) {
    free(buf);
    return res;
  }
```

```

  struct sfuse_dirent *ents = (struct sfuse_dirent *)buf;
  uint32_t max_entries = buf_size / sizeof(*ents);

  // 이름이 일치하는 엔트리 탐색 및 삭제
  for (uint32_t i = 0; i < max_entries; i++) {
    if (ents[i].ino != 0 && strcmp(ents[i].name, name) == 0) {
      ents[i].ino = 0;
      ents[i].name[0] = '\0';

      // 수정된 디렉터리 데이터를 디스크에 기록
      for (uint32_t b = 0; b < SFUSE_NDIR_BLOCKS; b++) {
        if (inode.direct[b] == 0)
          continue;

        res = write_block(fd, inode.direct[b],
                          (char *)buf + b * SFUSE_BLOCK_SIZE);
        if (res < 0) {
          free(buf);
          return res;
        }
      }

      free(buf);
      return 0;
    }
  }

  // 해당 이름의 엔트리가 없을 경우 메모리 해제 후 오류 반환
  free(buf);
  return -ENOENT;
}
```

8.28 src/disk.c 파일 참조

원시 디바이스에 직접 접근하는 저수준 I/O 연산 구현
#include "disk.h"
#include <errno.h>
#include <sys/types.h>
#include <unistd.h>
disk.c에 대한 include 의존 그래프



함수

- ssize_t disk_read (int fd, void *buf, size_t count, off_t off)

원시 디바이스의 지정된 offset에서 데이터를 읽는다.

- ssize_t disk_write (int fd, const void *buf, size_t count, off_t off)

  원시 디바이스의 지정된 offset에 데이터를 기록한다.

8.28.1 상세한 설명

원시 디바이스에 직접 접근하는 저수준 I/O 연산 구현
이 파일은 disk.h에서 선언된 disk_read()와 disk_write()를 구현하며, 주어진 원시 디바이스에 직접 접근하여 데이터를 읽고 쓰는 기능을 제공한다.

주의

read_block()과 write_block() 함수는 중복 코드를 방지하기 위해 별도의 파일(block.c)에 구현되어 있다.

disk.c 파일에서 정의되었습니다.

8.28.2 함수 문서화

8.28.2.1 disk_read()

```
ssize_t disk_read (
            int fd,
            void * buf,
            size_t count,
            off_t off)
```

원시 디바이스의 지정된 offset에서 데이터를 읽는다.
원시 디바이스에서 특정 offset으로부터 데이터를 읽는다.
본 함수는 lseek를 통해 디스크 내 원하는 위치(offset)로 이동한 뒤, read를 통해 데이터를 읽는다. offset은 파일의 시작 위치로부터 바이트 단위로 계산된다.

매개변수

fd	디바이스 파일 디스크립터
buf	읽은 데이터를 저장할 버퍼 포인터
count	읽을 데이터의 크기(바이트 단위)
off	읽기 시작할 위치 (바이트 단위 오프셋)

반환값

성공 시 실제로 읽은 바이트 수 반환. 실패 시 음수(-errno) 반환.

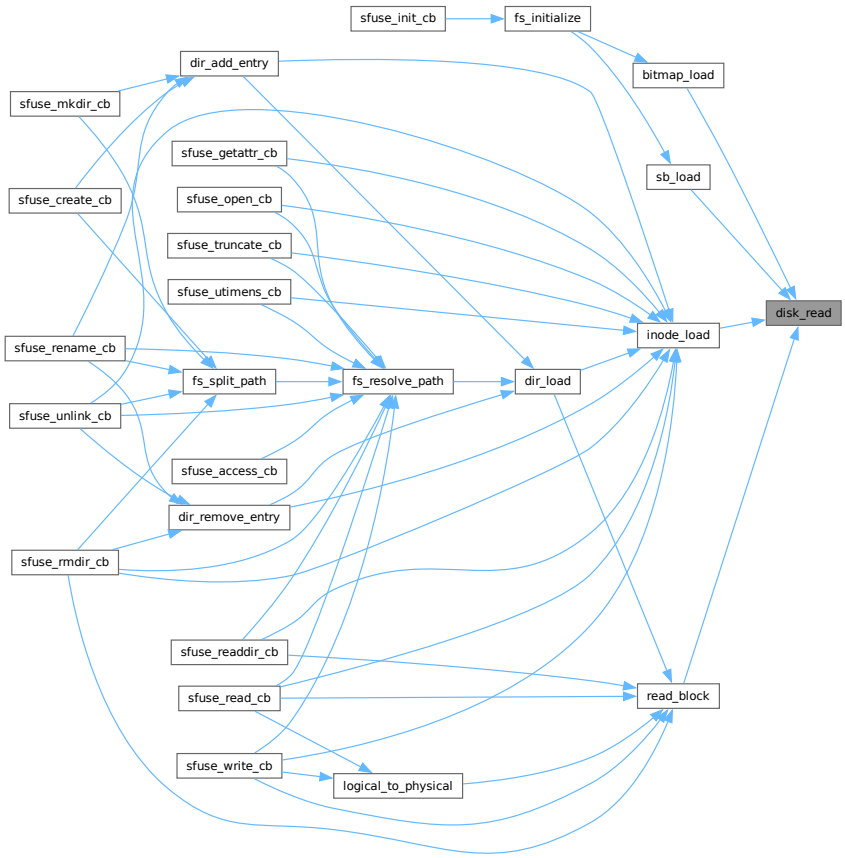
주의

반환된 바이트 수는 요청한 count 값보다 작을 수 있다.

disk.c 파일의 32 번째 라인에서 정의되었습니다.

```
{
 // 지정된 위치로 디스크 포인터 이동
 if (lseek(fd, off, SEEK_SET) < 0)
   return -errno; // lseek 실패 시 errno 반환

 // 데이터를 실제로 읽음
 ssize_t ret = read(fd, buf, count);
 if (ret < 0)
   return -errno; // 읽기 실패 시 errno 반환

 return ret; // 읽기 성공 시 읽은 바이트 수 반환
}
```

이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.28.2.2 disk_write()

```
ssize_t disk_write (
            int fd,
            const void * buf,
            size_t count,
            off_t off)
```

원시 디바이스의 지정된 offset에 데이터를 기록한다.
원시 디바이스의 특정 오프셋(offset)에 데이터를 쓴다.
본 함수는 lseek를 통해 디스크 내 원하는 위치(offset)로 이동한 뒤, write를 통해 데이터를 쓴다. offset은 파일의 시작 위치로부터 바이트 단위로 계산된다.

매개변수

fd	디바이스 파일 디스크립터
buf	기록할 데이터가 저장된 버퍼 포인터
count	기록할 데이터 크기(바이트 단위)
off	기록을 시작할 위치 (바이트 단위 오프셋)

반환값

성공 시 실제로 기록된 바이트 수 반환. 실패 시 음수(-errno) 반환.

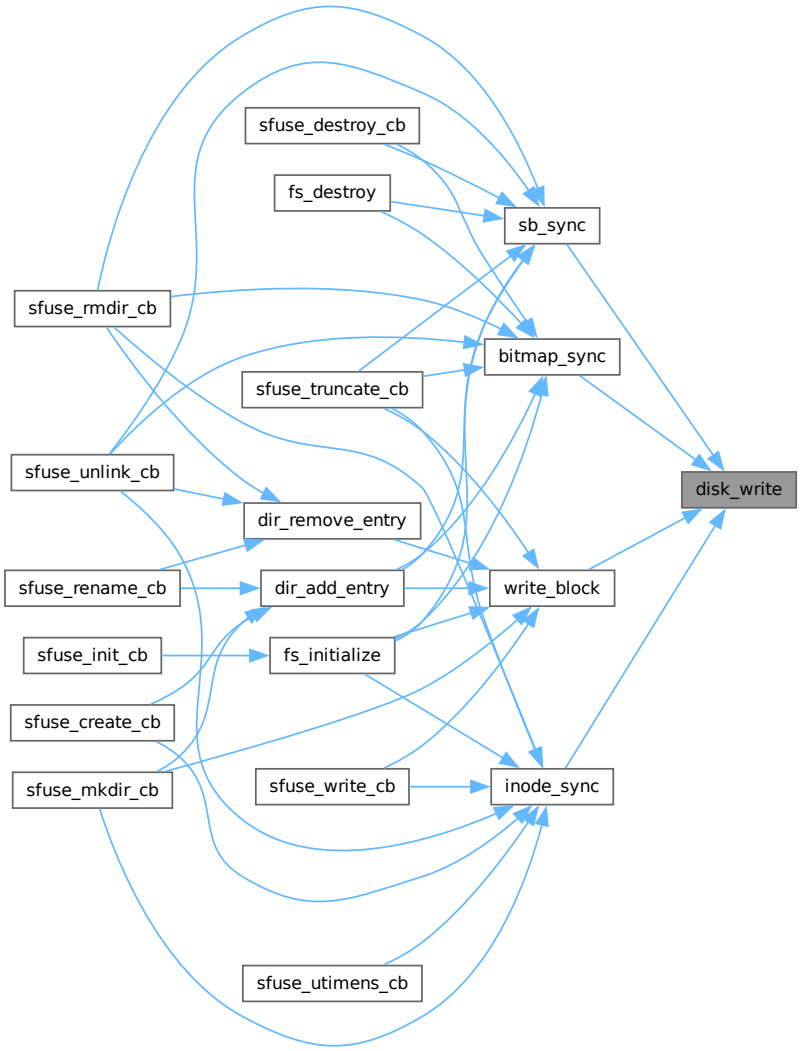
주의

기록된 바이트 수가 요청한 count 값보다 작을 수 있다. 디스크가 꽉 차는 등의 상황에서는 ENOSPC 오류가 발생할 수 있다.

disk.c 파일의 61 번째 라인에서 정의되었습니다.

```
00061                                                   {
00062   // 지정된 위치로 디스크 포인터 이동
00063   if (lseek(fd, off, SEEK_SET) < 0)
00064     return -errno; // lseek 실패 시 errno 반환
00065
00066   // 데이터를 실제로 기록함
00067   ssize_t ret = write(fd, buf, count);
00068   if (ret < 0)
00069     return -errno; // 기록 실패 시 errno 반환
00070
00071   return ret; // 기록 성공 시 기록한 바이트 수 반환
00072 }
```

이 함수를 호출하는 함수들에 대한 그래프입니다.:



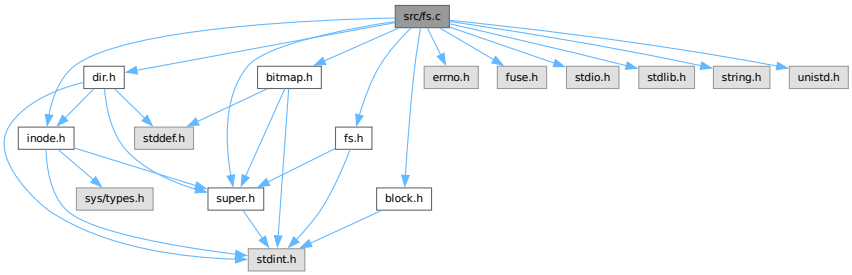
8.29 disk.c

이 파일의 문서화 페이지로 가기

```
#include "disk.h"
#include <errno.h>
#include <sys/types.h>
#include <unistd.h>

ssize_t disk_read(int fd, void *buf, size_t count, off_t off) {
  // 지정된 위치로 디스크 포인터 이동
  if (lseek(fd, off, SEEK_SET) < 0)
    return -errno; // lseek 실패 시 errno 반환

  // 데이터를 실제로 읽음
  ssize_t ret = read(fd, buf, count);
  if (ret < 0)
    return -errno; // 읽기 실패 시 errno 반환

  return ret; // 읽기 성공 시 읽은 바이트 수 반환
}

ssize_t disk_write(int fd, const void *buf, size_t count, off_t off) {
  // 지정된 위치로 디스크 포인터 이동
  if (lseek(fd, off, SEEK_SET) < 0)
    return -errno; // lseek 실패 시 errno 반환

  // 데이터를 실제로 기록함
  ssize_t ret = write(fd, buf, count);
  if (ret < 0)
    return -errno; // 기록 실패 시 errno 반환

  return ret; // 기록 성공 시 기록한 바이트 수 반환
}

// NOTE: read_block()과 write_block() 함수는 중복 코드를 방지하기 위해
// block.c에구현되어 있다.
```

8.30 src/fs.c 파일 참조

SFUSE 파일시스템 초기화, 관리 및 경로 분석 함수의 구현
#include "fs.h"
#include "bitmap.h"
#include "block.h"
#include "dir.h"
#include "inode.h"
#include "super.h"
#include <errno.h>
#include <fuse.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
fs.c에 대한 include 의존 그래프



함수

- struct sfuse_fs * get_fs_context (void)

현재 FUSE 컨텍스트에서 사용 중인 SFUSE 파일 시스템 컨텍스트를 반환한다.

- int fs_initialize (int backing_fd)

  SFUSE 파일 시스템을 초기화하고 슈퍼블록 및 메타데이터를 준비한다.
- void fs_destroy (void *private_data)

  SFUSE 파일 시스템을 종료하고 리소스를 정리하는 함수.
- int fs_resolve_path (struct sfuse_fs *fs, const char *path, uint32_t *ino_out)

  주어진 파일 또는 디렉터리 경로의 inode 번호를 찾는다.
- char * fs_split_path (const char *fullpath, uint32_t *parent_ino)

  전체 경로(fullpath)를 부모 경로와 파일명(또는 디렉터리명)으로 분리한다.

8.30.1 상세한 설명

SFUSE 파일시스템 초기화, 관리 및 경로 분석 함수의 구현
본 파일은 SFUSE 파일시스템을 관리하는 핵심 모듈로서, 슈퍼블록 초기화, inode 관리, 블록 관리, 경로 분석 및 해석 기능 등을 포함하며, 실제 블록 디바이스와의 동기화 작업을 수행하는 함수들을 구현한다. 파일시스템 생성 및 마운트 시 주요 데이터를 초기화하거나 기존 데이터를 불러와 사용자의 요청에 따라 디스크 상의 메타데이터를 관리한다.
fs.c 파일에서 정의되었습니다.

8.30.2 함수 문서화

8.30.2.1 fs_destroy()

void fs_destroy (
void * private_data)

SFUSE 파일 시스템을 종료하고 리소스를 정리하는 함수.
SFUSE 파일 시스템을 정리하고 언마운트한다.
파일 시스템 종료 시 호출되며, 메모리에 존재하는 파일 시스템의 메타데이터(슈퍼블록, 블록 및 inode 비트맵)를 디스크에 동기화하고, 할당된 메모리 리소스를 해제한다.

매개변수

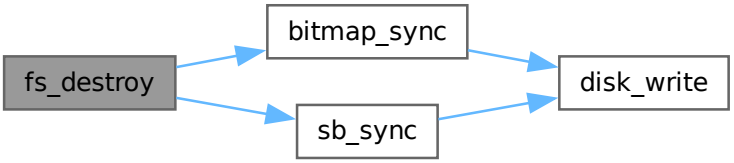
private_data	종료될 파일 시스템 컨텍스트(struct sfuse_fs)의 포인터.

fs.c 파일의 377 번째 라인에서 정의되었습니다.

```
{
// 전달받은 private_data 포인터를 struct sfuse_fs 타입으로 변환
struct sfuse_fs *fs = private_data;

// 비트맵 데이터를 디스크에 동기화하기 위한 메모리 크기 계산
// 블록 비트맵 크기: 전체 블록 수(blocks_count)를 비트맵 표현을 위해 바이트로
// 환산
size_t bmap_bytes = fs->sb.blocks_count / 8;

// inode 비트맵 크기: 전체 inode 수(inodes_count)를 비트맵 표현을 위해
// 바이트로 환산
size_t imap_bytes = fs->sb.inodes_count / 8;

// 메모리에 존재하는 블록 비트맵 상태를 디스크에 동기화한다.
// 이 작업을 통해 파일 시스템 종료 시 블록의 사용 상태가 디스크에 정확히
// 반영된다.
bitmap_sync(fs->backing_fd, fs->sb.block_bitmap_start, fs->block_map,
            bmap_bytes);

// 메모리에 존재하는 inode 비트맵 상태를 디스크에 동기화한다.
// 파일 시스템 종료 시 inode의 사용 여부를 디스크에 저장한다.
bitmap_sync(fs->backing_fd, fs->sb.inode_bitmap_start, fs->inode_map,
            imap_bytes);

// 슈퍼블록 상태를 디스크에 동기화하여 최신의 파일 시스템 메타데이터를
// 유지한다.
sb_sync(fs->backing_fd, &fs->sb);

// 파일 시스템의 종료 작업 이후 메모리에 할당된 비트맵 메모리를 해제하여,
// 메모리 누수를 방지한다.
free(fs->block_map); // 블록 비트맵 메모리 해제
free(fs->inode_map); // inode 비트맵 메모리 해제
```

00409 }

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



8.30.2.2 fs_initialize()

int fs_initialize (
int backing_fd)

SFUSE 파일 시스템을 초기화하고 슈퍼블록 및 메타데이터를 준비한다.
SFUSE 파일 시스템을 초기화하고 슈퍼블록 및 비트맵을 로드한다.
이 함수는 주어진 블록 장치 파일 디스크립터를 이용하여 SFUSE 파일 시스템을 초기화한다. 먼저 장치의 크기를 계산하고, 슈퍼블록이 존재하지 않으면 파일 시스템을 포맷하여 새로 생성하며, 슈퍼블록 및 루트 디렉터리를 초기화한다. 만약 슈퍼블록이 이미 존재한다면 이를 로드하고 기존 비트맵 데이터를 불러온다.

매개변수

backing_fd	파일 시스템을 저장하는 장치의 파일 디스크립터.

반환값

성공 시 0, 실패 시 음수의 에러 코드.

SFUSE 파일 시스템을 초기화하고 슈퍼블록 및 메타데이터를 준비한다.
파일 시스템의 슈퍼블록, inode 테이블, 비트맵 등을 초기화하거나 기존 데이터를 로드하여 파일 시스템 사용 준비를 완료한다.

매개변수

backing_fd	파일 시스템을 저장하는 블록 장치 파일 디스크립터.

반환값

성공 시 0, 실패 시 음수의 에러 코드.

fs.c 파일의 75 번째 라인에서 정의되었습니다.

```
00075                                   {
00076   // 현재 사용 중인 파일 시스템 컨텍스트 획득
00077   struct sfuse_fs *fs = get_fs_context();
00078
00079   // 얻은 블록 장치 파일 디스크립터 저장
00080   fs->backing_fd = backing_fd;
00081
00082   /*
00083   블록 장치(backing_fd)의 크기를 바이트 단위로 얻는다.
00084   파일의 크기를 얻으려면 파일의 가장 끝으로 파일 오프셋(offset)을 이동시킨다.
00085   이때 사용되는 함수가 바로 `lseek'이다.
00086
```

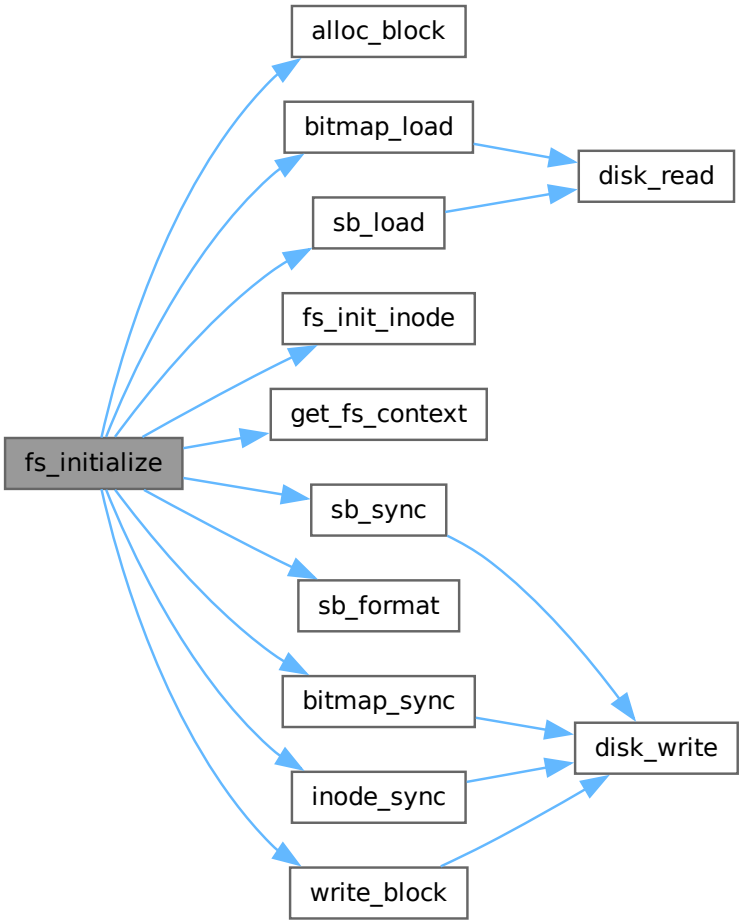
```
lseek 함수 원형:
off_t lseek(int fd, off_t offset, int whence);

인자 설명:
- fd: 오프셋을 이동시킬 파일 디스크립터.
- offset: 이동할 오프셋의 크기.
- whence: 오프셋 이동 기준을 설정하는 플래그.
  - SEEK_SET: 파일의 시작 지점으로부터 offset 바이트 이동.
  - SEEK_CUR: 현재 위치에서 offset 바이트 이동.
  - SEEK_END: 파일 끝에서부터 offset 바이트 이동.

여기서는 파일의 크기를 알고자 하므로,
SEEK_END를 이용해 파일의 끝으로 이동(offset은 0)을 지정한다.
그러면 lseek는 파일의 총 크기(바이트 단위)를 반환한다.
*/
off_t device_size = lseek(backing_fd, 0, SEEK_END);

/*
장치 크기(device_size)는 바이트 단위이기 때문에,
이를 SFUSE 파일 시스템이 사용하는 블록 크기(SFUSE_BLOCK_SIZE)로 나누어
블록 장치가 가진 전체 블록의 수를 계산한다.

예시로, 만약 장치 크기가 8192바이트이고 블록 크기가 1024바이트라면,
총 블록 수(total_blocks)는 8192 / 1024 = 8이 된다.

total_blocks 값은 파일 시스템의 메타데이터와 데이터 블록을 배치할 때
사용된다.
*/
uint32_t total_blocks = device_size / SFUSE_BLOCK_SIZE;

// 슈퍼블록을 디스크에서 로드(load) 시도.
// sb_load()가 성공하면 이미 파일 시스템이 포맷되어 있다는 뜻이다.
// sb_load()는 디스크에서 슈퍼블록 데이터를 읽어 fs->sb 구조체에 저장한다.
// 성공 시 0을 반환하며, 실패 시 음수를 반환한다.
//
// sb_load()가 실패하면 (즉, 음수 반환 시),
// 이는 슈퍼블록이 존재하지 않거나 손상된 상태를 나타내므로,
// 새로운 슈퍼블록을 포맷하고 초기화하는 과정을 수행해야 한다.
if (sb_load(backing_fd, &fs->sb) < 0) {

  // 슈퍼블록 포맷: 슈퍼블록을 깨끗한 초기 상태로 설정.
  // 이 함수는 슈퍼블록의 모든 값을 기본값 또는 초기 상태로 설정한다.
  sb_format(&fs->sb);

  // 슈퍼블록에 파일 시스템의 전체 블록 개수(total_blocks)를 설정한다.
  // 이 값은 파일 시스템이 관리할 수 있는 블록의 최대 개수를 나타낸다.
  fs->sb.blocks_count = total_blocks;

  // inode 개수 설정: 파일 시스템이 관리할 수 있는 inode의 총 개수를 설정.
  // inode는 파일이나 디렉터리 같은 객체의 메타데이터를 저장하는 단위이다.
  fs->sb.inodes_count = SFUSE_INODES_COUNT;

  // --- 메타데이터 영역의 크기 계산 시작 ---

  // 1. 블록 비트맵(block bitmap)의 크기를 블록 단위로 계산
  //
  // 블록 비트맵은 각 블록이 사용 중인지 여부를 비트 단위로 관리한다.
  // 하나의 블록을 1비트(bit)로 관리하므로,
  // 전체 블록 수를 8로 나누면 필요한 바이트(byte) 수가 된다.
  //
  // 이 바이트 수를 블록 단위(SFUSE_BLOCK_SIZE)로 변환하기 위해 다음 수식을
  // 사용:
  //
  // (fs->sb.blocks_count / 8 + SFUSE_BLOCK_SIZE - 1) / SFUSE_BLOCK_SIZE
  //
  // 이 수식은 '올림(ceil)'의 효과를 내는 표현으로,
  // 정확히 나누어 떨어지지 않을 경우 필요한 블록을 추가로 확보하기 위한
  // 계산법이다.
  uint32_t block_bitmap_blocks =
      (fs->sb.blocks_count / 8 + SFUSE_BLOCK_SIZE - 1) / SFUSE_BLOCK_SIZE;

  // 2. inode 비트맵(inode bitmap)의 크기를 블록 단위로 계산
  //
  // inode 비트맵은 inode들이 사용 중인지 여부를 관리하는 비트맵이다.
  // 전체 inode 개수를 8로 나누면 바이트 수가 되며,
  // 이것을 블록 단위로 변환하는 계산법은 블록 비트맵 계산법과 동일하다.
  uint32_t inode_bitmap_blocks =
      (fs->sb.inodes_count / 8 + SFUSE_BLOCK_SIZE - 1) / SFUSE_BLOCK_SIZE;

  // 3. inode 테이블(inode table)의 크기를 블록 단위로 계산
  //
  // inode 테이블은 실제 inode 구조체들을 저장하는 영역이다.
  // 각 inode는 'struct sfuse_inode'의 크기만큼 메모리를 차지하므로,
  // 전체 inode 개수에 inode 크기를 곱하면 inode 테이블의 총 크기가 바이트
  // 단위로 나온다.
  //
  // 이것을 다시 블록 크기(SFUSE_BLOCK_SIZE)로 나누어 몇 개의 블록이 필요한지
```

```
// 계산한다. 여기서도 블록이 정수로 딱 떨어지지 않을 수 있으므로, 올림
// 효과를 내도록 처리한다.
uint32_t inode_table_blocks =
    (fs->sb.inodes_count * sizeof(struct sfuse_inode) + SFUSE_BLOCK_SIZE -
     1) /
    SFUSE_BLOCK_SIZE;

// 슈퍼블록에서 관리하는 각 영역의 위치(블록 번호)를 설정한다.
// SFUSE 파일 시스템의 디스크 레이아웃은 일반적으로 다음과 같다:
//
//    파일 시스템 블록 구조:
// +-------------------------+
// | 블록 0 (슈퍼블록)        |
// +-------------------------+
// | 블록 1 (예비 블록)       |
// +-------------------------+
// | 블록 2 ~ N               |
// | (block bitmap)           |
// | [블록 사용 여부 표시]    |
// +-------------------------+
// | inode bitmap             |
// | [inode 사용 여부 표시]   |
// +-------------------------+
// | inode tables             |
// | [파일 메타데이터 저장]   |
// +-------------------------+
// | data block               |
// | [실제 사용자 데이터 저장]|
// +-------------------------+
//
// block 비트맵의 시작 위치는 슈퍼블록(블록 0)과 예비 블록(블록 1) 뒤에 위치
fs->sb.block_bitmap_start = 2;

// inode 비트맵은 block 비트맵 다음에 위치
fs->sb.inode_bitmap_start = fs->sb.block_bitmap_start + block_bitmap_blocks;

// inode 테이블은 inode 비트맵 다음에 위치
fs->sb.inode_table_start = fs->sb.inode_bitmap_start + inode_bitmap_blocks;

// 실제 사용자 데이터 블록은 inode 테이블 다음에 위치
fs->sb.data_block_start = fs->sb.inode_table_start + inode_table_blocks;

// 데이터 블록 시작 위치 이후의 블록만 실제 데이터 저장을 위해 사용
// 가능하므로, 전체 블록 수(total_blocks)에서 데이터 블록 시작 위치를 빼면
// 여유 블록 수를 구할 수 있다.
fs->sb.free_blocks = total_blocks - fs->sb.data_block_start;

// inode 중 루트 디렉터리를 제외한 나머지 inode들이 사용 가능
fs->sb.free_inodes = fs->sb.inodes_count - 1;

// 위에서 설정한 슈퍼블록 데이터를 디스크에 즉시 동기화하여 저장
if (sb_sync(backing_fd, &fs->sb) < 0)
  return -EIO; // 동기화 실패 시 입출력 에러 반환

// ---- 메모리에 블록 비트맵과 inode 비트맵을 관리할 공간을 확보 ----

// 비트맵 크기 계산:
// 비트맵은 비트 단위로 블록이나 inode의 사용 여부를 관리한다.
// 1 바이트(byte)는 8 비트(bit)를 가지므로, 블록과 inode의 개수를 각각 8로
// 나누면 비트맵이 필요한 메모리 크기가 바이트 단위로 나온다.
size_t bmap_bytes = fs->sb.blocks_count / 8;
size_t imap_bytes = fs->sb.inodes_count / 8;

// 위에서 계산한 크기만큼 메모리를 할당하고, 그 공간을 0으로 초기화한다.
// calloc(1, size)는 size 크기만큼 메모리를 할당한 후 모든 비트를 0으로
// 설정한다. 이렇게 하면 비트맵이 초기 상태에서는 모든 블록과 inode가 "사용
// 가능"(free)으로 표시된다.
fs->block_map = calloc(1, bmap_bytes);
fs->inode_map = calloc(1, imap_bytes);

// 메모리 할당 실패 시 메모리 부족 에러 반환
if (!fs->block_map || !fs->inode_map)
  return -ENOMEM;

// 비트맵의 초기 상태(모든 블록 및 inode가 사용 가능 상태)를 디스크에 동기화
bitmap_sync(backing_fd, fs->sb.block_bitmap_start, fs->block_map,
            bmap_bytes);
bitmap_sync(backing_fd, fs->sb.inode_bitmap_start, fs->inode_map,
            imap_bytes);

// ---- 루트 디렉터리 inode 초기화 ----

// 루트 inode 번호 설정 (1번으로 정의됨)
uint32_t root = SFUSE_ROOT_INO;

// inode 비트맵에서 루트 inode를 사용 중으로 표시하는 코드
// 비트맵(bitmap)은 각 inode의 사용 여부를 관리하는 데이터 구조로,
```

```
    // 1비트가 1개의 inode의 상태를 나타낸다.
    // 비트값이 0이면 해당 inode는 사용 가능(free), 1이면 사용
    // 중(allocated)이다.

    // inode 비트맵은 바이트(byte) 배열 형태로 관리된다.
    // 배열의 각 요소(바이트)마다 8개의 inode 상태를 표현할 수 있다.
    // 특정 inode가 속한 바이트의 위치는 inode 번호를 8로 나눈 값(root / 8)이며,
    // 바이트 내의 정확한 비트 위치는 inode 번호를 8로 나눈 나머지(root % 8)가
    // 된다.

    // 예시:
    // 루트 inode(root)가 5라면, inode_map[0] (바이트의 첫 번째 요소)의 5번째
    // 비트를 설정한다.

    // 이 코드의 동작 방식:
    // 1. (1 « (root % 8)): 1을 root % 8 만큼 왼쪽으로 시프트(shift)하여 원하는
    // 비트 위치로 설정한다.
    // 2. inode_map[root / 8] |= (...): OR 연산으로 해당 비트를 설정하여 inode를
    // '사용 중'으로 표시한다.
    //
    // 비트 연산을 사용하는 이유는 매우 빠르고 공간을 절약하기 때문이다.
    fs->inode_map[root / 8] |= (1 « (root % 8));

    // 위에서 inode 비트맵에 루트 inode를 할당했기 때문에,
    // 슈퍼블록에서 관리하는 사용 가능한(free) inode의 개수도 하나 감소시켜야
    // 한다.
    fs->sb.free_inodes--;

    // 루트 inode 구조체 초기화 (초기화 전에 메모리 전체를 0으로 설정)
    struct sfuse_inode root_inode;
    memset(&root_inode, 0, sizeof(root_inode));

    // 루트 inode 메타데이터 설정 (디렉터리, 접근 권한 0755, 소유자 UID/GID
    // 설정)
    fs_init_inode(&fs->sb, root, S_IFDIR | 0755, fuse_get_context()->uid,
                  fuse_get_context()->gid, &root_inode);

    // ---- 루트 디렉터리의 데이터 블록 할당 및 초기화 ----

    // 블록 할당 함수를 호출하여, 첫 번째 데이터 블록 인덱스를 할당 받음
    int blk_index = alloc_block(&fs->sb, fs->block_map);
    if (blk_index < 0)
      return -ENOSPC; // 사용 가능한 블록 공간 부족 시 에러 반환

    // 실제 블록 번호는 데이터 블록의 시작 위치에서 blk_index만큼 더한 값
    uint32_t phys_block = fs->sb.data_block_start + blk_index;

    // inode가 관리하는 직접(direct) 블록 포인터 중 첫 번째 항목을 설정
    root_inode.direct[0] = phys_block;
    root_inode.size =
        SFUSE_BLOCK_SIZE; // 루트 디렉터리 크기를 블록 크기만큼 설정

    // 데이터 블록 내에 루트 디렉터리의 기본 엔트리(“.” 및 “..”)를 생성하기 위한
    // 초기화
    uint8_t block[SFUSE_BLOCK_SIZE] = {0}; // 블록 크기만큼 0으로 초기화
    struct sfuse_dirent *entries =
        (struct sfuse_dirent *)block; // 디렉터리 엔트리 구조체로 사용

    // “.”(현재 디렉터리)는 루트 inode를 가리킴
    entries[0].ino = root;
    strcpy(entries[0].name, “.”);

    // “..”(상위 디렉터리) 역시 루트 inode를 가리킴 (루트 디렉터리는 자신을
    // 가리킴)
    entries[1].ino = root;
    strcpy(entries[1].name, “..”);

    // 구성된 디렉터리 데이터를 디스크의 실제 블록에 기록
    if (write_block(backing_fd, phys_block, block) < 0)
      return -EIO; // 블록 기록 실패 시 입출력 에러 반환

    // 구성된 루트 inode를 디스크에 저장하여 동기화
    if (inode_sync(backing_fd, &fs->sb, root, &root_inode) < 0)
      return -EIO; // inode 동기화 실패 시 입출력 에러 반환

    // ---- 모든 메타데이터의 최종 상태를 디스크에 동기화 ----

    // 블록 비트맵과 inode 비트맵의 최종 상태를 디스크에 저장
    bitmap_sync(backing_fd, fs->sb.block_bitmap_start, fs->block_map,
                bmap_bytes);
    bitmap_sync(backing_fd, fs->sb.inode_bitmap_start, fs->inode_map,
                imap_bytes);

    // 슈퍼블록의 최종 상태를 디스크에 저장하여 파일 시스템 초기화 완료
    sb_sync(backing_fd, &fs->sb);

  } else {
```

```
    // 슈퍼블록이 이미 존재(정상 로드 성공) 시, 기존 메타데이터를 디스크에서
    // 로드

    size_t bmap_bytes = fs->sb.blocks_count / 8;
    size_t imap_bytes = fs->sb.inodes_count / 8;

    fs->block_map = calloc(1, bmap_bytes);
    fs->inode_map = calloc(1, imap_bytes);

    // 기존 비트맵 데이터를 디스크에서 메모리로 로드하여 파일 시스템 재구성
    bitmap_load(backing_fd, fs->sb.block_bitmap_start, fs->block_map,
                bmap_bytes);
    bitmap_load(backing_fd, fs->sb.inode_bitmap_start, fs->inode_map,
                imap_bytes);
  }

  // 초기화 과정이 모두 정상적으로 완료되었으므로 성공(0)을 반환
  return 0;
}
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



이 함수를 호출하는 함수들에 대한 그래프입니다.:

8.30.2.3 fs_resolve_path()

```
int fs_resolve_path (
            struct sfuse_fs * fs,
            const char * path,
            uint32_t * ino_out)
```

주어진 파일 또는 디렉터리 경로의 inode 번호를 찾는다.
주어진 경로를 아이노드 번호로 변환한다.
이 함수는 SFUSE 파일 시스템 내에서 주어진 경로를 순차적으로 분석하여 해당 경로에 대응하는 inode 번호를 찾고 결과를 반환한다.
예를 들어 경로 "/home/user/file.txt"가 주어지면, "/" → "home" → "user" → "file.txt" 순서로 디렉터리를 순차적으로 탐색하여 최종적으로 "file.txt"의 inode 번호를 찾는다.

매개변수

fs	파일 시스템 컨텍스트(struct sfuse_fs)의 포인터.
path	inode 번호를 찾을 경로 문자열.
ino_out	찾은 inode 번호를 저장할 포인터.

반환값

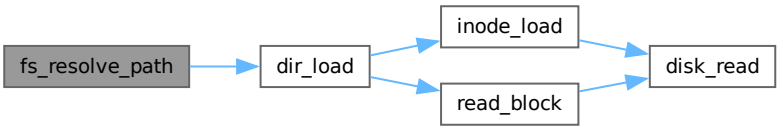
성공 시 0, 실패 시 음수의 에러 코드 반환.

fs.c 파일의 426 번째 라인에서 정의되었습니다.

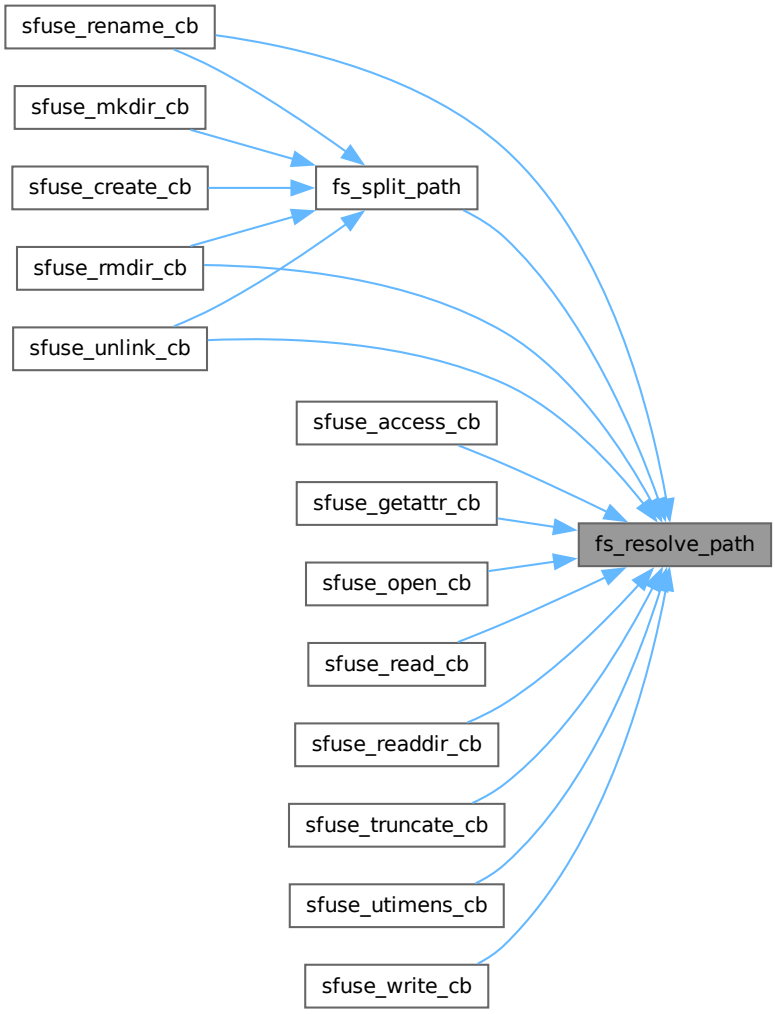
```
00426                                                   {
00427   // 시작 지점을 루트 디렉터리의 inode로 설정
00428   uint32_t cur = SFUSE_ROOT_INO;
00429
00430   // 예외 처리: 빈 문자열 또는 루트 디렉터리('/')가 들어오면 바로 루트 inode
00431   // 반환
00432   if (!path || path[0] == '\0' || strcmp(path, "/") == 0) {
00433     *ino_out = cur;
00434     return 0;
00435   }
00436
00437   // 경로 문자열을 수정 가능하도록 복제(메모리 복사)
00438   char *dup = strdup(path);
00439   if (!dup)
00440     return -ENOMEM; // 메모리 부족 시 에러 반환
00441
00442   // 경로 문자열을 "/" 기준으로 토큰(token)으로 나눈다.
00443   // 예시: "/home/user/file" → ["home", "user", "file"]
00444   char *token = strtok(dup, "/");
00445
00446   // 모든 토큰에 대해 순차적으로 디렉터리를 탐색
00447   while (token) {
00448     // 디렉터리 데이터를 메모리에 로드하기 위한 버퍼 크기를 계산한다.
00449     // 해당 버퍼는 디렉터리 엔트리 정보를 저장하는 데 사용된다.
00450     // SFUSE_NDIR_BLOCKS: 디렉터리 inode가 직접 참조할 수 있는 블록의 최대 개수.
00451     //                    직접 참조 블록 포인터 수 (12개)
00452     // SFUSE_BLOCK_SIZE:  각 블록의 크기(바이트 단위)()
00453     //
00454     // 따라서 디렉터리 데이터를 로드할 수 있는 전체 버퍼 크기는 다음과 같다:
00455     //
```

```
    //   buf_size = (직접 참조 블록 개수) × (각 블록 크기)
    //
    // 수식으로 표현하면:
    //   buf_size = SFUSE_NDIR_BLOCKS * SFUSE_BLOCK_SIZE;
    //
    // 예시:
    //   만약 SFUSE_NDIR_BLOCKS가 12이고 SFUSE_BLOCK_SIZE가 1024 바이트라면,
    //   buf_size = 12 × 1024 = 12288 바이트(12KB)가 된다.
    size_t buf_size = SFUSE_NDIR_BLOCKS * SFUSE_BLOCK_SIZE;
    void *buf = malloc(buf_size);
    if (!buf) {
      free(dup);
      return -ENOMEM; // 메모리 부족 시 에러 반환
    }

    // 현재 디렉터리의 모든 엔트리(entry)를 디스크에서 메모리로 로드한다.
    // dir_load는 cur(현재 inode 번호)에 해당하는 디렉터리 데이터를 버퍼에
    // 채운다.
    if (dir_load(fs->backing_fd, &fs->sb, cur, buf) < 0) {
      free(buf);
      free(dup);
      return -ENOENT; // 디렉터리를 읽을 수 없는 경우 존재하지 않음 에러 반환
    }

    // 버퍼의 데이터를 디렉터리 엔트리 배열로 캐스팅하여 접근 가능하도록 설정
    struct sfuse_dirent *ents = (struct sfuse_dirent *)buf;

    // 디렉터리 엔트리의 최대 개수를 계산
    size_t max_entries = buf_size / sizeof(*ents);

    // 다음 디렉터리 또는 파일의 inode 번호를 찾기 위한 임시 변수
    uint32_t next_ino = 0;

    // 현재 디렉터리 내의 엔트리 중에서 이름이 토큰과 일치하는 엔트리를 찾는다.
    for (size_t i = 0; i < max_entries; i++) {
      // ents[i].ino가 0이면 빈 엔트리이므로 무시
      // 토큰과 같은 이름의 엔트리를 찾으면 next_ino에 inode 번호 저장 후 반복문
      // 종료
      if (ents[i].ino != 0 && strcmp(ents[i].name, token) == 0) {
        next_ino = ents[i].ino;
        break;
      }
    }

    // 버퍼 사용이 끝났으므로 메모리 해제
    free(buf);

    // 해당하는 이름을 가진 엔트리를 찾지 못한 경우, 경로가 존재하지 않음을 의미
    if (next_ino == 0) {
      free(dup);
      return -ENOENT; // 경로 존재하지 않음 에러 반환
    }

    // 다음 토큰 탐색을 위해 현재 inode 번호를 방금 찾은 inode 번호로 업데이트
    cur = next_ino;

    // 다음 토큰으로 넘어감
    token = strtok(NULL, "/");
  }

  // 모든 토큰 탐색이 완료되면, 최종 inode 번호를 결과로 반환
  free(dup);      // 경로 문자열 복제본의 메모리 해제
  *ino_out = cur; // 최종 inode 번호 저장
  return 0;       // 성공적으로 경로를 해석함
}
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.30.2.4 fs_split_path()

```
char * fs_split_path (
            const char * fullpath,
            uint32_t * parent_ino)
```

전체 경로(fullpath)를 부모 경로와 파일명(또는 디렉터리명)으로 분리한다.
전체 경로를 부모 디렉터리의 아이노드와 마지막 구성요소의 이름으로 분할한다.
주어진 전체 경로에서 마지막 경로 구성 요소를 파일 또는 디렉터리 이름으로 분리하고, 나머지 앞부분 경로는 부모 디렉터리로 간주하여 해당하는 부모 디렉터리의 inode 번호를 찾는다.
예시: "/home/user/file.txt" → parent_ino: "/home/user"의 inode 번호 반환값: "file.txt"
"/file.txt" → parent_ino: 루트 디렉터리의 inode 번호 ("/") 반환값: "file.txt"

매개변수

fullpath	전체 파일 또는 디렉터리 경로 문자열
parent_ino	부모 디렉터리의 inode 번호가 저장될 포인터

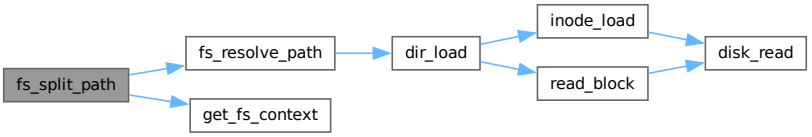
반환값

성공 시 파일명 또는 디렉터리명 문자열 포인터 반환 (반드시 호출한 곳에서 free 해야 함) 실패 시 NULL 반환

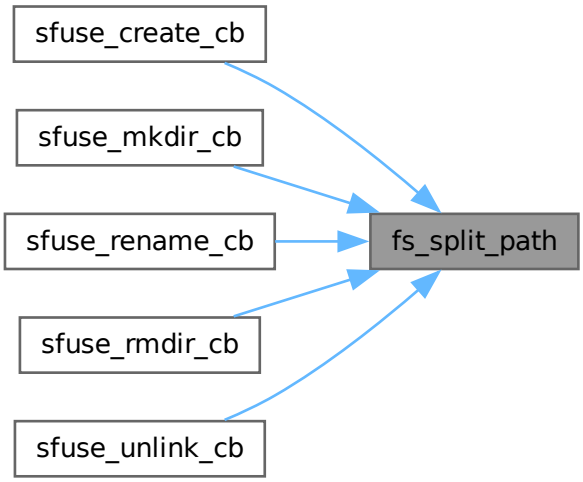
fs.c 파일의 543 번째 라인에서 정의되었습니다.

```
{
  char *dup = strdup(fullpath); // 경로를 수정하기 위해 복제본 생성
  if (!dup)
    return NULL; // 메모리 부족 시 실패 처리

  char *slash = strrchr(dup, '/'); // 마지막 '/' 문자를 찾아 경로 분리 지점 설정
  char *name;

  if (!slash) {
    // '/' 문자가 없을 경우 (예: "file.txt")
    *parent_ino = SFUSE_ROOT_INO; // 부모는 루트 디렉터리로 설정
    name = dup;                   // 전체 경로가 곧 파일명
  } else if (slash == dup) {
    // 경로가 루트에 바로 붙은 경우 (예: "/file.txt")
    *parent_ino = SFUSE_ROOT_INO; // 부모는 루트 디렉터리
    name = strdup(slash + 1);     // '/' 다음부터 이름 복사 ("file.txt")
    free(dup);                    // 최초 복제한 문자열은 더 이상 필요 없음
  } else {
    // 그 외 일반적인 경우 (예: "/home/user/file.txt")
    *slash =
        '\0'; // '/' 위치에서 문자열을 종료시켜 부모 경로만 남김 ("/home/user")

    // 남은 부모 경로(dup)를 이용하여 부모 디렉터리의 inode 번호를 찾는다.
    if (fs_resolve_path(get_fs_context(), dup, parent_ino) < 0) {
      free(dup);   // 부모 경로 탐색 실패 시 메모리 해제
      return NULL; // 실패 반환
    }

    // '/' 뒤의 문자열("file.txt")을 이름으로 복사하여 반환
    name = strdup(slash + 1);
    free(dup); // 부모 경로 탐색 완료 후 메모리 해제
  }

  return name; // 호출한 곳에서 반드시 free(name) 수행 필요
}
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.30.2.5 get_fs_context()

struct sfuse_fs * get_fs_context (
void)

현재 FUSE 컨텍스트에서 사용 중인 SFUSE 파일 시스템 컨텍스트를 반환한다.
FUSE 콜백 함수 내에서 전역 파일 시스템 컨텍스트를 얻는다.
본 함수는 FUSE 라이브러리에서 제공하는 fuse_get_context()를 통해 얻은 private_data 필드를 SFUSE 파일 시스템 컨텍스트 구조체 포인터로 반환한다.
SFUSE 파일 시스템 컨텍스트(struct sfuse_fs)는 파일 시스템의 상태 정보와 메타데이터를 포함한다.
추가 설명:

- 컨텍스트(context): 파일 시스템 연산 처리 시 필요한 사용자 및 상태 정보를 저장하는 구조체로, 요청자(UID, GID, PID)의 정보와 같은 메타데이터를 포함한다.
- fuse_get_context(): 현재 처리 중인 파일 시스템 연산의 컨텍스트를 반환하는 FUSE 제공 함수.
- private_data: 파일 시스템 개발자가 정의한 데이터를 저장하는 포인터로, SFUSE에서는 파일 시스템의 슈퍼블록, 비트맵, 블록 정보 등 상태 데이터와 메타데이터 접근을 위해 사용한다.

반환값

struct sfuse_fs* 현재 파일 시스템의 컨텍스트 포인터

참고

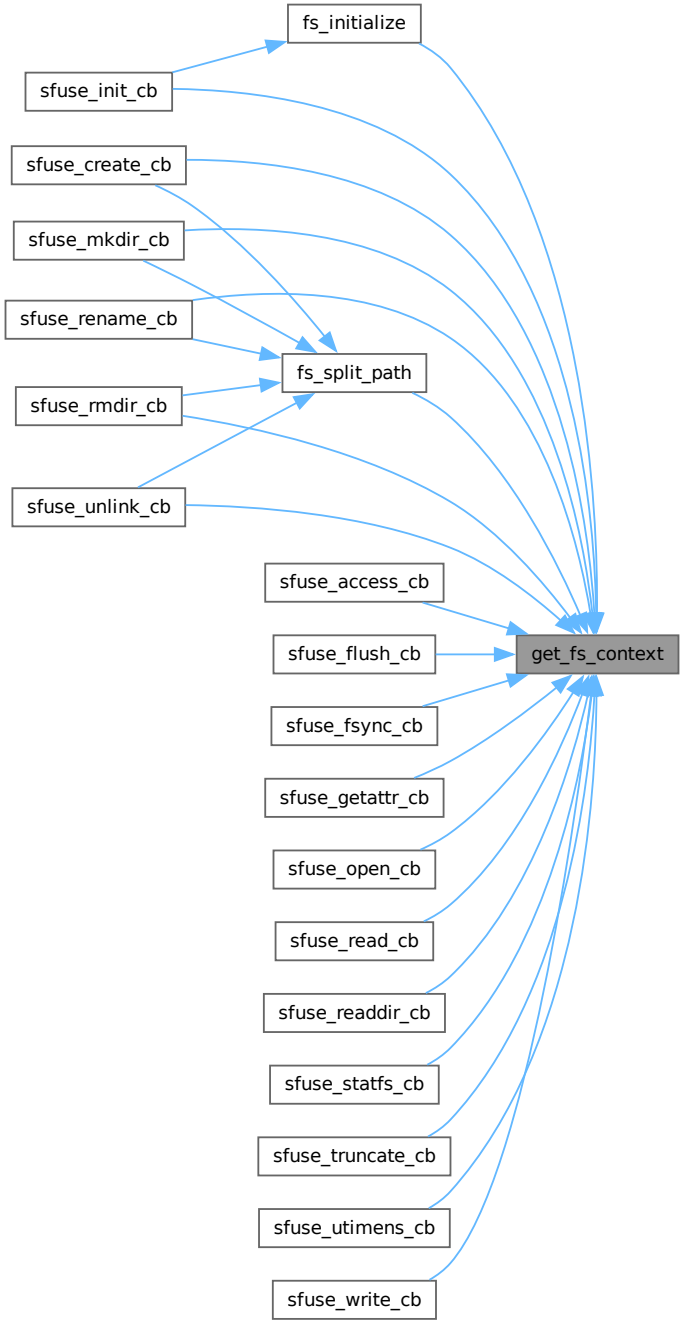
fuse_get_context()

struct sfuse_fs

fs.c 파일의 51 번째 라인에서 정의되었습니다.

```
00051                                   {
00052   return (struct sfuse_fs *)fuse_get_context()->private_data;
00053 }
```

이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.31 fs.c

이 파일의 문서화 페이지로 가기

```
00001
00011
00012 #include "fs.h"
00013 #include "bitmap.h"
00014 #include "block.h"
00015 #include "dir.h"
00016 #include "inode.h"
```

```
#include "super.h"
#include <errno.h>
#include <fuse.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

struct sfuse_fs *get_fs_context(void) {
  return (struct sfuse_fs *)fuse_get_context()->private_data;
}

int fs_initialize(int backing_fd) {
  // 현재 사용 중인 파일 시스템 컨텍스트 획득
  struct sfuse_fs *fs = get_fs_context();

  // 얻은 블록 장치 파일 디스크립터 저장
  fs->backing_fd = backing_fd;

  /*
  블록 장치(backing_fd)의 크기를 바이트 단위로 얻는다.
  파일의 크기를 얻으려면 파일의 가장 끝으로 파일 오프셋(offset)을 이동시킨다.
  이때 사용되는 함수가 바로 `lseek'이다.

  lseek 함수 원형:
  off_t lseek(int fd, off_t offset, int whence);

  인자 설명:
  - fd: 오프셋을 이동시킬 파일 디스크립터.
  - offset: 이동할 오프셋의 크기.
  - whence: 오프셋 이동 기준을 설정하는 플래그.
    - SEEK_SET: 파일의 시작 지점으로부터 offset 바이트 이동.
    - SEEK_CUR: 현재 위치에서 offset 바이트 이동.
    - SEEK_END: 파일 끝에서부터 offset 바이트 이동.

  여기서는 파일의 크기를 알고자 하므로,
  SEEK_END를 이용해 파일의 끝으로 이동(offset은 0)을 지정한다.
  그러면 lseek는 파일의 총 크기(바이트 단위)를 반환한다.
 */
  off_t device_size = lseek(backing_fd, 0, SEEK_END);

  /*
  장치 크기(device_size)는 바이트 단위이기 때문에,
  이를 SFUSE 파일 시스템이 사용하는 블록 크기(SFUSE_BLOCK_SIZE)로 나누어
  블록 장치가 가진 전체 블록의 수를 계산한다.

  예시로, 만약 장치 크기가 8192바이트이고 블록 크기가 1024바이트라면,
  총 블록 수(total_blocks)는 8192 / 1024 = 8이 된다.

  total_blocks 값은 파일 시스템의 메타데이터와 데이터 블록을 배치할 때
  사용된다.
  */
  uint32_t total_blocks = device_size / SFUSE_BLOCK_SIZE;

  // 슈퍼블록을 디스크에서 로드(load) 시도.
  // sb_load()가 성공하면 이미 파일 시스템이 포맷되어 있다는 뜻이다.
  // sb_load()는 디스크에서 슈퍼블록 데이터를 읽어 fs->sb 구조체에 저장한다.
  // 성공 시 0을 반환하며, 실패 시 음수를 반환한다.
  //
  // sb_load()가 실패하면 (즉, 음수 반환 시),
  // 이는 슈퍼블록이 존재하지 않거나 손상된 상태를 나타내므로,
  // 새로운 슈퍼블록을 포맷하고 초기화하는 과정을 수행해야 한다.
  if (sb_load(backing_fd, &fs->sb) < 0) {

    // 슈퍼블록 포맷: 슈퍼블록을 깨끗한 초기 상태로 설정.
    // 이 함수는 슈퍼블록의 모든 값을 기본값 또는 초기 상태로 설정한다.
    sb_format(&fs->sb);

    // 슈퍼블록에 파일 시스템의 전체 블록 개수(total_blocks)를 설정한다.
    // 이 값은 파일 시스템이 관리할 수 있는 블록의 최대 개수를 나타낸다.
    fs->sb.blocks_count = total_blocks;

    // inode 개수 설정: 파일 시스템이 관리할 수 있는 inode의 총 개수를 설정.
    // inode는 파일이나 디렉터리 같은 객체의 메타데이터를 저장하는 단위이다.
    fs->sb.inodes_count = SFUSE_INODES_COUNT;

    // --- 메타데이터 영역의 크기 계산 시작 ---

    // 1. 블록 비트맵(block bitmap)의 크기를 블록 단위로 계산
    //
    // 블록 비트맵은 각 블록이 사용 중인지 여부를 비트 단위로 관리한다.
    // 하나의 블록을 1비트(bit)로 관리하므로,
    // 전체 블록 수를 8로 나누면 필요한 바이트(byte) 수가 된다.
    //
    // 이 바이트 수를 블록 단위(SFUSE_BLOCK_SIZE)로 변환하기 위해 다음 수식을
    // 사용:
    //
```

```
// (fs->sb.blocks_count / 8 + SFUSE_BLOCK_SIZE - 1) / SFUSE_BLOCK_SIZE
//
// 이 수식은 '올림(ceil)'의 효과를 내는 표현으로,
// 정확히 나누어 떨어지지 않을 경우 필요한 블록을 추가로 확보하기 위한
// 계산법이다.
uint32_t block_bitmap_blocks =
    (fs->sb.blocks_count / 8 + SFUSE_BLOCK_SIZE - 1) / SFUSE_BLOCK_SIZE;

// 2. inode 비트맵(inode bitmap)의 크기를 블록 단위로 계산
//
// inode 비트맵은 inode들이 사용 중인지 여부를 관리하는 비트맵이다.
// 전체 inode 개수를 8로 나누면 바이트 수가 되며,
// 이것을 블록 단위로 변환하는 계산법은 블록 비트맵 계산법과 동일하다.
uint32_t inode_bitmap_blocks =
    (fs->sb.inodes_count / 8 + SFUSE_BLOCK_SIZE - 1) / SFUSE_BLOCK_SIZE;

// 3. inode 테이블(inode table)의 크기를 블록 단위로 계산
//
// inode 테이블은 실제 inode 구조체들을 저장하는 영역이다.
// 각 inode는 'struct sfuse_inode'의 크기만큼 메모리를 차지하므로,
// 전체 inode 개수에 inode 크기를 곱하면 inode 테이블의 총 크기가 바이트
// 단위로 나온다.
//
// 이것을 다시 블록 크기(SFUSE_BLOCK_SIZE)로 나누어 몇 개의 블록이 필요한지
// 계산한다. 여기서도 블록이 정수로 딱 떨어지지 않을 수 있으므로, 올림
// 효과를 내도록 처리한다.
uint32_t inode_table_blocks =
    (fs->sb.inodes_count * sizeof(struct sfuse_inode) + SFUSE_BLOCK_SIZE -
     1) /
    SFUSE_BLOCK_SIZE;

// 슈퍼블록에서 관리하는 각 영역의 위치(블록 번호)를 설정한다.
// SFUSE 파일 시스템의 디스크 레이아웃은 일반적으로 다음과 같다:
//
//    파일 시스템 블록 구조:
// +-------------------------+
// | 블록 0 (슈퍼블록)        |
// +-------------------------+
// | 블록 1 (예비 블록)       |
// +-------------------------+
// | 블록 2 ~ N               |
// | (block bitmap)           |
// | [블록 사용 여부 표시]    |
// +-------------------------+
// | inode bitmap             |
// | [inode 사용 여부 표시]   |
// +-------------------------+
// | inode tables             |
// | [파일 메타데이터 저장]   |
// +-------------------------+
// | data block               |
// | [실제 사용자 데이터 저장]|
// +-------------------------+
//
// block 비트맵의 시작 위치는 슈퍼블록(블록 0)과 예비 블록(블록 1) 뒤에 위치
fs->sb.block_bitmap_start = 2;

// inode 비트맵은 block 비트맵 다음에 위치
fs->sb.inode_bitmap_start = fs->sb.block_bitmap_start + block_bitmap_blocks;

// inode 테이블은 inode 비트맵 다음에 위치
fs->sb.inode_table_start = fs->sb.inode_bitmap_start + inode_bitmap_blocks;

// 실제 사용자 데이터 블록은 inode 테이블 다음에 위치
fs->sb.data_block_start = fs->sb.inode_table_start + inode_table_blocks;

// 데이터 블록 시작 위치 이후의 블록만 실제 데이터 저장을 위해 사용
// 가능하므로, 전체 블록 수(total_blocks)에서 데이터 블록 시작 위치를 빼면
// 여유 블록 수를 구할 수 있다.
fs->sb.free_blocks = total_blocks - fs->sb.data_block_start;

// inode 중 루트 디렉터리를 제외한 나머지 inode들이 사용 가능
fs->sb.free_inodes = fs->sb.inodes_count - 1;

// 위에서 설정한 슈퍼블록 데이터를 디스크에 즉시 동기화하여 저장
if (sb_sync(backing_fd, &fs->sb) < 0)
  return -EIO; // 동기화 실패 시 입출력 에러 반환

// ---- 메모리에 블록 비트맵과 inode 비트맵을 관리할 공간을 확보 ----

// 비트맵 크기 계산:
// 비트맵은 비트 단위로 블록이나 inode의 사용 여부를 관리한다.
// 1 바이트(byte)는 8 비트(bit)를 가지므로, 블록과 inode의 개수를 각각 8로
// 나누면 비트맵이 필요한 메모리 크기가 바이트 단위로 나온다.
size_t bmap_bytes = fs->sb.blocks_count / 8;
size_t imap_bytes = fs->sb.inodes_count / 8;
```

```
// 위에서 계산한 크기만큼 메모리를 할당하고, 그 공간을 0으로 초기화한다.
// calloc(1, size)는 size 크기만큼 메모리를 할당한 후 모든 비트를 0으로
// 설정한다. 이렇게 하면 비트맵이 초기 상태에서는 모든 블록과 inode가 "사용
// 가능"(free)으로 표시된다.
fs->block_map = calloc(1, bmap_bytes);
fs->inode_map = calloc(1, imap_bytes);

// 메모리 할당 실패 시 메모리 부족 에러 반환
if (!fs->block_map || !fs->inode_map)
  return -ENOMEM;

// 비트맵의 초기 상태(모든 블록 및 inode가 사용 가능 상태)를 디스크에 동기화
bitmap_sync(backing_fd, fs->sb.block_bitmap_start, fs->block_map,
            bmap_bytes);
bitmap_sync(backing_fd, fs->sb.inode_bitmap_start, fs->inode_map,
            imap_bytes);

// ---- 루트 디렉터리 inode 초기화 ----

// 루트 inode 번호 설정 (1번으로 정의됨)
uint32_t root = SFUSE_ROOT_INO;

// inode 비트맵에서 루트 inode를 사용 중으로 표시하는 코드
// 비트맵(bitmap)은 각 inode의 사용 여부를 관리하는 데이터 구조로,
// 1비트가 1개의 inode의 상태를 나타낸다.
// 비트값이 0이면 해당 inode는 사용 가능(free), 1이면 사용
// 중(allocated)이다.

// inode 비트맵은 바이트(byte) 배열 형태로 관리된다.
// 배열의 각 요소(바이트)마다 8개의 inode 상태를 표현할 수 있다.
// 특정 inode가 속한 바이트의 위치는 inode 번호를 8로 나눈 값(root / 8)이며,
// 바이트 내의 정확한 비트 위치는 inode 번호를 8로 나눈 나머지(root % 8)가
// 된다.

// 예시:
// 루트 inode(root)가 5라면, inode_map[0] (바이트의 첫 번째 요소)의 5번째
// 비트를 설정한다.

// 이 코드의 동작 방식:
// 1. (1 « (root % 8)): 1을 root % 8 만큼 왼쪽으로 시프트(shift)하여 원하는
// 비트 위치로 설정한다.
// 2. inode_map[root / 8] |= (...): OR 연산으로 해당 비트를 설정하여 inode를
// '사용 중'으로 표시한다.
//
// 비트 연산을 사용하는 이유는 매우 빠르고 공간을 절약하기 때문이다.
fs->inode_map[root / 8] |= (1 « (root % 8));

// 위에서 inode 비트맵에 루트 inode를 할당했기 때문에,
// 슈퍼블록에서 관리하는 사용 가능한(free) inode의 개수도 하나 감소시켜야
// 한다.
fs->sb.free_inodes--;

// 루트 inode 구조체 초기화 (초기화 전에 메모리 전체를 0으로 설정)
struct sfuse_inode root_inode;
memset(&root_inode, 0, sizeof(root_inode));

// 루트 inode 메타데이터 설정 (디렉터리, 접근 권한 0755, 소유자 UID/GID
// 설정)
fs_init_inode(&fs->sb, root, S_IFDIR | 0755, fuse_get_context()->uid,
              fuse_get_context()->gid, &root_inode);

// ---- 루트 디렉터리의 데이터 블록 할당 및 초기화 ----

// 블록 할당 함수를 호출하여, 첫 번째 데이터 블록 인덱스를 할당 받음
int blk_index = alloc_block(&fs->sb, fs->block_map);
if (blk_index < 0)
  return -ENOSPC; // 사용 가능한 블록 공간 부족 시 에러 반환

// 실제 블록 번호는 데이터 블록의 시작 위치에서 blk_index만큼 더한 값
uint32_t phys_block = fs->sb.data_block_start + blk_index;

// inode가 관리하는 직접(direct) 블록 포인터 중 첫 번째 항목을 설정
root_inode.direct[0] = phys_block;
root_inode.size =
    SFUSE_BLOCK_SIZE; // 루트 디렉터리 크기를 블록 크기만큼 설정

// 데이터 블록 내에 루트 디렉터리의 기본 엔트리("." 및 ".")를 생성하기 위한
// 초기화
uint8_t block[SFUSE_BLOCK_SIZE] = {0}; // 블록 크기만큼 0으로 초기화
struct sfuse_dirent *entries =
    (struct sfuse_dirent *)block; // 디렉터리 엔트리 구조체로 사용

// "."(현재 디렉터리)는 루트 inode를 가리킴
entries[0].ino = root;
strcpy(entries[0].name, ".");

// ".."(상위 디렉터리) 역시 루트 inode를 가리킴 (루트 디렉터리는 자신을
```

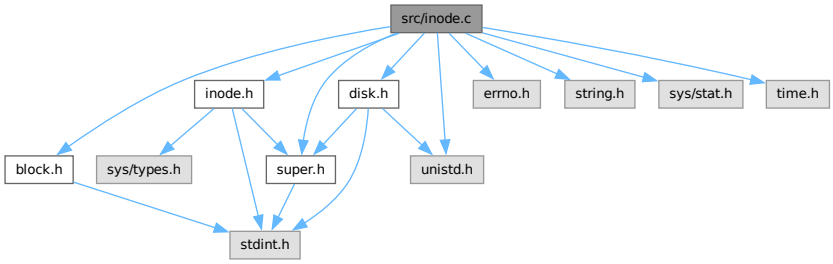
```
    // 가리킴)
    entries[1].ino = root;
    strcpy(entries[1].name, "..");

    // 구성된 디렉터리 데이터를 디스크의 실제 블록에 기록
    if (write_block(backing_fd, phys_block, block) < 0)
      return -EIO; // 블록 기록 실패 시 입출력 에러 반환

    // 구성된 루트 inode를 디스크에 저장하여 동기화
    if (inode_sync(backing_fd, &fs->sb, root, &root_inode) < 0)
      return -EIO; // inode 동기화 실패 시 입출력 에러 반환

    // ---- 모든 메타데이터의 최종 상태를 디스크에 동기화 ----

    // 블록 비트맵과 inode 비트맵의 최종 상태를 디스크에 저장
    bitmap_sync(backing_fd, fs->sb.block_bitmap_start, fs->block_map,
                bmap_bytes);
    bitmap_sync(backing_fd, fs->sb.inode_bitmap_start, fs->inode_map,
                imap_bytes);

    // 슈퍼블록의 최종 상태를 디스크에 저장하여 파일 시스템 초기화 완료
    sb_sync(backing_fd, &fs->sb);

  } else {
    // 슈퍼블록이 이미 존재(정상 로드 성공) 시, 기존 메타데이터를 디스크에서
    // 로드

    size_t bmap_bytes = fs->sb.blocks_count / 8;
    size_t imap_bytes = fs->sb.inodes_count / 8;

    fs->block_map = calloc(1, bmap_bytes);
    fs->inode_map = calloc(1, imap_bytes);

    // 기존 비트맵 데이터를 디스크에서 메모리로 로드하여 파일 시스템 재구성
    bitmap_load(backing_fd, fs->sb.block_bitmap_start, fs->block_map,
                bmap_bytes);
    bitmap_load(backing_fd, fs->sb.inode_bitmap_start, fs->inode_map,
                imap_bytes);
  }

  // 초기화 과정이 모두 정상적으로 완료되었으므로 성공(0)을 반환
  return 0;
}

void fs_destroy(void *private_data) {
  // 전달받은 private_data 포인터를 struct sfuse_fs 타입으로 변환
  struct sfuse_fs *fs = private_data;

  // 비트맵 데이터를 디스크에 동기화하기 위한 메모리 크기 계산
  // 블록 비트맵 크기: 전체 블록 수(blocks_count)를 비트맵 표현을 위해 바이트로
  // 환산
  size_t bmap_bytes = fs->sb.blocks_count / 8;

  // inode 비트맵 크기: 전체 inode 수(inodes_count)를 비트맵 표현을 위해
  // 바이트로 환산
  size_t imap_bytes = fs->sb.inodes_count / 8;

  // 메모리에 존재하는 블록 비트맵 상태를 디스크에 동기화한다.
  // 이 작업을 통해 파일 시스템 종료 시 블록의 사용 상태가 디스크에 정확히
  // 반영된다.
  bitmap_sync(fs->backing_fd, fs->sb.block_bitmap_start, fs->block_map,
              bmap_bytes);

  // 메모리에 존재하는 inode 비트맵 상태를 디스크에 동기화한다.
  // 파일 시스템 종료 시 inode의 사용 여부를 디스크에 저장한다.
  bitmap_sync(fs->backing_fd, fs->sb.inode_bitmap_start, fs->inode_map,
              imap_bytes);

  // 슈퍼블록 상태를 디스크에 동기화하여 최신의 파일 시스템 메타데이터를
  // 유지한다.
  sb_sync(fs->backing_fd, &fs->sb);

  // 파일 시스템의 종료 작업 이후 메모리에 할당된 비트맵 메모리를 해제하여,
  // 메모리 누수를 방지한다.
  free(fs->block_map); // 블록 비트맵 메모리 해제
  free(fs->inode_map); // inode 비트맵 메모리 해제
}

int fs_resolve_path(struct sfuse_fs *fs, const char *path, uint32_t *ino_out) {
  // 시작 지점을 루트 디렉터리의 inode로 설정
  uint32_t cur = SFUSE_ROOT_INO;

  // 예외 처리: 빈 문자열 또는 루트 디렉터리('/')가 들어오면 바로 루트 inode
  // 반환
  if (!path || path[0] == '\0' || strcmp(path, "/") == 0) {
    *ino_out = cur;
    return 0;
```

```
}

// 경로 문자열을 수정 가능하도록 복제(메모리 복사)
char *dup = strdup(path);
if (!dup)
  return -ENOMEM; // 메모리 부족 시 에러 반환

// 경로 문자열을 "/" 기준으로 토큰(token)으로 나눈다.
// 예시: "/home/user/file" → ["home", "user", "file"]
char *token = strtok(dup, "/");

// 모든 토큰에 대해 순차적으로 디렉터리를 탐색
while (token) {
  // 디렉터리 데이터를 메모리에 로드하기 위한 버퍼 크기를 계산한다.
  // 해당 버퍼는 디렉터리 엔트리 정보를 저장하는 데 사용된다.
  // SFUSE_NDIR_BLOCKS: 디렉터리 inode가 직접 참조할 수 있는 블록의 최대 개수.
  //                    직접 참조 블록 포인터 수 (12개)
  // SFUSE_BLOCK_SIZE:  각 블록의 크기(바이트 단위)()
  //
  // 따라서 디렉터리 데이터를 로드할 수 있는 전체 버퍼 크기는 다음과 같다:
  //
  //   buf_size = (직접 참조 블록 개수) × (각 블록 크기)
  //
  // 수식으로 표현하면:
  //   buf_size = SFUSE_NDIR_BLOCKS * SFUSE_BLOCK_SIZE;
  //
  // 예시:
  //   만약 SFUSE_NDIR_BLOCKS가 12이고 SFUSE_BLOCK_SIZE가 1024 바이트라면,
  //   buf_size = 12 × 1024 = 12288 바이트(12KB)가 된다.
  size_t buf_size = SFUSE_NDIR_BLOCKS * SFUSE_BLOCK_SIZE;
  void *buf = malloc(buf_size);
  if (!buf) {
    free(dup);
    return -ENOMEM; // 메모리 부족 시 에러 반환
  }

  // 현재 디렉터리의 모든 엔트리(entry)를 디스크에서 메모리로 로드한다.
  // dir_load는 cur(현재 inode 번호)에 해당하는 디렉터리 데이터를 버퍼에
  // 채운다.
  if (dir_load(fs->backing_fd, &fs->sb, cur, buf) < 0) {
    free(buf);
    free(dup);
    return -ENOENT; // 디렉터리를 읽을 수 없는 경우 존재하지 않음 에러 반환
  }

  // 버퍼의 데이터를 디렉터리 엔트리 배열로 캐스팅하여 접근 가능하도록 설정
  struct sfuse_dirent *ents = (struct sfuse_dirent *)buf;

  // 디렉터리 엔트리의 최대 개수를 계산
  size_t max_entries = buf_size / sizeof(*ents);

  // 다음 디렉터리 또는 파일의 inode 번호를 찾기 위한 임시 변수
  uint32_t next_ino = 0;

  // 현재 디렉터리 내의 엔트리 중에서 이름이 토큰과 일치하는 엔트리를 찾는다.
  for (size_t i = 0; i < max_entries; i++) {
    // ents[i].ino가 0이면 빈 엔트리이므로 무시
    // 토큰과 같은 이름의 엔트리를 찾으면 next_ino에 inode 번호 저장 후 반복문
    // 종료
    if (ents[i].ino != 0 && strcmp(ents[i].name, token) == 0) {
      next_ino = ents[i].ino;
      break;
    }
  }

  // 버퍼 사용이 끝났으므로 메모리 해제
  free(buf);

  // 해당하는 이름을 가진 엔트리를 찾지 못한 경우, 경로가 존재하지 않음을 의미
  if (next_ino == 0) {
    free(dup);
    return -ENOENT; // 경로 존재하지 않음 에러 반환
  }

  // 다음 토큰 탐색을 위해 현재 inode 번호를 방금 찾은 inode 번호로 업데이트
  cur = next_ino;

  // 다음 토큰으로 넘어감
  token = strtok(NULL, "/");
}

// 모든 토큰 탐색이 완료되면, 최종 inode 번호를 결과로 반환
free(dup);      // 경로 문자열 복제본의 메모리 해제
*ino_out = cur; // 최종 inode 번호 저장
return 0;       // 성공적으로 경로를 해석함
}
```

```
char *fs_split_path(const char *fullpath, uint32_t *parent_ino) {
  char *dup = strdup(fullpath); // 경로를 수정하기 위해 복제본 생성
  if (!dup)
    return NULL; // 메모리 부족 시 실패 처리

  char *slash = strrchr(dup, '/'); // 마지막 '/' 문자를 찾아 경로 분리 지점 설정
  char *name;

  if (!slash) {
    // '/' 문자가 없을 경우 (예: "file.txt")
    *parent_ino = SFUSE_ROOT_INO; // 부모는 루트 디렉터리로 설정
    name = dup;                   // 전체 경로가 곧 파일명
  } else if (slash == dup) {
    // 경로가 루트에 바로 붙은 경우 (예: "/file.txt")
    *parent_ino = SFUSE_ROOT_INO; // 부모는 루트 디렉터리
    name = strdup(slash + 1);     // '/' 다음부터 이름 복사 ("file.txt")
    free(dup);                    // 최초 복제한 문자열은 더 이상 필요 없음
  } else {
    // 그 외 일반적인 경우 (예: "/home/user/file.txt")
    *slash =
        '\0'; // '/' 위치에서 문자열을 종료시켜 부모 경로만 남김 ("/home/user")

    // 남은 부모 경로(dup)를 이용하여 부모 디렉터리의 inode 번호를 찾는다.
    if (fs_resolve_path(get_fs_context(), dup, parent_ino) < 0) {
      free(dup);   // 부모 경로 탐색 실패 시 메모리 해제
      return NULL; // 실패 반환
    }

    // '/' 뒤의 문자열("file.txt")을 이름으로 복사하여 반환
    name = strdup(slash + 1);
    free(dup); // 부모 경로 탐색 완료 후 메모리 해제
  }

  return name; // 호출한 곳에서 반드시 free(name) 수행 필요
}
```

8.32 src/inode.c 파일 참조

아이노드 로드, 동기화 및 블록 주소 변환 함수 구현
#include "inode.h"
#include "block.h"
#include "disk.h"
#include "super.h"
#include <errno.h>
#include <string.h>
#include <sys/stat.h>
#include <time.h>
#include <unistd.h>
inode.c에 대한 include 의존 그래프



함수

- void fs_init_inode (const struct sfuse_super *sb, uint32_t ino, mode_t mode, uid_t uid, gid_t gid, struct sfuse_inode *inode)

  < 파일 타입 매크로 (S_ISDIR)

- int inode_load (int fd, const struct sfuse_super *sb, uint32_t ino, struct sfuse_inode *inode)

  아이노드를 디스크에서 읽어 메모리에 로드한다.
- int inode_sync (int fd, const struct sfuse_super *sb, uint32_t ino, const struct sfuse_inode *inode)

  메모리에 저장된 아이노드를 디스크에 기록(동기화)한다.
- int logical_to_physical (int fd, const struct sfuse_super *sb, const struct sfuse_inode *inode, uint32_t lbn, void *buf, uint32_t *pbn_out)

  논리 블록 번호(Logical Block Number, LBN)를 물리 블록 번호(Physical Block Number, PBN)로 변환한다.

8.32.1 상세한 설명

아이노드 로드, 동기화 및 블록 주소 변환 함수 구현
이 파일은 SFUSE 파일 시스템에서 아이노드를 디스크로부터 읽거나 기록하는 함수, 아이노드를 초기화하는 함수, 논리적 블록 번호를 물리적 블록 번호로 변환하는 함수의 구현을 제공한다.
inode.c 파일에서 정의되었습니다.

8.32.2 함수 문서화

8.32.2.1 fs_init_inode()

```
void fs_init_inode (
            const struct sfuse_super * sb,
            uint32_t ino,
            mode_t mode,
            uid_t uid,
            gid_t gid,
            struct sfuse_inode * inode)
```

< 파일 타입 매크로 (S_ISDIR)
새로운 아이노드를 기본 값으로 초기화한다.
< 블록 읽기/쓰기 (read_block/write_block) 디스크 읽기/쓰기 함수 (disk_read/disk_write)
새로운 아이노드를 기본 값으로 초기화한다.
이 함수는 파일 시스템 내에서 새로운 아이노드를 생성할 때 호출된다. 아이노드는 파일 또는 디렉터리와 같은 객체의 메타데이터를 관리하는 핵심적인 구조체로, 이 함수는 아이노드를 사용 가능한 기본값으로 설정하여 초기 상태를 정의한다.
세부적으로는 다음의 작업을 수행한다:

- 아이노드 메모리 영역을 모두 0으로 초기화한다.
- 파일의 접근 권한(mode), 소유자(uid), 그룹(gid)을 설정한다.
- 파일 크기를 0으로 설정한다 (새로 생성된 파일이므로 초기 크기는 0이다).
- 접근(atime), 수정(mtime), 상태 변경(ctime) 시간을 현재 시각으로 설정한다.
- 모든 Direct 블록 포인터를 0으로 초기화하여, 아직 데이터를 가리키지 않도록 설정한다.
- Single indirect, Double indirect 블록 포인터 역시 0으로 설정하여, 추가적인 데이터 블록 참조가 없음을 나타낸다.
- 링크 수(links)를 초기화한다. 만약 디렉터리라면 기본적으로 2개(자기 자신 '.'과 상위 디렉터리 '..'), 파일이라면 1개로 설정한다.

매개변수

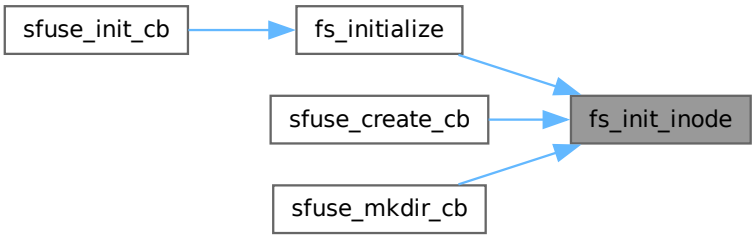
sb	슈퍼블록 정보 포인터 (미사용 파라미터)
ino	초기화할 아이노드 번호 (미사용 파라미터)
mode	새 아이노드의 파일 타입(파일/디렉터리 등)과 접근 권한
uid	새 아이노드를 소유할 사용자의 사용자 ID

gid	새 아이노드를 소유할 사용자의 그룹 ID
inode	실제로 초기화할 아이노드 구조체 포인터

inode.c 파일의 47 번째 라인에서 정의되었습니다.

```
00048                                                   {
00049   (void)sb;  // 미사용 파라미터로 인한 컴파일러 경고 방지
00050   (void)ino; // 미사용 파라미터로 인한 컴파일러 경고 방지
00051
00052   // inode 메모리를 0으로 초기화하여 깨끗한 상태로 시작
00053   memset(inode, 0, sizeof(*inode));
00054
00055   // 파일 타입과 접근 권한(mode)을 설정
00056   inode->mode = mode;
00057
00058   // 아이노드 소유자의 사용자(uid)와 그룹(gid) ID를 설정
00059   inode->uid = uid;
00060   inode->gid = gid;
00061
00062   // 새로 생성된 파일/디렉터리이므로 크기를 0으로 설정
00063   inode->size = 0;
00064
00065   // 접근(atime), 수정(mtime), 상태 변경(ctime) 시간을 현재 시각으로 초기화
00066   inode->atime = inode->mtime = inode->ctime = (uint32_t)time(NULL);
00067
00068   // Direct 블록 포인터를 모두 0으로 초기화하여 데이터 블록을 아직 가리키지 않음
00069   for (int i = 0; i < SFUSE_NDIR_BLOCKS; i++)
00070     inode->direct[i] = 0;
00071
00072   // Single indirect, Double indirect 블록 포인터도 데이터가 없음을 나타내기
00073   // 위해 0 설정
00074   inode->indirect = 0;
00075   inode->double_indirect = 0;
00076
00077   // 디렉터리면 기본 링크 수를 2로, 파일이면 1로 설정
00078   inode->links = (S_ISDIR(mode) ? 2 : 1);
00079 }
```

이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.32.2.2 inode_load()

```
int inode_load (
                int fd,
                const struct sfuse_super * sb,
                uint32_t ino,
                struct sfuse_inode * inode)
```

아이노드를 디스크에서 읽어 메모리에 로드한다.
디스크에서 지정된 아이노드를 읽어 메모리에 로드한다.
파일 시스템에서 파일과 디렉터리의 메타데이터(크기, 권한, 소유자, 데이터 블록 위치 등)는 아이노드(inode)에 저장된다. 이 함수는 디스크에 저장된 아이노드 데이터를 읽어 메모리의 inode 구조체에 로드한다.
아이노드를 로드하는 과정은 다음과 같다:

1. 아이노드 번호(ino)의 유효성을 검사한다.
   - ino 번호는 0이 아니어야 하고, 슈퍼블록의 inodes_count보다 작아야 한다.
2. 디스크 내의 아이노드 위치(offset)를 계산한다.
   - 슈퍼블록의 inode_table_start는 아이노드 테이블의 시작 위치를 나타내는 블록 번호이다.
   - 아이노드의 크기(sizeof(*inode))에 아이노드 번호를 곱해 정확한 위치를 계산한다.
3. 계산된 위치에서 아이노드 데이터를 읽어 inode 버퍼에 저장한다.
   - 읽은 데이터의 크기가 아이노드 구조체 크기와 일치하지 않으면 입출력 오류(EIO)를 반환한다.

매개변수

fd	디바이스 파일 디스크립터
sb	슈퍼블록 정보 포인터 (아이노드 테이블의 위치와 총 개수 정보 포함)
ino	읽을 아이노드 번호 (0번 아이노드는 무효로 간주됨)
inode	디스크에서 읽은 아이노드를 저장할 메모리 버퍼 포인터

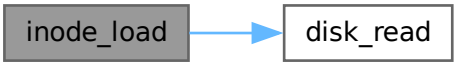
반환값

성공 시 0 반환, 실패 시 음수의 오류 코드 반환 -EINVAL : 유효하지 않은 아이노드 번호(ino가 0 이거나 유효 범위를 초과) -EIO : 아이노드 데이터를 읽는 중 입출력 오류 발생

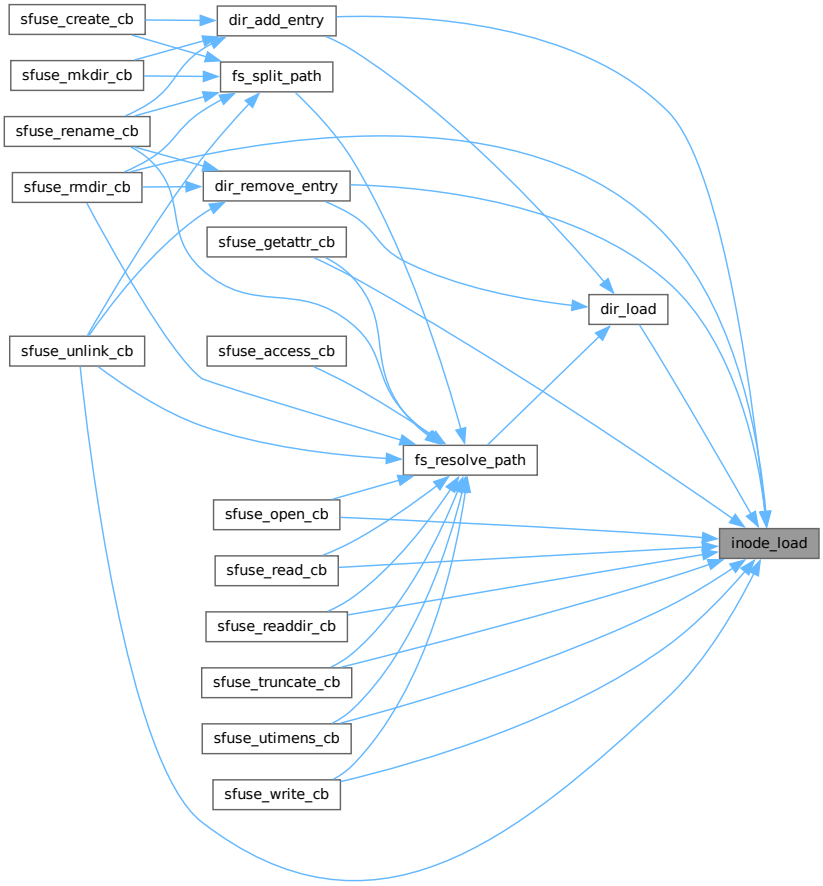
inode.c 파일의 113 번째 라인에서 정의되었습니다.

```
{
/* [1단계] 아이노드 번호(ino) 유효성 검사 */
if (ino == 0 || ino >= sb->inodes_count)
  return -EINVAL; // ino가 유효하지 않음 (0이거나 최대 범위 초과)

/* [2단계] 디스크에서 아이노드가 저장된 위치(offset) 계산 */
off_t off = ((off_t)sb->inode_table_start * SFUSE_BLOCK_SIZE) +
            (ino * sizeof(*inode));
/*
 * inode_table_start 블록 번호를 실제 바이트 단위로 변환하기 위해 블록
 * 크기(SFUSE_BLOCK_SIZE)를 곱함. 거기에 inode 번호(ino)에 inode
 * 크기(sizeof(*inode))를 곱하여 정확한 위치 계산.
 */

/* [3단계] 디스크에서 아이노드 데이터를 읽어 메모리에 로드 */
ssize_t ret = disk_read(fd, inode, sizeof(*inode), off);
if (ret < 0)
  return (int)ret; // 디스크 읽기 오류 발생 시 해당 오류 코드 반환

// 읽은 데이터 크기가 아이노드 크기와 불일치 시 입출력 오류 반환
if ((size_t)ret != sizeof(*inode))
  return -EIO;

return 0; // 성공적으로 아이노드 로드 완료
}
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.32.2.3 inode_sync()

```
int inode_sync (
            int fd,
            const struct sfuse_super * sb,
            uint32_t ino,
            const struct sfuse_inode * inode)
```

메모리에 저장된 아이노드를 디스크에 기록(동기화)한다.
메모리에 저장된 아이노드 정보를 디스크에 기록한다.
파일 시스템은 아이노드를 메모리 내에서 수정한 후, 변경된 내용을 디스크에 영구적으로 저장해야 한다.
이 함수는 메모리 상의 아이노드를 디스크의 올바른 위치에 정확하게 기록하여 파일의 메타데이터가 유지되도록 한다.
아이노드를 디스크에 기록하는 과정은 다음과 같다:

1. 아이노드 번호(ino)의 유효성을 검사한다.
   - ino 번호는 반드시 0보다 크고 슈퍼블록의 총 아이노드 개수(inodes_count)보다 작아야 한다.
2. 기록할 아이노드 데이터의 디스크 내 위치(offset)를 계산한다.
   - 아이노드 테이블의 시작 위치(inode_table_start)를 블록 단위에서 바이트 단위로 변환하고, 여기에 아이노드 번호(ino)와 아이노드의 크기(sizeof(*inode))를 곱하여 정확한 위치를 얻는다.
3. 메모리 상의 아이노드 데이터를 계산된 위치에 디스크로 기록한다.

- 기록 후, 실제로 기록된 바이트 수가 아이노드의 크기와 정확히 일치하는지 확인한다.
- 기록된 바이트 수가 일치하지 않거나 오류가 발생하면 적절한 오류 코드를 반환한다.

매개변수

fd	디바이스 파일 디스크립터 (디스크 접근을 위한 식별자)
sb	슈퍼블록 정보 (아이노드 테이블 위치 정보 포함)
ino	디스크에 기록할 아이노드 번호
inode	메모리에 있는, 디스크에 기록될 아이노드 데이터 포인터

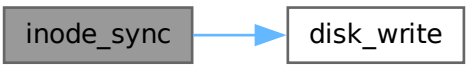
반환값

성공 시 0 반환, 실패 시 음수의 오류 코드 반환 -EINVAL : 유효하지 않은 아이노드 번호(0이거나 범위 초과) -EIO : 디스크 기록 과정에서 입출력 오류가 발생

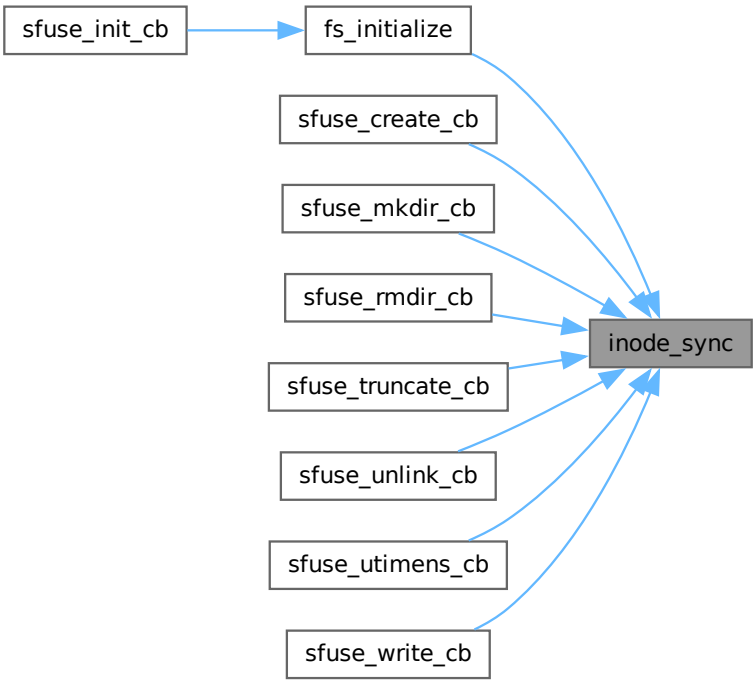
inode.c 파일의 173 번째 라인에서 정의되었습니다.

```
00174                                   {
00175   /* [1단계] 아이노드 번호(ino)의 유효성 검사 */
00176   if (ino == 0 || ino >= sb->inodes_count)
00177     return -EINVAL; // 아이노드 번호가 유효하지 않을 경우
00178
00179   /* [2단계] 아이노드 데이터를 기록할 디스크의 정확한 위치(offset) 계산 */
00180   off_t off = ((off_t)sb->inode_table_start * SFUSE_BLOCK_SIZE) +
00181               (ino * sizeof(*inode));
00182   /*
00183    * inode_table_start는 블록 번호를 나타내므로 바이트 단위로 변환하기 위해
00184    * SFUSE_BLOCK_SIZE를 곱한다.
00185    * 아이노드 번호(ino)에 아이노드 크기를 곱해 정확한 바이트 오프셋을 결정한다.
00186    */
00187
00188   /* [3단계] 메모리의 아이노드를 디스크의 해당 위치에 기록 */
00189   ssize_t ret = disk_write(fd, inode, sizeof(*inode), off);
00190   if (ret < 0)
00191     return (int)ret; // 디스크 기록 오류가 발생했을 때 오류 코드 반환
00192
00193   if ((size_t)ret != sizeof(*inode))
00194     return -EIO; // 기록된 데이터의 크기가 아이노드 크기와 다르면 입출력 오류
00195                  // 반환
00196
00197   return 0; // 성공적으로 아이노드를 디스크에 기록함
00198 }
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.32.2.4 logical_to_physical()

```
int logical_to_physical (
            int fd,
            const struct sfuse_super * sb,
            const struct sfuse_inode * inode,
            uint32_t lbn,
            void * buf,
            uint32_t * pbn_out)
```

논리 블록 번호(Logical Block Number, LBN)를 물리 블록 번호(Physical Block Number, PBN)로 변환한다.
논리 블록 번호(Logical Block Number)를 물리 블록 번호(Physical Block Number)로 변환한다.
파일은 여러 블록에 걸쳐 데이터를 저장하며, 파일 내에서의 블록 위치는 논리적(LBN)으로 표현된다. 그러나 실제 디스크는 논리 블록을 직접적으로 알지 못하므로, 물리적인 블록 번호(PBN)로 변환해 접근해야 한다. 이 함수는 아이노드에 저장된 블록 주소를 참조하여 LBN을 PBN으로 변환해주는 역할을 수행한다.
LBN에서 PBN 변환 방법:

- [Direct Block 처리]
  - LBN이 SFUSE_NDIR_BLOCKS(보통 12개)보다 작으면, direct 배열에서 직접 찾아 반환한다.
- [Single Indirect Block 처리]
  - Direct 블록으로 표현할 수 있는 범위를 넘어가면, Single indirect 블록에서 찾는다.
  - Single indirect 블록이 가리키는 블록 하나에는 여러 개의 블록 주소를 담을 수 있다.

- [Double Indirect Block 처리]
  - Single indirect 블록 범위도 초과하면, Double indirect 블록을 참조해야 한다.
  - Double indirect 블록은 두 단계의 포인터를 거쳐 주소를 참조한다.

매개변수

fd	디바이스 파일 디스크립터 (디스크 접근용)
sb	슈퍼블록 정보 (현재 사용하지 않음)
inode	대상 파일의 아이노드 포인터
lbn	변환하려는 논리 블록 번호
buf	블록 데이터를 읽기 위한 임시 버퍼 (크기: SFUSE_BLOCK_SIZE)
pbn_out	변환된 물리 블록 번호를 저장할 출력 포인터

반환값

성공 시 0, 실패 시 음수의 오류 코드 반환 -ENOENT : 요청한 블록이 할당되지 않아 존재하지 않음 -EIO : 블록 데이터를 디스크에서 읽을 때 입출력 오류 발생

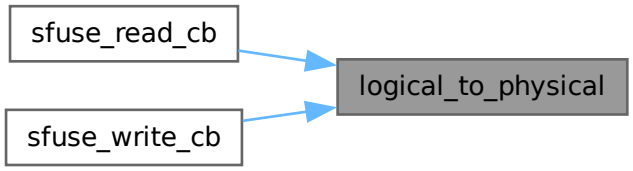
inode.c 파일의 235 번째 라인에서 정의되었습니다.

```
{

(void)sb; // 미사용 파라미터로 인한 컴파일러 경고 방지

/* --- [1단계: Direct 블록 확인] --- */
if (lbn < SFUSE_NDIR_BLOCKS) {
  // 요청한 논리 블록 번호가 Direct 블록의 범위 내에 있음
  *pbn_out = inode->direct[lbn]; // 직접적인 배열 참조로 즉시 얻을 수 있음
  return 0;
}

/* --- [2단계: Single Indirect 블록 확인] --- */
lbn -= SFUSE_NDIR_BLOCKS; // Direct 블록의 범위를 초과했으므로, Single
                          // indirect 범위로 보정

uint32_t entries = SFUSE_BLOCK_SIZE / sizeof(uint32_t);
// 하나의 블록에 저장 가능한 블록 주소의 개수를 계산
// (예: 4096 바이트 블록 / 4 바이트 주소 = 1024개의 블록 주소 저장 가능)

if (lbn < entries) {
  // Single indirect 블록에서 찾을 수 있는 범위 내에 있음
  if (inode->indirect == 0)
    return -ENOENT; // Single indirect 블록 자체가 아직 할당되지 않은 상태

  // indirect 블록 번호가 존재하면, 해당 블록을 디스크에서 읽는다
  if (read_block(fd, inode->indirect, buf) < 0)
    return -EIO; // indirect 블록을 읽는 도중 오류가 발생하면 입출력 오류 반환

  // indirect 블록에서 블록 주소 배열을 가져옴
  uint32_t *ptrs = (uint32_t *)buf;
  *pbn_out = ptrs[lbn]; // indirect 블록의 주소 배열에서 요청한 논리 블록의
                        // 물리 블록 번호를 얻음
  return 0;
}

/* --- [3단계: Double Indirect 블록 확인] --- */
lbn -= entries; // Single indirect 블록의 범위를 초과하였으므로, Double
                // indirect 블록을 처리하기 위해 보정

uint32_t idx1 = lbn / entries; // Double indirect 첫 번째 레벨의 인덱스 계산
uint32_t idx2 = lbn % entries; // Double indirect 두 번째 레벨의 인덱스 계산

if (inode->double_indirect == 0)
  return -ENOENT; // Double indirect 블록 자체가 할당되지 않은 상태

// double indirect의 첫 번째 레벨 블록을 읽는다.
if (read_block(fd, inode->double_indirect, buf) < 0)
  return -EIO; // 첫 번째 레벨 블록을 읽을 때 오류 발생 시 입출력 오류 반환

uint32_t *ptrs1 = (uint32_t *)buf;
uint32_t block2 = ptrs1[idx1]; // 두 번째 레벨 블록 번호를 얻음

if (block2 == 0)
  return -ENOENT; // 두 번째 레벨 블록이 할당되지 않은 경우
```

```

  // 두 번째 레벨 블록을 디스크에서 읽는다.
  if (read_block(fd, block2, buf) < 0)
    return -EIO; // 두 번째 레벨 블록을 읽을 때 오류 발생 시 입출력 오류 반환

  uint32_t *ptrs2 = (uint32_t *)buf;
  *pbn_out = ptrs2[idx2]; // 최종적으로 변환된 물리 블록 번호를 얻음

  return 0; // 성공적으로 물리 블록 번호 변환 완료
}
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:

이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.33 inode.c

이 파일의 문서화 페이지로 가기

```

#include "inode.h"
#include "block.h"
#include "disk.h"
#include "super.h"
#include <errno.h>
#include <string.h>
#include <sys/stat.h>
#include <time.h>
#include <unistd.h>

void fs_init_inode(const struct sfuse_super *sb, uint32_t ino, mode_t mode,
                   uid_t uid, gid_t gid, struct sfuse_inode *inode) {
  (void)sb;  // 미사용 파라미터로 인한 컴파일러 경고 방지
  (void)ino; // 미사용 파라미터로 인한 컴파일러 경고 방지

  // inode 메모리를 0으로 초기화하여 깨끗한 상태로 시작
  memset(inode, 0, sizeof(*inode));

  // 파일 타입과 접근 권한(mode)을 설정
  inode->mode = mode;

  // 아이노드 소유자의 사용자(uid)와 그룹(gid) ID를 설정
  inode->uid = uid;
  inode->gid = gid;

  // 새로 생성된 파일/디렉터리이므로 크기를 0으로 설정
  inode->size = 0;

```

```
  // 접근(atime), 수정(mtime), 상태 변경(ctime) 시간을 현재 시각으로 초기화
  inode->atime = inode->mtime = inode->ctime = (uint32_t)time(NULL);

  // Direct 블록 포인터를 모두 0으로 초기화하여 데이터 블록을 아직 가리키지 않음
  for (int i = 0; i < SFUSE_NDIR_BLOCKS; i++)
    inode->direct[i] = 0;

  // Single indirect, Double indirect 블록 포인터도 데이터가 없음을 나타내기
  // 위해 0 설정
  inode->indirect = 0;
  inode->double_indirect = 0;

  // 디렉터리면 기본 링크 수를 2로, 파일이면 1로 설정
  inode->links = (S_ISDIR(mode) ? 2 : 1);
}

int inode_load(int fd, const struct sfuse_super *sb, uint32_t ino,
               struct sfuse_inode *inode) {
  /* [1단계] 아이노드 번호(ino) 유효성 검사 */
  if (ino == 0 || ino >= sb->inodes_count)
    return -EINVAL; // ino가 유효하지 않음 (0이거나 최대 범위 초과)

  /* [2단계] 디스크에서 아이노드가 저장된 위치(offset) 계산 */
  off_t off = ((off_t)sb->inode_table_start * SFUSE_BLOCK_SIZE) +
              (ino * sizeof(*inode));
  /*
   * inode_table_start 블록 번호를 실제 바이트 단위로 변환하기 위해 블록
   * 크기(SFUSE_BLOCK_SIZE)를 곱함. 거기에 inode 번호(ino)에 inode
   * 크기(sizeof(*inode))를 곱하여 정확한 위치 계산.
   */

  /* [3단계] 디스크에서 아이노드 데이터를 읽어 메모리에 로드 */
  ssize_t ret = disk_read(fd, inode, sizeof(*inode), off);
  if (ret < 0)
    return (int)ret; // 디스크 읽기 오류 발생 시 해당 오류 코드 반환

  // 읽은 데이터 크기가 아이노드 크기와 불일치 시 입출력 오류 반환
  if ((size_t)ret != sizeof(*inode))
    return -EIO;

  return 0; // 성공적으로 아이노드 로드 완료
}

int inode_sync(int fd, const struct sfuse_super *sb, uint32_t ino,
               const struct sfuse_inode *inode) {
  /* [1단계] 아이노드 번호(ino)의 유효성 검사 */
  if (ino == 0 || ino >= sb->inodes_count)
    return -EINVAL; // 아이노드 번호가 유효하지 않을 경우

  /* [2단계] 아이노드 데이터를 기록할 디스크의 정확한 위치(offset) 계산 */
  off_t off = ((off_t)sb->inode_table_start * SFUSE_BLOCK_SIZE) +
              (ino * sizeof(*inode));
  /*
   * inode_table_start는 블록 번호를 나타내므로 바이트 단위로 변환하기 위해
   * SFUSE_BLOCK_SIZE를 곱한다.
   * 아이노드 번호(ino)에 아이노드 크기를 곱해 정확한 바이트 오프셋을 결정한다.
   */

  /* [3단계] 메모리의 아이노드를 디스크의 해당 위치에 기록 */
  ssize_t ret = disk_write(fd, inode, sizeof(*inode), off);
  if (ret < 0)
    return (int)ret; // 디스크 기록 오류가 발생했을 때 오류 코드 반환

  if ((size_t)ret != sizeof(*inode))
    return -EIO; // 기록된 데이터의 크기가 아이노드 크기와 다르면 입출력 오류
                 // 반환

  return 0; // 성공적으로 아이노드를 디스크에 기록함
}

int logical_to_physical(int fd, const struct sfuse_super *sb,
                        const struct sfuse_inode *inode, uint32_t lbn,
                        void *buf, uint32_t *pbn_out) {

  (void)sb; // 미사용 파라미터로 인한 컴파일러 경고 방지

  /* --- [1단계: Direct 블록 확인] --- */
  if (lbn < SFUSE_NDIR_BLOCKS) {
    // 요청한 논리 블록 번호가 Direct 블록의 범위 내에 있음
    *pbn_out = inode->direct[lbn]; // 직접적인 배열 참조로 즉시 얻을 수 있음
    return 0;
  }

  /* --- [2단계: Single Indirect 블록 확인] --- */
  lbn -= SFUSE_NDIR_BLOCKS; // Direct 블록의 범위를 초과했으므로, Single
                            // indirect 범위로 보정
```

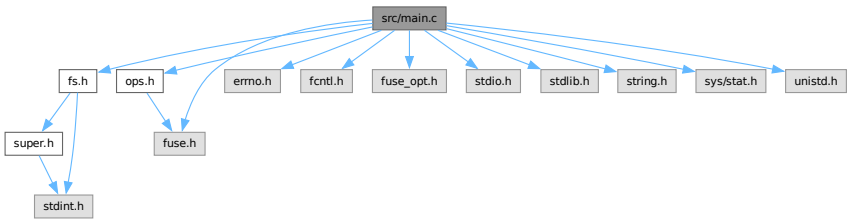
```
uint32_t entries = SFUSE_BLOCK_SIZE / sizeof(uint32_t);
// 하나의 블록에 저장 가능한 블록 주소의 개수를 계산
// (예: 4096 바이트 블록 / 4 바이트 주소 = 1024개의 블록 주소 저장 가능)

if (lbn < entries) {
  // Single indirect 블록에서 찾을 수 있는 범위 내에 있음
  if (inode->indirect == 0)
    return -ENOENT; // Single indirect 블록 자체가 아직 할당되지 않은 상태

  // indirect 블록 번호가 존재하면, 해당 블록을 디스크에서 읽는다
  if (read_block(fd, inode->indirect, buf) < 0)
    return -EIO; // indirect 블록을 읽는 도중 오류가 발생하면 입출력 오류 반환

  // indirect 블록에서 블록 주소 배열을 가져옴
  uint32_t *ptrs = (uint32_t *)buf;
  *pbn_out = ptrs[lbn]; // indirect 블록의 주소 배열에서 요청한 논리 블록의
                        // 물리 블록 번호를 얻음
  return 0;
}

/* --- [3단계: Double Indirect 블록 확인] --- */
lbn -= entries; // Single indirect 블록의 범위를 초과하였으므로, Double
                // indirect 블록을 처리하기 위해 보정

uint32_t idx1 = lbn / entries; // Double indirect 첫 번째 레벨의 인덱스 계산
uint32_t idx2 = lbn % entries; // Double indirect 두 번째 레벨의 인덱스 계산

if (inode->double_indirect == 0)
  return -ENOENT; // Double indirect 블록 자체가 할당되지 않은 상태

// double indirect의 첫 번째 레벨 블록을 읽는다.
if (read_block(fd, inode->double_indirect, buf) < 0)
  return -EIO; // 첫 번째 레벨 블록을 읽을 때 오류 발생 시 입출력 오류 반환

uint32_t *ptrs1 = (uint32_t *)buf;
uint32_t block2 = ptrs1[idx1]; // 두 번째 레벨 블록 번호를 얻음

if (block2 == 0)
  return -ENOENT; // 두 번째 레벨 블록이 할당되지 않은 경우

// 두 번째 레벨 블록을 디스크에서 읽는다.
if (read_block(fd, block2, buf) < 0)
  return -EIO; // 두 번째 레벨 블록을 읽을 때 오류 발생 시 입출력 오류 반환

uint32_t *ptrs2 = (uint32_t *)buf;
*pbn_out = ptrs2[idx2]; // 최종적으로 변환된 물리 블록 번호를 얻음

return 0; // 성공적으로 물리 블록 번호 변환 완료
}
```

8.34 src/main.c 파일 참조

SFUSE 파일시스템의 메인 진입점

```
#include "fs.h"
#include "ops.h"
#include <errno.h>
#include <fcntl.h>
#include <fuse.h>
#include <fuse_opt.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <unistd.h>
```

main.c에 대한 include 의존 그래프



함수

- int main (int argc, char *argv[ ])
  프로그램 메인 함수

8.34.1 상세한 설명

SFUSE 파일시스템의 메인 진입점
블록 디바이스를 사용하여 SFUSE 파일시스템을 마운트하는 프로그램의 메인 함수를 구현한다. 파일시스템의 초기화, 디바이스 유효성 검사 및 FUSE 인자 설정 후 실행된다.
main.c 파일에서 정의되었습니다.

8.34.2 함수 문서화

8.34.2.1 main()

```
int main (
            int argc,
            char * argv[])
```

프로그램 메인 함수
SFUSE 파일시스템을 지정된 블록 디바이스에 마운트하고, FUSE를 통해 사용자의 추가 옵션과 함께 운영한다. 블록 디바이스 유효성을 확인하고 메모리 및 자원을 관리한다.

매개변수

argc	명령줄 인자의 개수
argv	명령줄 인자 배열: • argv[0]: 프로그램 이름 • argv[1]: 블록 디바이스 경로 • argv[2]: 파일시스템 마운트 지점 • argv[3...n]: FUSE 추가 옵션들

반환값

프로그램 성공 시 EXIT_SUCCESS 반환, 실패 시 EXIT_FAILURE 반환

마운트할 블록 디바이스의 경로를 저장하는 포인터
SFUSE 파일시스템을 마운트할 디렉토리 경로를 저장하는 포인터
블록 디바이스 파일을 열고 파일 디스크립터를 획득한다.
open 함수는 파일이나 디바이스를 열 때 사용하며, O_RDWR는 읽기 및 쓰기 모드, O_SYNC는 데이터를 즉시 디스크에 쓰는 동기화 모드를 설정한다.

열린 디바이스가 올바른 블록 디바이스인지 확인한다.
fstat 함수는 파일 디스크립터의 상태를 stat 구조체에 저장하며, S_ISBLK 매크로를 사용하여 블록 디바이스 여부를 검사한다.
SFUSE 파일시스템 관리를 위한 구조체(sfuse_fs)에 메모리를 할당하고 초기화한다. calloc 사용하여 메모리를 0으로 초기화한다.
SFUSE 파일시스템에서 사용할 디바이스 파일 디스크립터를 저장
FUSE 라이브러리에 전달할 인자 구조체를 초기화한다. FUSE_ARGS_INIT 매크로는 구조체를 안전하게 초기화하는데 사용된다.
FUSE에 프로그램 이름과 마운트 포인트를 기본 인자로 설정하여 전달한다. argv[0]은 프로그램 이름을 나타내며, argv[2]는 파일시스템 마운트 지점이다.
명령줄에서 추가로 입력된 모든 옵션을 FUSE 인자에 추가한다. 이를 통해 사용자는 FUSE의 다양한 기능을 옵션으로 활성화할 수 있다.
필수 옵션을 추가하여 SFUSE 운영에 필요한 기본 설정을 활성화한다. -o: FUSE에 옵션을 전달할 때 사용하는 플래그이다. 여러 옵션을 쉼표로 구분하여 함께 전달할 수 있다.

- allow_other: 다른 사용자가 마운트된 파일시스템에 접근할 수 있도록 허용한다.
- default_permissions: 파일시스템의 기본적인 권한 관리를 FUSE가 수행하도록 -d: 파일시스템의 동작과 관련된 자세한 디버깅 정보를 출력한다. 한다.

FUSE 메인 루프를 실행하여 파일시스템의 실제 마운트 및 운영을 시작한다. sfuse_ops는 파일시스템 작업을 처리하는 함수 포인터를 포함한 구조체이다.
사용한 모든 자원을 정리하고 메모리를 해제한 뒤 프로그램을 종료한다. fuse_opt_free_args: FUSE 인자 관련 메모리 해제 close: 파일 디스크립터 닫기 free: 동적 할당된 메모리 해제
main.c 파일의 38 번째 라인에서 정의되었습니다.

```
00038                                   {
00039   if (argc >= 2 && (!strcmp(argv[1], "-h") || !strcmp(argv[1], "--help"))) {
00040     fprintf(stdout,
00041             "사용법 : sudo %s <device> <mountpoint> [options]\n"
00042             "(사용예: sudo ./sfuse /dev/sdx /mnt/partition -f -s -d -o "
00043             "allow_other,default_permissions\n"
00044             "\n기본 옵션들:\n"
00045             "  -f: FUSE 파일시스템을 포그라운드에서 실행한다.\n"
00046             "  -s: 싱글 스레드 모드로 실행한다(디버깅 용이).\n"
00047             "  -d: 디버그 모드를 활성화하여 파일시스템의 상세한 동작 정보를 "
00048             "출력한다.\n"
00049             "  -o allow_other: 모든 사용자가 파일시스템에 접근 가능하도록 "
00050             "허용한다.\n"
00051             "  -o default_permissions: 기본 UNIX 권한 체크를 활성화하여 파일 "
00052             "접근 권한을 관리한다.\n"
00053             "\n추가로 사용할 수 있는 FUSE 옵션들:\n"
00054             "  -h, --help: FUSE 사용법과 옵션 목록을 출력한다.\n"
00055             "  -V, --version: FUSE 라이브러리의 버전 정보를 출력한다.\n"
00056             "  -o ro: 읽기 전용으로 파일시스템을 마운트한다.\n"
00057             "  -o rw: 읽기 및 쓰기 모드로 마운트한다(기본값).\n"
00058             "  -o fsname=NAME: 파일시스템의 이름을 설정한다.\n"
00059             "  -o uid=N: 파일의 소유자 UID를 설정한다.\n"
00060             "  -o gid=N: 파일의 그룹 GID를 설정한다.\n"
00061             "  -o umask=MASK: 파일 및 디렉토리 생성 시 적용할 권한 마스크를 "
00062             "설정한다.\n"
00063             "  -o nonempty: 마운트 지점이 비어있지 않아도 마운트를 허용한다.\n"
00064             "  -o direct_io: 커널 페이지 캐시를 우회하여 직접 입출력을 "
00065             "활성화한다.\n"
00066             "  -o auto_unmount: 프로그램 종료 시 자동으로 마운트를 해제한다.\n",
00067             argv[0]);
00068     return EXIT_SUCCESS;
00069   }
00070
00071   if (argc < 3) {
00072     fprintf(stderr, "사용법: sudo %s <device> <mountpoint> [options]\n",
00073             argv[0]);
00074     return EXIT_FAILURE;
00075   }
00076
00078   const char *dev_path = argv[1];
00080   const char *mountpoint = argv[2];
00081
00089   int backing_fd = open(dev_path, O_RDWR | O_SYNC);
00090   if (backing_fd < 0) {
00091     perror("디바이스 열기 실패");
00092     return EXIT_FAILURE;
00093   }
00094
00101   struct stat st;
00102   if (fstat(backing_fd, &st) < 0 || !S_ISBLK(st.st_mode)) {
00103     fprintf(stderr, "%s는 블록 디바이스가 아닙니다.\n", dev_path);
00104     close(backing_fd);
```

```
    return EXIT_FAILURE;
  }

  struct sfuse_fs *fs = calloc(1, sizeof(*fs));
  if (!fs) {
    perror("메모리 할당 실패");
    close(backing_fd);
    return EXIT_FAILURE;
  }

  fs->backing_fd = backing_fd;

  struct fuse_args args = FUSE_ARGS_INIT(0, NULL);

  fuse_opt_add_arg(&args, argv[0]);
  fuse_opt_add_arg(&args, mountpoint);

  for (int i = 3; i < argc; i++) {
    fuse_opt_add_arg(&args, argv[i]);
  }

  fuse_opt_add_arg(&args, "-o");
  fuse_opt_add_arg(&args, "allow_other");
  fuse_opt_add_arg(&args, "default_permissions");
  fuse_opt_add_arg(&args, "-d");

  int ret = fuse_main(args.argc, args.argv, &sfuse_ops, fs);

  fuse_opt_free_args(&args);
  close(backing_fd);
  free(fs);

  return ret;
}
```

8.35 main.c

이 파일의 문서화 페이지로 가기

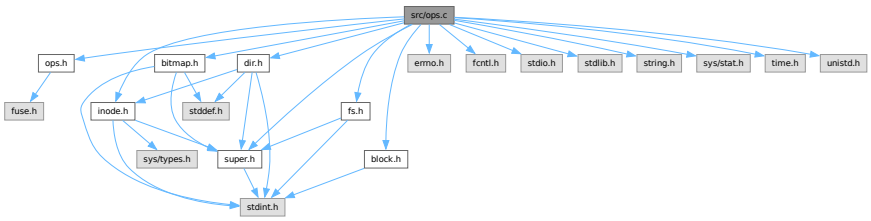
```
#include "fs.h"
#include "ops.h"
#include <errno.h>
#include <fcntl.h>
#include <fuse.h>
#include <fuse_opt.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
  if (argc >= 2 && (!strcmp(argv[1], "-h") || !strcmp(argv[1], "--help"))) {
    fprintf(stdout,
            "사용법 : sudo %s <device> <mountpoint> [options]\n"
            "(사용예: sudo ./sfuse /dev/sdx /mnt/partition -f -s -d -o "
            "allow_other,default_permissions\n"
            "\n기본 옵션들:\n"
            "  -f: FUSE 파일시스템을 포그라운드에서 실행한다.\n"
            "  -s: 싱글 스레드 모드로 실행한다(디버깅 용이).\n"
            "  -d: 디버그 모드를 활성화하여 파일시스템의 상세한 동작 정보를 "
            "출력한다.\n"
            "  -o allow_other: 모든 사용자가 파일시스템에 접근 가능하도록 "
            "허용한다.\n"
            "  -o default_permissions: 기본 UNIX 권한 체크를 활성화하여 파일 "
            "접근 권한을 관리한다.\n"
            "\n추가로 사용할 수 있는 FUSE 옵션들:\n"
            "  -h, --help: FUSE 사용법과 옵션 목록을 출력한다.\n"
            "  -V, --version: FUSE 라이브러리의 버전 정보를 출력한다.\n"
            "  -o ro: 읽기 전용으로 파일시스템을 마운트한다.\n"
            "  -o rw: 읽기 및 쓰기 모드로 마운트한다(기본값).\n"
            "  -o fsname=NAME: 파일시스템의 이름을 설정한다.\n"
            "  -o uid=N: 파일의 소유자 UID를 설정한다.\n"
            "  -o gid=N: 파일의 그룹 GID를 설정한다.\n"
            "  -o umask=MASK: 파일 및 디렉토리 생성 시 적용할 권한 마스크를 "
            "설정한다.\n"
            "  -o nonempty: 마운트 지점이 비어있지 않아도 마운트를 허용한다.\n"
            "  -o direct_io: 커널 페이지 캐시를 우회하여 직접 입출력을 "
            "활성화한다.\n"
            "  -o auto_unmount: 프로그램 종료 시 자동으로 마운트를 해제한다.\n",
            argv[0]);
    return EXIT_SUCCESS;
```

```
}

if (argc < 3) {
  fprintf(stderr, "사용법: sudo %s <device> <mountpoint> [options]\n",
          argv[0]);
  return EXIT_FAILURE;
}

const char *dev_path = argv[1];
const char *mountpoint = argv[2];

int backing_fd = open(dev_path, O_RDWR | O_SYNC);
if (backing_fd < 0) {
  perror("디바이스 열기 실패");
  return EXIT_FAILURE;
}

struct stat st;
if (fstat(backing_fd, &st) < 0 || !S_ISBLK(st.st_mode)) {
  fprintf(stderr, "%s는 블록 디바이스가 아닙니다.\n", dev_path);
  close(backing_fd);
  return EXIT_FAILURE;
}

struct sfuse_fs *fs = calloc(1, sizeof(*fs));
if (!fs) {
  perror("메모리 할당 실패");
  close(backing_fd);
  return EXIT_FAILURE;
}

fs->backing_fd = backing_fd;

struct fuse_args args = FUSE_ARGS_INIT(0, NULL);

fuse_opt_add_arg(&args, argv[0]);
fuse_opt_add_arg(&args, mountpoint);

for (int i = 3; i < argc; i++) {
  fuse_opt_add_arg(&args, argv[i]);
}

fuse_opt_add_arg(&args, "-o");
fuse_opt_add_arg(&args, "allow_other");
fuse_opt_add_arg(&args, "default_permissions");
fuse_opt_add_arg(&args, "-d");

int ret = fuse_main(args.argc, args.argv, &sfuse_ops, fs);

fuse_opt_free_args(&args);
close(backing_fd);
free(fs);

return ret;
}
```

8.36 src/ops.c 파일 참조

#include "ops.h"
#include "bitmap.h"
#include "block.h"
#include "dir.h"
#include "fs.h"
#include "inode.h"
#include "super.h"
#include <errno.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <time.h>
#include <unistd.h>

ops.c에 대한 include 의존 그래프



함수

- static void * sfuse_init_cb (struct fuse_conn_info *conn, struct fuse_config *cfg)
- static void sfuse_destroy_cb (void *private_data)
- static int sfuse_getattr_cb (const char *path, struct stat *stbuf, struct fuse_file_info *fi)
- static int sfuse_access_cb (const char *path, int mask)
- static int sfuse_readdir_cb (const char *path, void *buf, fuse_fill_dir_t filler, off_t offset, struct fuse_file_info *fi, enum fuse_readdir_flags flags)
- static int sfuse_open_cb (const char *path, struct fuse_file_info *fi)
- static int sfuse_read_cb (const char *path, char *buf, size_t size, off_t offset, struct fuse_file_info *fi)
- static int sfuse_write_cb (const char *path, const char *buf, size_t size, off_t offset, struct fuse↩ _file_info *fi)
- static int sfuse_create_cb (const char *path, mode_t mode, struct fuse_file_info *fi)
- static int sfuse_mkdir_cb (const char *path, mode_t mode)
- static int sfuse_unlink_cb (const char *path)
- static int sfuse_rmdir_cb (const char *path)
- static int sfuse_rename_cb (const char *oldpath, const char *newpath, unsigned int flags)
- static int sfuse_truncate_cb (const char *path, off_t size, struct fuse_file_info *fi)
- static int sfuse_utimens_cb (const char *path, const struct timespec tv[2], struct fuse_file_info *fi)
- static int sfuse_flush_cb (const char *path, struct fuse_file_info *fi)
- static int sfuse_fsync_cb (const char *path, int datasync, struct fuse_file_info *fi)
- static int sfuse_statfs_cb (const char *path, struct statvfs *stbuf)
- static int sfuse_listxattr_cb (const char *path, char *list, size_t size)
- static int sfuse_getxattr_cb (const char *path, const char *name, char *value, size_t size)

변수

- const struct fuse_operations sfuse_ops

8.36.1 함수 문서화

8.36.1.1 sfuse_access_cb()

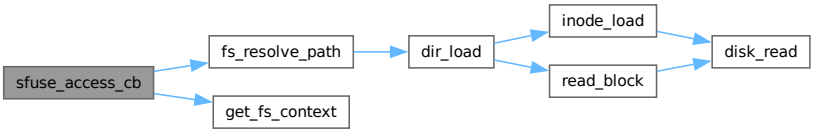
int sfuse_access_cb (
const char * path,
int mask) [static]

ops.c 파일의 71 번째 라인에서 정의되었습니다.

```
{
  struct sfuse_fs *fs = get_fs_context();
  uint32_t ino;
  if (fs_resolve_path(fs, path, &ino) < 0)
    return -ENOENT;
  (void)mask;
  return 0;
```

00078 }

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



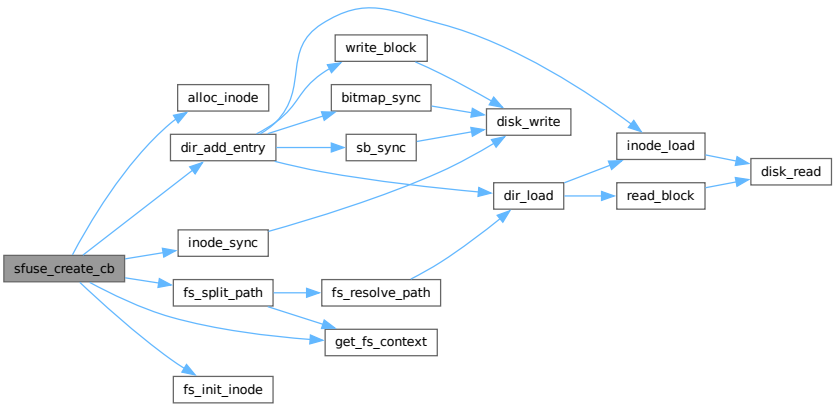
8.36.1.2 sfuse_create_cb()

int sfuse_create_cb (
const char * path,
mode_t mode,
struct fuse_file_info * fi) [static]

ops.c 파일의 242 번째 라인에서 정의되었습니다.

```
00243                                                   {
00244   struct sfuse_fs *fs = get_fs_context();
00245   uint32_t parent;
00246   char *name = fs_split_path(path, &parent);
00247   int ino = alloc_inode(&fs->sb, fs->inode_map);
00248   if (ino < 0) {
00249     free(name);
00250     return -ENOSPC;
00251   }
00252   struct sfuse_inode newnode;
00253   fs_init_inode(&fs->sb, ino, mode, fuse_get_context()->uid,
00254                 fuse_get_context()->gid, &newnode);
00255   inode_sync(fs->backing_fd, &fs->sb, ino, &newnode);
00256   dir_add_entry(fs->backing_fd, &fs->sb, parent, name, ino, fs->block_map,
00257                 fs->inode_map, &fs->sb);
00258   free(name);
00259   fi->fh = ino;
00260   return 0;
00261 }
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



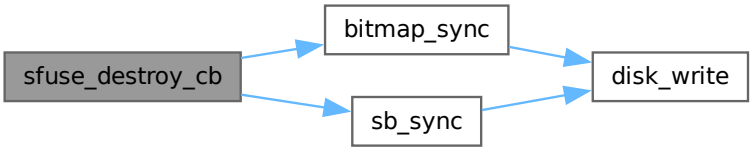
8.36.1.3 sfuse_destroy_cb()

void sfuse_destroy_cb (
void * private_data) [static]

ops.c 파일의 34 번째 라인에서 정의되었습니다.

```
{
struct sfuse_fs *fs = private_data;
size_t bmap_bytes = fs->sb.blocks_count / 8;
size_t imap_bytes = fs->sb.inodes_count / 8;
bitmap_sync(fs->backing_fd, fs->sb.block_bitmap_start, fs->block_map,
            bmap_bytes);
bitmap_sync(fs->backing_fd, fs->sb.inode_bitmap_start, fs->inode_map,
            imap_bytes);
sb_sync(fs->backing_fd, &fs->sb);
free(fs->block_map);
free(fs->inode_map);
}
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



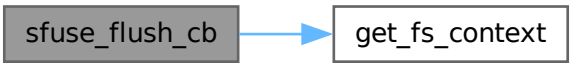
8.36.1.4 sfuse_flush_cb()

```
int sfuse_flush_cb (
            const char * path,
            struct fuse_file_info * fi)   [static]
```

ops.c 파일의 498 번째 라인에서 정의되었습니다.

```
{
(void)path;
(void)fi;
struct sfuse_fs *fs = get_fs_context();
if (fsync(fs->backing_fd) < 0)
  return -errno;
return 0;
}
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



8.36.1.5 sfuse_fsync_cb()

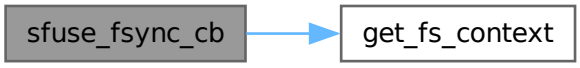
```
int sfuse_fsync_cb (
            const char * path,
            int datasync,
            struct fuse_file_info * fi)   [static]
```

ops.c 파일의 508 번째 라인에서 정의되었습니다.

```
{
(void)path;
```

```
(void)fi;
struct sfuse_fs *fs = get_fs_context();
int res = datasync ? fdatasync(fs->backing_fd) : fsync(fs->backing_fd);
if (res < 0)
  return -errno;
return 0;
}
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



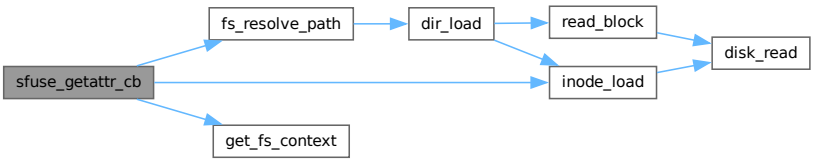
8.36.1.6 sfuse_getattr_cb()

```
int sfuse_getattr_cb (
            const char * path,
            struct stat * stbuf,
            struct fuse_file_info * fi)    [static]
```

ops.c 파일의 48 번째 라인에서 정의되었습니다.

```
{
(void)fi;
struct sfuse_fs *fs = get_fs_context();
uint32_t ino;
if (fs_resolve_path(fs, path, &ino) < 0)
  return -ENOENT;
struct sfuse_inode inode;
if (inode_load(fs->backing_fd, &fs->sb, ino, &inode) < 0)
  return -EIO;
memset(stbuf, 0, sizeof(*stbuf));
stbuf->st_mode = inode.mode;
stbuf->st_nlink = S_ISDIR(inode.mode) ? 2 : 1; // 디렉터리는 링크수 2로 고정
stbuf->st_uid = inode.uid;
stbuf->st_gid = inode.gid;
stbuf->st_size = inode.size;
stbuf->st_atime = inode.atime;
stbuf->st_mtime = inode.mtime;
stbuf->st_ctime = inode.ctime;
return 0;
}
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



8.36.1.7 sfuse_getxattr_cb()

```
int sfuse_getxattr_cb (
            const char * path,
            const char * name,
```

char * value,
size_t size) [static]

ops.c 파일의 556 번째 라인에서 정의되었습니다.

```
00557                                   {
00558   return -ENODATA; // 해당 속성 없음 처리 (안정적으로 처리됨)
00559 }
```

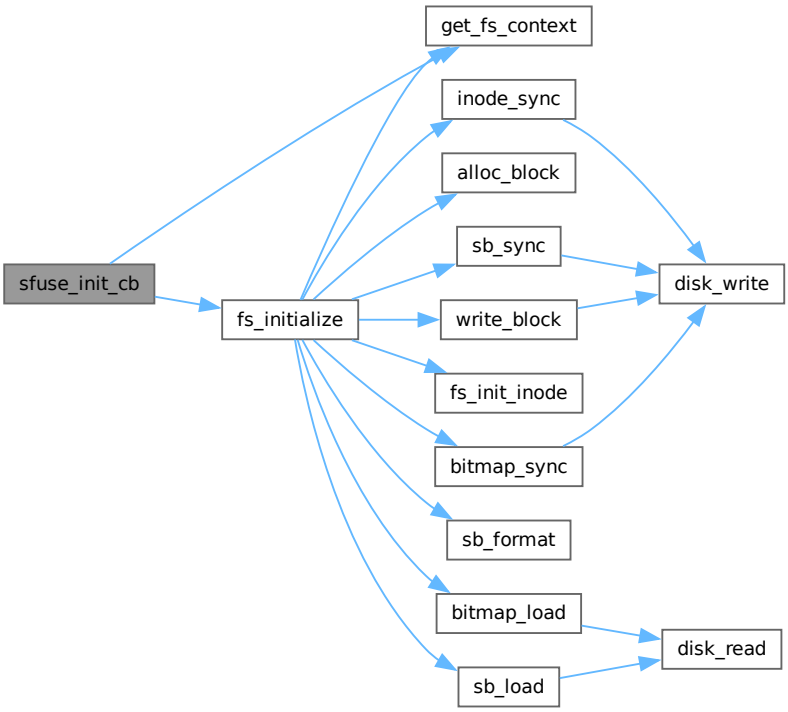
8.36.1.8 sfuse_init_cb()

void * sfuse_init_cb (
struct fuse_conn_info * conn,
struct fuse_config * cfg) [static]

ops.c 파일의 20 번째 라인에서 정의되었습니다.

```
00021                                   {
00022   (void)conn;
00023   (void)cfg;
00024   struct sfuse_fs *fs = get_fs_context();
00025   // 파일시스템 초기화 (슈퍼블록/비트맵 로드, 루트 아이노드 설정)
00026   if (fs_initialize(fs->backing_fd) < 0) {
00027     fprintf(stderr, "[SFUSE] FS initialization failed\n");
00028     exit(EXIT_FAILURE);
00029   }
00030   return fs;
00031 }
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



8.36.1.9 sfuse_listxattr_cb()

int sfuse_listxattr_cb (
const char * path,
char * list,
size_t size) [static]

ops.c 파일의 551 번째 라인에서 정의되었습니다.

```
{
  return 0; // 빈 리스트 반환 (성공적으로 처리됨)
}
```

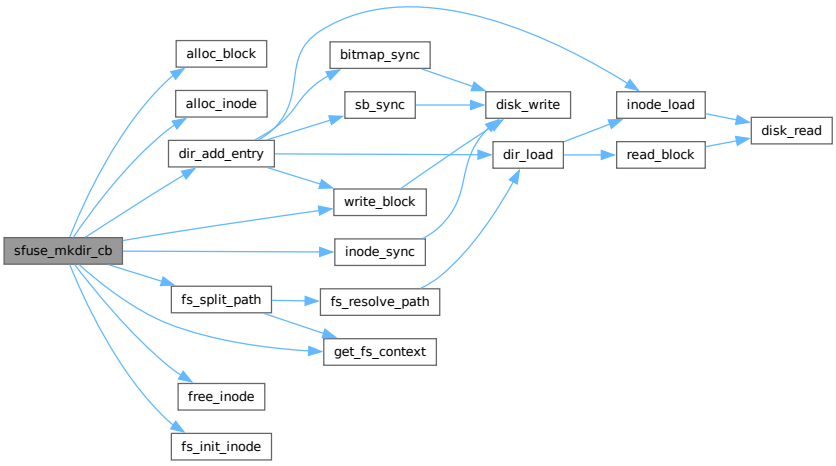
8.36.1.10 sfuse_mkdir_cb()

int sfuse_mkdir_cb (
const char * path,
mode_t mode) [static]

ops.c 파일의 264 번째 라인에서 정의되었습니다.

```
{
  struct sfuse_fs *fs = get_fs_context();
  uint32_t parent;
  char *name = fs_split_path(path, &parent);
  if (!name)
    return -EINVAL;

  int ino = alloc_inode(&fs->sb, fs->inode_map);
  if (ino < 0) {
    free(name);
    return -ENOSPC;
  }

  struct sfuse_inode dir_inode;
  fs_init_inode(&fs->sb, ino, mode | S_IFDIR, fuse_get_context()->uid,
                fuse_get_context()->gid, &dir_inode);

  int blk_index = alloc_block(&fs->sb, fs->block_map);
  if (blk_index < 0) {
    free_inode(&fs->sb, fs->inode_map, ino);
    free(name);
    return -ENOSPC;
  }

  dir_inode.direct[0] = fs->sb.data_block_start + blk_index;
  dir_inode.size = SFUSE_BLOCK_SIZE;

  uint8_t block[SFUSE_BLOCK_SIZE] = {0};
  struct sfuse_dirent *entries = (struct sfuse_dirent *)block;
  entries[0].ino = ino;
  strcpy(entries[0].name, ".");
  entries[1].ino = parent;
  strcpy(entries[1].name, "..");
  write_block(fs->backing_fd, dir_inode.direct[0], block);

  inode_sync(fs->backing_fd, &fs->sb, ino, &dir_inode);
  dir_add_entry(fs->backing_fd, &fs->sb, parent, name, ino, fs->block_map,
                fs->inode_map, &fs->sb);

  free(name);
  return 0;
}
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



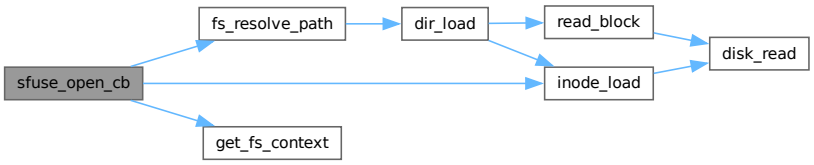
8.36.1.11 sfuse_open_cb()

```
int sfuse_open_cb (
                const char * path,
                struct fuse_file_info * fi)   [static]
```

ops.c 파일의 146 번째 라인에서 정의되었습니다.

```
00146                                                   {
00147   struct sfuse_fs *fs = get_fs_context();
00148   uint32_t ino;
00149
00150   if (fs_resolve_path(fs, path, &ino) < 0)
00151     return -ENOENT;
00152
00153   struct sfuse_inode inode;
00154   if (inode_load(fs->backing_fd, &fs->sb, ino, &inode) < 0)
00155     return -EIO;
00156
00157   // 디렉터리도 정상적으로 open 가능하게 처리
00158   return 0;
00159 }
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



8.36.1.12 sfuse_read_cb()

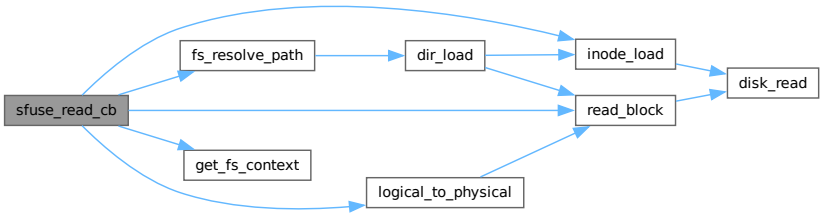
```
int sfuse_read_cb (
                const char * path,
                char * buf,
                size_t size,
                off_t offset,
                struct fuse_file_info * fi)   [static]
```

ops.c 파일의 162 번째 라인에서 정의되었습니다.

```
00163                                                   {
00164   (void)fi;
00165   struct sfuse_fs *fs = get_fs_context();
00166   uint32_t ino;
00167   if (fs_resolve_path(fs, path, &ino) < 0)
00168     return -ENOENT;
00169   struct sfuse_inode inode;
00170   inode_load(fs->backing_fd, &fs->sb, ino, &inode);
00171   if (S_ISDIR(inode.mode))
00172     return -EISDIR;
00173   if (offset >= inode.size)
00174     return 0;
00175   // 파일 크기를 넘어서는 부분은 읽지 않음
00176   size_t to_read = size;
00177   if (offset + to_read > inode.size)
00178     to_read = inode.size - offset;
00179   size_t done = 0;
00180   uint32_t pbn;
00181   uint8_t tmp[SFUSE_BLOCK_SIZE];
00182   // 오프셋부터 필요한 바이트만큼 읽기
00183   while (done < to_read) {
00184     off_t cur = offset + done;
00185     uint32_t lbn = cur / SFUSE_BLOCK_SIZE;
00186     size_t boff = cur % SFUSE_BLOCK_SIZE;
00187     size_t chunk = SFUSE_BLOCK_SIZE - boff;
00188     if (chunk > to_read - done)
00189       chunk = to_read - done;
00190     logical_to_physical(fs->backing_fd, &fs->sb, &inode, lbn, tmp, &pbn);
00191     read_block(fs->backing_fd, pbn, tmp);
00192     memcpy(buf + done, tmp + boff, chunk);
```

```
    done += chunk;
  }
  return done;
}
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



8.36.1.13 sfuse_readdir_cb()

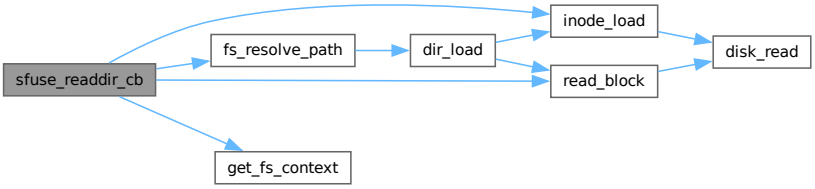
```
int sfuse_readdir_cb (
            const char * path,
            void * buf,
            fuse_fill_dir_t filler,
            off_t offset,
            struct fuse_file_info * fi,
            enum fuse_readdir_flags flags)   [static]
```

ops.c 파일의 81 번째 라인에서 정의되었습니다.

```
{
  struct sfuse_fs *fs = get_fs_context();
  uint32_t ino;

  if (fs_resolve_path(fs, path, &ino) < 0)
    return -ENOENT;

  struct sfuse_inode inode;
  if (inode_load(fs->backing_fd, &fs->sb, ino, &inode) < 0)
    return -EIO;

  size_t current_offset = 0;

  if (current_offset >= offset)
    if (filler(buf, ".", NULL, ++current_offset, 0))
      return 0;

  if (current_offset >= offset)
    if (filler(buf, "..", NULL, ++current_offset, 0))
      return 0;

  size_t entries_per_block = SFUSE_BLOCK_SIZE / sizeof(struct sfuse_dirent);
  size_t total_entries = inode.size / sizeof(struct sfuse_dirent);

  for (int b = 0; b < SFUSE_NDIR_BLOCKS; b++) {
    uint32_t blkno = inode.direct[b];
    if (!blkno)
      continue;

    if (blkno < fs->sb.data_block_start || blkno >= fs->sb.blocks_count)
      return -EIO;

    uint8_t block[SFUSE_BLOCK_SIZE];
    if (read_block(fs->backing_fd, blkno, block) < 0)
      return -EIO;

    struct sfuse_dirent *entries = (struct sfuse_dirent *)block;

    for (size_t i = 0; i < entries_per_block; i++) {
      size_t idx = b * entries_per_block + i;
      if (idx >= total_entries)
        break;

      if (!entries[i].ino || !strlen(entries[i].name))
        continue;
```

```
00128
00129       if (!strcmp(entries[i].name, ".") || !strcmp(entries[i].name, ".."))
00130         continue;
00131
00132       current_offset++;
00133
00134       if (current_offset <= offset)
00135         continue;
00136
00137       if (filler(buf, entries[i].name, NULL, current_offset, 0))
00138         return 0;
00139     }
00140   }
00141
00142   return 0;
00143 }
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



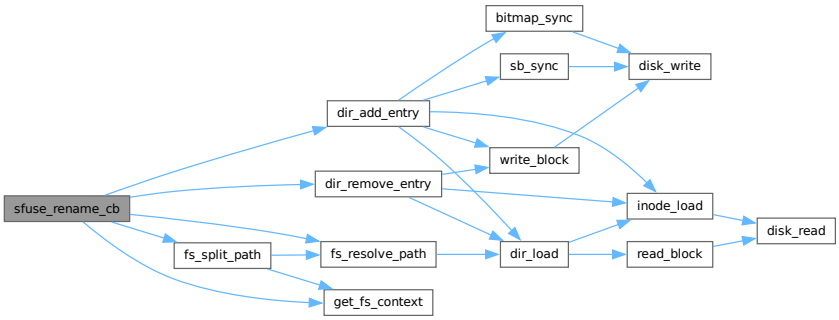
8.36.1.14 sfuse_rename_cb()

```
int sfuse_rename_cb (
                const char * oldpath,
                const char * newpath,
                unsigned int flags)   [static]
```

ops.c 파일의 407 번째 라인에서 정의되었습니다.

```
00408                                   {
00409   (void)flags;
00410   struct sfuse_fs *fs = get_fs_context();
00411   uint32_t oldp, newp;
00412   char *oldn = fs_split_path(oldpath, &oldp);
00413   char *newn = fs_split_path(newpath, &newp);
00414   uint32_t ino;
00415   fs_resolve_path(fs, oldpath, &ino);
00416   // 기존 경로의 디렉터리 엔트리 제거 및 새 경로로 추가
00417   dir_remove_entry(fs->backing_fd, &fs->sb, oldp, oldn);
00418   dir_add_entry(fs->backing_fd, &fs->sb, newp, newn, ino, fs->block_map,
00419               fs->inode_map, &fs->sb);
00420   free(oldn);
00421   free(newn);
00422   return 0;
00423 }
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



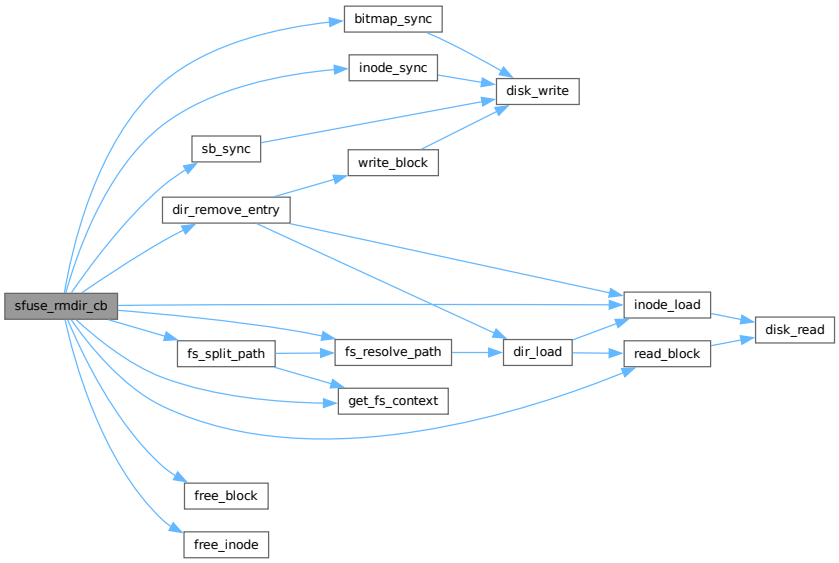
8.36.1.15 sfuse_rmdir_cb()

int sfuse_rmdir_cb (
const char * path) [static]

ops.c 파일의 354 번째 라인에서 정의되었습니다.

```
00354                                   {
00355   struct sfuse_fs *fs = get_fs_context();
00356   uint32_t parent;
00357   char *name = fs_split_path(path, &parent);
00358   if (!name)
00359     return -EINVAL;
00360
00361   uint32_t ino;
00362   if (fs_resolve_path(fs, path, &ino) < 0) {
00363     free(name);
00364     return -ENOENT;
00365   }
00366
00367   struct sfuse_inode inode;
00368   inode_load(fs->backing_fd, &fs->sb, ino, &inode);
00369   if (!S_ISDIR(inode.mode)) {
00370     free(name);
00371     return -ENOTDIR;
00372   }
00373
00374   uint8_t block[SFUSE_BLOCK_SIZE];
00375   read_block(fs->backing_fd, inode.direct[0], block);
00376   struct sfuse_dirent *ents = (struct sfuse_dirent *)block;
00377   int entries_per_block = SFUSE_BLOCK_SIZE / sizeof(struct sfuse_dirent);
00378
00379   for (int i = 0; i < entries_per_block; i++) {
00380     if (ents[i].ino && strcmp(ents[i].name, ".") &&
00381         strcmp(ents[i].name, "..")) {
00382       free(name);
00383       return -ENOTEMPTY;
00384     }
00385   }
00386
00387   free_block(&fs->sb, fs->block_map, inode.direct[0] - fs->sb.data_block_start);
00388   free_inode(&fs->sb, fs->inode_map, ino);
00389   dir_remove_entry(fs->backing_fd, &fs->sb, parent, name);
00390
00391   struct sfuse_inode parent_inode;
00392   inode_load(fs->backing_fd, &fs->sb, parent, &parent_inode);
00393   parent_inode.mtime = parent_inode.ctime = time(NULL);
00394   inode_sync(fs->backing_fd, &fs->sb, parent, &parent_inode);
00395
00396   bitmap_sync(fs->backing_fd, fs->sb.block_bitmap_start, fs->block_map,
00397               fs->sb.blocks_count / 8);
00398   bitmap_sync(fs->backing_fd, fs->sb.inode_bitmap_start, fs->inode_map,
00399               fs->sb.inodes_count / 8);
00400   sb_sync(fs->backing_fd, &fs->sb);
00401
00402   free(name);
00403   return 0;
00404 }
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



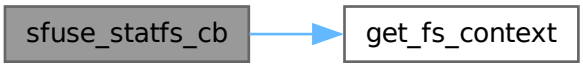
8.36.1.16 sfuse_statfs_cb()

int sfuse_statfs_cb (
const char * path,
struct statvfs * stbuf) [static]

ops.c 파일의 521 번째 라인에서 정의되었습니다.

```
00521                                                    {
00522   struct sfuse_fs *fs = get_fs_context();
00523
00524   memset(stbuf, 0, sizeof(struct statvfs));
00525
00526   /* VSFS의 슈퍼블록(sb)을 참조하여 채우기 */
00527   stbuf->f_bsize = SFUSE_BLOCK_SIZE; // 블록 크기 (VSFS의 블록 크기)
00528   stbuf->f_frsize =
00529       SFUSE_BLOCK_SIZE; // 프래그먼트 크기 (대부분 블록 크기와 같음)
00530
00531   /* 블록 개수 설정 (슈퍼블록 값 활용) */
00532   stbuf->f_blocks =
00533       fs->sb.blocks_count - fs->sb.data_block_start; // 데이터 블록의 전체 개수
00534   stbuf->f_bfree = fs->sb.free_blocks;              // 빈 블록 개수 (여유 공간)
00535   stbuf->f_bavail = fs->sb.free_blocks; // 일반 사용자가 쓸 수 있는 빈 블록 수
00536
00537   /* 아이노드 정보 설정 (VSFS 슈퍼블록 값 활용) */
00538   stbuf->f_files = fs->sb.inodes_count; // 전체 아이노드 개수
00539   stbuf->f_ffree = fs->sb.free_inodes;  // 빈 아이노드 개수
00540   stbuf->f_favail = fs->sb.free_inodes; // 일반 사용자 사용 가능 아이노드 수
00541
00542   /* FS 고유 식별자 및 기타 정보 */
00543   stbuf->f_fsid = 0x53465553;        // 임의 FSID (예: "SFUS" ASCII 코드값)
00544   stbuf->f_flag = ST_NOSUID;         // 마운트 옵션에 따라 설정 가능
00545   stbuf->f_namemax = SFUSE_NAME_LEN; // 최대 파일 이름 길이
00546
00547   return 0;
00548 }
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



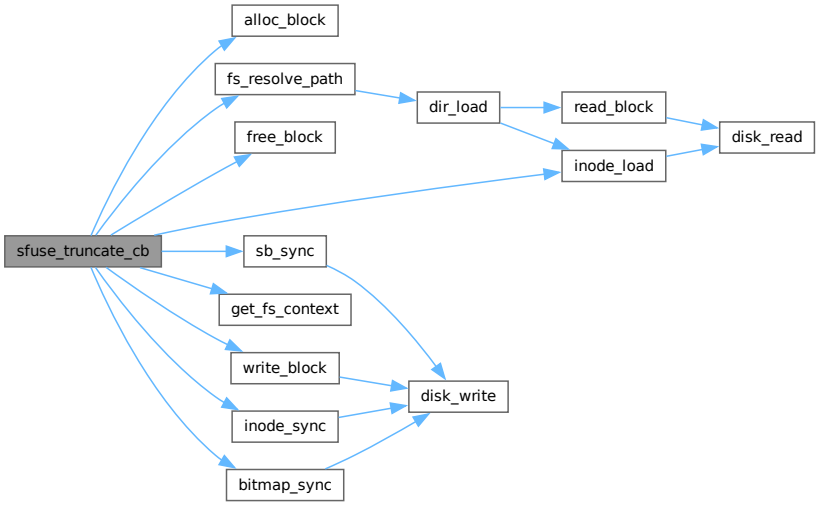
8.36.1.17 sfuse_truncate_cb()

```
int sfuse_truncate_cb (
            const char * path,
            off_t size,
            struct fuse_file_info * fi)   [static]
```

ops.c 파일의 426 번째 라인에서 정의되었습니다.

```
00427                                                   {
00428   struct sfuse_fs *fs = get_fs_context();
00429   uint32_t ino;
00430
00431   if (fs_resolve_path(fs, path, &ino) < 0)
00432     return -ENOENT;
00433
00434   struct sfuse_inode inode;
00435   if (inode_load(fs->backing_fd, &fs->sb, ino, &inode) < 0)
00436     return -EIO;
00437
00438   if (size > inode.size) {
00439     // 파일 크기 확장 시 새로 추가되는 영역 0으로 초기화
00440     uint8_t zero_block[SFUSE_BLOCK_SIZE] = {0};
00441     while (inode.size < size) {
00442       int blk_index = alloc_block(&fs->sb, fs->block_map);
00443       if (blk_index < 0)
00444         return -ENOSPC;
00445       uint32_t phys_block = fs->sb.data_block_start + blk_index;
00446       size_t direct_index = inode.size / SFUSE_BLOCK_SIZE;
00447       if (direct_index >= SFUSE_NDIR_BLOCKS)
00448         return -EFBIG; // 직접 블록만 지원한다고 가정
00449       inode.direct[direct_index] = phys_block;
00450       write_block(fs->backing_fd, phys_block, zero_block);
00451       inode.size += SFUSE_BLOCK_SIZE;
00452     }
00453   } else if (size < inode.size) {
00454     // 파일 크기 축소 시 블록 해제
00455     while (inode.size > size) {
00456       size_t direct_index = (inode.size - 1) / SFUSE_BLOCK_SIZE;
00457       if (direct_index < SFUSE_NDIR_BLOCKS && inode.direct[direct_index]) {
00458         free_block(&fs->sb, fs->block_map,
00459                    inode.direct[direct_index] - fs->sb.data_block_start);
00460         inode.direct[direct_index] = 0;
00461       }
00462       inode.size -= SFUSE_BLOCK_SIZE;
00463     }
00464   }
00465
00466   inode.size = size;
00467   inode.mtime = inode.ctime = time(NULL);
00468   inode_sync(fs->backing_fd, &fs->sb, ino, &inode);
00469
00470   bitmap_sync(fs->backing_fd, fs->sb.block_bitmap_start, fs->block_map,
00471               fs->sb.blocks_count / 8);
00472   sb_sync(fs->backing_fd, &fs->sb);
00473   return 0;
00474 }
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



8.36.1.18 sfuse_unlink_cb()

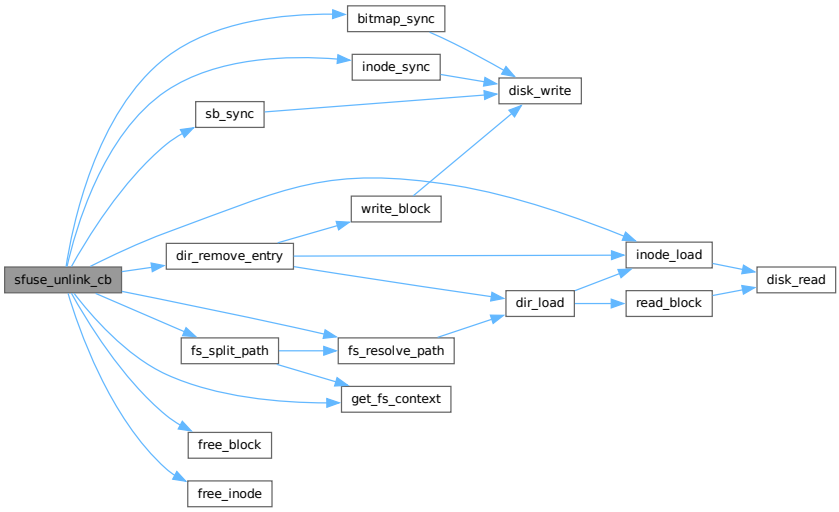
int sfuse_unlink_cb (
const char * path) [static]

ops.c 파일의 308 번째 라인에서 정의되었습니다.

```
00308                                   {
00309  struct sfuse_fs *fs = get_fs_context();
00310  uint32_t parent;
00311  char *name = fs_split_path(path, &parent);
00312  if (!name)
00313    return -EINVAL;
00314
00315  uint32_t ino;
00316  if (fs_resolve_path(fs, path, &ino) < 0) {
00317    free(name);
00318    return -ENOENT;
00319  }
00320
00321  struct sfuse_inode inode;
00322  inode_load(fs->backing_fd, &fs->sb, ino, &inode);
00323  if (S_ISDIR(inode.mode)) {
00324    free(name);
00325    return -EISDIR;
00326  }
00327
00328  for (int i = 0; i < SFUSE_NDIR_BLOCKS; i++) {
00329    if (inode.direct[i])
00330      free_block(&fs->sb, fs->block_map,
00331                 inode.direct[i] - fs->sb.data_block_start);
00332  }
00333
00334  free_inode(&fs->sb, fs->inode_map, ino);
00335  dir_remove_entry(fs->backing_fd, &fs->sb, parent, name);
00336
00337  // 상위 디렉터리 inode 정확히 업데이트
00338  struct sfuse_inode parent_inode;
00339  inode_load(fs->backing_fd, &fs->sb, parent, &parent_inode);
00340  parent_inode.mtime = parent_inode.ctime = time(NULL);
00341  inode_sync(fs->backing_fd, &fs->sb, parent, &parent_inode);
00342
00343  bitmap_sync(fs->backing_fd, fs->sb.block_bitmap_start, fs->block_map,
00344              fs->sb.blocks_count / 8);
00345  bitmap_sync(fs->backing_fd, fs->sb.inode_bitmap_start, fs->inode_map,
00346              fs->sb.inodes_count / 8);
00347  sb_sync(fs->backing_fd, &fs->sb);
00348
00349  free(name);
00350  return 0;
```

00351 }

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



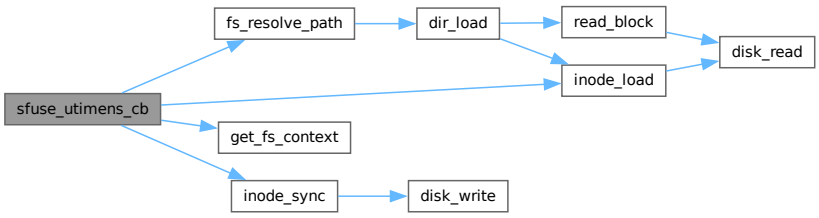
8.36.1.19 sfuse_utimens_cb()

int sfuse_utimens_cb (
const char * path,
const struct timespec tv[2],
struct fuse_file_info * fi) [static]

ops.c 파일의 477 번째 라인에서 정의되었습니다.

```
00478                                                   {
00479   struct sfuse_fs *fs = get_fs_context();
00480   uint32_t ino;
00481
00482   if (fs_resolve_path(fs, path, &ino) < 0)
00483     return -ENOENT;
00484
00485   struct sfuse_inode inode;
00486   if (inode_load(fs->backing_fd, &fs->sb, ino, &inode) < 0)
00487     return -EIO;
00488
00489   inode.atime = tv[0].tv_sec;
00490   inode.mtime = tv[1].tv_sec;
00491   inode.ctime = time(NULL); // ctime은 현재 시간으로 업데이트
00492   inode_sync(fs->backing_fd, &fs->sb, ino, &inode);
00493
00494   return 0;
00495 }
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



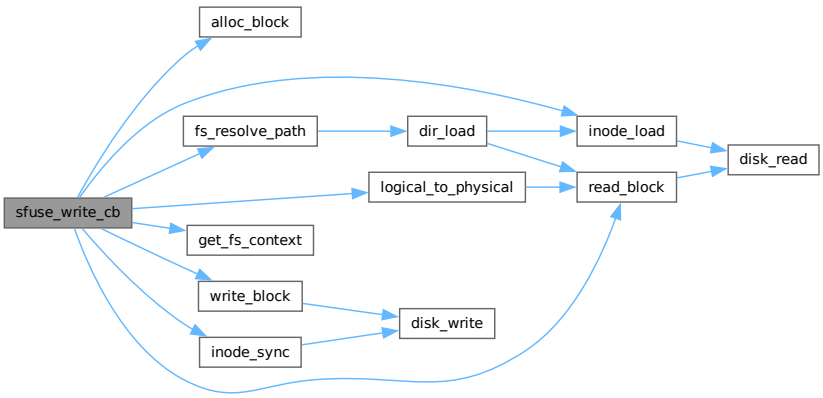
8.36.1.20 sfuse_write_cb()

```
int sfuse_write_cb (
            const char * path,
            const char * buf,
            size_t size,
            off_t offset,
            struct fuse_file_info * fi)    [static]
```

ops.c 파일의 199 번째 라인에서 정의되었습니다.

```
00200                                                   {
00201   (void)fi;
00202   struct sfuse_fs *fs = get_fs_context();
00203   uint32_t ino;
00204   if (fs_resolve_path(fs, path, &ino) < 0)
00205     return -ENOENT;
00206   struct sfuse_inode inode;
00207   inode_load(fs->backing_fd, &fs->sb, ino, &inode);
00208   if (S_ISDIR(inode.mode))
00209     return -EISDIR;
00210   size_t written = 0;
00211   uint32_t pbn;
00212   uint8_t tmp[SFUSE_BLOCK_SIZE];
00213   // 데이터 쓰기: 필요한 블록을 할당하거나 찾아서 부분 갱신
00214   while (written < size) {
00215     off_t cur = offset + written;
00216     uint32_t lbn = cur / SFUSE_BLOCK_SIZE;
00217     size_t boff = cur % SFUSE_BLOCK_SIZE;
00218     size_t chunk = SFUSE_BLOCK_SIZE - boff;
00219     if (chunk > size - written)
00220       chunk = size - written;
00221     if (logical_to_physical(fs->backing_fd, &fs->sb, &inode, lbn, tmp, &pbn) <
00222         0) {
00223       // 아직 물리 블록 할당 안 된 경우 새 블록 할당
00224       int new_off = alloc_block(&fs->sb, fs->block_map);
00225       pbn = fs->sb.data_block_start + new_off;
00226       if (lbn < SFUSE_NDIR_BLOCKS)
00227         inode.direct[lbn] = pbn;
00228     }
00229     read_block(fs->backing_fd, pbn, tmp);
00230     memcpy(tmp + boff, buf + written, chunk);
00231     write_block(fs->backing_fd, pbn, tmp);
00232     written += chunk;
00233   }
00234   if (offset + written > inode.size)
00235     inode.size = offset + written;
00236   inode.mtime = inode.ctime = (uint32_t)time(NULL);
00237   inode_sync(fs->backing_fd, &fs->sb, ino, &inode);
00238   return written;
00239 }
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



8.36.2 변수 문서화

8.36.2.1 sfuse_ops

const struct fuse_operations sfuse_ops

초기값:

```
= {
    .init = sfuse_init_cb,
    .destroy = sfuse_destroy_cb,
    .getattr = sfuse_getattr_cb,
    .access = sfuse_access_cb,
    .readdir = sfuse_readdir_cb,
    .open = sfuse_open_cb,
    .read = sfuse_read_cb,
    .write = sfuse_write_cb,
    .create = sfuse_create_cb,
    .mkdir = sfuse_mkdir_cb,
    .unlink = sfuse_unlink_cb,
    .rmdir = sfuse_rmdir_cb,
    .rename = sfuse_rename_cb,
    .truncate = sfuse_truncate_cb,
    .utimens = sfuse_utimens_cb,
    .flush = sfuse_flush_cb,
    .fsync = sfuse_fsync_cb,
    .statfs = sfuse_statfs_cb,
    .getxattr = sfuse_getxattr_cb,
    .listxattr = sfuse_listxattr_cb,
}
```

ops.c 파일의 562 번째 라인에서 정의되었습니다.

```
562                                 {
563     .init = sfuse_init_cb,
564     .destroy = sfuse_destroy_cb,
565     .getattr = sfuse_getattr_cb,
566     .access = sfuse_access_cb,
567     .readdir = sfuse_readdir_cb,
568     .open = sfuse_open_cb,
569     .read = sfuse_read_cb,
570     .write = sfuse_write_cb,
571     .create = sfuse_create_cb,
572     .mkdir = sfuse_mkdir_cb,
573     .unlink = sfuse_unlink_cb,
574     .rmdir = sfuse_rmdir_cb,
575     .rename = sfuse_rename_cb,
576     .truncate = sfuse_truncate_cb,
577     .utimens = sfuse_utimens_cb,
578     .flush = sfuse_flush_cb,
579     .fsync = sfuse_fsync_cb,
580     .statfs = sfuse_statfs_cb,
581     .getxattr = sfuse_getxattr_cb,
582     .listxattr = sfuse_listxattr_cb,
583 };
```

8.37 ops.c

이 파일의 문서화 페이지로 가기

```
00001 // ops.c: FUSE 콜백 구현
00002 // ops.c: FUSE 콜백 구현
00003 #include "ops.h"
00004 #include "bitmap.h"
00005 #include "block.h"
00006 #include "dir.h"
00007 #include "fs.h"
00008 #include "inode.h"
00009 #include "super.h"
00010 #include <errno.h>
00011 #include <fcntl.h>
00012 #include <stdio.h>
00013 #include <stdlib.h>
00014 #include <string.h>
00015 #include <sys/stat.h>
00016 #include <time.h>
00017 #include <unistd.h>
00018
00019 /* FUSE 초기화 콜백 (FUSE3 API) */
00020 static void *sfuse_init_cb(struct fuse_conn_info *conn,
00021                            struct fuse_config *cfg) {
00022   (void)conn;
00023   (void)cfg;
00024   struct sfuse_fs *fs = get_fs_context();
00025   // 파일시스템 초기화 (슈퍼블록/비트맵 로드, 루트 아이노드 설정)
```

```
  if (fs_initialize(fs->backing_fd) < 0) {
    fprintf(stderr, "[SFUSE] FS initialization failed\n");
    exit(EXIT_FAILURE);
  }
  return fs;
}

/* FUSE 종료 콜백 */
static void sfuse_destroy_cb(void *private_data) {
  struct sfuse_fs *fs = private_data;
  size_t bmap_bytes = fs->sb.blocks_count / 8;
  size_t imap_bytes = fs->sb.inodes_count / 8;
  bitmap_sync(fs->backing_fd, fs->sb.block_bitmap_start, fs->block_map,
              bmap_bytes);
  bitmap_sync(fs->backing_fd, fs->sb.inode_bitmap_start, fs->inode_map,
              imap_bytes);
  sb_sync(fs->backing_fd, &fs->sb);
  free(fs->block_map);
  free(fs->inode_map);
}

/* getattr */
static int sfuse_getattr_cb(const char *path, struct stat *stbuf,
                            struct fuse_file_info *fi) {
  (void)fi;
  struct sfuse_fs *fs = get_fs_context();
  uint32_t ino;
  if (fs_resolve_path(fs, path, &ino) < 0)
    return -ENOENT;
  struct sfuse_inode inode;
  if (inode_load(fs->backing_fd, &fs->sb, ino, &inode) < 0)
    return -EIO;
  memset(stbuf, 0, sizeof(*stbuf));
  stbuf->st_mode = inode.mode;
  stbuf->st_nlink = S_ISDIR(inode.mode) ? 2 : 1; // 디렉터리는 링크수 2로 고정
  stbuf->st_uid = inode.uid;
  stbuf->st_gid = inode.gid;
  stbuf->st_size = inode.size;
  stbuf->st_atime = inode.atime;
  stbuf->st_mtime = inode.mtime;
  stbuf->st_ctime = inode.ctime;
  return 0;
}

/* access */
static int sfuse_access_cb(const char *path, int mask) {
  struct sfuse_fs *fs = get_fs_context();
  uint32_t ino;
  if (fs_resolve_path(fs, path, &ino) < 0)
    return -ENOENT;
  (void)mask;
  return 0;
}

/* readdir */
static int sfuse_readdir_cb(const char *path, void *buf, fuse_fill_dir_t filler,
                            off_t offset, struct fuse_file_info *fi,
                            enum fuse_readdir_flags flags) {
  struct sfuse_fs *fs = get_fs_context();
  uint32_t ino;

  if (fs_resolve_path(fs, path, &ino) < 0)
    return -ENOENT;

  struct sfuse_inode inode;
  if (inode_load(fs->backing_fd, &fs->sb, ino, &inode) < 0)
    return -EIO;

  size_t current_offset = 0;

  if (current_offset >= offset)
    if (filler(buf, ".", NULL, ++current_offset, 0))
      return 0;

  if (current_offset >= offset)
    if (filler(buf, "..", NULL, ++current_offset, 0))
      return 0;

  size_t entries_per_block = SFUSE_BLOCK_SIZE / sizeof(struct sfuse_dirent);
  size_t total_entries = inode.size / sizeof(struct sfuse_dirent);

  for (int b = 0; b < SFUSE_NDIR_BLOCKS; b++) {
    uint32_t blkno = inode.direct[b];
    if (!blkno)
      continue;

    if (blkno < fs->sb.data_block_start || blkno >= fs->sb.blocks_count)
```

```
      return -EIO;

    uint8_t block[SFUSE_BLOCK_SIZE];
    if (read_block(fs->backing_fd, blkno, block) < 0)
      return -EIO;

    struct sfuse_dirent *entries = (struct sfuse_dirent *)block;

    for (size_t i = 0; i < entries_per_block; i++) {
      size_t idx = b * entries_per_block + i;
      if (idx >= total_entries)
        break;

      if (!entries[i].ino || !strlen(entries[i].name))
        continue;

      if (!strcmp(entries[i].name, ".") || !strcmp(entries[i].name, ".."))
        continue;

      current_offset++;

      if (current_offset <= offset)
        continue;

      if (filler(buf, entries[i].name, NULL, current_offset, 0))
        return 0;
    }
  }

  return 0;
}

/* open */
static int sfuse_open_cb(const char *path, struct fuse_file_info *fi) {
  struct sfuse_fs *fs = get_fs_context();
  uint32_t ino;

  if (fs_resolve_path(fs, path, &ino) < 0)
    return -ENOENT;

  struct sfuse_inode inode;
  if (inode_load(fs->backing_fd, &fs->sb, ino, &inode) < 0)
    return -EIO;

  // 디렉터리도 정상적으로 open 가능하게 처리
  return 0;
}

/* read */
static int sfuse_read_cb(const char *path, char *buf, size_t size, off_t offset,
                         struct fuse_file_info *fi) {
  (void)fi;
  struct sfuse_fs *fs = get_fs_context();
  uint32_t ino;
  if (fs_resolve_path(fs, path, &ino) < 0)
    return -ENOENT;
  struct sfuse_inode inode;
  inode_load(fs->backing_fd, &fs->sb, ino, &inode);
  if (S_ISDIR(inode.mode))
    return -EISDIR;
  if (offset >= inode.size)
    return 0;
  // 파일 크기를 넘어서는 부분은 읽지 않음
  size_t to_read = size;
  if (offset + to_read > inode.size)
    to_read = inode.size - offset;
  size_t done = 0;
  uint32_t pbn;
  uint8_t tmp[SFUSE_BLOCK_SIZE];
  // 오프셋부터 필요한 바이트만큼 읽기
  while (done < to_read) {
    off_t cur = offset + done;
    uint32_t lbn = cur / SFUSE_BLOCK_SIZE;
    size_t boff = cur % SFUSE_BLOCK_SIZE;
    size_t chunk = SFUSE_BLOCK_SIZE - boff;
    if (chunk > to_read - done)
      chunk = to_read - done;
    logical_to_physical(fs->backing_fd, &fs->sb, &inode, lbn, tmp, &pbn);
    read_block(fs->backing_fd, pbn, tmp);
    memcpy(buf + done, tmp + boff, chunk);
    done += chunk;
  }
  return done;
}

/* write */
static int sfuse_write_cb(const char *path, const char *buf, size_t size,
```

```
                        off_t offset, struct fuse_file_info *fi) {
  (void)fi;
  struct sfuse_fs *fs = get_fs_context();
  uint32_t ino;
  if (fs_resolve_path(fs, path, &ino) < 0)
    return -ENOENT;
  struct sfuse_inode inode;
  inode_load(fs->backing_fd, &fs->sb, ino, &inode);
  if (S_ISDIR(inode.mode))
    return -EISDIR;
  size_t written = 0;
  uint32_t pbn;
  uint8_t tmp[SFUSE_BLOCK_SIZE];
  // 데이터 쓰기: 필요한 블록을 할당하거나 찾아서 부분 갱신
  while (written < size) {
    off_t cur = offset + written;
    uint32_t lbn = cur / SFUSE_BLOCK_SIZE;
    size_t boff = cur % SFUSE_BLOCK_SIZE;
    size_t chunk = SFUSE_BLOCK_SIZE - boff;
    if (chunk > size - written)
      chunk = size - written;
    if (logical_to_physical(fs->backing_fd, &fs->sb, &inode, lbn, tmp, &pbn) <
        0) {
      // 아직 물리 블록 할당 안 된 경우 새 블록 할당
      int new_off = alloc_block(&fs->sb, fs->block_map);
      pbn = fs->sb.data_block_start + new_off;
      if (lbn < SFUSE_NDIR_BLOCKS)
        inode.direct[lbn] = pbn;
    }
    read_block(fs->backing_fd, pbn, tmp);
    memcpy(tmp + boff, buf + written, chunk);
    write_block(fs->backing_fd, pbn, tmp);
    written += chunk;
  }
  if (offset + written > inode.size)
    inode.size = offset + written;
  inode.mtime = inode.ctime = (uint32_t)time(NULL);
  inode_sync(fs->backing_fd, &fs->sb, ino, &inode);
  return written;
}

/* create */
static int sfuse_create_cb(const char *path, mode_t mode,
                           struct fuse_file_info *fi) {
  struct sfuse_fs *fs = get_fs_context();
  uint32_t parent;
  char *name = fs_split_path(path, &parent);
  int ino = alloc_inode(&fs->sb, fs->inode_map);
  if (ino < 0) {
    free(name);
    return -ENOSPC;
  }
  struct sfuse_inode newnode;
  fs_init_inode(&fs->sb, ino, mode, fuse_get_context()->uid,
                fuse_get_context()->gid, &newnode);
  inode_sync(fs->backing_fd, &fs->sb, ino, &newnode);
  dir_add_entry(fs->backing_fd, &fs->sb, parent, name, ino, fs->block_map,
                fs->inode_map, &fs->sb);
  free(name);
  fi->fh = ino;
  return 0;
}

/* mkdir */
static int sfuse_mkdir_cb(const char *path, mode_t mode) {
  struct sfuse_fs *fs = get_fs_context();
  uint32_t parent;
  char *name = fs_split_path(path, &parent);
  if (!name)
    return -EINVAL;

  int ino = alloc_inode(&fs->sb, fs->inode_map);
  if (ino < 0) {
    free(name);
    return -ENOSPC;
  }

  struct sfuse_inode dir_inode;
  fs_init_inode(&fs->sb, ino, mode | S_IFDIR, fuse_get_context()->uid,
                fuse_get_context()->gid, &dir_inode);

  int blk_index = alloc_block(&fs->sb, fs->block_map);
  if (blk_index < 0) {
    free_inode(&fs->sb, fs->inode_map, ino);
    free(name);
    return -ENOSPC;
  }
```

```c

  dir_inode.direct[0] = fs->sb.data_block_start + blk_index;
  dir_inode.size = SFUSE_BLOCK_SIZE;

  uint8_t block[SFUSE_BLOCK_SIZE] = {0};
  struct sfuse_dirent *entries = (struct sfuse_dirent *)block;
  entries[0].ino = ino;
  strcpy(entries[0].name, ".");
  entries[1].ino = parent;
  strcpy(entries[1].name, "..");
  write_block(fs->backing_fd, dir_inode.direct[0], block);

  inode_sync(fs->backing_fd, &fs->sb, ino, &dir_inode);
  dir_add_entry(fs->backing_fd, &fs->sb, parent, name, ino, fs->block_map,
                fs->inode_map, &fs->sb);

  free(name);
  return 0;
}

/* unlink */
static int sfuse_unlink_cb(const char *path) {
  struct sfuse_fs *fs = get_fs_context();
  uint32_t parent;
  char *name = fs_split_path(path, &parent);
  if (!name)
    return -EINVAL;

  uint32_t ino;
  if (fs_resolve_path(fs, path, &ino) < 0) {
    free(name);
    return -ENOENT;
  }

  struct sfuse_inode inode;
  inode_load(fs->backing_fd, &fs->sb, ino, &inode);
  if (S_ISDIR(inode.mode)) {
    free(name);
    return -EISDIR;
  }

  for (int i = 0; i < SFUSE_NDIR_BLOCKS; i++) {
    if (inode.direct[i])
      free_block(&fs->sb, fs->block_map,
                 inode.direct[i] - fs->sb.data_block_start);
  }

  free_inode(&fs->sb, fs->inode_map, ino);
  dir_remove_entry(fs->backing_fd, &fs->sb, parent, name);

  // 상위 디렉터리 inode 정확히 업데이트
  struct sfuse_inode parent_inode;
  inode_load(fs->backing_fd, &fs->sb, parent, &parent_inode);
  parent_inode.mtime = parent_inode.ctime = time(NULL);
  inode_sync(fs->backing_fd, &fs->sb, parent, &parent_inode);

  bitmap_sync(fs->backing_fd, fs->sb.block_bitmap_start, fs->block_map,
              fs->sb.blocks_count / 8);
  bitmap_sync(fs->backing_fd, fs->sb.inode_bitmap_start, fs->inode_map,
              fs->sb.inodes_count / 8);
  sb_sync(fs->backing_fd, &fs->sb);

  free(name);
  return 0;
}

/* rmdir */
static int sfuse_rmdir_cb(const char *path) {
  struct sfuse_fs *fs = get_fs_context();
  uint32_t parent;
  char *name = fs_split_path(path, &parent);
  if (!name)
    return -EINVAL;

  uint32_t ino;
  if (fs_resolve_path(fs, path, &ino) < 0) {
    free(name);
    return -ENOENT;
  }

  struct sfuse_inode inode;
  inode_load(fs->backing_fd, &fs->sb, ino, &inode);
  if (!S_ISDIR(inode.mode)) {
    free(name);
    return -ENOTDIR;
  }

```

```
uint8_t block[SFUSE_BLOCK_SIZE];
read_block(fs->backing_fd, inode.direct[0], block);
struct sfuse_dirent *ents = (struct sfuse_dirent *)block;
int entries_per_block = SFUSE_BLOCK_SIZE / sizeof(struct sfuse_dirent);

for (int i = 0; i < entries_per_block; i++) {
  if (ents[i].ino && strcmp(ents[i].name, ".") &&
      strcmp(ents[i].name, "..")) {
    free(name);
    return -ENOTEMPTY;
  }
}

free_block(&fs->sb, fs->block_map, inode.direct[0] - fs->sb.data_block_start);
free_inode(&fs->sb, fs->inode_map, ino);
dir_remove_entry(fs->backing_fd, &fs->sb, parent, name);

struct sfuse_inode parent_inode;
inode_load(fs->backing_fd, &fs->sb, parent, &parent_inode);
parent_inode.mtime = parent_inode.ctime = time(NULL);
inode_sync(fs->backing_fd, &fs->sb, parent, &parent_inode);

bitmap_sync(fs->backing_fd, fs->sb.block_bitmap_start, fs->block_map,
            fs->sb.blocks_count / 8);
bitmap_sync(fs->backing_fd, fs->sb.inode_bitmap_start, fs->inode_map,
            fs->sb.inodes_count / 8);
sb_sync(fs->backing_fd, &fs->sb);

free(name);
return 0;
}

/* rename */
static int sfuse_rename_cb(const char *oldpath, const char *newpath,
                           unsigned int flags) {
(void)flags;
struct sfuse_fs *fs = get_fs_context();
uint32_t oldp, newp;
char *oldn = fs_split_path(oldpath, &oldp);
char *newn = fs_split_path(newpath, &newp);
uint32_t ino;
fs_resolve_path(fs, oldpath, &ino);
// 기존 경로의 디렉터리 엔트리 제거 및 새 경로로 추가
dir_remove_entry(fs->backing_fd, &fs->sb, oldp, oldn);
dir_add_entry(fs->backing_fd, &fs->sb, newp, newn, ino, fs->block_map,
              fs->inode_map, &fs->sb);
free(oldn);
free(newn);
return 0;
}

/* truncate */
static int sfuse_truncate_cb(const char *path, off_t size,
                             struct fuse_file_info *fi) {
struct sfuse_fs *fs = get_fs_context();
uint32_t ino;

if (fs_resolve_path(fs, path, &ino) < 0)
  return -ENOENT;

struct sfuse_inode inode;
if (inode_load(fs->backing_fd, &fs->sb, ino, &inode) < 0)
  return -EIO;

if (size > inode.size) {
  // 파일 크기 확장 시 새로 추가되는 영역 0으로 초기화
  uint8_t zero_block[SFUSE_BLOCK_SIZE] = {0};
  while (inode.size < size) {
    int blk_index = alloc_block(&fs->sb, fs->block_map);
    if (blk_index < 0)
      return -ENOSPC;
    uint32_t phys_block = fs->sb.data_block_start + blk_index;
    size_t direct_index = inode.size / SFUSE_BLOCK_SIZE;
    if (direct_index >= SFUSE_NDIR_BLOCKS)
      return -EFBIG; // 직접 블록만 지원한다고 가정
    inode.direct[direct_index] = phys_block;
    write_block(fs->backing_fd, phys_block, zero_block);
    inode.size += SFUSE_BLOCK_SIZE;
  }
} else if (size < inode.size) {
  // 파일 크기 축소 시 블록 해제
  while (inode.size > size) {
    size_t direct_index = (inode.size - 1) / SFUSE_BLOCK_SIZE;
    if (direct_index < SFUSE_NDIR_BLOCKS && inode.direct[direct_index]) {
      free_block(&fs->sb, fs->block_map,
                 inode.direct[direct_index] - fs->sb.data_block_start);
      inode.direct[direct_index] = 0;
```

```
    }
    inode.size -= SFUSE_BLOCK_SIZE;
  }
}

inode.size = size;
inode.mtime = inode.ctime = time(NULL);
inode_sync(fs->backing_fd, &fs->sb, ino, &inode);

bitmap_sync(fs->backing_fd, fs->sb.block_bitmap_start, fs->block_map,
            fs->sb.blocks_count / 8);
sb_sync(fs->backing_fd, &fs->sb);
return 0;
}

/* utimens */
static int sfuse_utimens_cb(const char *path, const struct timespec tv[2],
                            struct fuse_file_info *fi) {
struct sfuse_fs *fs = get_fs_context();
uint32_t ino;

if (fs_resolve_path(fs, path, &ino) < 0)
  return -ENOENT;

struct sfuse_inode inode;
if (inode_load(fs->backing_fd, &fs->sb, ino, &inode) < 0)
  return -EIO;

inode.atime = tv[0].tv_sec;
inode.mtime = tv[1].tv_sec;
inode.ctime = time(NULL); // ctime은 현재 시간으로 업데이트
inode_sync(fs->backing_fd, &fs->sb, ino, &inode);

return 0;
}

/* flush */
static int sfuse_flush_cb(const char *path, struct fuse_file_info *fi) {
(void)path;
(void)fi;
struct sfuse_fs *fs = get_fs_context();
if (fsync(fs->backing_fd) < 0)
  return -errno;
return 0;
}

/* fsync */
static int sfuse_fsync_cb(const char *path, int datasync,
                          struct fuse_file_info *fi) {
(void)path;
(void)fi;
struct sfuse_fs *fs = get_fs_context();
int res = datasync ? fdatasync(fs->backing_fd) : fsync(fs->backing_fd);
if (res < 0)
  return -errno;
return 0;
}

/* statfs */
/* SFUSE statfs 콜백 구현 (파일 시스템 용량 정보 제공) */
static int sfuse_statfs_cb(const char *path, struct statvfs *stbuf) {
struct sfuse_fs *fs = get_fs_context();

memset(stbuf, 0, sizeof(struct statvfs));

/* VSFS의 슈퍼블록(sb)을 참조하여 채우기 */
stbuf->f_bsize = SFUSE_BLOCK_SIZE; // 블록 크기 (VSFS의 블록 크기)
stbuf->f_frsize =
    SFUSE_BLOCK_SIZE; // 프래그먼트 크기 (대부분 블록 크기와 같음)

/* 블록 개수 설정 (슈퍼블록 값 활용) */
stbuf->f_blocks =
    fs->sb.blocks_count - fs->sb.data_block_start; // 데이터 블록의 전체 개수
stbuf->f_bfree = fs->sb.free_blocks;              // 빈 블록 개수 (여유 공간)
stbuf->f_bavail = fs->sb.free_blocks; // 일반 사용자가 쓸 수 있는 빈 블록 수

/* 아이노드 정보 설정 (VSFS 슈퍼블록 값 활용) */
stbuf->f_files = fs->sb.inodes_count; // 전체 아이노드 개수
stbuf->f_ffree = fs->sb.free_inodes;  // 빈 아이노드 개수
stbuf->f_favail = fs->sb.free_inodes; // 일반 사용자 사용 가능 아이노드 수

/* FS 고유 식별자 및 기타 정보 */
stbuf->f_fsid = 0x53465553;        // 임의 FSID (예: "SFUS" ASCII 코드값)
stbuf->f_flag = ST_NOSUID;         // 마운트 옵션에 따라 설정 가능
stbuf->f_namemax = SFUSE_NAME_LEN; // 최대 파일 이름 길이

return 0;
```

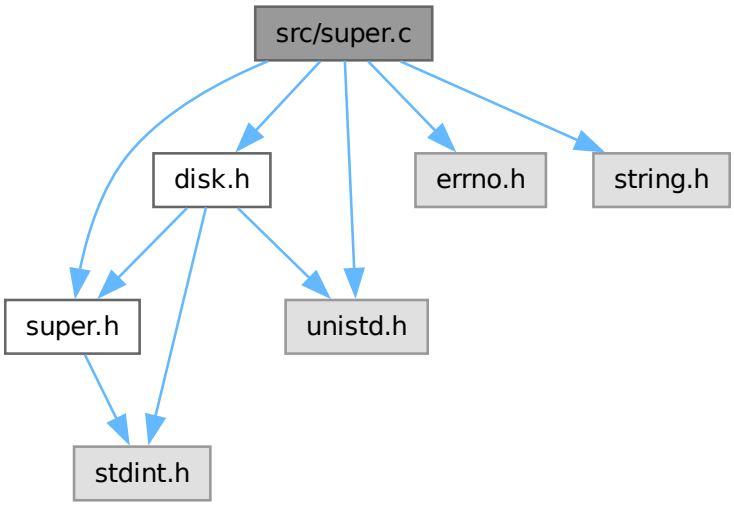
```
}

// 확장 속성을 지원하지 않음: 빈 목록 반환으로 안정적으로 처리
static int sfuse_listxattr_cb(const char *path, char *list, size_t size) {
  return 0; // 빈 리스트 반환 (성공적으로 처리됨)
}

// 확장 속성을 지원하지 않음: ENODATA(ENOATTR) 반환으로 안정적으로 처리
static int sfuse_getxattr_cb(const char *path, const char *name, char *value,
                             size_t size) {
  return -ENODATA; // 해당 속성 없음 처리 (안정적으로 처리됨)
}

// fuse_operations 구조체에 추가하여 최종 적용
const struct fuse_operations sfuse_ops = {
    .init = sfuse_init_cb,
    .destroy = sfuse_destroy_cb,
    .getattr = sfuse_getattr_cb,
    .access = sfuse_access_cb,
    .readdir = sfuse_readdir_cb,
    .open = sfuse_open_cb,
    .read = sfuse_read_cb,
    .write = sfuse_write_cb,
    .create = sfuse_create_cb,
    .mkdir = sfuse_mkdir_cb,
    .unlink = sfuse_unlink_cb,
    .rmdir = sfuse_rmdir_cb,
    .rename = sfuse_rename_cb,
    .truncate = sfuse_truncate_cb,
    .utimens = sfuse_utimens_cb,
    .flush = sfuse_flush_cb,
    .fsync = sfuse_fsync_cb,
    .statfs = sfuse_statfs_cb,
    .getxattr = sfuse_getxattr_cb,
    .listxattr = sfuse_listxattr_cb,
};
```

8.38 src/super.c 파일 참조

SFUSE 파일 시스템의 슈퍼블록 관리 함수 구현

#include "super.h"
#include "disk.h"
#include <errno.h>
#include <string.h>
#include <unistd.h>

super.c에 대한 include 의존 그래프



함수

- int sb_load (int fd, struct sfuse_super *sb)
  디스크에서 슈퍼블록을 읽어오는 함수
- int sb_sync (int fd, const struct sfuse_super *sb)
  슈퍼블록 구조체를 디스크에 쓰기(동기화)하는 함수
- void sb_format (struct sfuse_super *sb)
  슈퍼블록 구조체를 기본값으로 초기화(포맷)하는 함수

8.38.1 상세한 설명

SFUSE 파일 시스템의 슈퍼블록 관리 함수 구현
이 소스 파일은 슈퍼블록을 로드, 동기화, 포맷하는 함수를 구현한다.
super.c 파일에서 정의되었습니다.

8.38.2 함수 문서화

8.38.2.1 sb_format()

void sb_format (
struct sfuse_super * sb)

슈퍼블록 구조체를 기본값으로 초기화(포맷)하는 함수
슈퍼블록 구조체를 기본 값으로 초기화하는 함수
이 함수는 파일 시스템을 처음 생성할 때 호출되며, 슈퍼블록 구조체에 초기 설정값을 채워서 파일 시스템의 메타데이터를 초기화한다.

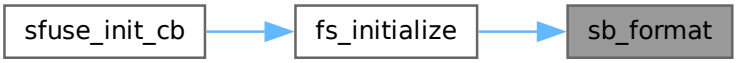
매개변수

sb	초기화할 슈퍼블록 구조체의 포인터

super.c 파일의 89 번째 라인에서 정의되었습니다.

```
{
memset(sb, 0, sizeof(*sb));
sb->magic = SFUSE_MAGIC;
sb->inodes_count = SFUSE_INODES_COUNT;
sb->blocks_count = SFUSE_BLOCKS_COUNT;
sb->free_inodes = sb->inodes_count - 1;
sb->free_blocks = sb->blocks_count - SFUSE_DATA_BLOCK_START;
sb->inode_bitmap_start = SFUSE_INODE_BITMAP_BLOCK;
sb->block_bitmap_start = SFUSE_BLOCK_BITMAP_BLOCK;
sb->inode_table_start = SFUSE_INODE_TABLE_BLOCK;
sb->data_block_start = SFUSE_DATA_BLOCK_START;
}
```

이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.38.2.2 sb_load()

```
int sb_load (
            int fd,
            struct sfuse_super * sb)
```

디스크에서 슈퍼블록을 읽어오는 함수
디스크의 정해진 위치에서 슈퍼블록을 읽어 메모리로 로드하고, 읽은 데이터가 정상적인지 유효성을 검사한다. 유효성 검사는 읽은 데이터 크기와 슈퍼블록 매직 넘버를 이용하여 수행된다.

매개변수

fd	디바이스 파일 디스크립터 (열린 블록 디바이스 파일)
sb	읽어온 슈퍼블록을 저장할 구조체 포인터

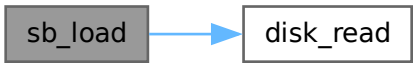
반환값

성공 시 0을 반환, 실패 시 음수 오류 코드 반환 -EIO: 디스크에서 읽은 바이트 수가 요청한 크기와 일치하지 않음 -EINVAL: 슈퍼블록의 매직 넘버가 예상 값과 다름 (슈퍼블록이 손상되었거나 올바르지 않은 디바이스) 기타 음수 값: disk_read 함수 자체가 반환한 오류 코드

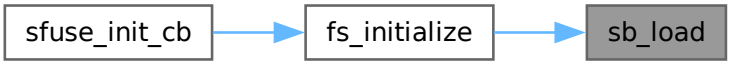
super.c 파일의 29 번째 라인에서 정의되었습니다.

```
{
ssize_t ret;

// 슈퍼블록을 디스크(SFUSE_SUPERBLOCK_OFFSET)에서 읽어 메모리(sb)에 저장한다.
ret = disk_read(fd, sb, sizeof(*sb), SFUSE_SUPERBLOCK_OFFSET);

// disk_read가 실패하면 반환된 음수 오류 값을 그대로 반환한다.
if (ret < 0)
  return ret;

// 요청한 슈퍼블록 크기만큼 정확히 읽었는지 확인한다.
if ((size_t)ret != sizeof(*sb))
  return -EIO; // 읽은 데이터 크기가 슈퍼블록 크기와 다름 (입출력 오류)

// 슈퍼블록의 매직 넘버를 검사하여 슈퍼블록 유효성 검사를 수행한다.
if (sb->magic != SFUSE_MAGIC)
  return -EINVAL; // 잘못된 매직 넘버

// 성공적으로 슈퍼블록을 읽고 유효성을 확인했으므로 0 반환.
return 0;
}
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.38.2.3 sb_sync()

```
int sb_sync (
            int fd,
            const struct sfuse_super * sb)
```

슈퍼블록 구조체를 디스크에 쓰기(동기화)하는 함수
슈퍼블록을 디스크에 동기화(쓰기)하는 함수
메모리 상의 슈퍼블록 구조체의 내용을 디스크의 지정된 위치에 기록하여, 파일 시스템의 메타데이터를 영구적으로 저장한다.

매개변수

fd	디바이스 파일 디스크립터
sb	디스크에 기록할 슈퍼블록 구조체의 포인터

반환값

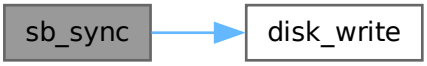
성공 시 0, 실패 시 음수 오류 코드 반환 -EIO: 기록된 바이트 수가 슈퍼블록의 크기와 일치하지 않음 기타 음수 값: disk_write 함수 자체에서 반환한 오류 코드

super.c 파일의 63 번째 라인에서 정의되었습니다.

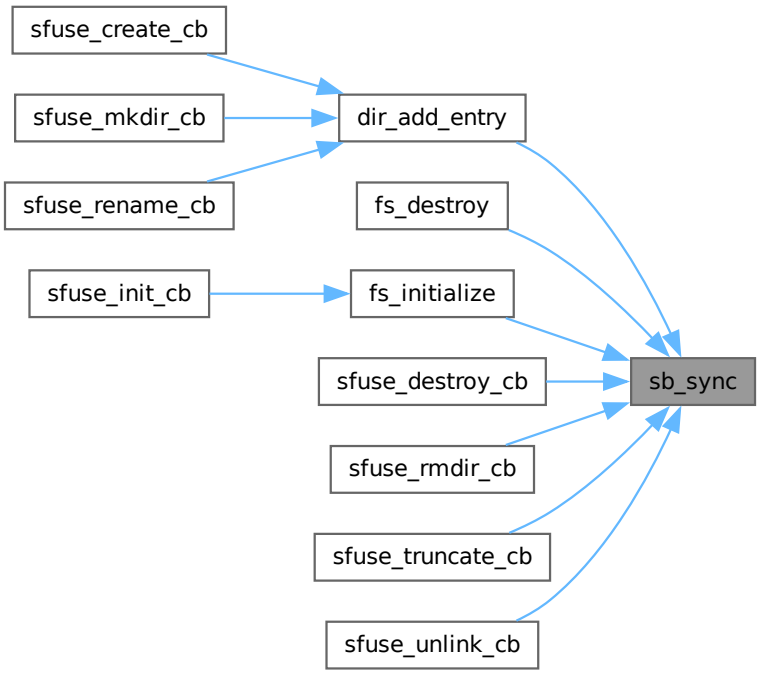
```
00063                                   {
00064   ssize_t ret;
00065
00066   // 슈퍼블록 내용을 디스크(SFUSE_SUPERBLOCK_OFFSET)에 기록
00067   ret = disk_write(fd, sb, sizeof(*sb), SFUSE_SUPERBLOCK_OFFSET);
00068
00069   // 쓰기 실패 시, disk_write 함수가 반환한 음수 오류 코드를 반환
00070   if (ret < 0)
00071     return ret;
00072
00073   // 요청한 슈퍼블록 크기만큼 정확히 기록되었는지 확인
00074   if ((size_t)ret != sizeof(*sb))
00075     return -EIO;
00076
00077   // 정상적으로 슈퍼블록이 디스크에 저장됨
00078   return 0;
```

```
}
```

이 함수 내부에서 호출하는 함수들에 대한 그래프입니다.:



이 함수를 호출하는 함수들에 대한 그래프입니다.:



8.39 super.c

이 파일의 문서화 페이지로 가기

```
#include "super.h"
#include "disk.h"
#include <errno.h>
#include <string.h>
#include <unistd.h>

int sb_load(int fd, struct sfuse_super *sb) {
  ssize_t ret;

  // 슈퍼블록을 디스크(SFUSE_SUPERBLOCK_OFFSET)에서 읽어 메모리(sb)에 저장한다.
  ret = disk_read(fd, sb, sizeof(*sb), SFUSE_SUPERBLOCK_OFFSET);

  // disk_read가 실패하면 반환된 음수 오류 값을 그대로 반환한다.
  if (ret < 0)
```

```
    return ret;

  // 요청한 슈퍼블록 크기만큼 정확히 읽었는지 확인한다.
  if ((size_t)ret != sizeof(*sb))
    return -EIO; // 읽은 데이터 크기가 슈퍼블록 크기와 다름 (입출력 오류)

  // 슈퍼블록의 매직 넘버를 검사하여 슈퍼블록 유효성 검사를 수행한다.
  if (sb->magic != SFUSE_MAGIC)
    return -EINVAL; // 잘못된 매직 넘버

  // 성공적으로 슈퍼블록을 읽고 유효성을 확인했으므로 0 반환.
  return 0;
}

int sb_sync(int fd, const struct sfuse_super *sb) {
  ssize_t ret;

  // 슈퍼블록 내용을 디스크(SFUSE_SUPERBLOCK_OFFSET)에 기록
  ret = disk_write(fd, sb, sizeof(*sb), SFUSE_SUPERBLOCK_OFFSET);

  // 쓰기 실패 시, disk_write 함수가 반환한 음수 오류 코드를 반환
  if (ret < 0)
    return ret;

  // 요청한 슈퍼블록 크기만큼 정확히 기록되었는지 확인
  if ((size_t)ret != sizeof(*sb))
    return -EIO;

  // 정상적으로 슈퍼블록이 디스크에 저장됨
  return 0;
}

void sb_format(struct sfuse_super *sb) {
  memset(sb, 0, sizeof(*sb));
  sb->magic = SFUSE_MAGIC;
  sb->inodes_count = SFUSE_INODES_COUNT;
  sb->blocks_count = SFUSE_BLOCKS_COUNT;
  sb->free_inodes = sb->inodes_count - 1;
  sb->free_blocks = sb->blocks_count - SFUSE_DATA_BLOCK_START;
  sb->inode_bitmap_start = SFUSE_INODE_BITMAP_BLOCK;
  sb->block_bitmap_start = SFUSE_BLOCK_BITMAP_BLOCK;
  sb->inode_table_start = SFUSE_INODE_TABLE_BLOCK;
  sb->data_block_start = SFUSE_DATA_BLOCK_START;
}
```

Index

alloc_block
    bitmap.c, 85
    bitmap.h, 22
alloc_inode
    bitmap.c, 86
    bitmap.h, 23
atime
    sfuse_inode, 17

backing_fd
    sfuse_fs, 16
benchmark/README.md, 84
benchmark/result.md, 21
benchmark/workflow.md, 21
bitmap.c
    alloc_block, 85
    alloc_inode, 86
    bitmap_load, 87
    bitmap_sync, 88
    free_block, 89
    free_inode, 90
bitmap.h
    alloc_block, 22
    alloc_inode, 23
    bitmap_load, 25
    bitmap_sync, 26
    free_block, 28
    free_inode, 29
bitmap_load
    bitmap.c, 87
    bitmap.h, 25
bitmap_sync
    bitmap.c, 88
    bitmap.h, 26
block.c
    read_block, 93
    write_block, 95
block.h
    read_block, 31
    write_block, 34
block_bitmap_start
    sfuse_super, 19
block_map
    sfuse_fs, 16
blocks_count
    sfuse_super, 19

ctime
    sfuse_inode, 17

data_block_start
    sfuse_super, 19
dir.c
    dir_add_entry, 98
    dir_load, 100
    dir_remove_entry, 101
dir.h
    dir_add_entry, 37
    dir_load, 40
    dir_remove_entry, 42
    SFUSE_NAME_LEN, 37
dir_add_entry
    dir.c, 98
    dir.h, 37
dir_load
    dir.c, 100
    dir.h, 40
dir_remove_entry
    dir.c, 101
    dir.h, 42
direct
    sfuse_inode, 18
disk.c
    disk_read, 106
    disk_write, 107
disk.h
    disk_read, 46
    disk_write, 47
disk_read
    disk.c, 106
    disk.h, 46
disk_write
    disk.c, 107
    disk.h, 47
double_indirect
    sfuse_inode, 18

fio 워크플로우, 7
free_block
    bitmap.c, 89
    bitmap.h, 28
free_blocks
    sfuse_super, 19
free_inode
    bitmap.c, 90
    bitmap.h, 29
free_inodes
    sfuse_super, 19
fs.c
    fs_destroy, 110

fs_initialize, 111
fs_resolve_path, 116
fs_split_path, 118
get_fs_context, 120
fs.h
fs_destroy, 51
fs_initialize, 52
fs_resolve_path, 57
fs_split_path, 61
get_fs_context, 63
SFUSE_ROOT_INO, 51
fs_destroy
fs.c, 110
fs.h, 51
fs_init_inode
inode.c, 128
inode.h, 67
fs_initialize
fs.c, 111
fs.h, 52
fs_resolve_path
fs.c, 116
fs.h, 57
fs_split_path
fs.c, 118
fs.h, 61

get_fs_context
fs.c, 120
fs.h, 63
gid
sfuse_inode, 18

HISTORY.md, 21

include/bitmap.h, 21, 30
include/block.h, 31, 36
include/dir.h, 36, 44
include/disk.h, 45, 49
include/fs.h, 50, 64
include/inode.h, 65, 75
include/ops.h, 76, 77
include/super.h, 77, 83
indirect
sfuse_inode, 18
ino
sfuse_dirent, 15
inode.c
fs_init_inode, 128
inode_load, 129
inode_sync, 131
logical_to_physical, 133
inode.h
fs_init_inode, 67
inode_load, 68
inode_sync, 70
logical_to_physical, 72
SFUSE_NDIR_BLOCKS, 66
inode_bitmap_start
sfuse_super, 19
inode_load
inode.c, 129
inode.h, 68
inode_map
sfuse_fs, 16
inode_sync
inode.c, 131
inode.h, 70
inode_table_start
sfuse_super, 20
inodes_count
sfuse_super, 20

links
sfuse_inode, 18
logical_to_physical
inode.c, 133
inode.h, 72

magic
sfuse_super, 20
main
main.c, 138
main.c
main, 138
mode
sfuse_inode, 18
mtime
sfuse_inode, 18

name
sfuse_dirent, 15

ops.c
sfuse_access_cb, 142
sfuse_create_cb, 143
sfuse_destroy_cb, 143
sfuse_flush_cb, 144
sfuse_fsync_cb, 144
sfuse_getattr_cb, 145
sfuse_getxattr_cb, 145
sfuse_init_cb, 146
sfuse_listxattr_cb, 146
sfuse_mkdir_cb, 147
sfuse_open_cb, 147
sfuse_ops, 157
sfuse_read_cb, 148
sfuse_readdir_cb, 149
sfuse_rename_cb, 150
sfuse_rmdir_cb, 151
sfuse_statfs_cb, 152
sfuse_truncate_cb, 153
sfuse_unlink_cb, 154
sfuse_utimens_cb, 155
sfuse_write_cb, 155
ops.h
sfuse_ops, 76

read_block

block.c, 93
block.h, 31
README.md, 84

sb
sfuse_fs, 16
sb_format
super.c, 165
super.h, 79
sb_load
super.c, 166
super.h, 80
sb_sync
super.c, 167
super.h, 81
SFUSE, 1
SFUSE HISTORY, 3
sfuse_access_cb
ops.c, 142
SFUSE_BLOCK_BITMAP_BLOCK
super.h, 78
SFUSE_BLOCK_SIZE
super.h, 78
SFUSE_BLOCKS_COUNT
super.h, 78
sfuse_create_cb
ops.c, 143
SFUSE_DATA_BLOCK_START
super.h, 79
sfuse_destroy_cb
ops.c, 143
sfuse_dirent, 15
ino, 15
name, 15
sfuse_flush_cb
ops.c, 144
sfuse_fs, 15
backing_fd, 16
block_map, 16
inode_map, 16
sb, 16
sfuse_fsync_cb
ops.c, 144
sfuse_getattr_cb
ops.c, 145
sfuse_getxattr_cb
ops.c, 145
sfuse_init_cb
ops.c, 146
sfuse_inode, 17
atime, 17
ctime, 17
direct, 18
double_indirect, 18
gid, 18
indirect, 18
links, 18
mode, 18
mtime, 18
size, 18
uid, 18
SFUSE_INODE_BITMAP_BLOCK
super.h, 79
SFUSE_INODE_TABLE_BLOCK
super.h, 79
SFUSE_INODES_COUNT
super.h, 79
sfuse_listxattr_cb
ops.c, 146
SFUSE_MAGIC
super.h, 79
sfuse_mkdir_cb
ops.c, 147
SFUSE_NAME_LEN
dir.h, 37
SFUSE_NDIR_BLOCKS
inode.h, 66
sfuse_open_cb
ops.c, 147
sfuse_ops
ops.c, 157
ops.h, 76
sfuse_read_cb
ops.c, 148
sfuse_readdir_cb
ops.c, 149
sfuse_rename_cb
ops.c, 150
sfuse_rmdir_cb
ops.c, 151
SFUSE_ROOT_INO
fs.h, 51
sfuse_statfs_cb
ops.c, 152
sfuse_super, 19
block_bitmap_start, 19
blocks_count, 19
data_block_start, 19
free_blocks, 19
free_inodes, 19
inode_bitmap_start, 19
inode_table_start, 20
inodes_count, 20
magic, 20
SFUSE_SUPERBLOCK_OFFSET
super.h, 79
sfuse_truncate_cb
ops.c, 153
sfuse_unlink_cb
ops.c, 154
sfuse_utimens_cb
ops.c, 155
sfuse_write_cb
ops.c, 155
size
sfuse_inode, 18
src/bitmap.c, 84, 91

src/block.c, 93, 96
src/dir.c, 97, 103
src/disk.c, 105, 108
src/fs.c, 109, 121
src/inode.c, 127, 135
src/main.c, 137, 140
src/ops.c, 141, 157
src/super.c, 164, 168
super.c
    sb_format, 165
    sb_load, 166
    sb_sync, 167
super.h
    sb_format, 79
    sb_load, 80
    sb_sync, 81
    SFUSE_BLOCK_BITMAP_BLOCK, 78
    SFUSE_BLOCK_SIZE, 78
    SFUSE_BLOCKS_COUNT, 78
    SFUSE_DATA_BLOCK_START, 79
    SFUSE_INODE_BITMAP_BLOCK, 79
    SFUSE_INODE_TABLE_BLOCK, 79
    SFUSE_INODES_COUNT, 79
    SFUSE_MAGIC, 79
    SFUSE_SUPERBLOCK_OFFSET, 79

uid
    sfuse_inode, 18

write_block
    block.c, 95
    block.h, 34

실험 결과, 5